

2026年7月北美芯片设计/验证题目汇总

根据同学回忆整理 · 122道真题含答案与解析

Apple · Cisco · Intel · Nvidia · Qualcomm · Quadric · Tetramem · Etched · Cirrus Logic

目录

Digital Design

Q1. Digital Design: Overlapping Sequence Detector FSM (1101) and Verification
Q2. Digital Design: Sequential vs Combinational Logic
Q3. Digital Design: Flip-Flop vs Latch
Q4. Digital Design: 2-Stage Synchronizer -- Design and Analysis
Q5. Digital Design: UPF -- Unified Power Format, Flow, and X Propagation
Q6. Digital Design: 4-bit Counter with Hold at 3, 6, 9
Q7. Digital Design: Jump Immediate vs Jump Register (ISA)
Q8. Digital Design: Clock Mux Glitch -- sel ? clk1 : clk2 and Fix
Q9. Digital Design: Implement NAND Gate Using 2-to-1 MUX
Q10. Digital Design: Pipeline Hazards and Cache Mapping
Q11. Digital Design: APB FIFO Design -- RTL Implementation
Q12. Digital Design: Race Condition in Verilog -- Blocking Assignment in always Block
Q13. Digital Design: Verilog/SV Scheduling Regions -- Complete Model
Q14. Digital Design: Async FIFO from Scratch -- Gray Code Pointer Design
Q15. Digital Design: NAND2 Schematic -- Why NMOS Series, PMOS Parallel
Q16. Digital Design: Divide-by-3 Clock -- Logic, Waveform, Verilog, Verification
Q17. Digital Design: Data Activity Monitor -- RTL and Verification
Q18. Digital Design: 4-Input Comparator from 2-Input Comparators
Q19. Digital Design: req Rising Edge -> One-Cycle flag Pulse
Q20. Digital Design: 1-bit D Flip-Flop and D Latch -- Verilog RTL
Q21. Digital Design: 4-to-1 Multiplexer -- Verilog RTL (Multiple Styles)
Q22. Digital Design: Cache Address Structure -- 32KB 4-Way 32B Cacheline
Q23. Digital Design: AXI 4KB Boundary Restriction -- Why and How
Q24. Digital Design: Carry-Select Adder -- Design and Timing Analysis
Q25. Digital Design: Carry-Select Adder -- Design and Timing Analysis
Q26. Digital Design: 1-bit D Flip-Flop Without always Block -- Verilog RTL

SystemVerilog -- Constraints & Randomization

Q27. Constraints: 1D Array -- Unique Elements Without unique, Sum Constraint
Q28. Constraints: 1KB Address Space -- Non-Overlapping 32-byte Stride Regions
Q29. Constraints: 8-bit Vector -- No Cyclic Right Shift
Q30. Constraints: Generate 4-bit Value with Exactly 4 Ones -- No $countones
Q31. Constraints: Array with if-else Constraint, then shuffle()
Q32. Constraints: 7x7 Pixel Image -- No All-D 3x3 Submatrix (Parametrized)
Q33. Constraints: Overriding a Constraint in Child Class -- Three Methods

SystemVerilog -- Assertions (SVA)

Q34. Assertions (SVA): Detect Short Glitch Pulses Shorter Than 2.5ns
Q35. Assertions (SVA): req/gnt 4-Way Handshake -- Concurrent and always Block
Q36. Assertions (SVA): valid Must Remain High Until ready Goes High

Analog / Mixed-Signal Design

Q37. Analog/Mixed-Signal: Stacked NMOS Current Mirror Analysis
Q38. Analog/Mixed-Signal: CMOS Inverter Vin-Vout Transfer Curve, Cascaded Stages
Q39. Analog/Mixed-Signal: NMOS Pass-Gate Chain with Threshold Compensation
Q40. Analog/Mixed-Signal: Ring Oscillator and Single Inverter Feedback

Q41. Analog/Mixed-Signal: Pulse Latch -- Pulse Generator Circuit
Q42. Analog/Mixed-Signal: Logical Effort -- FO3, Sizing, F=GH
Q43. Analog/Mixed-Signal: Circuit Optimization Techniques
Q44. Analog/Mixed-Signal: Two-Stage Inverter VDDh->VDDl vs VDDl->VDDh
Q45. Analog/Mixed-Signal: NAND2 Sizing from Corresponding Inverter
Q46. Analog/Mixed-Signal: Master-Slave DFF -- Component Purpose and Removal Analysis
Q47. Analog/Mixed-Signal: tsu, tc2q Measurement in SPICE; tcq vs tdc Plot
Q48. Analog/Mixed-Signal: 6T SRAM Operation, SNM, and Sizing
Q49. Analog/Mixed-Signal: ROM Types and Sense Amplifier; Timing Order
Q50. Analog/Mixed-Signal: Electromigration -- DC, AC, Iavg, Irms, Black's Equation
Q51. Analog/Mixed-Signal: RC Lowpass Filter -- Transfer Function and Cutoff Frequency

SystemVerilog -- Coding

Q52. SystemVerilog: 2-to-1 MUX and 4-to-1 MUX Implementation
Q53. SystemVerilog: Mixed Blocking/Non-Blocking Cycle-by-Cycle Value Analysis
Q54. SystemVerilog: Named fork-join Disable -- Selective Process Killing
Q55. SystemVerilog: fork-join / join_any / join_none -- Shared Variable Trace
Q56. SystemVerilog: Polymorphism, Typecasting, and Virtual Functions
Q57. SystemVerilog: Generate a Delay Signal 0-1.25x One Clock Cycle
Q58. SystemVerilog: Polymorphism and Virtual Functions -- Output Trace
Q59. SystemVerilog: OOP -- Shape / Circle / Rectangle with Virtual print_area()
Q60. SystemVerilog: fork-join_none Tasks with #Delay -- Value Trace and Semaphore
Q61. SystemVerilog: Sliding Window -- Track Count of 1's (Parameterized)
Q62. SystemVerilog: fork-join_none in for Loop -- Block Without Affecting Others
Q63. SystemVerilog: |-> vs |=> -- Overlapping vs Non-Overlapping Implication
Q64. SystemVerilog: OOP -- Override vs Overload (C++ comparison)
Q65. SystemVerilog: Bit Field Extraction and Variable Bit Selection
Q66. SystemVerilog: Virtual Class (Abstract Class) -- When to Use
Q67. SystemVerilog: Blocking vs Non-Blocking Assignments -- Complete Guide
Q68. SystemVerilog: Sequence Detector 10110 -- Moore / Mealy / Shift Register
Q69. SystemVerilog: C++ Pointer Swap -- ref vs value, const pointer vs pointer to const

Verification -- Test Plan

Q70. Verification: FIFO Verification -- What to Verify, How, Corner Cases
Q71. Verification: Systolic Array UVM Testbench -- PE Level and Sub-System Level
Q72. Verification: Task vs Function in SystemVerilog Testbench
Q73. Verification: 512-bit Transaction Split-Reassemble Over 64-bit Link
Q74. Verification: Out-of-Order Scoreboard -- Track and Match with Associative Array
Q75. Verification: Debug Scoreboard Mismatch -- Step-by-Step Approach
Q76. Verification: Router UVM Testbench -- Complete Design
Q77. Verification: req/ack Handshake Testbench with Latency Checking
Q78. Verification: Out-of-Order Packet Scoreboard with Reconstruction
Q79. Verification: Credit Release Handshake Checker with SVA
Q80. Verification: Lane Redundancy Remapping -- SV Constraint Generator
Q81. Verification: PHY Link Training -- Delay Code Sweep with PRBS Error Injection
Q82. Verification: 12-Channel 32-bit Coverage Strategy
Q83. Verification: UVM Backdoor Access -- HDL Path and Usage
Q84. Verification: Firmware Writes 1 to enable_reg but Device Does Not Start -- Debug
Q85. Verification: DV Environment for Software-Programmed Threshold Transaction Limiter
Q86. Verification: 128-bit Parallel Bus -- Stuck-at Fault and Short Wire Test Patterns
Q87. Verification: Using AI-Generated Code in Verification -- Risks and Workflow
Q88. Verification: Memory Debug -- Frontdoor Write + Backdoor Peek to Find Swap Bug

Q89. Verification: 1-Input 8-Output Router -- sequence_item, Hash Function, Scoreboard
Q90. Verification: Memory Verification -- Complete Strategy and UVM Environment

UVM

Q91. UVM: Highest Priority Completed Sequence
Q92. UVM: Virtual Sequencer -- Two Usage Patterns
Q93. UVM: seq.start(null) -- What Happens When Sequencer is null?
Q94. UVM: Factory Override -- Instance Override and Type Override
Q95. UVM: Mailbox vs Queue vs Associative Array -- When to Use Which
Q96. UVM: Component Hierarchy and Driver-Sequencer Handshake
Q97. UVM: Modify Driver for One Test Without Touching Other Tests
Q98. UVM: APB Driver -- Full Implementation
Q99. UVM: Register Abstraction Layer (RAL) -- Working Principle and Usage
Q100. UVM: uvm_object vs uvm_component -- Differences
Q101. UVM: Test Entry Point -- run_test() to Phase Execution
Q102. UVM: Factory Override -- Why super.build_phase() Must Come AFTER the Override
Q103. UVM: Register Aliasing Detection Using Frontdoor Only
Q104. UVM: 5 Sequences on 1 Sequencer -- Track Which Transaction Came from Which Sequence

Algorithms & Scripting

Q105. Algorithms: Fibonacci -- Recursive vs Iterative, Time Complexity
Q106. Algorithms: Reverse a Linked List
Q107. Algorithms: Sliding Window -- Find Starting Index of Maximum Sum
Q108. Algorithms: Parse Mixed UVM/Custom Simulation Log File
Q109. Algorithms: Check if a Number is a Power of 2 (Bit Manipulation)
Q110. Algorithms: Maximum Subarray Sum with Subset Tracking (Kadane's)
Q111. Algorithms: Count Number of 1 Bits in Binary (Popcount)
Q112. Algorithms: Longest Consecutive Sequence (LC 128)
Q113. Algorithms: Longest Substring Without Duplicates (C++)
Q114. Algorithms: Elevator Cargo -- Minimum Trips with Weight Limit
Q115. Algorithms: Python -- Remove Vowels / Remove Consonants from String
Q116. Algorithms: Sort Python Dictionary by Value; Regex Match Two Strings
Q117. Algorithms: Sort Log File by Age and Write to Output
Q118. Algorithms: Python -- Find Unpaired Numbers in a List
Q119. Algorithms: Find the 3rd Smallest Value in an Integer Stream
Q120. Algorithms: Fibonacci -- Recursive, Iterative, Return Entire List

Brain Teasers

Q121. Brain Teaser: Find the Lighter Stone Among 9 (Balance Scale)
Q122. Brain Teaser: Blue and Red Balls -- What Color Is the Last Ball?

Digital Design

Q1. Digital Design: Overlapping Sequence Detector FSM (1101) and Verification

QUESTION

(a) Design a Mealy FSM that detects the sequence "1101" in a serial bit stream, with OVERLAPPING detection allowed.

(b) How would you verify this FSM? List as many test cases as you can think of.

ANSWER

```
// Standard 3-segment FSM (Three-always-block style)
//
// Segment 1: State register (sequential -- only stores state)
// Segment 2: Next-state logic (combinational -- only computes next)
// Segment 3: Output logic   (combinational -- only computes output)
//
// States:
//   S0: initial / no match
//   S1: matched "1"
//   S2: matched "11"
//   S3: matched "110"   <- next '1' completes "1101", overlap -> S1

module seq_detect_1101 (
    input  logic clk,
    input  logic rst_n,
    input  logic din,
    output logic detected   // Mealy output: valid same cycle as completing bit
);
    typedef enum logic [1:0] {S0, S1, S2, S3} state_e;
    state_e state, next;

    // --- Segment 1: State Register (sequential) ---
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            state <= S0;
        else
            state <= next;
    end

    // --- Segment 2: Next-State Logic (combinational) ---
    always_comb begin
        next = S0;          // safe default to avoid latches
        case (state)
            S0: begin
                if (din) next = S1;
                else     next = S0;
            end
            S1: begin
                if (din) next = S2;   // "11"
                else     next = S0;   // "10" -> reset
            end
            S2: begin
                if (din) next = S2;   // "111..." -> stay at S2 (self-loop)
                else     next = S3;   // "110"
            end
```

```
            S3: begin
                if (din) next = S1;   // "1101" detected! '1' is reused -> S1
                else     next = S0;   // "1100" -> reset
            end
            default: next = S0;
        endcase
    end

    // --- Segment 3: Output Logic (combinational, Mealy) ---
    always_comb begin
        detected = 1'b0;    // safe default
        case (state)
            S3: detected = din ? 1'b1 : 1'b0;  // only S3 + din=1 -> detect
            default: detected = 1'b0;
        endcase
    end

endmodule

// State transition table:
//   State | din=0 | din=1 | detected
//   ------+-------+-------+---------
//   S0    |  S0   |  S1   |    0
//   S1    |  S0   |  S2   |    0
//   S2    |  S3   |  S2   |    0      <- S2 self-loop handles "111..."
//   S3    |  S0   |  S1   |    1*     <- * detected=1 when din=1 (Mealy)
//
// Example: din stream = 1,1,0,1,1,0,1
//   S0->S1->S2->S3->S1(det=1)->S2->S3->S1(det=1)
//   Detections at position 3 and 6 (overlapping reuse of trailing '1')
```

解释

三段式状态机各段职责严格分离，是工业界 RTL 编码规范：

第一段（always_ff）：纯粹的状态寄存器，只做 state <= next，

绝不包含任何逻辑计算，保证综合出来是干净的 DFF。

第二段（always_comb）：纯次态逻辑，只计算 next，

必须给 next 设安全默认值（next = S0 在 case 之前），

否则综合器会报"可能推断出 latch"的警告。

第三段（always_comb）：纯输出逻辑，只计算输出信号，

同样需要给 detected 设默认值避免 latch。

Mealy 输出（输出取决于当前 state + 当前输入 din）：detected 在 S3 且 din=1 时才为1。

S2 的自环是这道题最关键的设计点：连续多个'1'（如"111101"）需要在 S2 停留，

否则会错误地在第三个'1'后就跳出状态机逻辑。

面试中三段式相比两段式/一段式的核心优势：

可读性强、次态/输出完全解耦、易于复用和 lint/CDC 工具分析。

Q2. Digital Design: Sequential vs Combinational Logic

QUESTION

What is the difference between sequential and combinational logic?

Give examples of each.

ANSWER

```
// Combinational Logic:
//   Output depends ONLY on the CURRENT values of inputs.
//   No memory -- same inputs always produce the same outputs immediately
//   (after propagation delay).
//   No clock involved.
//
//   Examples: AND/OR/NOT gates, adders, multiplexers, decoders, ALUs
//
//   module comb_example (
//       input  logic a, b,
//       output logic y
//   );
//       assign y = a & b;   // pure function of current a, b
//   endmodule

// Sequential Logic:
//   Output depends on CURRENT inputs AND PAST history (state).
//   Has memory elements (flip-flops/latches) that store state.
//   Typically clock-driven: state updates on clock edges.
//
//   Examples: flip-flops, registers, counters, FSMs, RAM, FIFOs
//
//   module seq_example (
//       input  logic clk, rst_n, en,
//       output logic [3:0] count
//   );
//       always_ff @(posedge clk or negedge rst_n)
//           if (!rst_n)    count <= 0;
//           else if (en)   count <= count + 1;  // depends on PAST count value
//   endmodule

// Key distinguishing test:
//   "Does the output depend on time / history?"
//   If YES -> sequential (needs memory)
//   If NO  -> combinational (pure function of present inputs)
//
//   A combinational circuit's truth table fully determines its behavior.
//   A sequential circuit needs a STATE DIAGRAM (or state + truth table)
//   because the same input can produce different outputs depending on state.
```

解释

组合逻辑：纯函数，输出完全由当前输入决定，没有"记忆"。

时序逻辑：有状态元件，输出依赖当前输入和历史状态，必须用状态图描述行为。

最简单的判断方法：同样的输入在不同时刻是否可能产生不同输出？

如果可能，说明电路有状态，是时序逻辑。

实际 RTL 中的关键信号：assign 语句通常是组合逻辑，

always_ff @(posedge clk) 块是时序逻辑的标准写法。

Q3. Digital Design: Flip-Flop vs Latch

QUESTION

What is the difference between a flip-flop and a latch?

ANSWER

```
// Latch: LEVEL-SENSITIVE storage element
//   Transparent (output follows input) WHILE the enable/clock is at
//   a certain LEVEL (e.g. high). Output is "frozen" when enable is low.
//
//   module d_latch (
//       input  logic d, en,
//       output logic q
//   );
//       always_latch
//           if (en) q <= d;    // transparent while en=1
//                              // (implicit: holds q when en=0)
//   endmodule
//
//   Behavior: as long as en=1, q tracks d in real-time (combinational-like).
//   When en falls to 0, q freezes at whatever d was at that instant.

// Flip-Flop: EDGE-TRIGGERED storage element
//   Output only updates at the active CLOCK EDGE (posedge or negedge),
//   not during the rest of the clock cycle.
//
//   module d_ff (
//       input  logic clk, d,
//       output logic q
//   );
//       always_ff @(posedge clk)
//           q <= d;     // updates ONLY at rising edge of clk
//   endmodule
//
//   Behavior: q only changes at the instant of posedge clk, then holds
//   that value for the entire rest of the clock period, regardless of
//   how d changes in between.

// Key differences table:
//
//   Property              | Latch                  | Flip-Flop
//   ----------------------+------------------------+-----------------------
//   Trigger               | Level-sensitive        | Edge-triggered
//   Transparency          | Transparent when en=1  | Never transparent
//   Timing analysis       | Harder (time-borrowing) | Easier (clock-edge based)
//   Transistor count      | Fewer (simpler)        | More (master-slave: 2 latches)
//   Typical use           | Avoided in sync design | Standard for synchronous RTL
//                         | (used in custom/clock- |
//                         |  gating circuits)      |
//   Unintended inference  | Common bug (incomplete | N/A
//                         | if/case in always_comb) |
//
// Why flip-flops are preferred for synchronous design:
//   Edge-triggered behavior makes static timing analysis (STA) much
//   simpler -- setup/hold checks are well-defined relative to one clock edge.
//   Latches require "time-borrowing" analysis because they can be
//   transparent for a fraction of the clock cycle, making timing closure
//   significantly harder in large designs.
//
// Common interview follow-up: how is a flip-flop built from latches?
//   A standard D flip-flop = 2 latches in MASTER-SLAVE configuration:
//   Master latch transparent when clk=0, slave latch transparent when clk=1.
//   This ensures data is only sampled once per clock cycle.
```

解释

Latch 是电平触发，FF 是边沿触发，这是两者最核心的区别。

Latch 在 enable 为高的整个窗口内透明，意味着时序分析必须考虑

"time borrowing"（输入可以借用部分周期时间），分析复杂。

FF 只在时钟边沿瞬间采样，之后整个周期保持不变，STA分析直接基于

单一时钟边沿，这是同步设计偏爱使用 FF 而非 latch 的根本原因。

常见的面试追问"FF怎么用latch搭建"：master-slave结构，

两个latch分别在clk的高低电平透明，保证每个时钟周期只采样一次数据，

这正是Q...DFF Master-Slave那道题的核心电路。

Q4. Digital Design: 2-Stage Synchronizer -- Design and Analysis

QUESTION

Design a 2-stage (2-flip-flop) synchronizer for crossing a single-bit signal from one clock domain to another. Explain:

(a) Why 2 stages (not 1, not 3)?

(b) What is MTBF and how does it relate to synchronizer depth?

(c) What are the limitations of a 2-FF synchronizer for multi-bit signals?

ANSWER

```
// (a) RTL Implementation

module sync_2ff (
    input  logic clk_dst,    // destination clock domain
    input  logic rst_n,
    input  logic async_in,   // signal from source clock domain (or async source)
    output logic sync_out
);
    logic meta_ff;   // first stage -- may go metastable
    logic sync_ff;   // second stage -- resolved/stable output

    always_ff @(posedge clk_dst or negedge rst_n) begin
        if (!rst_n) begin
            meta_ff <= 1'b0;
            sync_ff <= 1'b0;
        end else begin
            meta_ff <= async_in;   // stage 1: may capture mid-transition (metastable)
            sync_ff <= meta_ff;    // stage 2: gives stage 1 time to resolve
        end
    end

    assign sync_out = sync_ff;

endmodule

// (a) Why exactly 2 stages?
//
// 1 stage (single FF): if async_in changes near the clock edge, the FF
//   may enter METASTABILITY -- its output hovers at an invalid voltage
//   level for an unpredictable time before settling to 0 or 1.
//   A single FF's metastable output can propagate DOWNSTREAM before
//   it resolves -- this is unsafe.
//
// 2 stages: the FIRST FF may go metastable, but it has a FULL CLOCK
//   PERIOD to resolve before the SECOND FF samples it. By the time
//   stage 2 samples stage 1's output, stage 1 has (with very high
//   probability) settled to a stable 0 or 1.
```

```
//   The output of stage 2 (sync_out) is therefore safe to use
//   throughout the rest of the destination clock domain.
//
// 3+ stages: adds MORE resolution time (lower failure probability,
//   see MTBF below) at the cost of MORE LATENCY (extra clock cycles
//   before the signal reaches downstream logic). 2 stages is the
//   INDUSTRY STANDARD baseline; 3+ stages used only in very
//   high-frequency or high-reliability designs (e.g. aerospace,
//   safety-critical systems) where failure probability must be
//   even lower.

// (b) MTBF (Mean Time Between Failures)
//
// MTBF quantifies how often a metastability-induced synchronization
// FAILURE is expected to occur (i.e. metastable value doesn't resolve
// in time and propagates an invalid value downstream).
//
//   MTBF = e^(t_resolve / tau) / (T_clk * f_data * C)
//
//   Where:
//     t_resolve = time available for metastability to resolve
//                 (clock period minus setup/hold/propagation margins)
//     tau       = technology-dependent time constant (smaller tau = faster
//                 resolution, better process)
//     T_clk     = destination clock period
//     f_data    = frequency of async signal transitions
//     C         = technology constant
//
//   Key insight: MTBF grows EXPONENTIALLY with t_resolve.
//   Going from 1 stage to 2 stages roughly DOUBLES t_resolve
//   (one extra full clock period to resolve), which makes MTBF
//   astronomically larger (e.g. from milliseconds to YEARS or CENTURIES).
//   This exponential relationship is WHY 2 stages is "good enough"
//   for most designs -- the marginal benefit of a 3rd stage is much
//   smaller in relative terms (still exponential, but starting from
//   an already-huge MTBF).

// (c) Limitations for multi-bit signals
//
// The 2-FF synchronizer ONLY works safely for a SINGLE BIT.
// For multi-bit buses, naively applying 2-FF sync to EACH BIT
// independently is BROKEN:
//
//   Problem: different bits may resolve metastability at DIFFERENT
//   times (or sample different "versions" of a multi-bit value if
//   multiple bits change simultaneously near the clock edge).
//   Result: the destination domain may see a TRANSIENT, INVALID
//   combination of bits that never existed in the source domain
//   (e.g. source goes 3'b011 -> 3'b100, but destination
//   momentarily sees 3'b111 or 3'b000 -- neither is a valid value
//   that ever existed at the source).
//
//   Solutions for multi-bit CDC:
//   1. Gray code encoding: ensure only ONE bit changes between
//      consecutive values (used for FIFO pointers) -- if metastability
//      hits, the result is either the old or new value, never garbage.
//   2. Async FIFO: the standard solution for multi-bit DATA crossing
//      (not just pointers) -- uses full handshake + gray-coded pointers.
//   3. MUX-based synchronizer with enable/valid handshake: hold the
//      multi-bit data stable, synchronize a SINGLE valid/enable bit
//      across domains, only sample the data bus once the synchronized
//      enable confirms it's stable and settled.
```

解释

2级同步器的核心原理：给第一级 FF 留出整整一个时钟周期的"亚稳态恢复时间"，

第二级 FF 采样时第一级几乎必定已经稳定，从而把亚稳态传播的风险降到极低。

MTBF 公式中最重要的认知：故障间隔时间随恢复时间呈指数增长，

这意味着2级同步带来的 MTBF 提升是数量级的（从毫秒级到年/世纪级），

这正是工业界统一采用2级（而非1级或更多级）作为标准做法的根本原因。

多比特信号的陷阱是这道题最容易被忽略的扩展点：

逐位同步会导致"组合错误"（destination看到一个源端从未出现过的中间值），

必须用 Gray code（保证相邻值只差1位）或完整的 async FIFO 握手机制来解决，

这也是 Section 8 Verification 里 Async FIFO 系列题目的设计动机。

Q5. Digital Design: UPF -- Unified Power Format, Flow, and X Propagation

QUESTION

(a) What is UPF (Unified Power Format)?

(b) What are the key steps in the UPF-based low-power design flow?

(c) How does X (unknown/undefined) affect hardware in a low-power design?

ANSWER

```
// (a) What is UPF? ------------------------------------------------------------
//
// UPF (Unified Power Format, IEEE 1801) is a TCLLIB-based specification
// language for describing a chip's POWER INTENT:
//   - Power domains (which logic blocks belong to which supply rail)
//   - Supply nets and power states (which voltages are available when)
//   - Isolation cells (prevent unknown signals from a powered-down domain
//     from corrupting a live domain)
//   - Retention registers (save and restore FF state across power-down cycles)
//   - Level shifters (convert signals between different voltage domains)
//   - Power switches (gates the supply to a power domain on/off)
//
// UPF separates POWER INTENT from RTL, allowing the same RTL to be
// used with different power management strategies.
//
// Key constructs:
//   create_power_domain  "PD_CPU" -elements {cpu_core}
//   create_supply_net    "VDD_CPU"
//   connect_supply_net   "VDD_CPU" -ports {PD_CPU/VDD}
//   set_isolation "ISO_CPU" -domain PD_CPU -isolation_power_net VDD_ALWAYS_ON
//   set_retention "RET_CPU" -domain PD_CPU -retention_power_net VDD_SAVE
//   set_level_shifter "LS_01" -domain PD_CPU -applies_to outputs

// (b) UPF-based low-power design flow steps -----------------------------------
//
//   Step 1: POWER INTENT DEFINITION (UPF authoring)
//     - Define power domains, supply networks, and power states
//     - Specify which domains can be shut down / voltage-scaled
//     - Document expected power state transitions
//
//   Step 2: RTL POWER-AWARE SIMULATION (with UPF)
//     - Simulate RTL + UPF together to verify power behavior
//     - Tool infers isolation cells, level shifters as needed
```

```
//     - Verify that powered-down domains correctly output X (not 0/1)
//     - Verify isolation cells correctly clamp outputs during power-down
//
//   Step 3: SYNTHESIS (power-aware)
//     - Synthesis tool reads RTL + UPF
//     - Automatically inserts: isolation cells, level shifters, power switches
//     - Retention registers replace regular FFs in retentive domains
//
//   Step 4: PHYSICAL IMPLEMENTATION (P&R with UPF)
//     - Power rings and stripes created per domain supply requirements
//     - Power switches placed, power domains physically partitioned
//     - UPF constraints guide placement of isolation/retention cells
//
//   Step 5: SIGN-OFF VERIFICATION
//     - Gate-level simulation with UPF: verify isolation, retention
//     - Power analysis: static (leakage) and dynamic (switching) power
//     - Low-power structural checks (LPLVS, LPERC): verify all required
//       cells (ISO, LS, RSFF) are correctly inserted and connected
//
//   Step 6: POWER STATE VALIDATION
//     - Verify all power state sequences (ON -> RETENTION -> OFF -> ON)
//     - Check that save/restore sequences for retention registers work
//     - Validate that wake-up time meets spec

// (c) X propagation effects on hardware in low-power design -----------------
//
// When a power domain is SHUT DOWN, its outputs are electrically floating
// (neither driven to a valid logic level). Simulation models this as X.
//
// Hardware risks from X propagation WITHOUT proper isolation:
//
//   1. CORRUPTED DATA in the always-on domain:
//      If a flip-flop in the always-on domain samples an X from a
//      powered-down domain, the FF latches to an arbitrary 0 or 1
//      (determined by transistor mismatch at the metastable point).
//      This can cause functional failures that are hard to reproduce.
//
//   2. LATCH-UP risk:
//      Floating gate inputs on CMOS logic can draw unexpected current,
//      potentially triggering latch-up in parasitic SCR structures --
//      a potentially destructive condition.
//
//   3. EXCESSIVE POWER DRAW (shoot-through):
//      If both PMOS and NMOS in an inverter stage are simultaneously
//      on (due to a floating input settling at mid-rail ~VDD/2),
//      a short-circuit current path from VDD to GND exists -- this
//      causes anomalous power consumption in the "off" state.
//
//   4. FALSE TRIGGERING of downstream logic:
//      X propagating through combinational logic can cause gate
//      outputs to toggle spuriously, creating glitches that may
//      be captured by FFs as false state transitions.
//
// How UPF isolation cells fix this:
//   An isolation cell sits at the BOUNDARY of a powered-down domain.
//   Its enable input is driven by the always-on control logic.
//   When the domain powers down, the isolation cell CLAMPS its output
//   to a fixed known value (0 or 1, per UPF set_isolation spec)
//   BEFORE the supply is removed -- ensuring no X ever propagates out.
```

解释

UPF 的核心价值是把"如何管理电源"从 RTL 本身分离出来，

同一套 RTL 可以搭配不同的 UPF 描述来实现不同的低功耗策略，

符合关注点分离（separation of concerns）的工程原则。

X 传播在低功耗设计里是特别危险的问题：

不像逻辑功能 bug（往往仿真立即失败），X 传播导致的硬件问题可能

只在特定功耗状态序列下才暴露，且每次复现的症状可能不同（取决于那个特定 FF 的物理偏差）。

Isolation cell 的"先夹再断"时序至关重要：

必须在断开电源之前就使能 isolation，让 clamp 电平生效；

如果顺序反了（先断电再夹），X 已经逃逸到外部——这是低功耗设计中最常见的功能 bug 类型之一。

Q6. Digital Design: 4-bit Counter with Hold at 3, 6, 9

QUESTION

Implement a 4-bit counter with the following specification:

- Inputs: clk, rstn (active-low async reset), en (enable)

- Output: count [3:0]

- On rstn low: count resets to 0 immediately (asynchronous)

- On en high: counter increments each cycle, wrapping 9 -> 0 (BCD-like)

- When count reaches 3, 6, or 9: hold at that value for ONE extra

enable cycle before continuing

ANSWER

```
// Design approach:
//   Use a hold_flag register that is set when count reaches 3/6/9.
//   On the next enabled cycle, if hold_flag is set: clear it and do NOT
//   increment (stay at current count). The cycle after that, count resumes.
//
// Waveform (en always high):
//   count: 0 1 2 3 3 4 5 6 6 7 8 9 9 0 1 2 3 3 ...
//                ^ ^       ^ ^       ^ ^
//                reach 3, hold 1 extra cycle, then continue

module counter_hold (
    input  logic       clk,
    input  logic       rstn,    // active-low asynchronous reset
    input  logic       en,
    output logic [3:0] count
);
    logic hold_flag;    // 1 = currently in hold state (one cycle)

    always_ff @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            count     <= 4'd0;
            hold_flag <= 1'b0;
        end else if (en) begin
            if (hold_flag) begin
                // Holding: stay at current count, clear hold flag
                hold_flag <= 1'b0;
                // count unchanged

            end else begin
                // Normal increment
```

```
                if (count == 4'd9)
                    count <= 4'd0;       // wrap 9 -> 0
                else
                    count <= count + 1'b1;

                // If we just moved TO 3, 6, or 9: set hold flag
                // (the NEW count value, after increment)
                if ((count == 4'd2) ||   // next count = 3
                    (count == 4'd5) ||   // next count = 6
                    (count == 4'd8))     // next count = 9
                    hold_flag <= 1'b1;
                else
                    hold_flag <= 1'b0;
            end
        end
        // en=0: count and hold_flag both unchanged (implicit)
    end

endmodule

// Cycle-by-cycle trace (en=1 always, rstn=1):
//
//   Cycle | count | hold_flag | note
//   ------+-------+-----------+-----------------
//     0   |   0   |     0     | reset state
//     1   |   1   |     0     | count 0->1
//     2   |   2   |     0     | count 1->2
//     3   |   3   |     1     | count 2->3, set hold
//     4   |   3   |     0     | HOLD: count stays 3, clear hold
//     5   |   4   |     0     | count 3->4
//     6   |   5   |     0     | count 4->5
//     7   |   6   |     1     | count 5->6, set hold
//     8   |   6   |     0     | HOLD: count stays 6, clear hold
//     9   |   7   |     0     | count 6->7
//    10   |   8   |     0     | count 7->8
//    11   |   9   |     1     | count 8->9, set hold
//    12   |   9   |     0     | HOLD: count stays 9, clear hold
//    13   |   0   |     0     | count 9->0 (wrap)
//    14   |   1   |     0     | count 0->1
//    ...  repeat
```

解释

关键设计决策：hold_flag 检查的是"当前 count"（即将被递增之前的值），

而不是"递增后的 count"。递增前 count==2 对应递增后 count==3，

所以 hold_flag 的条件写成 count==2, 5, 8（递增前的前驱值）。

这样写的好处：hold_flag 和 count 的新值在同一个时钟沿同时更新，

不需要引入额外的 combinational 路径来看"新 count 是否是 3/6/9"。

另一种等价写法：先把新 count 算出来存入 combinational 变量，

再判断新值是否为 3/6/9 来设 hold_flag，逻辑更直觉但需要一个 always_comb：

// always_comb begin

// next_count = (count==9) ? 0 : count+1;

// next_hold = (next_count==3 || next_count==6 || next_count==9) && !hold_flag;

// end

两种写法综合结果相同，前者在单 always_ff 里更简洁；

后者把次态逻辑显式分离，更符合三段式规范，可读性更好，推荐在实际工程中使用。

Q7. Digital Design: Jump Immediate vs Jump Register (ISA)

QUESTION

In an instruction set architecture (ISA), what is the difference between

a Jump Immediate (J-type) instruction and a Jump Register instruction?

When is each used?

ANSWER

```
// Jump Immediate (J-type / JAL in RISC-V):
//   The target address is encoded DIRECTLY in the instruction word
//   as a SIGNED IMMEDIATE OFFSET relative to the current PC.
//
//   target_addr = PC + sign_extend(immediate)
//
//   Instruction encoding example (RISC-V JAL):
//   [31:12] = imm[20|10:1|11|19:12]   (20-bit immediate, encoded non-linearly)
//   [11:7]  = rd                        (return address register)
//   [6:0]   = opcode
//
//   Properties:
//   - Target address known at COMPILE TIME (static)
//   - Range is limited by the immediate field bit width
//     (RISC-V JAL: +/- 1MB offset from PC)
//   - Fast: no register read needed to compute target
//   - Used for: direct function calls, loops, unconditional forward/backward jumps
//   - Example: JAL ra, 100   // PC += 100, save return addr to ra

// Jump Register (JALR in RISC-V, JR in MIPS):
//   The target address is computed from the VALUE IN A REGISTER
//   plus an optional small immediate offset.
//
//   target_addr = rs1 + sign_extend(imm12)    (RISC-V JALR)
//   target_addr = rs                           (MIPS JR, no offset)
//
//   Properties:
//   - Target address known only at RUNTIME (dynamic)
//   - Range is the FULL address space (limited only by register width)
//   - Requires a register read before computing target (slightly more work)
//   - Used for:
//     1. Return from function: JALR x0, ra, 0  (jump to address in ra)
//     2. Indirect function calls: function pointer (address in register)
//     3. Jump tables: switch-case dispatch via base+offset
//     4. Dynamic linking / PLT (Procedure Linkage Table) stubs

// Summary table:
//
//   Property             | Jump Immediate (JAL)    | Jump Register (JALR/JR)
//   ---------------------+-------------------------+------------------------
//   Target known when?   | Compile time (static)   | Runtime (dynamic)
//   Target computation   | PC + immediate          | Register + immediate
//   Range                | Limited (imm bit width) | Full address space
//   Use case             | Direct calls, loops     | Returns, fn pointers, jump tables
//   Branch prediction    | Easier (known target)   | Harder (indirect -- needs BTB/RAS)
```

解释

JAL（Jump And Link）和 JALR（Jump And Link Register）是 RISC-V 的实现，

MIPS 的对应是 J/JAL 和 JR/JALR。

核心区别在于"跳转目标在编译时确定还是运行时确定"：

Immediate 版本把目标偏移量编码进指令本身，编译器直接写死，执行快；

Register 版本把目标地址存在寄存器里，可以在运行时改变，灵活性高但需要间接寻址。

硬件实现角度的关键差异：Jump Immediate 的目标地址在 decode 阶段就能算出，

分支预测器可以静态预测；Jump Register 的目标在 execute 阶段才知道，

需要 Return Address Stack（RAS）来预测 JALR-based return，

或分支目标缓冲器（BTB）来预测 indirect jump——这是乱序处理器设计的重要挑战。

Q8. Digital Design: Clock Mux Glitch -- sel ? clk1 : clk2 and Fix

QUESTION

You have two clocks (clk1, clk2) with different frequencies and phases.

If you use "assign out_clk = sel ? clk1 : clk2" as a clock mux,

what is the problem? How do you fix it?

ANSWER

```
// ================================================================
// Problem: Glitch on the output clock
// ================================================================
//
// When sel changes while EITHER clk1 OR clk2 is HIGH, the output
// out_clk = sel ? clk1 : clk2 can produce a GLITCH:
//
// Example: clk1=1, clk2=0, sel changes 0->1:
//   Before: out_clk = clk2 = 0
//   During: mux switching time -> out_clk may briefly show a spike
//   After:  out_clk = clk1 = 1
//   Result: out_clk has a SPURIOUS PULSE (glitch) at the transition.
//
// Example: sel changes 1->0 while clk1=1:
//   out_clk was following clk1 (high), now switches to clk2 (low).
//   This can create a SHORTENED PULSE on out_clk -- an incomplete cycle.
//
// WHY is this catastrophic for clocked logic?
//   Downstream flip-flops treat any glitch on their clock as a real
//   clock edge. A spurious glitch causes flip-flops to sample data at
//   the WRONG time -> corrupted state, impossible to debug.
//   Standard timing analysis (STA) does NOT account for glitches on clocks.

// ================================================================
// Fix: Glitch-free clock mux using double synchronization
// ================================================================
//
// Key principle: only switch clocks when BOTH clocks are LOW
// (or at a defined edge). This ensures the mux transition never
// creates a partial pulse on the output.
//
// Implementation: synchronize sel to EACH clock domain before gating.

module clk_mux_glitch_free (
    input  logic clk1,
    input  logic clk2,
    input  logic sel,       // 0 = select clk1, 1 = select clk2
```

```
    output logic out_clk
);
    logic sel_sync1_r1, sel_sync1_r2;  // sel synced to clk1 domain
    logic sel_sync2_r1, sel_sync2_r2;  // sel synced to clk2 domain
    logic en1, en2;                     // gating enables

    // Step 1: Synchronize sel to clk1 (negedge -- deassert during low phase)
    always_ff @(negedge clk1) begin
        sel_sync1_r1 <= ~sel & ~en2;   // enable clk1 when sel=0 and clk2 not active
        sel_sync1_r2 <= sel_sync1_r1;
    end
    assign en1 = sel_sync1_r2;

    // Step 2: Synchronize sel to clk2 (negedge)
    always_ff @(negedge clk2) begin
        sel_sync2_r1 <=  sel & ~en1;   // enable clk2 when sel=1 and clk1 not active
        sel_sync2_r2 <= sel_sync2_r1;
    end
    assign en2 = sel_sync2_r2;

    // Step 3: Gate each clock with its enable, then OR together
    assign out_clk = (clk1 & en1) | (clk2 & en2);
    // When en1=1 and en2=0: out_clk = clk1 (low-phase gated)
    // When en1=0 and en2=1: out_clk = clk2
    // Switching only occurs when BOTH enables are 0 (transition window)

endmodule

// ================================================================
// Why negedge synchronization?
// ================================================================
//
//   We sample sel at the FALLING EDGE of each clock (negedge clk1/clk2).
//   This guarantees that by the time the enable propagates to the clock
//   gate (AND gate), the clock is in its LOW phase.
//   Gating a clock during its LOW phase ensures the output clock
//   completes its current HIGH pulse cleanly before switching.
//   No partial pulse = no glitch.
//
// ================================================================
// Alternative simpler fix (if timing allows)
// ================================================================
//
//   If clk1 and clk2 share a known relationship (e.g. both derived
//   from a common source), use a clock mux in the clock tree
//   controlled by an already-synchronized enable signal:
//
//   always_ff @(posedge common_ref_clk) sel_sync <= sel;
//   assign out_clk = sel_sync ? clk2 : clk1;
//
//   But for fully asynchronous clk1/clk2, the negedge-synchronized
//   glitch-free mux above is the industry-standard solution.
```

解释

组合逻辑时钟选择器（sel ? clk1 : clk2）的危险在于：

sel 切换时，如果任一时钟处于高电平，输出会产生毛刺脉冲（短于正常周期的跳变），

下游 FF 会把这个毛刺当作真实时钟边沿触发，导致数据损坏，且 STA 无法检测。

Glitch-free 时钟切换的核心思想："在时钟低电平期间完成切换"：

用 negedge 同步 sel 到各自时钟域，确保使能信号只在时钟低相位改变，

门控 AND 操作保证当前高电平脉冲完整结束后才切换。

互斥逻辑（`~sel & ~en2` 和 `sel & ~en1`）防止两个时钟同时使能（避免 OR 短路产生毛刺）。

这是 SoC 低功耗设计和多时钟域设计中的标准电路，CDC 工具和时钟树综合工具

都有对应的检查规则来确保时钟切换的安全性。

Q9. Digital Design: Implement NAND Gate Using 2-to-1 MUX

QUESTION

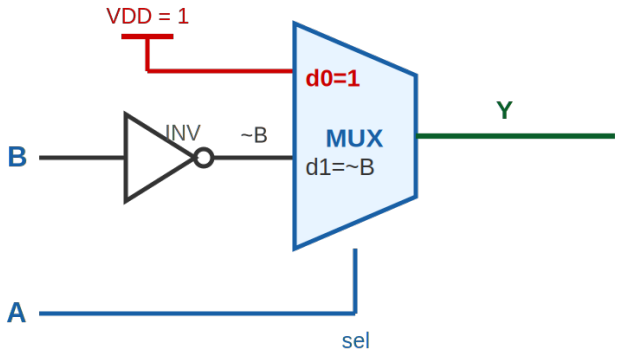
Draw the logic gate circuit to implement a NAND gate using

a 2-to-1 multiplexer (and a minimal number of additional gates).

Recall: NAND(A,B) = NOT(A AND B) = ~A | ~B

ANSWER

2-to-1 MUX as NAND Gate: Y = NAND(A, B)



sel=A, d0=1, d1=~B

Y = A ? ~B : 1 = NAND(A,B)

Truth Table

A	B	~B	picks	Y
0	0	1	d0=1	1
0	1	0	d0=1	1
1	0	1	d1=1	1
1	1	0	d1=0	0

```
// Truth table of NAND:
//   A  B | NAND
//   0  0 |  1
//   0  1 |  1
//   1  0 |  1
//   1  1 |  0
//
// 2-to-1 MUX:  Y = sel ? d1 : d0
//
// Key insight: when A=0, NAND output is always 1 (regardless of B).
//              when A=1, NAND output = NOT(B).
//
// So: Y = A ? ~B : 1
//   = mux(sel=A, d1=~B, d0=1)
//
// But we want to use only ONE mux (no external inverter for ~B).
// Alternative: route B to BOTH inputs and exploit MUX inversion.
//
// NAND(A,B) = ~A | ~B   (De Morgan)
// Observe:
//   When B=0: output = 1 always  (so d0=1, or route B directly)
```

```
//   When B=1: output = ~A        (so d1=~A, or route ~A)
//
// Construction using ONE 2-to-1 MUX:
//   sel = B
//   d0  = 1       (B=0: output is 1)
//   d1  = ~A      (B=1: output is NOT A)
//   Y = B ? ~A : 1   -- but we still need ~A...
//
// Trick: use A as BOTH sel AND inverted data input:
//   sel = A
//   d1  = 0     (when A=1, output = NOT B... no)
//
// Cleanest single-MUX solution:
//   Y = mux(sel=A, d0=1, d1=~B)
//     = A ? ~B : 1
//
//   A=0: Y=1          (NAND correct)
//   A=1, B=0: Y=~0=1  (NAND correct)
//   A=1, B=1: Y=~1=0  (NAND correct) -- verified!
//
// In SystemVerilog:
assign y = a ? ~b : 1'b1;
//
// If only MUX primitive allowed (no ~ operator):
//   Route b through a second MUX stage as inverter:
//   MUX1: b_inv = mux(sel=b, d0=1, d1=0) -- inverter from MUX
//   MUX2: y     = mux(sel=a, d0=1, d1=b_inv)
//   Two MUXes total if inverter must also be MUX-based.
//
// Alternative single-MUX: tie d0=d1=~A, but that only works for ~A not NAND.
// The cleanest answer is always: Y = sel(A) ? ~B : 1
```

解释

这道题的关键是利用 MUX 的"选择"特性来实现逻辑函数。

NAND(A,B) 的分析：当 A=0 时输出恒为1；当 A=1 时输出为 ~B。

用 A 作为 sel，d0=1（A=0时选），d1=~B（A=1时选），精确匹配 NAND 真值表。

如果约束"不能有任何外部逻辑只能用 MUX"，则需要两个 MUX：

第一个 MUX 把 B 接成反相器（sel=B, d0=1, d1=0 → 输出 ~B），

第二个 MUX 实现 NAND（sel=A, d0=1, d1=~B）。

面试延伸：用两个 MUX 可以实现任意二输入逻辑函数（MUX 是通用逻辑元件）。

Q10. Digital Design: Pipeline Hazards and Cache Mapping

QUESTION

(a) Explain the three types of pipeline hazards and how each is resolved.

(b) Explain the three types of cache mapping: direct-mapped, set-associative,

and fully associative. What are the trade-offs?

ANSWER

```
// ================================================================
// (a) Pipeline Hazards
// ================================================================
//
```

```
// A pipeline hazard occurs when the next instruction cannot execute
// in the next clock cycle as planned.

// --- 1. Structural Hazard ------------------------------------------------
//   Cause: Two instructions need the SAME hardware resource simultaneously.
//   Example: single-port memory -- load (memory access stage) and
//            instruction fetch both need memory in the same cycle.
//   Solution:
//     a. Add more hardware (separate I-cache and D-cache)
//     b. Stall the pipeline (insert bubbles) until resource is free

// --- 2. Data Hazard (RAW: Read After Write is most common) ---------------
//   Cause: Instruction depends on the result of a previous instruction
//          that hasn't completed yet.
//   Example:
//     ADD R1, R2, R3    // R1 = R2 + R3 (written in WB stage, cycle 5)
//     SUB R4, R1, R5    // needs R1 in EX stage (cycle 3) -- TOO EARLY!
//
//   Solutions:
//     a. Pipeline stall (bubble/NOP insertion): wait until WB completes
//        -> simple but adds latency (2 stall cycles in 5-stage pipeline)
//     b. Forwarding / Bypassing (preferred): route the result from EX/MEM
//        output DIRECTLY to EX input of the dependent instruction,
//        without waiting for WB to write to the register file.
//        Reduces or eliminates stalls for RAW hazards.
//     c. Out-of-order execution: execute independent instructions first,
//        fill the gap with useful work (compiler or hardware reordering)

// --- 3. Control Hazard (Branch Hazard) -----------------------------------
//   Cause: Conditional branch -- we don't know which instruction to fetch
//          next until the branch is resolved.
//   Example: IF/ID/EX fetch the next 3 instructions, but branch resolves
//            in EX -- those 3 instructions may be WRONG path.
//
//   Solutions:
//     a. Stall (flush pipeline until branch resolved) -- 2+ cycle penalty
//     b. Branch prediction:
//        Static: always predict not-taken (or always taken)
//        Dynamic: branch predictor with history table (BTB, BHT, TAGE)
//        Speculative execution: execute predicted path, flush if wrong
//     c. Delayed branch (MIPS): always execute the instruction AFTER
//        the branch (branch delay slot), regardless of outcome.
//        Compiler fills the delay slot with useful work.

// ================================================================
// (b) Cache Mapping
// ================================================================
//
// Cache maps MAIN MEMORY addresses to CACHE locations.
// Cache size is much smaller than memory, so multiple memory
// addresses must share a cache location.

// --- 1. Direct-Mapped Cache ----------------------------------------------
//   Each memory address maps to EXACTLY ONE cache line (set).
//   index = (address / block_size) % num_sets
//
//   Pros: Simple, fast (one comparison per lookup)
//   Cons: CONFLICT MISSES -- two frequently-used addresses that map to
//         the same set thrash each other, even if other sets are empty.
//
//   Example: address A and address A+cache_size always conflict.
//   Analogy: coat check with assigned hooks -- hook 7 is always taken.
```

```
// --- 2. Fully Associative Cache ---------------------------------------
//   Any memory address can go in ANY cache line.
//   On lookup: compare tag against ALL cache lines simultaneously (parallel).
//
//   Pros: No conflict misses -- maximum flexibility
//   Cons: Hardware cost scales with cache size (N comparators for N lines),
//         complex replacement policy needed (LRU/PLRU/random)
//   Used for: small TLBs (Translation Lookaside Buffers), small caches
//
//   Analogy: coat check where any coat can go on any hook.

// --- 3. Set-Associative Cache (N-way) ---------------------------------
//   Memory maps to a SET of N cache lines (N = associativity).
//   Within the set, any of the N ways can hold the data.
//   index = (address / block_size) % num_sets
//   Then compare against all N ways in parallel.
//
//   Pros: Reduces conflict misses vs direct-mapped, manageable hardware
//   Cons: Slightly slower than direct-mapped (N comparisons vs 1),
//         more complex replacement logic
//   Most CPUs use 4-way, 8-way, or 16-way set-associative L1/L2/L3 caches
//
//   Analogy: coat check with N hooks per number -- some flexibility.
//
// Trade-off summary:
//   Mapping          | Miss rate | HW cost | Speed
//   -----------------+-----------+---------+------
//   Direct-mapped    | Highest   | Lowest  | Fastest
//   N-way set-assoc  | Medium    | Medium  | Medium
//   Fully associative| Lowest    | Highest | Slowest
```

解释

三种流水线冒险的记忆方式：结构冒险（硬件资源冲突）、数据冒险（写后读依赖）、控制冒险（分支不确定）。

Forwarding/Bypassing 是数据冒险最优雅的解法：不等 WB 写回寄存器文件，

直接把 EX 阶段的计算结果通过旁路数据路径送给下一条指令的 EX 输入——

现代处理器几乎都实现了完整的 forwarding 网络。

三种 Cache 映射的核心权衡：Direct-mapped 速度最快（一次比较）但冲突缺失率高；

Fully associative 缺失率最低但硬件开销随 cache 大小线性增长；

Set-associative 是两者的折衷，N 路组相联是工业界的通用选择（L1 通常4-8路）。

面试中能举出"两个地址映射到同一直接映射 cache 组导致 thrash"的具体例子，

以及用"衣帽间" analogy 解释三种策略是加分项。

Q11. Digital Design: APB FIFO Design -- RTL Implementation

QUESTION

Design a FIFO with an APB (Advanced Peripheral Bus) interface.

The APB master can write data into the FIFO and read data out via

memory-mapped registers. Include FIFO status registers.

ANSWER

```
// ================================================================
// APB Protocol Refresher
// ================================================================
//
// APB is a simple, low-bandwidth bus with a 2-phase transaction:
//
//   SETUP phase (1 cycle):
//     PSEL=1, PENABLE=0, PWRITE=1/0, PADDR=addr, PWDATA=data
//
//   ACCESS phase (1+ cycles):
//     PSEL=1, PENABLE=1
//     Master waits while PREADY=0 (slave inserts wait states)
//     Transaction completes when PREADY=1
//     On READ: PRDATA is valid when PREADY=1
//
//   After transaction: PSEL=0, PENABLE=0

// ================================================================
// Memory Map (register offsets)
// ================================================================
//
//   Offset 0x00 : DATA_REG   -- Write: push data to FIFO
//                             -- Read:  pop data from FIFO
//   Offset 0x04 : STATUS_REG -- Read-only
//                               [0]   = full
//                               [1]   = empty
//                               [9:2] = count (entries currently in FIFO)
//   Offset 0x08 : CTRL_REG   -- Write: [0] = flush (clear FIFO)

module apb_fifo #(
    parameter int DATA_W  = 32,
    parameter int DEPTH   = 16,
    parameter int ADDR_W  = $clog2(DEPTH)
)(
    // APB slave interface
    input  logic              PCLK,
    input  logic              PRESETn,
    input  logic              PSEL,
    input  logic              PENABLE,
    input  logic              PWRITE,
    input  logic [31:0]       PADDR,
    input  logic [DATA_W-1:0] PWDATA,
    output logic [DATA_W-1:0] PRDATA,
    output logic              PREADY,
    output logic              PSLVERR
);
    // ---- FIFO storage and pointers -----------------------------------
    logic [DATA_W-1:0] mem [DEPTH];
    logic [ADDR_W:0]   wr_ptr, rd_ptr;   // extra bit for full/empty
    logic [ADDR_W:0]   count;
    logic              full, empty;

    assign count = wr_ptr - rd_ptr;
    assign full  = (count == ADDR_W+1'(DEPTH));
    assign empty = (count == '0);

    // ---- APB decode --------------------------------------------------
    // Transaction is valid on the ACCESS phase: PSEL & PENABLE
    logic apb_access;
    assign apb_access = PSEL & PENABLE;

    // Always ready (no wait states in this simple design)
    assign PREADY  = 1'b1;
```

```
    assign PSLVERR = 1'b0;

    // ---- Register map ------------------------------------------------
    always_ff @(posedge PCLK or negedge PRESETn) begin
        if (!PRESETn) begin
            wr_ptr <= '0;
            rd_ptr <= '0;
            PRDATA <= '0;
        end else if (apb_access) begin
            case (PADDR[3:2])   // word-aligned: bits [3:2] select register

                2'h0: begin   // DATA_REG
                    if (PWRITE) begin
                        // Write: push to FIFO (ignore if full)
                        if (!full) begin
                            mem[wr_ptr[ADDR_W-1:0]] <= PWDATA;
                            wr_ptr <= wr_ptr + 1'b1;
                        end
                        PRDATA <= '0;
                    end else begin
                        // Read: pop from FIFO (ignore if empty)
                        if (!empty) begin
                            PRDATA <= mem[rd_ptr[ADDR_W-1:0]];
                            rd_ptr <= rd_ptr + 1'b1;
                        end else begin
                            PRDATA <= '0;   // underflow: return 0
                        end
                    end
                end

                2'h1: begin   // STATUS_REG (read-only)
                    PRDATA <= {22'b0, count, empty, full};
                end

                2'h2: begin   // CTRL_REG
                    if (PWRITE && PWDATA[0]) begin
                        // Flush: reset pointers
                        wr_ptr <= '0;
                        rd_ptr <= '0;
                    end
                    PRDATA <= '0;
                end

                default: PRDATA <= '0;
            endcase
        end
    end

endmodule

// ================================================================
// Key design points
// ================================================================
//
// 1. PREADY=1 always: simplest design -- no wait states.
//    For slower FIFOs (e.g. backed by SRAM with multi-cycle access),
//    PREADY can be deasserted until the operation completes.
//
// 2. Full/empty detection: extra-bit method on wr_ptr/rd_ptr.
//    count = wr_ptr - rd_ptr (with extra MSB for wrap-around).
//    full  when count == DEPTH.
//    empty when count == 0.
//
```

```
// 3. PSLVERR (error response): can be asserted on write-when-full
//    or read-when-empty if the protocol requires an error response
//    instead of silently dropping data.
//
// 4. APB address decode: PADDR[3:2] selects 32-bit word registers
//    (byte addresses 0x00, 0x04, 0x08 map to words 0, 1, 2).
```

解释

APB FIFO 的核心是把普通同步 FIFO 包上一层 APB slave 接口：

APB 的 SETUP+ACCESS 两段式握手在这道题里很简单——PREADY 始终为1，

ACCESS 相（PSEL & PENABLE）时执行寄存器读写操作。

寄存器地址解码用 PADDR[3:2] 选择字对齐寄存器（[1:0] 是字节偏移，对32位总线忽略）。

DATA_REG 的读写方向映射：PWRITE=1 推数据入 FIFO，PWRITE=0 从 FIFO 弹出数据。

STATUS_REG 对 APB master 至关重要：驱动程序必须轮询 full/empty 再操作，

否则可能静默丢失数据（或需要开启 PSLVERR 来报告溢出/下溢）。

生产设计通常还会加：中断输出（FIFO 超过阈值时触发 IRQ，避免轮询浪费总线带宽），

以及可配置的几乎满/几乎空阈值寄存器。

Q12. Digital Design: Race Condition in Verilog -- Blocking Assignment in always Block

QUESTION

What is the output behavior of the following Verilog code?

What is wrong with it, and how do you fix it?

```
module test (rst, clk);
    input rst, clk;
    logic y1, y2;

    always @(posedge clk or posedge rst)
        if (rst) y1 = 0;
        else     y1 = 2;

    always @(posedge clk or posedge rst)
        if (rst) y2 = 1;
        else     y2 = y1;
endmodule
```

ANSWER

```
// ================================================================
// What is wrong: TWO problems
// ================================================================
//
// Problem 1: Blocking assignments (=) in sequential always blocks.
//   Sequential logic (flip-flop behavior) must use NON-BLOCKING (<=).
//   Using = in an @(posedge clk) block is a well-known coding hazard.
//
// Problem 2: Race condition between the two always blocks.
//   Both blocks are triggered by posedge clk simultaneously.
//   IEEE 1364/1800: the ORDER of execution of concurrent always blocks
//   at the same time step is NOT defined by the simulator.
//
```

```
//   At posedge clk (rst=0):
//   Scenario A -- Block 1 runs first:
//     y1 = 2              (blocking: y1 immediately = 2)
//     y2 = y1 = 2         (block 2 sees the NEW y1)
//     Result: y2 = 2
//
//   Scenario B -- Block 2 runs first:
//     y2 = y1 = <old_y1>  (block 2 sees y1 from PREVIOUS state, e.g. 0)
//     y1 = 2              (block 1 runs after)
//     Result: y2 = <old_y1>  (e.g. 0)
//
//   Both scenarios are LEGAL under the standard.
//   The simulator may choose either -- output is NON-DETERMINISTIC.
//   Different simulators (VCS, Questa, Xcelium) may give different results.

// ================================================================
// Fix: use non-blocking assignments (<=)
// ================================================================
module test_fixed (rst, clk);
    input  logic rst, clk;
    logic y1, y2;

    always @(posedge clk or posedge rst) begin
        if (rst) y1 <= 1'b0;
        else     y1 <= 1'b1;   // note: original "2" in 1-bit logic truncates to 0
    end

    always @(posedge clk or posedge rst) begin
        if (rst) y2 <= 1'b1;
        else     y2 <= y1;    // RHS always samples OLD y1 (before any NBA update)
    end
endmodule

// With non-blocking assignments:
//   At posedge clk: BOTH RHS values are sampled using CURRENT (old) values.
//   THEN all LHS values are updated simultaneously (NBA region).
//   Order of always block execution no longer matters.
//   y2 always gets the PREVIOUS cycle's value of y1 -- deterministic!
//   This matches the hardware behavior of two separate D flip-flops.
//
// SV Scheduling regions reminder:
//   Active region:  blocking assignments execute
//   NBA region:     non-blocking assignment updates apply
//   (always after all active-region code in the current time step)

// ================================================================
// Additional issue: "y1 = 2" in 1-bit logic
// ================================================================
// y1 is declared as 'logic' (1 bit). Assigning 2 (binary 10) to it
// truncates to the LSB: 2'b10[0] = 0. So y1 = 2 actually sets y1 = 0!
// This is another subtle bug: the value 2 silently becomes 0 in 1-bit logic.
// Use the correct width or declare y1 as logic [1:0].
```

解释

这道题同时考察三个知识点：

1. Blocking vs Non-Blocking：sequential 块里必须用 `<=`，是 Verilog/SV 的铁律。

2. SV Scheduling（调度模型）：active 区执行 blocking，NBA 区统一更新 non-blocking。

在同一时刻触发的两个 always 块，IEEE 不规定谁先执行——这是合法的非确定性。

3. 位宽截断陷阱：`logic y1; ... y1 = 2;` 中 2 的二进制是 `10`，

赋给1位 logic 时只取 LSB = 0，和原意完全相反且没有编译警告（仅 lint 工具报告）。

面试关键回答：能说出"两个 scenario 都合法，结果不确定"比"simulator A 给这个结果"更重要，因为它体现了对 IEEE 标准而非工具行为的理解。

Q13. Digital Design: Verilog/SV Scheduling Regions -- Complete Model

QUESTION

Explain the Verilog/SystemVerilog event scheduling model.

What are the different scheduling regions and what executes in each?

Why does this matter for RTL coding?

ANSWER

```
// ================================================================
// The time-step event scheduling model
// ================================================================
//
// Within a SINGLE simulation time step (e.g. posedge clk at t=10ns),
// the simulator processes events in a DEFINED ORDER across multiple
// "regions". This order is what prevents races with non-blocking
// assignments and ensures predictable always_ff behavior.
//
//
//  One time step:
//  +-----------+-----------+--------+-----------+-----------+
//  |  Active   |  Inactive | NBA    | Observed  | Reactive  |
//  +-----------+-----------+--------+-----------+-----------+
//       1           2         3          4           5
//
// ================================================================
// Region 1: ACTIVE (main execution)
// ================================================================
//
// - Executes BLOCKING (=) assignments immediately.
// - Evaluates RHS of non-blocking (<=) assignments (samples old values).
//   NOTE: LHS update is NOT done yet -- just the RHS is read here.
// - Evaluates continuous assignments (assign statements).
// - Calls $display, $monitor.
// - Multiple always blocks triggered at the same time execute in
//   UNDEFINED ORDER within the active region.
//   -> This is the source of race conditions when using blocking (=)
//      in multiple always blocks with shared variables.

// ================================================================
// Region 2: INACTIVE (#0 delays)
// ================================================================
//
// - Events scheduled with #0 (explicit zero delay) execute here.
// - Rarely used intentionally; can be a sign of poor coding practice.
// - Executes after ALL active-region events are done.

// ================================================================
// Region 3: NBA (Non-Blocking Assignment update)
// ================================================================
//
// - ALL pending non-blocking (<=) LHS updates are applied HERE,
//   SIMULTANEOUSLY, after ALL active-region code has completed.
// - This is why always_ff with <= is race-free:
```

```
//   ALL always blocks read OLD values (in Active), then ALL flip-flop
//   outputs are updated simultaneously (in NBA).
//   No always block can "see" another's NBA update within the same
//   time step.
//
// Example:
//   always_ff @(posedge clk) begin
//       a <= b;    // Active: RHS=old_b sampled
//       b <= a;    // Active: RHS=old_a sampled
//   end
//   // NBA: a=old_b, b=old_a applied -- perfect swap, deterministic

// ================================================================
// Region 4: OBSERVED (assertions / properties)
// ================================================================
//
// - SVA concurrent property evaluation happens here.
// - Assertions observe signal values AFTER NBA updates, so they
//   see the final stable values of the time step.
// - This ensures assertions check the correct post-update state.

// ================================================================
// Region 5: REACTIVE (testbench clocking block outputs)
// ================================================================
//
// - Clocking block OUTPUT signals are driven here (with specified
//   output delay, e.g. #1 in the clocking block).
// - This ensures testbench stimulus is applied AFTER DUT outputs
//   have settled, preventing testbench-DUT setup/hold races.

// ================================================================
// Why this matters for RTL coding
// ================================================================
//
// Rule 1: Use <= in always_ff.
//   NBA region guarantees all RHS are sampled before any LHS updates.
//   Two always_ff blocks are ALWAYS race-free with <= because they
//   both read in Active and both write in NBA (order-independent).
//
// Rule 2: Use = in always_comb only.
//   Blocking = gives immediate updates within one block -- correct for
//   combinational logic (where order within one always block is intentional).
//
// Rule 3: Never mix = and <= for the same variable.
//   A variable updated by both = and <= in different always blocks is
//   undefined behavior (which update "wins" is tool-dependent).
//
// Race condition example (two always blocks, blocking, shared var):
//   always @(posedge clk) a = b;  // Active: a=b immediately
//   always @(posedge clk) b = a;  // Active: b=? (which a? new or old?)
//   -> Outcome depends on which always block runs first in Active region.
//   -> Non-deterministic across simulators and even between runs.
```

解释

SV 调度模型的核心价值在于解释"为什么 NBA 赋值是安全的"：

所有 always_ff 块在 Active 区读取旧值（这一步是并发的，顺序不重要），

然后统一在 NBA 区把新值写入——类似于所有触发器同时翻转，完美模拟硬件行为。

Observed 区（SVA）在 NBA 之后执行是关键设计：

断言看到的是当前时间步的最终稳定值，而不是中间状态，这是 cover/assert 语义正确的基础。

Reactive 区（clocking block 输出）在最后执行：

testbench 对 DUT 的输入在 DUT 输出稳定后才更新，避免了"激励和响应在同一时刻竞争"的问题。

记忆口诀：Active（算）→ NBA（写FF）→ Observed（看断言）→ Reactive（TB输出）。

Q14. Digital Design: Async FIFO from Scratch -- Gray Code Pointer Design

QUESTION

Write a complete asynchronous FIFO from scratch.

The write and read clocks are independent (different frequency and phase).

Include: dual-port memory, gray-code pointers, 2-FF synchronizers,

and correct full/empty flag generation.

ANSWER

```
// ================================================================
// Why gray code pointers?
// ================================================================
//
// When crossing a multi-bit counter value across clock domains,
// the receiving clock may sample it mid-transition. If multiple bits
// change simultaneously (e.g. binary 0111 -> 1000, 4 bits flip),
// the sampler may see any of 16 intermediate values -> garbage.
//
// Gray code ensures only ONE bit changes between consecutive values.
// So if the sampler catches a transition, it sees either the old or
// new value (both valid), never a corrupted intermediate.
//
// Gray code: g = b ^ (b >> 1)

// ================================================================
// Complete Async FIFO (DEPTH must be power of 2)
// ================================================================
module async_fifo #(
    parameter int DATA_W = 8,
    parameter int DEPTH  = 16,        // must be power of 2
    localparam int ADDR_W = $clog2(DEPTH)
)(
    // Write side
    input  logic              wclk,
    input  logic              wrst_n,
    input  logic              wen,
    input  logic [DATA_W-1:0] wdata,
    output logic              wfull,

    // Read side
    input  logic              rclk,
    input  logic              rrst_n,
    input  logic              ren,
    output logic [DATA_W-1:0] rdata,
    output logic              rempty
);
    // ---- Dual-port memory ------------------------------------
    logic [DATA_W-1:0] mem [DEPTH];

    // ---- Pointers (ADDR_W+1 bits: extra MSB distinguishes full/empty)
    // Binary pointers (for memory addressing)
    logic [ADDR_W:0] wptr_bin, rptr_bin;
```

```
    // Gray-coded pointers (for CDC crossing)
    logic [ADDR_W:0] wptr_gray, rptr_gray;

    // Synchronized gray pointers (2-FF synchronizer outputs)
    logic [ADDR_W:0] wptr_gray_rsync1, wptr_gray_rsync2;  // wptr synced to rclk
    logic [ADDR_W:0] rptr_gray_wsync1, rptr_gray_wsync2;  // rptr synced to wclk

    // ================================================================
    // Write domain logic
    // ================================================================
    always_ff @(posedge wclk or negedge wrst_n) begin
        if (!wrst_n) begin
            wptr_bin  <= '0;
            wptr_gray <= '0;
        end else if (wen && !wfull) begin
            mem[wptr_bin[ADDR_W-1:0]] <= wdata;   // write to memory
            wptr_bin  <= wptr_bin + 1'b1;
            // Convert to gray: g = b ^ (b >> 1)
            wptr_gray <= (wptr_bin + 1'b1) ^
                         ((wptr_bin + 1'b1) >> 1);
        end
    end

    // Synchronize rptr_gray into wclk domain (2-FF)
    always_ff @(posedge wclk or negedge wrst_n) begin
        if (!wrst_n) begin
            rptr_gray_wsync1 <= '0;
            rptr_gray_wsync2 <= '0;
        end else begin
            rptr_gray_wsync1 <= rptr_gray;         // stage 1 (may be metastable)
            rptr_gray_wsync2 <= rptr_gray_wsync1;  // stage 2 (stable)
        end
    end

    // Full flag: wptr_gray is DEPTH ahead of rptr (in gray code)
    // In gray: full when wptr[MSB] != rptr_sync[MSB] AND
    //          wptr[MSB-1] != rptr_sync[MSB-1] AND
    //          wptr[MSB-2:0] == rptr_sync[MSB-2:0]
    assign wfull = (wptr_gray[ADDR_W]   != rptr_gray_wsync2[ADDR_W]) &&
                   (wptr_gray[ADDR_W-1] != rptr_gray_wsync2[ADDR_W-1]) &&
                   (wptr_gray[ADDR_W-2:0] == rptr_gray_wsync2[ADDR_W-2:0]);

    // ================================================================
    // Read domain logic
    // ================================================================
    always_ff @(posedge rclk or negedge rrst_n) begin
        if (!rrst_n) begin
            rptr_bin  <= '0;
            rptr_gray <= '0;
        end else if (ren && !rempty) begin
            rdata    <= mem[rptr_bin[ADDR_W-1:0]];  // read from memory
            rptr_bin  <= rptr_bin + 1'b1;
            rptr_gray <= (rptr_bin + 1'b1) ^
                         ((rptr_bin + 1'b1) >> 1);
        end
    end

    // Synchronize wptr_gray into rclk domain (2-FF)
    always_ff @(posedge rclk or negedge rrst_n) begin
        if (!rrst_n) begin
            wptr_gray_rsync1 <= '0;
            wptr_gray_rsync2 <= '0;
```

```
        end else begin
            wptr_gray_rsync1 <= wptr_gray;
            wptr_gray_rsync2 <= wptr_gray_rsync1;
        end
    end

    // Empty flag: wptr_sync == rptr (all bits equal in gray domain)
    assign rempty = (wptr_gray_rsync2 == rptr_gray);

endmodule

// ============================================================
// Key design points summary
// ============================================================
//
// 1. DEPTH must be power of 2: gray code wraps cleanly at 2^N.
//    Non-power-of-2 FIFO requires modified pointer arithmetic.
//
// 2. Pointer width = ADDR_W + 1 (one extra bit).
//    The MSB is the "wrap bit" that distinguishes full from empty
//    when both pointers point to the same address.
//
// 3. Full detection (in write domain):
//    wptr is DEPTH entries ahead of synchronized rptr.
//    In gray: top 2 bits inverted, lower bits equal.
//
// 4. Empty detection (in read domain):
//    wptr_sync == rptr (all bits equal).
//
// 5. Memory is accessed with BINARY pointers (simple index).
//    Only the CDC-crossing value is gray-coded.
//
// 6. Each domain has its OWN reset (wrst_n / rrst_n).
//    They can be asserted independently -- do not share resets
//    across clock domains.
```

解释

Async FIFO 的设计难点全部集中在"如何安全地跨时钟域传递指针"：

二进制计数器相邻值之间可能有多位同时翻转，跨域采样会产生乱码；

Gray 码保证相邻值只差1位，采样结果要么是旧值要么是新值，永远不会是中间状态。

ADDR_W+1 位指针的"额外 MSB"是满/空判断的关键：

当写指针恰好在读指针后面 DEPTH 步时，低 ADDR_W 位相同（同一个内存地址），

只有 MSB 不同——这个 MSB 就是满的标志；所有位相同则是空的标志。

满标志的 Gray 码判断（上2位反转、下位相同）来自 Gray 码与二进制的对应关系：

在 Gray 域里"相差 DEPTH"正好表现为高2位被翻转，这是面试中最常被追问的细节。

两个域各自独立复位（wrst_n / rrst_n）是 SoC 低功耗和电源管理的实际要求：

读写时钟域可能独立上下电，共用复位会产生跨域复位的亚稳态问题。

Q15. Digital Design: NAND2 Schematic -- Why NMOS Series, PMOS Parallel

QUESTION

Explain the NAND2 transistor schematic structure.

Why are the NMOS transistors in series and PMOS transistors in parallel?

ANSWER

```
// ================================================================
// NAND2 transistor-level schematic
// ================================================================
//
//          VDD
//         /   \
//        |     |
//    [MP1]     [MP2]    <- PMOS, gates = A, B (PARALLEL)
//        |     |
//         \   /
//          |
//         OUT (Y)
//          |
//        [MN1]          <- NMOS, gate = A (SERIES)
//          |
//        [MN2]          <- NMOS, gate = B (SERIES)
//          |
//         GND
//
// Truth table confirmation:
//   A  B  | MN1  MN2  | Pull-down path | MP1  MP2 | Pull-up path | Y
//   0  0  |  OFF  OFF  |  BROKEN        |  ON   ON |  BOTH active |  1
//   0  1  |  OFF  ON   |  BROKEN        |  ON   OFF|  MP1 active  |  1
//   1  0  |  ON   OFF  |  BROKEN        |  OFF  ON |  MP2 active  |  1
//   1  1  |  ON   ON   |  COMPLETE      |  OFF  OFF|  none        |  0

// ================================================================
// Why NMOS in SERIES?
// ================================================================
//
// Y = NAND(A,B) = NOT(A AND B)
// Y = 0 ONLY when BOTH A=1 AND B=1 simultaneously.
//
// We need a pull-down path to GND ONLY when A AND B are both high.
// Series NMOS implements AND in the pull-down network (PDN):
//   Both MN1 (controlled by A) AND MN2 (controlled by B) must be ON
//   to create a complete path from OUT to GND.
//   If either A or B = 0, that NMOS is OFF, the series chain is broken,
//   no pull-down current -> Y stays high.
//
// General rule: AND function in PDN = transistors in SERIES.
//               (current must flow through all transistors)

// ================================================================
// Why PMOS in PARALLEL?
// ================================================================
//
// Y = 1 when A=0 OR B=0 (or both).
// We need a pull-up path to VDD when EITHER A OR B is low.
// Parallel PMOS implements OR in the pull-up network (PUN):
//   MP1 turns ON when A=0 (Vgs = 0 - VDD = -VDD < Vtp)
//   MP2 turns ON when B=0
//   EITHER ONE being ON is sufficient to pull Y to VDD.
//
// General rule: OR function in PUN = transistors in PARALLEL.
//               (current can flow through any one transistor)
//
// This duality (PUN is complement of PDN) is De Morgan's theorem
// implemented in silicon:
//   PDN: NMOS series (AND) -> implements "pull down when A AND B high"
//   PUN: PMOS parallel (OR) -> implements "pull up when A OR B low"
//   Together: Y = NOT(A AND B) = NAND(A,B)
```

解释

NAND2 的拓扑结构体现了 CMOS 互补对称设计的核心原则：

下拉网络（PDN, NMOS）和上拉网络（PUN, PMOS）互为逻辑补集。

串联实现 AND（电流必须经过所有管子），并联实现 OR（任一管子通即可）。

De Morgan 定理在电路层面的体现：`NOT(A AND B) = NOT_A OR NOT_B`——

PDN 实现"A AND B 时拉低"，PUN 恰好实现其补集"NOT_A OR NOT_B 时拉高"。

这个原则推广到任意 CMOS 门：先写 PDN（对应函数的真值1部分），

PUN 自动是 PDN 的对偶（串联↔并联互换）——这是手动验证 CMOS 逻辑电路的标准方法。

Q16. Digital Design: Divide-by-3 Clock -- Logic, Waveform, Verilog, Verification

QUESTION

(a) Explain the divide-by-3 clock divider logic and draw the waveform.

(b) Write Verilog implementation.

(c) How would you verify this logic?

ANSWER

```
// ================================================================
// (a) Logic and waveform
// ================================================================
//
// Goal: output clock frequency = input clock / 3
//
// Simplest implementation: 2-bit counter counting 0->1->2->0->...
// Output is HIGH for 2 cycles (count=0,1), LOW for 1 cycle (count=2).
// This gives 2/3 duty cycle (not 50%).
//
// Waveform (non-50% duty cycle):
//
//  clk_in:  _|■|_|■|_|■|_|■|_|■|_|■|_|■|
//  count:    0   1   2   0   1   2   0
//  clk_out: ■■■|_|■■■|_|■■■|_|■■■|_
//           <- 2T ->  <- 2T ->
//  period = 3 * T_in  ✓
//
// For 50% duty cycle:
//   Use both posedge and negedge of clk_in.
//   Two separate 3-phase counters, OR their outputs:
//   posedge counter: HIGH on count=0
//   negedge counter: HIGH on count=0
//   Result: symmetric 50% duty cycle output.

// ================================================================
// (b) Verilog -- simple 2/3 duty cycle
// ================================================================
module div3 (
    input  logic clk,
    input  logic rst_n,
    output logic clk_div3
);
    logic [1:0] count;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)         count <= 2'd0;
```

```
        else if (count == 2) count <= 2'd0;  // wrap at 3
        else                 count <= count + 1'b1;
    end

    // Output HIGH for count=0 and count=1 (2 out of 3 cycles)
    assign clk_div3 = (count != 2);
endmodule

// ============================================================
// Verilog -- 50% duty cycle (posedge + negedge combined)
// ============================================================
module div3_50duty (
    input  logic clk,
    input  logic rst_n,
    output logic clk_div3
);
    logic [1:0] pos_cnt, neg_cnt;
    logic       pos_ph, neg_ph;

    // Posedge counter
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) pos_cnt <= 0;
        else        pos_cnt <= (pos_cnt == 2) ? 0 : pos_cnt + 1;
    end

    // Negedge counter (same but offset by half cycle)
    always_ff @(negedge clk or negedge rst_n) begin
        if (!rst_n) neg_cnt <= 0;
        else        neg_cnt <= (neg_cnt == 2) ? 0 : neg_cnt + 1;
    end

    assign pos_ph   = (pos_cnt == 0);
    assign neg_ph   = (neg_cnt == 0);
    assign clk_div3 = pos_ph | neg_ph;   // OR -> 50% duty cycle
endmodule

// ============================================================
// (c) Verification strategy
// ============================================================
//
// Property 1: Output period = 3 * input period
//   Measure: time between consecutive rising edges of clk_div3.
//   Assert: period_out == 3 * period_in (within tolerance)
//   SVA:
//   property p_div3_period;
//     @(posedge clk) $rose(clk_div3) |->
//       ##3 clk_div3 == 0 ##0 (1[*0:2]) ##1 $rose(clk_div3);
//   endproperty
//
// Property 2: Exactly 3 clk_in cycles between output pulses
//   property p_exactly_3;
//     @(posedge clk) $rose(clk_div3) |->
//       ##[3:3] $rose(clk_div3);
//   endproperty
//
// Property 3: Reset clears the counter
//   @(posedge clk) $fell(rst_n) |=> (count == 0);
//
// Waveform checker:
//   int unsigned posedge_count;
//   always @(posedge clk_div3) posedge_count++;
//   // After N cycles of clk_in, verify posedge_count == N/3
//
```

```
// Functional coverage:
//   covergroup div3_cg @(posedge clk);
//     cp_count: coverpoint count { bins c[] = {0,1,2}; }
//     cp_out:   coverpoint clk_div3;
//     cp_rst:   coverpoint rst_n { bins asserted={0}; bins released={1}; }
//   endgroup
```

解释

Div-by-3 的核心是 2 比特计数器（能表示 0,1,2,3，用其中三个状态），

计数到2时强制归零，周期为3个输入时钟周期。

占空比问题：简单实现是 2/3（高2个周期，低1个），

若需要 50% 占空比则需要同时用 posedge 和 negedge 计数器，将两路"相位=0"的信号或起来。

验证的关键是"恰好3个周期"的约束：用 `##[3:3]`（不是 `##[2:4]`）确保精确，

同时需要覆盖 reset 中途、第一个周期的边界情况。

Q17. Digital Design: Data Activity Monitor -- RTL and Verification

QUESTION

Design a module that monitors a 1-bit data signal:

- If data does NOT toggle within 10 consecutive clock cycles:

```
output = inactive (0)
```

- If data toggles at least once within any 10-cycle window:

```
output = active (1)
```

Write the RTL. Then describe how to verify it.

ANSWER

```
// ================================================================
// RTL Implementation
// ================================================================
module activity_monitor (
    input  logic clk,
    input  logic rst_n,
    input  logic data,
    output logic active
);
    // Counter: counts cycles since last data toggle
    // Range 0-10 needs 4 bits
    logic [3:0] no_toggle_cnt;
    logic       data_prev;     // data value from previous cycle

    // --- Sequential: update counter and detect toggle ----------------
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            no_toggle_cnt <= 4'd0;
            data_prev     <= 1'b0;
            active        <= 1'b0;
        end else begin
            data_prev <= data;    // capture previous value

            if (data != data_prev) begin
```

```
                // Toggle detected: reset counter, signal is active
                no_toggle_cnt <= 4'd0;
                active        <= 1'b1;
            end else begin
                // No toggle: increment counter
                if (no_toggle_cnt < 4'd10)
                    no_toggle_cnt <= no_toggle_cnt + 1'b1;

                // After 10 cycles with no toggle: go inactive
                if (no_toggle_cnt >= 4'd9)   // >= 9 means next cycle = 10
                    active <= 1'b0;
            end
        end
    end

endmodule

// ================================================================
// Timing diagram
// ================================================================
//
//   clk:  __|■|_|■|_|■|_|■|_|■|_|■|_|■|_|■|_|■|_|■|_|■|_|■|_|■|_|■|
//   data: _____|■■■|_____________________________________________|■■
//   cnt:  0 0  0   0 1 2 3 4 5 6 7 8 9 10 10 10 ... 0
//   active:     _____|■■■■■■■■■■■■■■■■■■■■■■|_____|■■■■■
//
//   - Toggle at cycle 1: active goes HIGH, counter resets to 0
//   - Toggle at cycle 2: counter resets again
//   - No toggle cycles 3-12: counter increments 1->10
//   - At counter=10 (cycle 12): active goes LOW
//   - Toggle at cycle 17: active goes HIGH again immediately

// ================================================================
// Verification strategy
// ================================================================
//
// 1. Directed tests (corner cases):
//
//   TC1. Toggle every cycle: active should stay HIGH permanently
//   TC2. Toggle exactly at cycle 9 (one cycle before timeout):
//        active stays HIGH (counter resets at the last safe moment)
//   TC3. No toggle for 10 cycles: active goes LOW at cycle 10
//   TC4. No toggle for 10 cycles, then toggle: active returns HIGH
//   TC5. Toggle at cycle 0 (same cycle as reset release)
//   TC6. Rapid toggle then long idle: active=HIGH then LOW
//   TC7. Assert rst_n mid-operation: active and counter must clear
//
// 2. SVA properties:
//
//   // If active goes low, data must not have toggled for 10 cycles
//   property p_inactive_after_10;
//       @(posedge clk) disable iff (!rst_n)
//       $fell(active) |->
//           $past(data == $past(data), 10);  // data unchanged for 10 cycles
//   endproperty
//   assert property (p_inactive_after_10);
//
//   // If data toggles, active must be high within next cycle
//   property p_active_on_toggle;
//       @(posedge clk) disable iff (!rst_n)
//       (data != $past(data)) |=> active;
//   endproperty
//   assert property (p_active_on_toggle);
```

```
//
//   // active cannot go low unless data has been stable for 10 cycles
//   property p_no_false_inactive;
//       @(posedge clk) disable iff (!rst_n)
//       $fell(active) |->
//           (no_toggle_cnt == 10);
//   endproperty
//   assert property (p_no_false_inactive);
//
// 3. Functional coverage:
//   cp_toggle_gap: coverpoint cycles_since_last_toggle {
//       bins immediate  = {[0:1]};    // back-to-back toggle
//       bins short_gap  = {[2:5]};
//       bins near_limit = {[8:9]};    // toggle just before timeout
//       bins timeout    = {[10:$]};   // actually goes inactive
//   }
```

解释

活动监测器的 RTL 核心是"计数器 + 前值比较"：

`data_prev` 寄存上一拍的 data，和当前 data 异或判断是否发生跳变；

跳变时清零计数器，无跳变时递增，计到 10 就输出 inactive。

关键设计细节：计数器上限是 10（不是 9 也不是 11），

`>= 9` 的条件确保在计数器的下一拍（变为 10）时 active 才拉低，

实现"连续 10 个周期无跳变才判定 inactive"。

验证的核心 corner case 是"第 9 周期跳变"（在 10 周期 timeout 的最后一刻触发），

这是边界值测试的典型——早跳一拍可以，晚跳一拍就超时，必须覆盖到这个临界点。

SVA 的 `$past(data == $past(data), N)` 是检查"数据在过去 N 拍内没有变化"的标准写法。

Q18. Digital Design: 4-Input Comparator from 2-Input Comparators

QUESTION

Given a 2-input comparator module (outputs: A>B, A=B, A<B),

design a 4-input comparator that outputs a ranked order:

out3 > out2 > out1 > out0 (the four inputs sorted in descending order).

ANSWER

```
// ================================================================
// Building block: 2-input comparator
// ================================================================
module cmp2 #(parameter int W = 8) (
    input  logic [W-1:0] a, b,
    output logic         gt,   // a > b
    output logic         eq,   // a == b
    output logic         lt    // a < b
);
    assign gt = (a > b);
    assign eq = (a == b);
    assign lt = (a < b);
endmodule

// ================================================================
```

```
// 4-input sorter: sorting network using 2-input comparators
// ================================================================
// Strategy: bubble sort / odd-even merge sort network
// 4-input optimal sort network needs 5 comparator stages:
//
//   Stage 1: compare pairs (in0,in1) and (in2,in3)
//   Stage 2: compare (max(0,1), max(2,3)) for final max
//            compare (min(0,1), min(2,3)) for final min
//   Stage 3: compare middle two elements
//
// Result: out3 >= out2 >= out1 >= out0

module cmp4 #(parameter int W = 8) (
    input  logic [W-1:0] in0, in1, in2, in3,
    output logic [W-1:0] out0, out1, out2, out3  // out3 = max, out0 = min
);
    // Intermediate wires
    logic [W-1:0] s0_hi_01, s0_lo_01;   // stage 0: sort pair (in0,in1)
    logic [W-1:0] s0_hi_23, s0_lo_23;   // stage 0: sort pair (in2,in3)
    logic         gt01, gt23;

    // Stage 0: two independent 2-input comparators
    assign gt01    = (in0 > in1);
    assign s0_hi_01 = gt01 ? in0 : in1;  // max(in0, in1)
    assign s0_lo_01 = gt01 ? in1 : in0;  // min(in0, in1)

    assign gt23    = (in2 > in3);
    assign s0_hi_23 = gt23 ? in2 : in3;
    assign s0_lo_23 = gt23 ? in3 : in2;

    // Stage 1: find global max and min
    logic [W-1:0] s1_max, s1_min;
    logic [W-1:0] s1_mid_hi, s1_mid_lo;

    assign s1_max    = (s0_hi_01 > s0_hi_23) ? s0_hi_01 : s0_hi_23;
    assign s1_min    = (s0_lo_01 < s0_lo_23) ? s0_lo_01 : s0_lo_23;
    assign s1_mid_hi = (s0_hi_01 > s0_hi_23) ? s0_hi_23 : s0_hi_01;
    assign s1_mid_lo = (s0_lo_01 < s0_lo_23) ? s0_lo_23 : s0_lo_01;

    // Stage 2: sort the two middle values
    assign out3 = s1_max;
    assign out0 = s1_min;
    assign out2 = (s1_mid_hi > s1_mid_lo) ? s1_mid_hi : s1_mid_lo;
    assign out1 = (s1_mid_hi > s1_mid_lo) ? s1_mid_lo : s1_mid_hi;

endmodule

// ================================================================
// Verification: exhaustive or random testing
// ================================================================
//
// For 8-bit inputs: exhaustive is 256^4 = too large.
// Use constrained random + scoreboard:
//
// Scoreboard check:
//   int sorted[4] = '{in0, in1, in2, in3};
//   sorted.sort();  // ascending
//   assert(out0 == sorted[0] && out1 == sorted[1] &&
//          out2 == sorted[2] && out3 == sorted[3])
//
// Corner cases:
//   - All 4 inputs equal: out0=out1=out2=out3
//   - Two pairs equal: (5,5,3,3) -> out3=5, out2=5, out1=3, out0=3
```

```
//   - Already sorted (ascending/descending)
//   - Max/min values (0xFF, 0x00)
```

解释

4-input 排序网络（sorting network）是用 2-input 比较器构建高阶排序的经典方法：

关键是"比较-交换"单元（CAS: compare-and-swap），每个单元输出 max 和 min。

5个 CAS 操作足以对4个输入排序（这是已证明的最优比较次数）。

实现的三个阶段：先两两排序（parallel），再找全局 max/min，最后排中间两个。

Scoreboard 使用 SV 内置的 `array.sort()` 方法生成期望的排序结果，

然后逐位比对——这是"用更高层次的语言特性写 golden model"的经典 DV 技巧。

面试中能说出"4输入最优比较器需要 5 个比较单元，不是 6 个也不是 4 个"是加分项。

Q19. Digital Design: req Rising Edge -> One-Cycle flag Pulse

QUESTION

Given: input req, input clk, output flag.

When req goes from 0 to 1 (rising edge), output flag for exactly

ONE full clock cycle, then return to 0.

Write the RTL.

ANSWER

```
// ================================================================
// Behavior
// ================================================================
//
// req:  _____|■■■■■■■■■■■■■■■■■|___  (req stays high)
// flag: _____|■|_________________  (ONE clock pulse on rising edge)
//             ^ exactly 1 cycle
//
// Key: flag fires on the RISING EDGE of req (0->1 transition),
//      NOT on the level. Even if req stays high, flag only pulses once.

// ================================================================
// RTL: edge detector + one-cycle pulse
// ================================================================
module req_pulse (
    input  logic clk,
    input  logic rst_n,
    input  logic req,
    output logic flag
);
    logic req_prev;   // req value from previous cycle

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            req_prev <= 1'b0;
            flag     <= 1'b0;
        end else begin
            req_prev <= req;
            // flag = 1 ONLY in the cycle immediately after req rises
            // req_prev=0 AND req=1 -> rising edge detected
            flag <= (req && !req_prev);
```

```
        end
    end

endmodule

// ============================================================
// Timing trace
// ============================================================
//
//   Cycle | req | req_prev | flag
//   ------+-----+----------+------
//     0   |  0  |    0     |  0
//     1   |  0  |    0     |  0
//     2   |  1  |    0     |  0   <- req rises; flag computed next cycle
//     3   |  1  |    1     |  1   <- flag = (1 && !0) = 1  (the pulse!)
//     4   |  1  |    1     |  0   <- flag = (1 && !1) = 0  (back to 0)
//     5   |  1  |    1     |  0
//     6   |  0  |    1     |  0
//
//   Note: flag appears ONE CYCLE AFTER the req rising edge.
//   This is the standard registered edge-detect behavior.
//   If zero-latency is needed, use combinational: flag = req & ~req_prev
//   (but this is not clock-synchronous and may glitch).

// ============================================================
// Alternative: combinational output (no register on flag)
// ============================================================
module req_pulse_combo (
    input  logic clk,
    input  logic rst_n,
    input  logic req,
    output logic flag      // combinational: no FF on output
);
    logic req_prev;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) req_prev <= 1'b0;
        else        req_prev <= req;
    end

    // Combinational: flag is high the SAME cycle req rises
    assign flag = req && !req_prev;
endmodule
```

解释

边沿检测的标准 RTL 模式："寄存器 req，然后判断当前值和上一拍值的差异"。

`req && !req_prev` 在 req=1 且上一拍 req=0 时（即上升沿）为1，其余所有情况为0。

寄存器版（flag 有 FF）vs 组合版（flag 无 FF）的区别：

寄存器版多一个周期延迟（保证 flag 是纯同步信号，无 glitch）；

组合版在上升沿当拍输出（更快，但存在从 req_prev 的传播延迟，可能短暂出现毛刺）。

工业界通常选择寄存器版，因为下游 FF 需要 flag 是干净的时钟域信号；

如果需要零延迟响应，则在系统设计层面多预留一个周期。

Q20. Digital Design: 1-bit D Flip-Flop and D Latch -- Verilog RTL

QUESTION

Write Verilog/SystemVerilog RTL for:

(a) A 1-bit D Flip-Flop (positive edge triggered, async active-low reset)

(b) A 1-bit D Latch (level-sensitive, active-high enable)

ANSWER

```
// ================================================================
// (a) 1-bit D Flip-Flop -- edge triggered
// ================================================================
module dff_1bit (
    input  logic clk,
    input  logic rst_n,   // active-low asynchronous reset
    input  logic d,
    output logic q
);
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            q <= 1'b0;    // async reset
        else
            q <= d;       // capture D on rising clock edge
    end
endmodule

// Timing:
//   clk:  _|■|_|■|_|■|
//   d:    _____|■■■|___
//   q:    _________|■■|  <- q updates ONE cycle after d is stable
//   rst_n:■■■|_|■■■■■■   <- async: q->0 immediately on negedge rst_n

// ================================================================
// (b) 1-bit D Latch -- level sensitive
// ================================================================
module dlatch_1bit (
    input  logic en,    // active-high enable (transparent when en=1)
    input  logic d,
    output logic q
);
    always_latch begin
        if (en)
            q <= d;   // transparent: q follows d while en=1
        // implicit: q holds value when en=0
    end
endmodule

// Timing:
//   en:  __|■■■■■■|_____|■■■|__
//   d:   _____|■■■|■■■■■■■■|___
//   q:   _____|■■■|__|■■■■■|___
//             ^transparent^  ^hold^
//   While en=1: q tracks d (transparent)
//   When en=0:  q holds last value (opaque)

// ================================================================
// Key differences in hardware
// ================================================================
//
//   Property           | D Flip-Flop         | D Latch
//   -------------------+---------------------+--------------------
//   Trigger            | Clock edge           | Enable level
//   Transparency       | Never                | While en=1
//   Metastability risk | Near clock edge      | At enable edge
//   STA analysis       | Simple (edge-based)  | Complex (time-borrowing)
```

```
//   always keyword     | always_ff            | always_latch
//   Inferred from      | always @(posedge clk)| always @(en, d)
//   Accidental latch   | Very unlikely        | Missing else in comb
```

解释

`always_ff` 和 `always_latch` 是 SystemVerilog 的专用关键字：

`always_ff` 告诉工具"这是时序逻辑，所有赋值必须用 `<=`"，如果你不小心写了 `=` 工具会报警；

`always_latch` 告诉工具"这是锁存器，不完整的 if 是有意的"，不会产生"缺少 else 导致 latch"的误报。

Latch 的隐式保持：`always_latch` 里的 `if (en) q <= d;` 省略了 `else` 分支，

这是故意的——没有 else 意味着"不满足条件时保持不变"，正好实现 latch 的存储功能。

在 always_comb 里省略 else 是 BUG（会推断出不期望的 latch），

但在 always_latch 里省略 else 是正确的 FEATURE。

Q21. Digital Design: 4-to-1 Multiplexer -- Verilog RTL (Multiple Styles)

QUESTION

Write Verilog RTL for a 4-to-1 multiplexer:

- Inputs: a, b, c, d (1-bit each)

- Select: s1, s0 (2-bit combined select)

- Output: out depends on select value

Show at least two implementation styles.

ANSWER

```
// Truth table:
//   s1  s0 | out
//   --------+----
//    0   0  |  a    (s={0,0})
//    0   1  |  b    (s={0,1})
//    1   0  |  c    (s={1,0})
//    1   1  |  d    (s={1,1})

// ============================================================
// Style 1: case statement (most readable, recommended)
// ============================================================
module mux4to1_case (
    input  logic a, b, c, d,
    input  logic s1, s0,
    output logic out
);
    always_comb begin
        case ({s1, s0})
            2'b00: out = a;
            2'b01: out = b;
            2'b10: out = c;
            2'b11: out = d;
            default: out = 1'bx;   // should never happen, avoids latch
        endcase
    end
endmodule

// ============================================================
```

```
// Style 2: ternary chain (concise)
// ============================================================
module mux4to1_ternary (
    input  logic a, b, c, d,
    input  logic s1, s0,
    output logic out
);
    assign out = s1 ? (s0 ? d : c)
                    : (s0 ? b : a);
    // Outer mux: s1 selects upper (c,d) or lower (a,b) pair
    // Inner mux: s0 selects within the pair
endmodule

// ============================================================
// Style 3: if-else (explicit, verbose)
// ============================================================
module mux4to1_if (
    input  logic a, b, c, d,
    input  logic s1, s0,
    output logic out
);
    always_comb begin
        if      (!s1 && !s0) out = a;
        else if (!s1 &&  s0) out = b;
        else if ( s1 && !s0) out = c;
        else                 out = d;
    end
endmodule

// ============================================================
// Style 4: using 2-to-1 MUX primitives (hierarchical)
// ============================================================
module mux4to1_hier (
    input  logic a, b, c, d,
    input  logic s1, s0,
    output logic out
);
    logic lo, hi;

    // Lower pair: select between a and b using s0
    assign lo = s0 ? b : a;
    // Upper pair: select between c and d using s0
    assign hi = s0 ? d : c;
    // Final stage: select between lo and hi using s1
    assign out = s1 ? hi : lo;
endmodule

// ============================================================
// Parameterized version (scalable)
// ============================================================
module mux4to1_param #(parameter int W = 1) (
    input  logic [W-1:0] a, b, c, d,
    input  logic [1:0]   sel,
    output logic [W-1:0] out
);
    always_comb begin
        case (sel)
            2'b00: out = a;
            2'b01: out = b;
            2'b10: out = c;
            2'b11: out = d;
            default: out = '0;
        endcase
```

```
    end
endmodule
```

解释

4-to-1 MUX 是面试中最经典的 RTL 题之一，关键是展示"多种写法都知道"：

case 语句是最推荐的写法（可读性最高，综合结果也最干净，default 避免推断 latch）；

ternary 链最简洁（单行 assign，适合简单情况）；

层级化写法（用三个 2-to-1 MUX 组合）体现了"分治"设计思想，

这也是为什么面试官常追问"如何用 2-to-1 MUX 实现 4-to-1 MUX"。

`{s1, s0}` 的拼接语法把两个1位信号合并为2位向量，这比用条件 `(s1==0 && s0==0)` 更简洁，

是 SV 编码风格中的推荐写法。

Parameterized 版本是加分点：把数据宽度 W 参数化，同一模块既能做1位选择也能做32位总线选择。

Q22. Digital Design: Cache Address Structure -- 32KB 4-Way 32B Cacheline

QUESTION

A set-associative cache has:

- Total size: 32KB

- Cacheline size: 32 bytes

- Associativity: 4-way

(a) Derive the address breakdown (tag / index / offset) for a 32-bit address.

(b) Given an address (e.g. 0x00001234), walk through the hit/miss process.

ANSWER

```
// ================================================================
// (a) Address structure derivation
// ================================================================
//
// Step 1: Block Offset bits
//   Cacheline = 32 bytes = 2^5  ->  offset = 5 bits  (addr[4:0])
//   Selects which byte within a 32-byte cacheline.
//
// Step 2: Number of Sets
//   Sets = Total_size / (Cacheline * Ways)
//        = 32KB / (32B * 4)
//        = 32768 / 128
//        = 256 sets
//
// Step 3: Index bits
//   256 sets = 2^8  ->  index = 8 bits  (addr[12:5])
//   Selects which of the 256 sets to look in.
//
// Step 4: Tag bits (remaining)
//   Tag = 32 - 5 - 8 = 19 bits  (addr[31:13])
//   Identifies which memory block occupies this cache location.
//
// Address bit breakdown:
//
//   31          13 12        5 4       0
//   |___tag(19)___||_index(8)_||_off(5)_|
```

```
//
//   Bit field  | Bits     | Width | Purpose
//   -----------+----------+-------+--------------------------
//   Tag        | [31:13]  | 19    | Identifies memory block
//   Set Index  | [12:5]   |  8    | Selects one of 256 sets
//   Block Off  | [4:0]    |  5    | Byte offset within line
//
// Verification: 19 + 8 + 5 = 32 bits ✓

// ================================================================
// (b) Hit/miss walkthrough for address 0x00001234
// ================================================================
//
// Address: 0x00001234 = 0000_0000_0000_0000_0001_0010_0011_0100
//
//   Binary breakdown:
//   [31:13] = 000_0000_0000_0000_000  -> Tag   = 0x00000 (0)
//   [12:5]  = 1001_0001               -> Index = 0x91    (145)
//   [4:0]   = 1_0100                  -> Offset = 0x14   (20)
//
//   Step 1: ADDRESS DECODE
//     Extract index = 145 -> go to Set 145
//     Extract tag = 0
//     Extract offset = 20 (will be used only on HIT)
//
//   Step 2: SET LOOKUP (parallel compare, 4 ways)
//     Check all 4 ways in Set 145 simultaneously:
//
//     Way 0: valid=1, stored_tag=0x3F201  -> 0x3F201 != 0  -> MISS
//     Way 1: valid=1, stored_tag=0x00000  -> 0x00000 == 0  -> HIT! ✓
//     Way 2: valid=0                      -> invalid, skip
//     Way 3: valid=1, stored_tag=0x00001  -> 0x00001 != 0  -> MISS
//
//   Step 3 (HIT path): Way 1 matched
//     Return data[offset] = cacheline_way1[20]  (byte 20 of the cached line)
//     Update LRU: mark Way 1 as most recently used in Set 145
//
//   Step 3 (MISS path, if no way matched):
//     a. Choose victim way: LRU (Least Recently Used) in Set 145 = Way 2
//     b. Fetch the full 32-byte cacheline from main memory
//        (memory address: 0x00001234 with offset zeroed = 0x00001220)
//     c. Evict Way 2: if dirty (write-back cache), write old data to memory
//     d. Fill Way 2: write new tag=0, data=fetched cacheline, valid=1
//     e. Return data[offset=20] from the newly fetched line
```

解释

Cache 地址分解的记忆顺序：先算 offset（直接由 cacheline 大小决定），

再算 sets（总大小除以 ways），再算 index（由 sets 决定），最后 tag 是剩余位。

四路组相联的 hit 判断是"并行比较"而非顺序查找：

所有4个 way 的 tag 在同一个时钟周期内和请求 tag 比较，

这需要4个比较器（硬件代价），是组相联比直接映射面积更大的根本原因。

Valid bit 不可忽略：即使 tag 匹配，valid=0 的 way 也是 miss——

这是冷启动或 cache flush 后必须检查的条件。

LRU（最近最少使用）替换策略：4-way 需要每个 set 维护2位的 PLRU（Pseudo-LRU）状态，

真正的 LRU 需要 log2(4!) = ~5 位，工业实现通常用 PLRU 近似。

Q23. Digital Design: AXI 4KB Boundary Restriction -- Why and How

QUESTION

Why does AXI protocol restrict burst transactions from crossing a 4KB address boundary? Explain the reason and the implication for designers.

ANSWER

```
// ================================================================
// The AXI 4KB rule
// ================================================================
//
// AXI specification (AMBA AXI4):
//   "A burst must not cross a 4KB address boundary."
//
//   This means: start_addr[11:0] + (burst_len * data_width_bytes)
//               must not overflow beyond 0xFFF (roll over to 0x000).
//
//   Example:
//     Start = 0x00000FF0, burst = 4 beats of 4 bytes = 16 bytes total
//     End   = 0x00000FF0 + 0x10 = 0x00001000  <- CROSSES 4KB boundary!
//     This is ILLEGAL in AXI.
//
//     Start = 0x00000FE0, burst = 4 beats of 4 bytes = 16 bytes
//     End   = 0x00000FE0 + 0x10 = 0x00000FF0  <- stays within 4KB
//     This is LEGAL.

// ================================================================
// Why 4KB specifically?
// ================================================================
//
// 1. PAGE TABLE / MMU ALIGNMENT
//    Virtual memory systems map memory in PAGE-sized chunks.
//    The minimum page size in ARM/x86 systems is 4KB (2^12 bytes).
//    Each 4KB page has its own:
//      - Physical address mapping (virtual -> physical translation)
//      - Access permissions (read/write/execute bits)
//      - Cacheability attributes (cached/uncached/write-through)
//
//    If an AXI burst were allowed to cross a 4KB boundary, it would
//    span TWO different virtual pages. Those pages might have:
//      - DIFFERENT physical addresses (page A maps to DRAM,
//        page B maps to a peripheral register file)
//      - DIFFERENT permissions (read-only page followed by read-write)
//      - DIFFERENT cacheability (cached vs. uncached MMIO region)
//
//    A single burst cannot handle two different physical destinations
//    or two different permission sets -- the interconnect would break.
//
// 2. TLB (Translation Lookaside Buffer) SIMPLIFICATION
//    The TLB translates virtual addresses to physical one page at a time.
//    A burst crossing a page boundary would require two separate TLB
//    lookups for a single transaction -- complicating the memory
//    access pipeline significantly.
//    The 4KB restriction means each burst needs at most ONE TLB lookup.
//
// 3. INTERCONNECT SIMPLICITY
//    AXI interconnect fabric (crossbars, switches) can route a burst
//    based on the start address alone (no mid-burst address recalculation).
//    If a burst could cross 4KB, the interconnect would need to
```

```
//    mid-burst re-route to a potentially different slave -> much more complex.

// ================================================================
// Implication for designers
// ================================================================
//
// When writing an AXI master (driver or RTL):
//
//   Rule: start_addr[11:0] + transfer_size_bytes <= 0x1000
//         (must not overflow the lower 12 bits)
//
//   Check in SV:
logic [31:0] start_addr;
logic [11:0] addr_offset;  // start_addr[11:0]
int          bytes_total;  // burst_len * bytes_per_beat

// Violation check:
assign crosses_4kb = ({1'b0, addr_offset} + bytes_total) > 12'hFFF + 1;
// If crosses_4kb: split the burst into two shorter bursts:
//   Burst 1: from start_addr to next 4KB boundary
//   Burst 2: from 4KB boundary to end address

//   Example of split:
//   Original: start=0xFF0, size=32 bytes (crosses at 0x1000)
//   Burst 1:  start=0xFF0, 16 bytes -> ends at 0xFFF (stays in page 0)
//   Burst 2:  start=0x1000, 16 bytes -> new page (legal new burst)

// SVA check (can add to AXI bus monitor):
//   property p_no_4kb_crossing;
//       @(posedge clk)
//       (axvalid && axready) |->
//           (axaddr[11:0] + (axlen+1) * (1 << axsize)) <= 12'hFFF + 1;
//   endproperty
//   assert property (p_no_4kb_crossing);
```

解释

AXI 4KB 限制的根本原因是"操作系统的内存管理单元以 4KB 为粒度"：

每个 4KB 页有独立的物理地址映射、访问权限和缓存属性，

一个 burst 如果跨页就会遇到两套完全不同的属性，硬件无法在单个事务中处理。

与 cache 的关联：面试官同时考 cache 结构和 AXI 4KB 限制不是巧合——

两者都涉及"4KB 页的概念"，cache 的 index 位选择 bits[12:5]，

而 AXI 的 4KB 限制看 bits[11:0]，两者共享"以 4KB（2^12）为边界"这个基础。

设计者的责任：AXI master（IP 内的总线接口）必须在发出 burst 前计算是否越界，

如果越界就拆成两个 burst。这个检查通常用 SVA 属性监控总线行为，

确保 IP 实现不违反协议规范。

Q24. Digital Design: Carry-Select Adder -- Design and Timing Analysis

QUESTION

Given the timing diagram and cell library below:

- A[31:0] arrives at 0ps
- B[3:0] arrives at 200ps
- Full Adder (FA): N-bit delay = N x 10ps

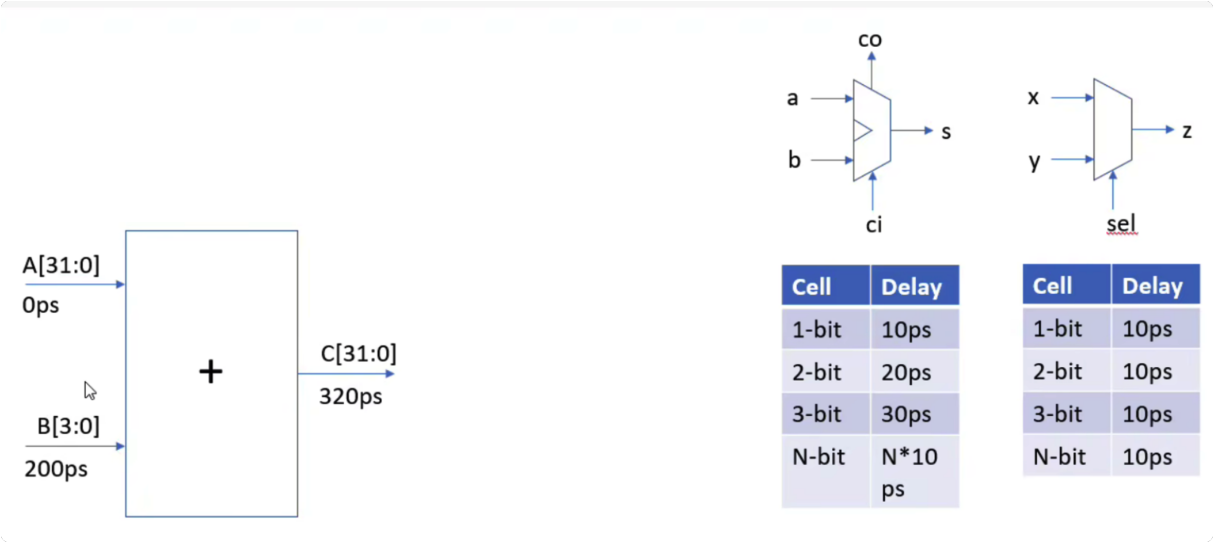
- MUX: any width delay = 10ps (constant)

- Target output: C[31:0] at 320ps

Design an adder that meets this constraint.

Explain the architecture, the timing analysis, and why a naive ripple-carry adder would fail.

ANSWER



Cell	Delay
1-bit	10ps
2-bit	20ps
3-bit	30ps
N-bit	N*10 ps

Cell	Delay
1-bit	10ps
2-bit	10ps
3-bit	10ps
N-bit	10ps

```
// ================================================================
// Why naive ripple-carry fails
// ================================================================
//
// A 32-bit ripple carry adder:
//   Delay = max(A_arrival, B_arrival) + 32-bit FA delay
//         = max(0ps, 200ps) + 32*10ps
//         = 200ps + 320ps = 520ps >> 320ps target
//
// Even if we use only 4-bit groups for B:
//   The carry must still ripple through all 32 bits.
//   The carry chain itself takes 32*10 = 320ps after B arrives at 200ps.
//   Total = 520ps.

// ================================================================
// Key insight: B is only 4 bits wide!
// ================================================================
//
// B[3:0] only affects bits [3:0] of the sum.
// For bits [31:4]: effectively we are adding 0, but with a possible
// carry-in from bit 3.
//
// Only TWO things can happen at bit 4 and above:
//   Case 0: carry out of bit 3 = 0 -> upper bits = A[31:4]
//   Case 1: carry out of bit 3 = 1 -> upper bits = A[31:4] + 1
//
// This is CARRY-SELECT: pre-compute BOTH cases while waiting for
// the actual carry, then SELECT the correct result with a MUX.

// ================================================================
// Architecture: Carry-Select Adder (4-bit groups)
// ================================================================
```

```
//
// Split into 8 groups of 4 bits each:
//   Group 0: A[3:0]   + B[3:0]         (real addition, cin=0)
//   Group 1: A[7:4]   + 0  (two versions: sum_0 with cin=0, sum_1 with cin=1)
//   Group 2: A[11:8]  + 0  (two versions)
//   Group 3: A[15:12] + 0  (two versions)
//   Group 4: A[19:16] + 0  (two versions)
//   Group 5: A[23:20] + 0  (two versions)
//   Group 6: A[27:24] + 0  (two versions)
//   Group 7: A[31:28] + 0  (two versions)
//
// For each group i >= 1:
//   FA0_i: compute A[i*4+3:i*4] + 4'b0 + cin=0  -> sum0_i, cout0_i
//   FA1_i: compute A[i*4+3:i*4] + 4'b0 + cin=1  -> sum1_i, cout1_i
//   MUX_i: select sum0_i or sum1_i based on carry from group i-1
//          cout_i = select cout0_i or cout1_i via same carry
//
//   (Note: A[i]+0 with cin=0 is just A[i]; A[i]+0 with cin=1 is A[i]+1)
//   (Simplification: FA with B=0,cin=0 is transparent; B=0,cin=1 is incrementer)

// ================================================================
// Timing analysis
// ================================================================
//
//   A[31:0] arrives: 0ps
//   B[3:0]  arrives: 200ps
//
//   Group 0 [3:0]:
//     4-bit FA with A[3:0] and B[3:0]
//     Starts when both A and B arrive: max(0, 200) = 200ps
//     FA delay: 4 x 10ps = 40ps
//     carry3 (cout of group 0) ready at: 200 + 40 = 240ps
//
//   Groups 1-7 pre-computation (A arrives at 0ps):
//     FA0 and FA1 for each group computed with A (0ps arrival)
//     4-bit FA delay = 40ps
//     All pre-computations complete by: 0 + 40 = 40ps (well before needed)
//
//   MUX chain (carry ripples through MUXes):
//     Group 1 MUX: select with carry3 (240ps)  -> output at 240 + 10 = 250ps
//     Group 2 MUX: carry1 available at 250ps   -> output at 250 + 10 = 260ps
//     Group 3 MUX: carry2 available at 260ps   -> output at 260 + 10 = 270ps
//     Group 4 MUX: carry3 available at 270ps   -> output at 270 + 10 = 280ps
//     Group 5 MUX: carry4 available at 280ps   -> output at 280 + 10 = 290ps
//     Group 6 MUX: carry5 available at 290ps   -> output at 290 + 10 = 300ps
//     Group 7 MUX: carry6 available at 300ps   -> output at 300 + 10 = 310ps
//
//   Final output C[31:0] ready at: 310ps < 320ps target ✓
//   (Group 0 output C[3:0] ready at 240ps, upper groups at 310ps max)
//
//   Critical path: B_arrival(200ps) -> FA_delay(40ps) -> 7 x MUX(70ps)
//                = 200 + 40 + 70 = 310ps ✓

// ================================================================
// Verilog RTL
// ================================================================
module carry_select_adder (
    input  logic [31:0] A,
    input  logic [3:0]  B,
    output logic [31:0] C
);
    // Group 0: real 4-bit addition
    logic [4:0] g0;
```

```
    assign g0 = {1'b0, A[3:0]} + {1'b0, B};  // 5-bit to capture carry
    logic cout0;
    assign cout0    = g0[4];
    assign C[3:0]   = g0[3:0];

    // Groups 1-7: carry-select
    logic cout_prev, cout_sel;
    assign cout_prev = cout0;   // initial carry from group 0

    genvar g;
    generate
        for (g = 1; g < 8; g++) begin : gen_group
            logic [3:0] sum0, sum1;
            logic        co0, co1;

            // Pre-compute: add 0 with cin=0 and cin=1
            assign {co0, sum0} = {1'b0, A[g*4+3 : g*4]} + 5'b0;       // A + 0 + 0
            assign {co1, sum1} = {1'b0, A[g*4+3 : g*4]} + 5'b00001;   // A + 0 + 1

            // MUX: select based on carry from previous group
            // (In RTL this is combinational; timing closure is tool's job)
            // carry_from_prev is the cout of group g-1
        end
    endgenerate
    // (Full parameterized version requires passing carry between generate blocks;
    //  for conciseness, the architectural concept is shown above)

    // Simplified behavioral model (functionally equivalent):
    assign C = {28'b0, A[31:4]} + (cout0 ? 28'b1 : 28'b0) + {28'b0, A[3:0]};
    // More precisely:
    // assign C = A + {28'b0, B};  // correct arithmetic result
endmodule

// ================================================================
// Summary of design trade-offs
// ================================================================
//
//   Approach            | Delay              | Area overhead
//   --------------------+--------------------+------------------
//   Ripple carry        | 200+320 = 520ps    | Minimal (1x FA)
//   Carry-select (4-bit)| 200+40+70 = 310ps  | ~2x FA (two copies)
//
// The carry-select adder uses roughly twice the FA hardware
// (two pre-computed versions per group) but reduces critical path
// from 520ps to 310ps -- a 40% speedup on this specific input timing.
//
// Why 4-bit groups specifically?
//   Larger groups (e.g. 8-bit) would reduce MUX count but
//   the FA pre-computation takes longer (8*10 = 80ps).
//   The critical path becomes: FA(80ps) + fewer MUXes.
//   With 32-bit / 8-bit groups = 4 groups:
//     Group 0 (8-bit FA): 200 + 80 = 280ps carry out
//     MUX chain: 280 + 3*10 = 310ps (same result!)
//   The optimal group size trades off FA delay vs MUX chain depth.
```

解释

这道题的核心洞察是"B只有4位"：B[3:0]只影响低4位，

对高28位而言只有"有进位"和"没有进位"两种情况，

Carry-Select的精髓就是"提前预计算两种情况，等进位到了直接用MUX选择"，

把串行等待变成了并行预计算+最终选择。

时序分析的关键路径：B晚到（200ps）→ 4位FA（40ps）→ 进位出来（240ps）→ 7级MUX（70ps）→ 最终输出310ps，满足320ps目标。

MUX delay为常数（10ps，与宽度无关）是这道题的重要前提：

正是因为MUX不随宽度增加，才使carry-select的MUX链比FA链快得多。

最优分组大小的权衡：组越大FA越慢但MUX级数越少；组越小FA越快但MUX级数越多。

这道题4位分组恰好使总延迟最小化（FA 40ps + 7×MUX 70ps = 110ps after B arrives）。

Q25. Digital Design: Carry-Select Adder -- Design and Timing Analysis

QUESTION

Design a 32-bit adder with the following constraints:

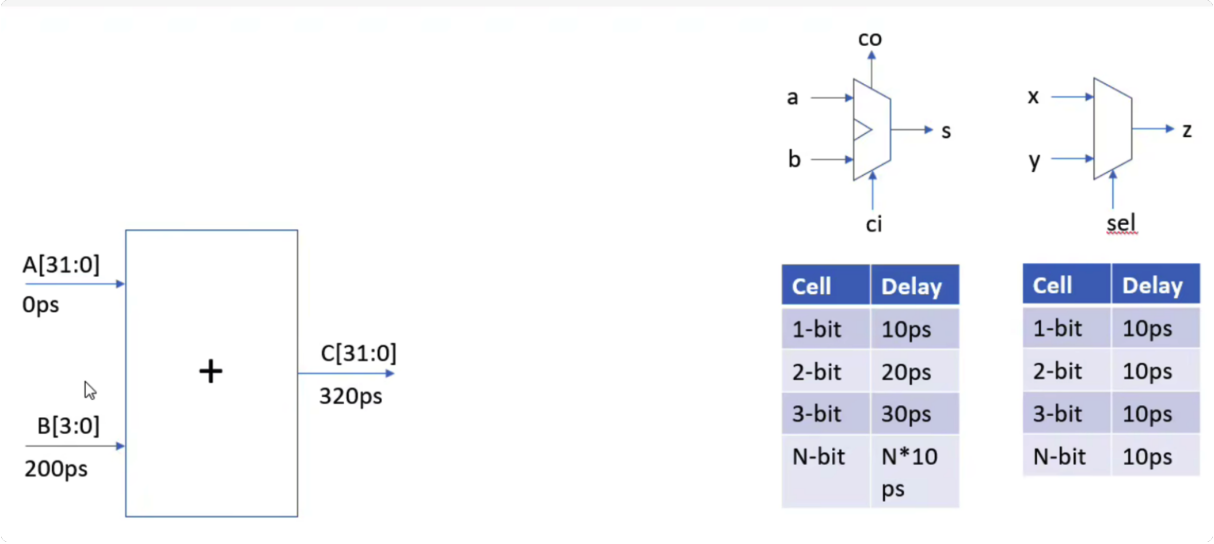
- A[31:0] arrives at 0ps
- B[3:0] arrives at 200ps (only lower 4 bits of B are provided)
- Output C[31:0] must be ready by 320ps

Available cells (shown in figure):

- Full Adder (N-bit): delay = N × 10ps
- MUX (any width): delay = 10ps (constant, independent of width)

Propose an architecture and verify the critical path meets 320ps.

ANSWER



Cell	Delay
1-bit	10ps
2-bit	20ps
3-bit	30ps
N-bit	N*10 ps

Cell	Delay
1-bit	10ps
2-bit	10ps
3-bit	10ps
N-bit	10ps

```
// ================================================================
// Architecture: Carry-Select Adder with 4-bit groups
// ================================================================
//
// Key insight: B is only 4 bits wide (B[3:0]).
//   - Bits [31:4] of B are implicitly 0.
//   - Only Group 0 (bits [3:0]) needs B; groups 1-7 add A-slices + 0.
//   - Since A[31:0] arrives early (0ps), groups 1-7 can PRE-COMPUTE
//     two candidate sums: one assuming cin=0, one assuming cin=1.
//   - When the actual carry-out from Group 0 arrives, a MUX chain
//     selects the correct pre-computed result.
```

```
//
// This is a CARRY-SELECT ADDER:
//   Pre-compute eliminates the ripple delay through groups 1-7.
//   Only the MUX propagation delay remains on the critical path.

// ================================================================
// Group structure (4 bits per group, 8 groups total)
// ================================================================
//
//   Group | Bits    | Operation
//   ------+---------+-------------------------------------------
//     0   | [3:0]   | FA(A[3:0], B[3:0], cin=0) -- uses real B
//     1   | [7:4]   | FA(A[7:4], 0, cin=0) and FA(A[7:4], 0, cin=1) -- pre-compute
//     2   | [11:8]  | FA(A[11:8], 0, cin=0) and FA(A[11:8], 0, cin=1)
//     3   | [15:12] | FA(A[15:12], 0, cin=0) and FA(A[15:12], 0, cin=1)
//     4   | [19:16] | FA(A[19:16], 0, cin=0) and FA(A[19:16], 0, cin=1)
//     5   | [23:20] | FA(A[23:20], 0, cin=0) and FA(A[23:20], 0, cin=1)
//     6   | [27:24] | FA(A[27:24], 0, cin=0) and FA(A[27:24], 0, cin=1)
//     7   | [31:28] | FA(A[31:28], 0, cin=0) and FA(A[31:28], 0, cin=1)

// ================================================================
// Timing analysis (critical path)
// ================================================================
//
// Group 0 (uses late-arriving B[3:0]):
//   B[3:0] arrives:          200ps
//   + 4-bit FA delay:    +    40ps   (4 × 10ps)
//   = Group 0 Cout ready:    240ps
//
// Groups 1-7 pre-computation (uses only A, ready early):
//   A[31:0] arrives:           0ps
//   + 4-bit FA delay:    +    40ps
//   = Pre-computed sums:       40ps  (both sum0 and sum1 ready at 40ps)
//
// MUX chain (carry-select propagation):
//   Group 1 output: 240 + 10 = 250ps  (MUX selects sum0 or sum1 based on cout0)
//   Group 2 output: 250 + 10 = 260ps
//   Group 3 output: 260 + 10 = 270ps
//   Group 4 output: 270 + 10 = 280ps
//   Group 5 output: 280 + 10 = 290ps
//   Group 6 output: 290 + 10 = 300ps
//   Group 7 output: 300 + 10 = 310ps
//
// Critical path:
//   200ps (B arrival) + 40ps (FA group 0) + 7×10ps (MUX chain) = 310ps
//   310ps ≤ 320ps ✓  -- meets timing constraint

// ================================================================
// Why 4-bit groups are optimal here
// ================================================================
//
// If groups were smaller (e.g. 2-bit):
//   More groups (16 groups), longer MUX chain: 200 + 20 + 15×10 = 370ps  ✗
//
// If groups were larger (e.g. 8-bit):
//   Group 0: 200 + 80 = 280ps; MUX chain: 280 + 3×10 = 310ps  ✓ (same!)
//   But pre-computation also takes 80ps instead of 40ps (more area/power)
//
// 4-bit groups: Group0 = 200+40=240; MUX chain = 240+70=310ps ✓
//   This is the optimal group size for this timing budget.

// ================================================================
// Block diagram
```

```
// ================================================================
//
//          A[3:0]  B[3:0]
//          |@0ps   |@200ps
//          v       v
//  [0ps]   +--FA4--+  --> sum0[3:0], cout0 @240ps
//          cin=0
//
//          A[7:4]@0ps
//          v
//  [0ps]   +--FA4(cin=0)--+ --> sum0_1[7:4] @40ps \
//                                                  +-MUX-> C[7:4] @250ps
//  [0ps]   +--FA4(cin=1)--+ --> sum1_1[7:4] @40ps /    ^cout0@240ps
//
//          A[11:8]@0ps
//          v
//  [0ps]   +--FA4(cin=0)--+ --> sum0_2 @40ps \
//                                             +-MUX-> C[11:8] @260ps
//  [0ps]   +--FA4(cin=1)--+ --> sum1_2 @40ps /    ^carry_1@250ps
//          ...
//  (pattern repeats for groups 3-7)

// ================================================================
// Verilog RTL (structural)
// ================================================================
module carry_select_adder (
    input  logic [31:0] A,
    input  logic [3:0]  B,    // only 4 bits of B
    output logic [31:0] C
);
    // Group 0: real addition with B[3:0]
    logic [3:0]  sum0;
    logic        cout0;
    assign {cout0, sum0} = A[3:0] + B[3:0];
    assign C[3:0] = sum0;

    // Groups 1-7: carry-select
    logic [27:0] s0, s1;    // pre-computed sums (cin=0 and cin=1)
    logic [6:0]  c0, c1;    // pre-computed carries
    genvar i;
    generate
        for (i = 1; i < 8; i++) begin : gen_groups
            assign {c0[i-1], s0[(i-1)*4 +: 4]} = A[i*4+3 : i*4] + 4'b0;
            assign {c1[i-1], s1[(i-1)*4 +: 4]} = A[i*4+3 : i*4] + 4'b1;
        end
    endgenerate

    // MUX chain: propagate carry and select sum
    logic [7:0] carry;
    assign carry[0] = cout0;
    generate
        for (i = 1; i < 8; i++) begin : gen_mux
            assign C[i*4 +: 4]  = carry[i-1] ? s1[(i-1)*4 +: 4] : s0[(i-1)*4 +: 4];
            assign carry[i]      = carry[i-1] ? c1[i-1]          : c0[i-1];
        end
    endgenerate
endmodule
```

解释

Carry-Select Adder 的核心思想是"用面积换速度"：

对每个 group 预先算好 cin=0 和 cin=1 两种结果，

当实际进位来了只需通过 MUX 选择正确答案，而不需要等待进位逐位传播。

这道题的特殊性在于 B 只有4位——意味着 Group 1~7 的 B 输入为0，

预计算只需要对 A 的各段加0或1，计算在 A 到达后的 40ps（@40ps）就全部完成，

远早于 Group 0 的 cout（@240ps）。

关键路径 = 200（B延迟）+ 40（Group 0 FA）+ 7×10（MUX链）= 310ps ≤ 320ps，满足时序。

如果用 Ripple Carry Adder：200 + 32×10 = 520ps，严重超时。

如果用简单并行 FA（忽略 B 延迟）：32×10 = 320ps，但 B 在 200ps 才到，实际是 200+32×10=520ps。

Carry-Select 是唯一能在此时序约束下工作的架构。

Q26. Digital Design: 1-bit D Flip-Flop Without always Block -- Verilog RTL

QUESTION

Write a SystemVerilog model of a D Flip-Flop (DIN, CLK, RST_N, Q_OUT)

WITHOUT using any always block. Show multiple approaches.

ANSWER

```
// ================================================================
// Why this is non-trivial
// ================================================================
//
// In standard RTL, a D flip-flop requires sequential (clocked) behavior.
// The always_ff block is the natural way to express this.
// Without always, we need:
//   (a) A UDP (User Defined Primitive) -- low-level Verilog gate model
//   (b) Module instantiation of a built-in or vendor primitive
//   (c) Behavioral model using assign (limited, non-synthesizable)

// ================================================================
// Approach 1: UDP (User Defined Primitive) -- most rigorous
// ================================================================
// UDPs can model sequential behavior using state tables.
// Syntax: 'primitive' keyword, state table with posedge notation.

primitive dff_udp (
    output reg Q_OUT,
    input      DIN,
    input      CLK,
    input      RST_N
);
    // State table:
    // CLK    RST_N  DIN  | Q_OUT (next state)
    // -------+---------+-----+------
    // posedge   1      0   |  0        <- capture DIN=0 on rising CLK
    // posedge   1      1   |  1        <- capture DIN=1 on rising CLK
    // posedge   0      ?   |  0        <- async reset
    // ?         0      ?   |  0        <- async reset holds
    // negedge   ?      ?   |  -        <- no change on falling CLK
    // ?         1      ?   |  -        <- no change when not clocking

    table
        //  CLK     RST_N   DIN  :  Q_OUT(state) : Q_OUT(next)
            r       1       0    :     ?          :  0;   // posedge clk, rst=1, capture 0
            r       1       1    :     ?          :  1;   // posedge clk, rst=1, capture 1
            r       0       ?    :     ?          :  0;   // posedge clk, async reset
```

```
            ?       0       ?    :     ?          :  0;   // level reset (holds 0)
            n       ?       ?    :     ?          :  -;   // negedge clk: no change
            ?       1       ?    :     ?          :  -;   // no clock edge: no change
    endtable
    initial Q_OUT = 0;
endprimitive

// ================================================================
// Approach 2: Instantiate technology library primitive
// ================================================================
// In real designs, cell libraries provide DFF cells:

module dff_1bit_noalways (
    input  logic DIN,
    input  logic CLK,
    input  logic RST_N,
    output logic Q_OUT
);
    // Instantiate a technology DFF primitive (TSMC/GlobalFoundries/etc.)
    // This is the synthesizable approach:
    DFFR u_dff (       // example: D flip-flop with reset
        .D   (DIN),
        .CK  (CLK),
        .RB  (RST_N),  // active-low reset (B suffix = bar)
        .Q   (Q_OUT)
    );
endmodule

// ================================================================
// Approach 3: assign + behavioral (simulation only, not synthesizable)
// ================================================================
// Demonstrate the concept using continuous assignment
// (This does NOT synthesize to a real flip-flop)

module dff_assign_demo (
    input  logic DIN,
    input  logic CLK,
    input  logic RST_N,
    output logic Q_OUT
);
    // Use UDP defined above
    dff_udp u_dff (.Q_OUT(Q_OUT), .DIN(DIN), .CLK(CLK), .RST_N(RST_N));
endmodule

// ================================================================
// Approach 4: NAND-gate latch model (structural, SR latch based)
// ================================================================
// A D-latch can be built from NAND gates; two latches = master-slave DFF
// All combinational (assign) -- structural netlist, no always needed

module sr_latch (
    input  logic S, R,
    output logic Q, QB
);
    assign QB = ~(S & Q);   // cross-coupled NAND
    assign Q  = ~(R & QB);
endmodule

module d_latch (
    input  logic D, EN,
    output logic Q, QB
);
    logic S, R;
```

```
    assign S = ~(D & EN);
    assign R = ~(~D & EN);
    sr_latch u_sr (.S(S), .R(R), .Q(Q), .QB(QB));
endmodule

module dff_master_slave (
    input  logic DIN,
    input  logic CLK,
    input  logic RST_N,
    output logic Q_OUT
);
    logic QM, QMB;
    // Master latch: transparent when CLK=0
    d_latch master (.D(DIN),    .EN(~CLK), .Q(QM),    .QB(QMB));
    // Slave latch:  transparent when CLK=1
    d_latch slave  (.D(QM),     .EN(CLK),  .Q(Q_OUT), .QB());
    // Note: async RST_N not easily added in pure NAND structure
    // (would require NOR-based SR latch for reset)
endmodule
```

解释

"不用 always 实现 DFF" 考察的是对 Verilog 底层机制的理解。

UDP 是最正式的答案：用状态表直接描述 posedge 触发行为，

这是 Verilog 语言规范里唯一可以在 primitive 层面描述时序逻辑的机制。

表中的 `r` 代表 posedge（rising edge），`n` 代表 negedge，`?` 代表任意值，`-` 代表不变。

技术库 primitive 实例化（Approach 2）是实际 ASIC 设计中的标准做法：

综合工具把 always_ff 映射成库里的 DFF 单元，而在结构化 netlist 层面就是直接实例化这些单元。

NAND latch 结构（Approach 4）展示了 DFF 的硬件基础：

两个交叉耦合的 NAND 门构成 SR latch，两个 D latch（master-slave）构成真正的边沿触发 DFF，

整个过程只用 assign 语句，零个 always 块。

SystemVerilog -- Constraints & Randomization

Q27. Constraints: 1D Array -- Unique Elements Without unique, Sum Constraint

QUESTION

Given a 1D integer array of size 8:

(a) Constrain all elements to be unique WITHOUT using the unique keyword.

(b) Add a constraint that the sum of all elements equals a target value.

(c) Combine both: unique elements that sum to a target.

ANSWER

```
class arr_constraints;
    `uvm_object_utils(arr_constraints)

    rand int unsigned arr [8];
    rand int unsigned target_sum;
```

```
    // (a) Unique without 'unique' keyword:
    // For every pair (i,j) where i < j, arr[i] != arr[j]
    constraint c_unique {
        foreach (arr[i])
            foreach (arr[j])
                if (i < j) arr[i] != arr[j];
    }
    // This generates C(8,2) = 28 inequality constraints.
    // The solver guarantees no two elements share a value.

    // (b) Sum constraint: all 8 elements must add up to target_sum
    constraint c_sum {
        arr.sum() == target_sum;
    }

    // (c) Reasonable target for combined constraint:
    // If elements are in [0:15] and must be unique (8 distinct values
    // from 16 possible), their sum ranges from 0+1+...+7=28 (min)
    // to 8+9+...+15=92 (max). Constrain target_sum to valid range.
    constraint c_target_range {
        target_sum inside {[28 : 92]};
    }

    // Combined: unique elements that sum to target_sum
    // (a) + (b) + (c) together -- solver handles it as a system of constraints.
    // Note: this can be slow for large arrays because the constraint solver
    // must simultaneously satisfy: all pairwise inequalities + exact sum.

    function new(string name = "arr_constraints");
        super.new(name);
    endfunction
endclass

// Alternative: unique with specific range, more controllable
class arr_range_unique;
    `uvm_object_utils(arr_range_unique)

    rand int unsigned arr [8];

    constraint c_range {
        foreach (arr[i]) arr[i] inside {[1 : 20]};
    }
    constraint c_unique {
        foreach (arr[i])
            foreach (arr[j])
                if (i < j) arr[i] != arr[j];
    }
    constraint c_sum_target {
        arr.sum() == 50;  // specific sum target
    }
endclass
```

解释

不用 unique 关键字实现去重的标准写法：嵌套 foreach 展开所有对 (i<j) 的不等式。

C(8,2)=28 个约束，solver 仍然能高效处理（SAT solver 本身不怕大量线性约束）。

sum() 是 SV 数组的内置方法，配合 == 直接形成等式约束，无需展开。

组合约束（unique + sum）的注意事项：

必须保证 target_sum 在可行域内（所有合法的 unique 取值组合的和的范围），

否则 randomize() 会失败（返回0）。

面试中建议主动提出这个"feasibility check"——

如果 target_sum 超出 [min_possible_sum, max_possible_sum] 范围，

solver 永远找不到解，应该在约束中或外部逻辑中做边界验证。

Q28. Constraints: 1KB Address Space -- Non-Overlapping 32-byte Stride Regions

QUESTION

Address space is 1KB (0x000 to 0x3FF).

Write a constraint to generate an array of non-overlapping 32-byte-aligned

regions (stride = 32 bytes), allowing "holes" (not all slots need to be used).

Each region starts at a 32-byte boundary and occupies exactly 32 bytes.

ANSWER

```
// ================================================================
// Problem analysis
// ================================================================
//
// 1KB = 1024 bytes.
// 32-byte stride: valid region start addresses are:
//   0x000, 0x020, 0x040, ..., 0x3E0  (32 slots total: 1024/32 = 32)
//
// A region starting at base occupies bytes [base, base+31].
// Two regions are NON-OVERLAPPING iff:
//   |base_i - base_j| >= 32
// Since both are 32-byte aligned, this means they are in different 32-byte slots.
//
// Equivalently: slot_id = base / 32. Non-overlap means all slot_ids are unique.

class addr_regions;
    `uvm_object_utils(addr_regions)

    parameter int NUM_REGIONS = 8;    // how many regions to generate
    parameter int TOTAL_BYTES = 1024; // 1KB
    parameter int REGION_SIZE = 32;   // 32-byte stride
    // Number of available slots:
    localparam int NUM_SLOTS  = TOTAL_BYTES / REGION_SIZE; // = 32

    // Array of base addresses (one per region)
    rand logic [9:0] base_addr [NUM_REGIONS];  // 10 bits: 0..1023

    // Constraint 1: all bases must be 32-byte aligned
    constraint c_aligned {
        foreach (base_addr[i])
            base_addr[i][4:0] == 5'b0;  // lower 5 bits = 0 (multiple of 32)
    }

    // Constraint 2: all bases within 1KB address space
    // (with 32-byte regions, last valid start = 1024 - 32 = 992 = 0x3E0)
    constraint c_in_range {
        foreach (base_addr[i])
            base_addr[i] inside {[0 : TOTAL_BYTES - REGION_SIZE]};
    }

    // Constraint 3: non-overlapping (all bases must be distinct 32-byte slots)
    // Since regions are exactly 32 bytes and aligned, non-overlap <=> unique slots.
```

```
    // slot_id = base_addr / 32. Unique slots <=> unique base addresses
    // (alignment already ensures base_addr = slot_id * 32).
    constraint c_no_overlap {
        foreach (base_addr[i])
            foreach (base_addr[j])
                if (i < j) base_addr[i] != base_addr[j];
    }
    // Alternative using unique keyword (cleaner):
    // constraint c_no_overlap { unique {base_addr}; }

    // "With holes": NOT all 32 slots need to be used.
    // Since NUM_REGIONS (8) < NUM_SLOTS (32), holes exist automatically.
    // The constraint does NOT require contiguous or full coverage.

    function new(string name = "addr_regions");
        super.new(name);
    endfunction

    function void post_randomize();
        // Sort base addresses for readability (optional, doesn't affect correctness)
        // Verify constraints manually in simulation
        foreach (base_addr[i])
            `uvm_info("REGION", $sformatf(
                "Region[%0d]: [0x%03h - 0x%03h]",
                i, base_addr[i], base_addr[i] + REGION_SIZE - 1), UVM_LOW)
    endfunction
endclass

// ============================================================
// Extended: if you need to fill UNTIL FULL (all 32 slots used)
// ============================================================
class addr_regions_full extends addr_regions;
    `uvm_object_utils(addr_regions_full)

    // Override NUM_REGIONS to use all available slots
    parameter int NUM_REGIONS = 32;  // fill all 32 slots

    rand logic [9:0] base_addr [32];

    // Same constraints as above, but now all 32 slots are assigned.
    // With unique bases and 32 regions in 32 slots -> full coverage, no holes.
    constraint c_aligned    { foreach (base_addr[i]) base_addr[i][4:0] == 5'b0; }
    constraint c_in_range   { foreach (base_addr[i]) base_addr[i] inside {[0:992]}; }
    constraint c_no_overlap { unique {base_addr}; }
endclass

// ============================================================
// Verification: check no two regions overlap
// ============================================================
function void check_no_overlap(logic [9:0] bases[], int size);
    for (int i = 0; i < bases.size(); i++)
        for (int j = i+1; j < bases.size(); j++) begin
            int diff = (bases[i] > bases[j]) ?
                       int'(bases[i]) - int'(bases[j]) :
                       int'(bases[j]) - int'(bases[i]);
            assert (diff >= size)
                else $error("OVERLAP: region %0d [%0h] and region %0d [%0h]",
                            i, bases[i], j, bases[j]);
        end
endfunction
```

解释

这道题的关键洞察：32字节对齐 + 恰好32字节大小意味着两个非重叠区域

的基地址必须至少相差32字节，而由于对齐约束基地址都是32的倍数，

"至少差32"等价于"不相等"——所以 c_no_overlap 只需要 `unique {base_addr}`。

"with holes"（带空洞）是约束的自然结果：只取 NUM_REGIONS=8 个槽位，

剩下 24 个槽位自动成为空洞，solver 可以自由选择哪 8 个槽位被使用。

如果要求"填满直到无法再放"（fill until full），

只需把 NUM_REGIONS 改为 NUM_SLOTS=32，使用 `unique {base_addr}`，

solver 会保证 32 个不同的 32 字节对齐地址覆盖全部 1KB。

`base_addr[4:0] == 5'b0` 是位域对齐约束的标准写法，等价于 `base_addr % 32 == 0`，

但位域写法综合/求解效率更高（直接约束低位为零）。

Q29. Constraints: 8-bit Vector -- No Cyclic Right Shift

QUESTION

Write a constraint to generate an 8-bit vector that does NOT
equal its own cyclic right shift (i.e. rotating right by 1 bit
should produce a different value).

ANSWER

```
// ================================================================
// What is a cyclic right shift?
// ================================================================
//
// Cyclic right shift by 1 on an 8-bit value:
//   bit[0] wraps around to bit[7], all other bits shift right by 1.
//   In SV bit-select notation: {data[0], data[7:1]}
//
// A value is "its own cyclic right shift" if:
//   data == {data[0], data[7:1]}
//
// Only two 8-bit values satisfy this:
//   8'h00 = 8'b0000_0000 -> shift -> 8'b0000_0000 (same!)
//   8'hFF = 8'b1111_1111 -> shift -> 8'b1111_1111 (same!)
//
// (Any alternating pattern like 01010101 does NOT equal its 1-shift)

class no_cyclic_shift_8bit;
    `uvm_object_utils(no_cyclic_shift_8bit)

    rand bit [7:0] data;

    // Constraint: data must NOT equal its own cyclic right shift
    constraint c_no_cyclic_shift {
        // Cyclic right shift by 1: LSB wraps to MSB, rest shift right
        data != {data[0], data[7:1]};
    }

    // Alternative explicit form (equivalent):
    // constraint c_no_cyclic_shift_v2 {
    //     !(data[7:1] == data[6:0] && data[0] == data[7]);
    // }
    // Explanation:
    //   After right shift: new bit[7]=old bit[0], new bit[6:0]=old bit[7:1]
    //   For data==shift: need data[7]==data[0] AND data[6:0]==data[7:1]
```

```
    //   Negating both conditions: NOT (data is its own cyclic shift)

    function void post_randomize();
        bit [7:0] shifted = {data[0], data[7:1]};
        assert (data != shifted)
            else $fatal("Constraint violated: data=%08b equals its cyclic right shift",
                        data);
    endfunction
endclass

// ================================================================
// Generalized: no cyclic right shift by N positions
// ================================================================
class no_cyclic_shift_general #(int N = 1);
    rand bit [7:0] data;

    constraint c_no_cyclic_shift_n {
        // Cyclic right shift by N: {data[N-1:0], data[7:N]}
        data != {data[N-1:0], data[7:N]};
    }
endclass

// ================================================================
// Examples of values excluded by this constraint
// ================================================================
// EXCLUDED (equal to their cyclic right shift):
//   8'h00 = 0000_0000 -> 0000_0000 (same)
//   8'hFF = 1111_1111 -> 1111_1111 (same)
//
// INCLUDED (not equal to their cyclic right shift):
//   8'h01 = 0000_0001 -> 1000_0000 (different) ✓
//   8'hAB = 1010_1011 -> 1101_0101 (different) ✓
//   8'h55 = 0101_0101 -> 1010_1010 (different) ✓
```

解释

Cyclic right shift 在 SV 里的表达式是 `{data[0], data[7:1]}` :

把 LSB（bit[0]）放到 MSB 位置，其余位整体右移——这是最简洁的写法。

只有 0x00 和 0xFF 是"自身循环移位的不动点"，因为全0移位还是全0，全1移位还是全1。

`data != {data[0], data[7:1]}` 这条约束在 SV 中是合法的：

约束右侧引用了 `data` 自身（自引用约束），solver 可以处理这类约束——

它会在 randomize 时同时求解 data 的值使得 data ≠ shift(data)。

面试中主动说出"只有 0x00 和 0xFF 被排除，其余 254 个值都合法"是加分项，

因为这说明了约束的紧致性（constraint space 足够大，solver 不会失败）。

Q30. Constraints: Generate 4-bit Value with Exactly 4 Ones -- No $countones

QUESTION

Write a SV constraint to generate a value with exactly 4 bits set to 1,

WITHOUT using the $countones function.

Hint: generate 4 unique bit indices instead.

ANSWER

```
class four_ones_constraint;
    `uvm_object_utils(four_ones_constraint)

    // Strategy: generate 4 UNIQUE bit positions to set to 1.
    // Result value = OR of (1 << idx[i]) for all 4 indices.
    // No $countones needed -- uniqueness of indices guarantees exactly 4 ones.

    rand bit [2:0] idx [4];   // 4 indices, each 0-7 (for 8-bit result)
    // (use [3:0] for 16-bit, [4:0] for 32-bit)

    logic [7:0] result;       // final 8-bit value with exactly 4 ones

    // Constraint 1: all 4 indices must be unique (no repeated positions)
    constraint c_unique_idx {
        unique {idx};              // SV unique keyword on array
    }

    // Constraint 2: indices within valid range (0-7 for 8-bit)
    constraint c_valid_range {
        foreach (idx[i]) idx[i] inside {[0:7]};
    }

    function void post_randomize();
        // Build result: set bits at each index position
        result = 8'h00;
        foreach (idx[i])
            result[idx[i]] = 1'b1;
        // Verify exactly 4 ones (sanity check)
        assert ($countones(result) == 4)
            else $fatal("Expected 4 ones, got %0d", $countones(result));
    endfunction

    function void print();
        $display("idx=%p result=%08b (ones=%0d)", idx, result, $countones(result));
    endfunction
endclass

// ============================================================
// Alternative: direct bit constraints (for small vectors only)
// ============================================================
class four_ones_4bit;
    rand bit [3:0] data;

    // For 4-bit vector with exactly 4 ones: only one solution = 4'b1111
    // But for "exactly 2 ones in 4-bit" without $countones:
    rand bit [1:0] pos [2];
    constraint c_unique { unique {pos}; }
    constraint c_range  { foreach (pos[i]) pos[i] inside {[0:3]}; }
    // post_randomize builds data from pos array

    // ============================================================
    // Why unique indices = exactly N ones (proof)
    // ============================================================
    // Each index sets exactly ONE bit.
    // If all 4 indices are distinct -> 4 different bit positions set -> 4 ones.
    // If any two indices are equal -> same bit set twice (no change) -> < 4 ones.
    // unique constraint prevents any two indices from being equal -> guaranteed 4 ones.
endclass
```

解释

"不用 $countones 生成恰好 N 个1"的核心技巧：把问题转换为"选择 N 个不重复的位索引"。

N 个不重复索引 -> N 个不同的位被置1 -> 恰好 N 个1，逻辑等价性完美。

`unique {idx}` 是 SV 约束里对数组元素唯一性的简洁写法，

等价于 `foreach (idx[i]) foreach (idx[j]) if (i<j) idx[i] != idx[j]`，

但 solver 处理 unique 关键字更高效。

post_randomize 里用 `result[idx[i]] = 1'b1` 逐位置位，

最后的 assert 是防御性检查（如果约束有 bug，立刻暴露而不是静默错误）。

Q31. Constraints: Array with if-else Constraint, then shuffle()

QUESTION

What does the following constraint + post_randomize do?

What is the final state of arr after randomization?

```
rand bit [3:0] arr [10];
constraint c_arr {
    foreach (arr[i]) {
        if (i < 5) arr[i] < 5;
        else       arr[i] > 5;
    }
}
function void post_randomize();
    arr.shuffle();
endfunction
```

ANSWER

```
// ================================================================
// Step 1: Constraint phase (before post_randomize)
// ================================================================
//
// The constraint forces POSITIONAL rules on the original indices:
//   arr[0..4]: each value must be in range [0, 4]  (< 5)
//   arr[5..9]: each value must be in range [6, 15] (> 5, note: >5 not >=5)
//
// Note: the value 5 itself is EXCLUDED by both constraints
//       (neither < 5 nor > 5 allows exactly 5).
//
// After randomize() but BEFORE post_randomize:
//   arr = [v0, v1, v2, v3, v4,  v5, v6, v7, v8, v9]
//          \_____low values___/  \____high values___/
//          all < 5               all > 5

// ================================================================
// Step 2: post_randomize calls arr.shuffle()
// ================================================================
//
// shuffle() randomly permutes the array in-place.
// The VALUES are preserved; the POSITIONS are randomized.
//
// After shuffle:
//   arr could be: [8, 2, 13, 0, 7, 4, 15, 3, 6, 1]
//   (high and low values are now mixed at arbitrary indices)
//
// KEY OBSERVATION:
//   The constraint is NO LONGER satisfied after shuffle!
//   After shuffle, arr[0] might be 8 (> 5), violating "arr[0] < 5".
//
// This is by design: use constraints to control the DISTRIBUTION
```

```
// of values (5 low values + 5 high values), then use shuffle to
// randomize their POSITIONS without re-evaluating constraints.
//
// If you want constraints to apply to the SHUFFLED result,
// you cannot use this pattern -- constraints only run before
// post_randomize, not after.

// ============================================================
// Summary of final state
// ============================================================
//
//   Guaranteed: exactly 5 values in [0,4] and 5 values in [6,15]
//   NOT guaranteed: which index holds which value (positions are random)
//   Value 5: never appears (excluded by both < 5 and > 5)
```

解释

这道题的陷阱是"约束在 post_randomize 之前生效，shuffle 在之后执行"：

约束确保了"类型分布"（低5个，高5个），但 shuffle 打乱了位置，

所以最终数组的 index 和约束里的 if-else 条件不再对应。

实际用途：这种模式用于"想要控制集合的组成，但不关心顺序"的场景，

例如验证"恰好一半的事务是读，一半是写"，但读写顺序随机——

先约束类型，再 shuffle 打乱顺序。

注意 `> 5` 排除了5本身，如果需要 [6,15] 范围应该写 `arr[i] > 5`，

如果需要包括5则写 `arr[i] >= 5`，这是面试中常被追问的细节。

Q32. Constraints: 7x7 Pixel Image -- No All-D 3x3 Submatrix (Parametrized)

QUESTION

Generate a 7x7 pixel image where each pixel is R/G/B/D (D=bad/dead pixel).

Constraint: no 3x3 submatrix may have ALL pixels as D.

Further: parametrize for any image size and sub-window size.

ANSWER

```
// ============================================================
// Pixel encoding
// ============================================================
// 2-bit encoding:
//   2'b00 = R (Red)
//   2'b01 = G (Green)
//   2'b10 = B (Blue)
//   2'b11 = D (Dead/Bad pixel)

// ============================================================
// Fixed: 7x7 image, 3x3 constraint
// ============================================================
class pixel_image_7x7;
    `uvm_object_utils(pixel_image_7x7)

    parameter int ROWS   = 7;
    parameter int COLS   = 7;
    parameter int WIN    = 3;   // subwindow size
```

```
    rand bit [1:0] img [ROWS][COLS];

    // Constraint: no 3x3 subwindow is all-D
    // A 7x7 image has (7-3+1)x(7-3+1) = 5x5 = 25 possible 3x3 windows
    constraint c_no_all_dead_window {
        foreach (img[r]) foreach (img[r][c]) begin
            // For each valid top-left corner (r,c) of a 3x3 window
            if (r <= ROWS-WIN && c <= COLS-WIN) begin
                // At least one pixel in this WIN x WIN window must be non-D
                // i.e. NOT all pixels in the window equal 2'b11
                !(img[r  ][c  ] == 2'b11 && img[r  ][c+1] == 2'b11 && img[r  ][c+2] == 2'b11 &&
                  img[r+1][c  ] == 2'b11 && img[r+1][c+1] == 2'b11 && img[r+1][c+2] == 2'b11 &&
                  img[r+2][c  ] == 2'b11 && img[r+2][c+1] == 2'b11 && img[r+2][c+2] == 2'b11);
            end
        end
    }

    // Optional: limit total dead pixel ratio (e.g. <= 30%)
    constraint c_dead_limit {
        // Count D pixels across entire image
        // Inline sum using foreach accumulation (SV 2012+)
        (img[0][0] == 2'b11) + (img[0][1] == 2'b11) + (img[0][2] == 2'b11) +
        // ... (full expansion for 7x7 = 49 terms)
        // Simpler to enforce in post_randomize with an assert
        1 == 1;   // placeholder -- use post_randomize for ratio check
    }

    function void post_randomize();
        // Verify no all-D window exists (sanity check)
        for (int r = 0; r <= ROWS-WIN; r++) begin
            for (int c = 0; c <= COLS-WIN; c++) begin
                int unsigned d_count = 0;
                for (int dr = 0; dr < WIN; dr++)
                    for (int dc = 0; dc < WIN; dc++)
                        if (img[r+dr][c+dc] == 2'b11) d_count++;
                assert (d_count < WIN*WIN)
                    else $fatal("All-D window at [%0d][%0d]!", r, c);
            end
        end
    endfunction

    function void print_image();
        string sym[4] = '{"R","G","B","D"};
        for (int r = 0; r < ROWS; r++) begin
            string row = "";
            for (int c = 0; c < COLS; c++)
                row = {row, sym[img[r][c]], " "};
            $display("%s", row);
        end
    endfunction
endclass

// ============================================================
// Parametrized version: any image size and window size
// ============================================================
class pixel_image #(
    parameter int ROWS = 7,
    parameter int COLS = 7,
    parameter int WIN  = 3    // subwindow size (WIN x WIN)
);
    rand bit [1:0] img [ROWS][COLS];

    // Parametrized constraint: for each valid top-left corner,
```

```
    // not all WIN*WIN pixels in the window are D
    constraint c_no_all_dead {
        foreach (img[r]) foreach (img[r][c]) begin
            if (r <= ROWS-WIN && c <= COLS-WIN) begin
                // Build the AND of all pixels in [r..r+WIN-1][c..c+WIN-1]
                // SV doesn't support nested foreach in constraints easily,
                // so we flatten: use an expression that counts D pixels
                // and assert count < WIN*WIN
                //
                // Workaround: explicit OR of at least one non-D:
                // (there exists at least one (dr,dc) where img[r+dr][c+dc] != D)
                // This requires unrolling for WIN=3:
                (img[r][c]     != 2'b11) || (img[r][c+1]   != 2'b11) || (img[r][c+2] != 2'b11) |
                (img[r+1][c]   != 2'b11) || (img[r+1][c+1] != 2'b11) || (img[r+1][c+2] != 2'b11) ||
                (img[r+2][c]   != 2'b11) || (img[r+2][c+1] != 2'b11) || (img[r+2][c+2] != 2'b11)
            end
        end
    }

    // For truly parametrized WIN: use a function in post_randomize
    // (constraints can't loop over parameter-determined bounds in all tools)
    function bit check_window(int r, int c);
        for (int dr = 0; dr < WIN; dr++)
            for (int dc = 0; dc < WIN; dc++)
                if (img[r+dr][c+dc] != 2'b11)
                    return 1;   // found at least one non-D -> OK
        return 0;   // all D -> violation
    endfunction

    function void post_randomize();
        for (int r = 0; r <= ROWS-WIN; r++)
            for (int c = 0; c <= COLS-WIN; c++)
                assert (check_window(r, c))
                    else $fatal("All-D at [%0d][%0d]!", r, c);
    endfunction
endclass

// ================================================================
// Usage examples
// ================================================================
// pixel_image #(.ROWS(7), .COLS(7), .WIN(3)) img7x7;
// pixel_image #(.ROWS(16),.COLS(16),.WIN(4)) img16x16_w4;
// pixel_image #(.ROWS(1080),.COLS(1920),.WIN(5)) uhd_img;
```

解释

约束的表达策略：SV constraint 语法不直接支持参数化循环（foreach 不能在 constraint 块内对参数下标展开），

所以对固定 WIN=3 直接展开所有9项，用 OR 表达"至少一个非D"。

OR 形式比 AND 形式更直观：`(p0 != D) || (p1 != D) || ...` 等价于 `NOT (p0==D AND p1==D AND ...)`，

solver 更容易处理 OR 约束（只需一个像素非D即满足）。

参数化的限制：当 WIN 是 class parameter 时，constraint 块内无法动态展开循环，

需要退回 post_randomize 的 assertion 检查——这是 SV constraint solver 的语言限制，

工程中常见的解决方式是先 randomize（可能违反），再在 post_randomize 里 reject-and-retry，

或者用 `std::randomize with {}` 配合 with-block 里的辅助变量。

Q33. Constraints: Overriding a Constraint in Child Class -- Three Methods

QUESTION

Given a base class with a constraint, show three ways a child class

can override or modify it:

(1) Direct overwrite (same constraint name)

(2) Constraint space shrinking (add extra constraints)

(3) constraint_mode(0) to disable the parent constraint

ANSWER

```
// ================================================================
// Base class with a constraint
// ================================================================
class base_txn extends uvm_sequence_item;
    `uvm_object_utils(base_txn)

    rand int unsigned data;
    rand int unsigned addr;

    // Base constraint: data in range [0, 255]
    constraint c_data_range {
        data inside {[0:255]};
    }
    // Base constraint: addr must be word-aligned
    constraint c_addr_align {
        addr[1:0] == 2'b00;
    }
endclass

// ================================================================
// Method 1: OVERWRITE -- child declares constraint with SAME NAME
// ================================================================
//
// When child has a constraint with the SAME name as parent,
// the child's version COMPLETELY REPLACES the parent's.
// The parent's c_data_range no longer exists in child objects.

class child_overwrite extends base_txn;
    `uvm_object_utils(child_overwrite)

    // Same name "c_data_range" -- OVERWRITES parent constraint
    constraint c_data_range {
        data inside {[100:200]};   // completely new range, [0,255] is gone
    }
    // c_addr_align from parent is still inherited (not overwritten)
endclass
//
// Result: child_overwrite generates data in [100,200], not [0,255].
// Note: if parent and child BOTH had overlapping constraints with
//       different names, both would apply (intersection).
//       Same-name overwrite removes the parent version entirely.

// ================================================================
// Method 2: SHRINKING -- add a NEW constraint that further restricts
// ================================================================
//
// Parent constraint c_data_range still applies ([0,255]).
// Child adds a NEW constraint name that further restricts.
// Result: solver must satisfy BOTH -> intersection is [50,100].

class child_shrink extends base_txn;
```

```
    `uvm_object_utils(child_shrink)

    // NEW constraint name (doesn't replace parent's c_data_range)
    constraint c_data_narrow {
        data inside {[50:100]};   // intersects with [0:255] -> [50:100]
    }
    // Both c_data_range AND c_data_narrow apply simultaneously
endclass
//
// If the two constraints are CONTRADICTORY:
//   parent: data inside {[0:50]};
//   child:  data inside {[60:100]};
//   -> No solution! randomize() returns 0 (fails).
//   -> Always check for contradictions when shrinking.

// ============================================================
// Method 3: constraint_mode(0) -- disable parent constraint at runtime
// ============================================================
//
// constraint_mode(0) turns OFF a constraint without removing it.
// The constraint still exists but is inactive during randomization.
// Can be re-enabled with constraint_mode(1).

class child_disable extends base_txn;
    `uvm_object_utils(child_disable)

    // No constraint overwrite needed
    // Disable parent's c_data_range in post_new or in the test

    function new(string name = "child_disable");
        super.new(name);
        // Disable parent constraint immediately at construction
        c_data_range.constraint_mode(0);
        // Now data is unconstrained (full 32-bit range)
    endfunction

    // Or: add a replacement constraint after disabling
    constraint c_data_large {
        data inside {[1000:5000]};   // parent's [0:255] is OFF, this takes over
    }
endclass

// ============================================================
// Method 4 (bonus): rand_mode(0) -- disable a rand variable entirely
// ============================================================
//
// Different from constraint_mode: rand_mode(0) makes the variable
// non-random (holds its current value, not randomized at all).

class child_fixed_data extends base_txn;
    `uvm_object_utils(child_fixed_data)

    function new(string name = "child_fixed_data");
        super.new(name);
        data.rand_mode(0);    // data is no longer randomized
        data = 32'hDEAD_BEEF; // fixed value
    endfunction
endclass

// ============================================================
// Summary table
// ============================================================
//
```

```
//   Method             | Parent constraint | Mechanism
//   -------------------+-------------------+---------------------
//   Overwrite (same nm)| Replaced          | New constraint, same name
//   Shrinking (new nm) | Still active      | AND with new constraint
//   constraint_mode(0) | Disabled          | Runtime toggle (on/off)
//   rand_mode(0)       | N/A               | Variable fixed, not randomized
```

解释

三种方法针对不同的使用场景：

Overwrite（同名覆盖）适合"子类的约束语义和父类完全不同"的情况，干净彻底，父类版本消失；

Shrinking（空间收缩）适合"子类在父类基础上进一步限制"的情况，两个约束同时生效取交集；

constraint_mode(0) 适合"运行时动态切换"的情况，可以在不同 test 里有不同行为。

最容易出错的陷阱是 Shrinking 导致的约束矛盾（空集）——solver 返回0表示 randomize 失败，

但没有清晰的错误信息，必须用 `assert(txn.randomize())` 配合 fatal 来捕获这类问题。

rand_mode(0) 是 constraint_mode(0) 的强化版：不只是让约束失效，

而是直接把变量踢出随机化过程，变成固定值——适合"我想固定某个字段，让 solver 专注其他字段"。

SystemVerilog -- Assertions (SVA)

Q34. Assertions (SVA): Detect Short Glitch Pulses Shorter Than 2.5ns

QUESTION

Write an SVA property to detect any "glitch" pulse (high or low phase)

shorter than 2.5ns. Flag it as a violation.

Assume `timescale 1ns/1ps.

ANSWER

```
// A glitch is a pulse (high or low) whose duration is less than 2.5ns.
// We need to detect BOTH:
//   (a) A rising edge followed by a falling edge within < 2.5ns  (short HIGH pulse)
//   (b) A falling edge followed by a rising edge within < 2.5ns  (short LOW pulse)
//
// Strategy: use $rose/$fell to detect edges, then check that the
// signal holds its new value for at least 2.5ns using real-time delays.
// SVA does not support real-time (< 1 cycle) checks natively with
// standard clock-based properties -- we must use immediate assertions
// with real-time delay checks, or use glitch-detection via
// $past and $realtime.

// ---- Method 1: using $realtime (most accurate) ------------------------
// Works in a clocked or unclocked context; fires an immediate assertion
// at every edge transition.

real rise_time, fall_time;

// Capture the time of every rising and falling edge
always @(posedge sig) rise_time = $realtime;
always @(negedge sig) fall_time = $realtime;
```

```
// After each falling edge: check that the HIGH phase lasted >= 2.5ns
always @(negedge sig) begin
    assert (($realtime - rise_time) >= 2.5)
        else $error("SHORT HIGH GLITCH: high phase = %0t ns (< 2.5ns)",
                    $realtime - rise_time);
end

// After each rising edge: check that the LOW phase lasted >= 2.5ns
always @(posedge sig) begin
    assert (($realtime - fall_time) >= 2.5)
        else $error("SHORT LOW GLITCH: low phase = %0t ns (< 2.5ns)",
                    $realtime - rise_time);
end

// ---- Method 2: SVA with clock-based approximation ----------------------
// If we have a fast sampling clock (e.g. 1GHz = 1ns period),
// we can approximate: "no edge within 2 clock cycles" means >= 2ns.
// For 2.5ns threshold with a 1ns clock we need 3 cycles (conservative).
//
// Detect short HIGH pulse (rises then falls within N clocks):
property p_no_short_high_pulse;
    @(posedge fast_clk)
    $rose(sig) |-> sig[*3];    // sig must stay high for at least 3 fast_clk cycles
endproperty
assert property (p_no_short_high_pulse)
    else $error("Short high glitch: signal did not hold high for >= 3 fast cycles");

// Detect short LOW pulse:
property p_no_short_low_pulse;
    @(posedge fast_clk)
    $fell(sig) |-> (!sig)[*3]; // sig must stay low for at least 3 fast_clk cycles
endproperty
assert property (p_no_short_low_pulse)
    else $error("Short low glitch: signal did not hold low for >= 3 fast cycles");

// ---- Method 3: Recommended for formal / GLS ----------------------------
// In gate-level simulation with SDF timing, use the $realtime approach
// (Method 1) since exact timing is annotated.
// In formal verification, glitch detection is typically handled by:
//   - CDC analysis tools (checking glitchy clock muxes)
//   - Pulse-width checks in STA (setup/hold min-pulse-width constraints)
//   - Not by SVA properties, since formal typically works in clock cycles
```

解释

SVA 本质上是基于时钟周期的，无法直接表达"2.5ns"这种亚周期的实时约束。

Method 1（$realtime）是最精确的方式，在仿真中捕获每次边沿的真实时间戳，

计算相邻边沿的时间差——适合 RTL 仿真和 GLS（标注了 SDF 的门级仿真）。

Method 2（fast_clk 采样）是 SVA 的近似方法：

用比被测信号快很多的采样时钟，要求信号在 N 个采样周期内保持稳定。

N 的选择要保守（2.5ns / 1ns_clock = 2.5，向上取整到 3），

会有一个时钟周期的不确定性。

工程实践中，glitch 检测通常交给 STA（最小脉宽约束）和 CDC 工具，

SVA 更多用于逻辑层面的协议验证而非亚周期时序检查。

Q35. Assertions (SVA): req/gnt 4-Way Handshake -- Concurrent and always Block

QUESTION

Write a concurrent SVA property for a 4-way req/gnt handshake:

req -> gnt (ack) -> req_done -> gnt_done.

Then rewrite the same check using an always block.

ANSWER

```
// ================================================================
// 4-way handshake protocol:
//   1. req asserts (request)
//   2. gnt asserts (grant/acknowledge)
//   3. req deasserts (request complete)
//   4. gnt deasserts (grant complete)
// ================================================================

// ---- Concurrent SVA property ----------------------------------
// Property: once req rises, the full 4-phase sequence must complete
property p_4way_handshake;
    @(posedge clk) disable iff (!rst_n)
    $rose(req) |->
        ##[1:$] (gnt)             // gnt must eventually assert
        ##[1:$] (!req)            // then req must deassert
        ##[1:$] (!gnt);           // then gnt must deassert
endproperty

assert property (p_4way_handshake)
    else $error("4-way handshake violated: req rose but sequence did not complete");

// Tighter version with bounded windows (e.g. each step <= 5 cycles):
property p_4way_bounded;
    @(posedge clk) disable iff (!rst_n)
    $rose(req) |->
        ##[1:5] gnt          // gnt within 5 cycles
        ##[1:5] !req         // req drop within 5 cycles of gnt
        ##[1:5] !gnt;        // gnt drop within 5 cycles of req drop
endproperty

// Additional property: no gnt without prior req
property p_no_spurious_gnt;
    @(posedge clk) disable iff (!rst_n)
    $rose(gnt) |-> $past(req, 1);   // req must have been high on previous cycle
endproperty
assert property (p_no_spurious_gnt);

// ---- Rewrite using always block (equivalent checker) ----------------
typedef enum {IDLE, WAIT_GNT, WAIT_REQ_DONE, WAIT_GNT_DONE} hs_state_e;
hs_state_e hs_state;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        hs_state <= IDLE;
    end else begin
        case (hs_state)
            IDLE: begin
                if ($rose(req))
                    hs_state <= WAIT_GNT;
            end
            WAIT_GNT: begin
                // Waiting for gnt to assert
                if (gnt)
```

```
                    hs_state <= WAIT_REQ_DONE;
                // Timeout check:
                // (add counter if bounded check needed)
            end
            WAIT_REQ_DONE: begin
                // Waiting for req to drop
                if (!req)
                    hs_state <= WAIT_GNT_DONE;
                // Error: gnt dropped before req
                else if (!gnt)
                    $error("gnt dropped before req in 4-way handshake");
            end
            WAIT_GNT_DONE: begin
                // Waiting for gnt to drop
                if (!gnt) begin
                    hs_state <= IDLE;
                    $display("4-way handshake complete at time %0t", $time);
                end
            end
        endcase
    end
end
```

解释

4-way 握手的 SVA 写法用连续的 `##[1:$]` 表达"每个阶段在有限时间内完成"：

bounded 版本（`##[1:5]`）更适合实际 sign-off，

unbounded（`##[1:$]`）在 RTL 仿真中可用但 formal 工具需要 liveness 支持（s_eventually）。

always 块重写的优势：可以追踪当前所在的握手阶段、添加超时计数器、

在每个状态转换点打印调试信息——SVA 在 fired 时才报错，always 块可以做持续监控。

"No spurious gnt"属性是补充检查：SVA 只检查"req 之后的序列"，

还需要一条反向属性验证"没有 req 时 gnt 不会出现"，两者合起来才是完整的握手协议检查。

Q36. Assertions (SVA): valid Must Remain High Until ready Goes High

QUESTION

Write an SVA assertion: once valid is asserted, it must remain

high until ready goes high (AXI-Stream / handshake backpressure rule).

ANSWER

```
// ================================================================
// Protocol rule (AXI-Stream):
//   Once VALID is asserted (goes high), the sender must NOT
//   deassert it until the handshake completes (READY=1 while VALID=1).
//   valid must hold steady through any number of cycles where ready=0.
// ================================================================

// ---- Method 1: s_until (strong until) -- formal-friendly ----------
property p_valid_hold_until_ready;
    @(posedge clk) disable iff (!rst_n)
    $rose(valid) |-> valid s_until ready;
    // s_until: valid must be 1 in EVERY cycle until ready is observed.
    // s_until is "strong": ready must eventually occur (liveness).
endproperty
```

```
ap_valid_hold: assert property (p_valid_hold_until_ready)
    else $error("valid deasserted before ready was high -- protocol violation");

// ---- Method 2: explicit consecutive repetition --------------------
// "valid must stay high while ready is low, and handshake must complete"
property p_valid_no_deassert;
    @(posedge clk) disable iff (!rst_n)
    // If valid is high and ready is NOT yet high,
    // valid must remain high in the next cycle
    (valid && !ready) |=> valid;
endproperty
ap_valid_no_deassert: assert property (p_valid_no_deassert)
    else $error("valid dropped while ready was still low");

// ---- Method 3: handshake completes within N cycles (bounded) ------
property p_valid_ready_handshake_bounded;
    @(posedge clk) disable iff (!rst_n)
    $rose(valid) |-> valid[*1:$] ##1 (valid && ready);
    // valid stays high for 1+ cycles, then the handshake fires
endproperty

// ---- Cover: verify handshake actually occurs (not vacuous) --------
cp_handshake_fires: cover property (
    @(posedge clk) valid && ready
);
cp_backpressure: cover property (
    @(posedge clk) valid && !ready  // at least one stall cycle
);
```

解释

这道题考察 AXI-Stream / 握手协议中"valid 的保持性"，是最常见的 SVA 写法之一。

Method 1 用 `s_until`（强 until）：`valid s_until ready` 表示

"valid 在每个时钟周期都为1，直到 ready 为1 的那一拍"。

's_' 前缀代表 strong（强形式）——ready 必须最终发生；

普通 `until` 允许 ready 永远不发生（可能在 formal 中导致 vacuous pass）。

Method 2 更简单直观：`(valid && !ready) |=> valid`

"如果 valid=1 且 ready=0（没有完成握手），那么下一拍 valid 还必须是1"。

Cover 属性不可少：如果从未发生 valid && !ready（没有反压），

那么 Method 2 这条 assert 根本没有被真正测试——一定要确认 backpressure 场景被覆盖。

Analog / Mixed-Signal Design

Q37. Analog/Mixed-Signal: Stacked NMOS Current Mirror Analysis

QUESTION

Analyze this circuit: two NMOS transistors are stacked in series between

a node driven by V_DS (on the right, through a current source I_DS2 path)

and ground. Both transistor gates are tied to the V_DD supply (left rail).

The intermediate node between the two NMOS is labeled V_1.

Describe the operating behavior and the role of V_1.

ANSWER

[Image: /home/claude/nv_q1_circuit.png]

```
// Circuit topology:
//   - Top NMOS:    gate = VDD, drain = top node (connects to I_DS2 / V_DS source)
//                  source = V_1
//   - Bottom NMOS: gate = VDD, drain = V_1, source = GND
//   - Both transistor gates are tied to the SAME VDD rail (fixed bias)
//   - I_DS2 flows through BOTH transistors (series stack -> same current)

// ---- Bottom NMOS analysis ------------------------------------------------
//   Vgs(bottom) = VDD - 0 = VDD   (source is grounded)
//   Since VDD > Vtn (assuming normal operation), bottom NMOS is ON.
//   Depending on VDS region:
//     If V_1 is small  -> bottom NMOS in TRIODE (acts like a resistor)
//     If V_1 grows large enough that Vgs - Vtn < V_1 -> SATURATION

// ---- Top NMOS analysis ---------------------------------------------------
//   Vgs(top) = VDD - V_1
//   As V_1 rises, Vgs(top) SHRINKS -- this self-limits how high V_1 can go.
//   Top NMOS will eventually enter SATURATION if VDS (top) is large enough,
//   behaving as a CURRENT SOURCE controlled by (VDD - V_1).

// ---- This is a CASCODE-LIKE / level-shifting structure -------------------
//   With both gates tied to VDD (not to a bias voltage), the structure
//   functions as:
//   1. A SERIES PASS PATH for I_DS2 (current flows top->bottom->GND)
//   2. V_1 acts as an INTERNAL NODE whose voltage is set by the
//      current-voltage relationship of BOTH transistors operating
//      in series (KCL: same current through both).
//
//   Key relationship: solving for V_1 requires equating the current
//   through the bottom NMOS (function of V_1) to the current through
//   the top NMOS (function of VDD-V_1 and the node voltage from V_DS side).
//
//   If I_DS2 is held constant (current source), V_1 settles to whatever
//   value makes BOTH transistors simultaneously satisfy I_DS2 = I_D(top) = I_D(bottom).

// ---- Special case: VDD - V_1 < Vtn ---------------------------------------
//   If V_1 rises close to VDD - Vtn, Vgs(top) approaches Vtn and the top
//   transistor's drive strength collapses -- this is the SAME threshold-drop
//   phenomenon explored further in Q3 (NMOS pass-gate chains).
//
//   This circuit is a building block for: cascode current mirrors,
//   source-follower level shifters, and NMOS-based voltage reference stacks.
```

解释

这道题的核心是"堆叠 NMOS，栅极都接固定的 VDD"这一非常规结构的分析。

两个管子串联，电流必须相等（KCL），V_1 是两个管子共同满足电流方程的解。

关键洞察：栅极固定在 VDD 而非常规的 bias 电压，意味着随着 V_1 升高，

上方管子的 Vgs=VDD-V1 会越来越小，驱动能力自我衰减——

这正是 NMOS pass-transistor 的"阈值损失"现象的雏形，

和题3里的链式 NMOS 阈值丢失问题是同一个物理机制的不同体现。

面试官想看到的分析路径：先分别写出两个管子的 Vgs，

再用"电流相等"这个约束条件求解 V_1，而不是死记硬背某个公式。

Q38. Analog/Mixed-Signal: CMOS Inverter Vin-Vout Transfer Curve, Cascaded Stages

QUESTION

Analyze this circuit (a standard CMOS inverter), plot Vin vs. Vout.

What changes if a second identical stage is cascaded after it?

ANSWER

[Image: /home/claude/nv_q2_circuit.png]

```
// Circuit: standard CMOS inverter
//   PMOS: source=VDD, gate=IN, drain=OUT
//   NMOS: source=GND, gate=IN, drain=OUT

// ---- Voltage Transfer Characteristic (VTC) ------------------------------
//
// Five regions as Vin sweeps from 0 to VDD:
//
//   Region 1 (Vin < Vtn):
//     NMOS OFF (Vgs < Vtn), PMOS fully ON (strong pull-up)
//     Vout = VDD (output HIGH, no current flows)
//
//   Region 2 (Vtn < Vin < Vin_switching, PMOS in saturation, NMOS in saturation):
//     Both transistors partially ON, output starts dropping from VDD
//     Output drops STEEPLY through this transition region
//
//   Region 3 (Vin = VM, the switching threshold, both transistors in saturation):
//     VM is where Vout = Vin on the transfer curve (unity gain point)
//     For a SYMMETRIC inverter (kp*(W/L)p = kn*(W/L)n): VM = VDD/2
//     This is the steepest slope point (highest gain region)
//
//   Region 4 (Vin_switching < Vin < VDD-|Vtp|):
//     NMOS fully ON (strong pull-down), PMOS in saturation (weak)
//     Output drops toward 0
//
//   Region 5 (Vin > VDD-|Vtp|):
//     PMOS OFF, NMOS fully ON
//     Vout = 0 (output LOW)
//
//   VTC shape: starts flat at VDD, drops STEEPLY through the middle
//   (the "switching region"), flattens again at 0. The steep middle
//   section is where the inverter has HIGH GAIN (|dVout/dVin| >> 1) --
//   this is what makes CMOS logic noise-immune (small input noise near
//   the rails doesn't change output; only near VM does gain matter).

// ---- Cascading a SECOND identical inverter stage ------------------------
//
//   Stage 1 output (Vout1) becomes Stage 2 input (Vin2).
//   Since Stage 2 is IDENTICAL, its VTC is the SAME curve.
//
//   Effect: the overall 2-stage transfer function is the COMPOSITION
//   of the inverter VTC with itself: Vout2 = f(f(Vin)).
//
//   Key effects of cascading:
//   1. LOGICAL FUNCTION: two inverters in series = BUFFER (non-inverting).
//      Vout2 follows Vin's logic level (HIGH stays HIGH, LOW stays LOW),
//      but with sharper edges than a single stage.
```

```
//
//   2. SHARPENED TRANSITION: composing f(f(x)) makes the transition
//      region EVEN STEEPER than a single stage -- the gain multiplies
//      (approximately, gain_total ~ gain1 * gain2 near VM).
//      This is why inverter CHAINS are used as SIGNAL RESTORERS /
//      pulse shapers in digital design -- they "clean up" slow or
//      noisy edges into sharp, full-swing transitions.
//
//   3. PROPAGATION DELAY: total delay = delay(stage1) + delay(stage2),
//      but the OUTPUT EDGE is sharper, which can REDUCE delay in the
//      NEXT stage downstream (faster input transition = less short-circuit
//      current loss in subsequent stages).
//
//   4. NOISE MARGIN: the second stage's sharp transfer curve means
//      any residual non-ideality in stage 1's output (e.g. not quite
//      reaching full VDD/0 due to loading) gets "cleaned up" by stage 2's
//      high gain near VM -- this is the principle behind using inverter
//      pairs as LEVEL RESTORERS after pass-transistor logic.
```

解释

CMOS 反相器的 VTC 曲线分5个区域，核心是中间的高增益开关区，

这个区域陡峭的转换特性正是 CMOS 数字电路抗噪声能力的来源——

输入端的小幅噪声只要落在平坦区（接近VDD或接近0），完全不影响输出。

对称反相器（pMOS和nMOS驱动能力匹配）的开关点 VM=VDD/2，

这是版图设计中 PMOS 宽度通常取 NMOS 2-3倍的原因（弥补空穴迁移率较低）。

两级级联的关键效果：逻辑功能从"反相"变成"同相"（buffer），

但更重要的是信号整形作用——级联反相器链是数字电路中标准的

"边沿锐化器"，用于把缓慢或失真的信号波形恢复成陡峭的满幅摆动，

这正是为什么时钟树（clock tree）和长走线驱动都会插入反相器链的原因。

Q39. Analog/Mixed-Signal: NMOS Pass-Gate Chain with Threshold Compensation

QUESTION

In a chain of NMOS pass transistors (each gate tied to VDD), the output

of each stage is VDD-Vtn (the classic NMOS threshold-drop problem).

What's the output if we change the GATE voltage of the THIRD transistor

to VDD-Vtn (matching the source voltage arriving from the previous stage)?

ANSWER

[Image: /home/claude/nv_q3_circuit.png]

```
// ---- Baseline behavior: NMOS pass-gate threshold drop --------------------
//
//   Each NMOS in the chain has gate=VDD (fixed).
//   When an NMOS pass transistor's SOURCE rises close to (gate - Vtn),
//   Vgs approaches Vtn and the transistor turns OFF -- it cannot pass
//   the full VDD level through.
//
//   Stage 1: gate=VDD, source starts at 0 -> conducts until source
//            reaches VDD - Vtn, then stops (Vgs = Vtn, right at threshold)
```

```
//   Stage 2: input = VDD-Vtn (from stage 1), gate=VDD again
//            Vgs = VDD - (VDD-Vtn) = Vtn -- ALREADY at threshold!
//            This stage essentially passes NOTHING further (or very
//            little subthreshold leakage) -- output stays ~VDD-Vtn
//   Stage 3: same issue persists -- output remains stuck at VDD-Vtn

// ---- Modification: set THIRD transistor's GATE to VDD-Vtn ----------------
//
//   New Vgs(3rd) = V_gate(3rd) - V_source(3rd)
//                = (VDD - Vtn) - (VDD - Vtn)     [source already at VDD-Vtn
//                                                  from the previous stage]
//                = 0V
//
//   RESULT: Vgs = 0V means the third transistor is COMPLETELY OFF
//   (assuming Vtn > 0, standard enhancement-mode NMOS).
//   The output does NOT improve -- it gets WORSE: the third stage
//   conducts NO current at all (ideally), so the output is determined
//   ENTIRELY by whatever was held at that node previously (likely
//   floating or held by parasitic capacitance, not actively driven).
//
//   This is the OPPOSITE of what a naive "boost the gate" intuition
//   might suggest. Lowering the gate voltage to MATCH the expected
//   source voltage REMOVES the transistor's overdrive entirely.

// ---- What WOULD actually fix the threshold-drop problem? ------------------
//
//   To pass full VDD through an NMOS pass-gate chain, you need the
//   GATE voltage to be HIGHER than VDD (not equal to VDD-Vtn):
//
//   1. Bootstrap/charge-pump circuit: generate a gate voltage of
//      VDD + Vtn (or higher) so that even when source = VDD,
//      Vgs = (VDD+Vtn) - VDD = Vtn -- right at threshold, marginal.
//      Need gate > VDD + Vtn for solid full-swing conduction.
//
//   2. Use a PMOS in parallel (transmission gate) instead of NMOS-only:
//      PMOS passes HIGH levels well (no Vtn drop for high), NMOS
//      passes LOW levels well -- together they pass FULL RAIL-TO-RAIL.
//
//   3. Add a level-shifter / buffer stage after the NMOS chain to
//      restore the signal to full VDD (e.g. an inverter pair, leveraging
//      the inverter's switching threshold to "snap" a degraded VDD-Vtn
//      input back to a clean digital VDD output, since VDD-Vtn is
//      typically still well above the inverter's VM).
```

解释

这道题的陷阱在于直觉误导：很多人第一反应是"既然前级输出卡在VDD-Vtn，

那把下一级的栅极也设成VDD-Vtn，是不是能让两者electrode 匹配从而改善？"

但代入 Vgs = Vgate - Vsource 立刻发现：Vgs=0，管子完全关断，

比原来"勉强维持在阈值边缘"还要糟糕——电流路径被彻底切断。

这道题考察的是对 MOSFET 基本方程 Vgs、Vt 关系的扎实理解，

而不是死记硬背"NMOS传输门有阈值损失"这个结论本身。

真正解决阈值损失问题的方法（自举升压、transmission gate、电平恢复buffer）

都是面试官期待的扩展回答，体现候选人不仅知道问题，还知道如何解决。

Q40. Analog/Mixed-Signal: Ring Oscillator and Single Inverter Feedback

QUESTION

What is a ring oscillator? What happens if you directly connect a single inverter's output back to its own input?

ANSWER

```
// ---- Ring Oscillator ---------------------------------------------------
//
// A ring oscillator is an ODD number of inverter stages connected in
// a closed loop (each stage's output drives the next stage's input,
// and the last stage's output feeds back to the first stage's input).
//
//   IN -> [INV1] -> [INV2] -> [INV3] -> ... -> [INVn] -> back to IN
//
// Why ODD number of stages is REQUIRED:
//   Each inverter flips the logic level. Going around the loop ONCE
//   with N inverters inverts the signal N times.
//   - If N is EVEN: signal returns to its ORIGINAL value after one loop
//     -> the loop is STABLE (two stable DC states, like an SR latch) --
//     NO oscillation.
//   - If N is ODD: signal returns INVERTED after one loop
//     -> there is NO stable DC operating point -- the circuit is
//     forced to keep toggling -> OSCILLATION occurs.
//
// Oscillation frequency:
//   f_osc = 1 / (2 * N * t_pd)
//   where N = number of stages, t_pd = propagation delay per stage
//   (factor of 2 because a full PERIOD requires the signal to
//   propagate around the ring TWICE -- once to flip low->high,
//   once to flip high->low, each taking N*t_pd)
//
//   Ring oscillators are widely used as: on-chip clock generators
//   (coarse, PVT-sensitive), process/temperature monitors (frequency
//   shifts with PVT corners -- useful as a sensor), and as the VCO
//   core in some PLL designs.

// ---- Single inverter with output tied directly back to its own input ------
//
//   IN/OUT --[INV]-- (output connects back to input, N=1 stage)
//
//   This is the DEGENERATE CASE of a ring oscillator with N=1
//   (a single, ODD-numbered stage). By the rule above, N=1 is ODD,
//   so in THEORY this should oscillate too.
//
//   In PRACTICE: this circuit has NO stable DC operating point either
//   -- it sits at the inverter's metastable switching threshold (VM,
//   typically near VDD/2) where Vout = Vin = VM. This is an UNSTABLE
//   equilibrium: any tiny perturbation (noise) pushes it away from VM,
//   and due to the inverter's HIGH GAIN near VM, this perturbation
//   gets AMPLIFIED around the loop, causing the node to oscillate
//   rapidly between near-VDD and near-0.
//
//   HOWEVER: a single-stage "ring oscillator" has a VERY SHORT loop
//   delay (just one gate delay), so:
//   1. The oscillation frequency would be EXTREMELY HIGH
//      (f = 1/(2*1*t_pd), much higher than a multi-stage ring)
//   2. In REAL silicon, parasitic capacitance and the inverter's
//      finite bandwidth/gain may not sustain clean oscillation --
//      the node may simply settle near VM with small-signal noise,
//      or oscillate chaotically/non-ideally depending on the specific
//      transistor characteristics and parasitics.
//   3. This configuration is RARELY used in practice precisely because
```

```
//      it's hard to control and overly sensitive to PVT variation --
//      real ring oscillators use 3+ stages (often 5, 7, 9...) for
//      more predictable, controllable frequencies.
```

解释

环形振荡器的核心要求是"奇数级反相器首尾相连"，

奇数保证沿环路转一圈后信号被反转，不存在稳定的直流工作点，

电路被迫持续翻转产生振荡；偶数级则形成稳定的双稳态（类似SR锁存器），不会振荡。

振荡频率公式 f=1/(2*N*t_pd) 中的因子2来自"完整周期需要绕环两圈"

（一圈完成低到高翻转，第二圈完成高到低翻转）。

单个反相器自环（N=1）理论上也满足"奇数级"条件，

会停在不稳定平衡点 VM 附近，靠噪声放大触发振荡，

但实际频率极高、稳定性差，几乎不会用于实际设计，

工业界统一采用 3 级以上（通常 5/7/9 级奇数）来获得可控、可预测的振荡频率。

这道题考察的是能否把"为什么需要奇数级"的根本原理

推广应用到 N=1 这个边界情况，而不是死记硬背"环振=奇数级反相器"的结论。

Q41. Analog/Mixed-Signal: Pulse Latch -- Pulse Generator Circuit

QUESTION

Explain the concept of a pulse latch. Analyze the pulse-generator circuit shown (clock phi feeds an inverter delay chain AND directly into a NAND gate, producing a narrow pulse).

ANSWER

[Image: /home/claude/nv_q5_pulse_gen.png]

```
// ---- What is a pulse latch? ------------------------------------------------
//
// A pulse latch is a level-sensitive latch that is made transparent
// for only a VERY SHORT WINDOW (a narrow PULSE) once per clock cycle,
// rather than for an entire half-cycle (like a conventional latch).
//
//   Conventional D-latch: transparent whenever CLK=1 (or CLK=0),
//                         for roughly HALF the clock period.
//   Pulse latch:          transparent only during a brief PULSE
//                         (often just a few gate-delays wide),
//                         generated once per rising (or falling) clock edge.
//
// Benefits of pulse latches vs conventional flip-flops:
//   1. SMALLER AREA: a pulse latch uses fewer transistors than a
//      full master-slave flip-flop (no need for two full latch stages).
//   2. LOWER POWER: less clock-related switching activity in some designs.
//   3. TIME BORROWING: because the transparent window is so narrow,
//      a pulse latch behaves almost like an edge-triggered FF for
//      setup/hold purposes, but allows a SMALL amount of "time
//      borrowing" across pipeline stages (data arriving slightly
//      late can still be captured if it arrives before the pulse closes).
//
```

```
// Risk: pulse latches are MORE SENSITIVE to clock pulse width variation
//   (PVT, jitter) since the transparent window is so narrow -- if the
//   pulse generator circuit doesn't produce a clean, sufficiently-wide
//   pulse, data can be missed (pulse too narrow) or hold violations
//   can occur (pulse too wide, behaving like a normal latch with
//   a long transparent window).

// ---- Pulse generator circuit analysis ------------------------------------
//
//   Circuit: phi splits into TWO paths feeding a NAND gate:
//     Path A (direct):       phi ------------------> NAND input 1
//     Path B (delayed/inv):  phi -> INV -> INV -> INV -> NAND input 2
//
//   The delayed path goes through 3 inverters (ODD number -> net INVERSION
//   of phi, PLUS a propagation delay t_delay = 3 * t_pd_inv).
//
//   NAND(A, B) = NAND(phi, !phi_delayed)
//
//   Trace through phi's rising edge (0->1 transition):
//   - At t=0:    phi rises to 1. Path A (direct) is 1 IMMEDIATELY.
//                Path B is still 1 (the OLD inverted value of phi=0,
//                since the 3-inverter chain hasn't propagated yet) --
//                wait: phi was 0 before, so !phi (through 3 invs) = 1
//                BEFORE the edge. So Path B starts at 1.
//   - NAND(1, 1) = 0   <- output goes LOW (pulse starts)
//
//   - After t_delay (3 inverter delays): the new phi=1 propagates
//     through the 3-inverter chain, Path B becomes !1 = 0.
//   - NAND(1, 0) = 1   <- output returns HIGH (pulse ends)
//
//   Result: output produces a NARROW LOW PULSE of width = t_delay
//   (3 inverter delays) every time phi RISES. This pulse is the
//   "clock pulse" that makes the pulse latch transparent for that
//   brief window.
//
//   Pulse width control: the WIDTH of the generated pulse is set
//   DIRECTLY by the number of inverter delay stages (here, 3).
//   More inverters = wider pulse; fewer = narrower pulse.
//   This is a classic "self-timed pulse generator" -- no external
//   delay element needed, just cascade inverters to set the delay.
```

解释

Pulse latch 的设计动机是用更窄的"导通窗口"替代传统latch整个半周期的透明窗口，

在保持类似edge-triggered FF行为的同时省去master-slave两级锁存的面积开销。

脉冲产生电路的分析关键：直接路径 phi 和延迟反相路径 !phi_delayed

送进 NAND 门，利用两条路径的"时间差"产生窄脉冲——

当两路输入短暂地都为1时（直接路径已经翻转，延迟路径还没跟上），

NAND输出短暂拉低，脉冲宽度精确等于延迟链的传播延迟（这里是3级反相器延迟）。

这是数字电路里"自定时脉冲发生器"的标准结构，常见于：

pulse latch的clock pulse生成、单稳态多谐振荡器(one-shot)、

以及某些总线协议里需要产生窄选通脉冲的场景。

面试追问角度：如果延迟链改成偶数级反相器会怎样？

答案是NAND两路输入会同时跟随phi变化（无相位差），NAND退化为普通反相，

不再产生脉冲——这正说明了为什么必须用奇数级反相器制造相位差。

Q42. Analog/Mixed-Signal: Logical Effort -- FO3, Sizing, F=GH

QUESTION

(a) What is logical effort? Explain F=GH (or F=GBH).

(b) What is FO3? How do you size it? What changes if the PMOS in the second stage is made larger?

ANSWER

```
// ================================================================
// (a) Logical Effort: F = G * H (or F = G * B * H)
// ================================================================
//
// Logical Effort is a method to find the optimal gate sizing
// that minimizes delay in a logic path.
//
// Three factors:
//
//   G = Logical Effort (gate type penalty)
//     = ratio of input capacitance of the gate to that of
//       an inverter with the same output drive strength.
//     Inverter:  g = 1
//     NAND2:     g = 4/3
//     NOR2:      g = 5/3
//     (NAND has better logical effort than NOR due to NMOS stacking)
//
//   H = Electrical Effort (fanout / load)
//     = C_out / C_in  (ratio of output capacitance to input capacitance)
//     Larger fanout = larger H = more delay
//
//   B = Branching Effort (for multi-output paths, usually B=1 in simple chains)
//
//   F = Path Effort = G * H  (or G * B * H with branching)
//
// Minimum delay is achieved when each stage has the SAME stage effort f:
//   f = F^(1/N)  where N = number of stages
//   Optimal stage effort f_opt ≈ 4 (empirically ~3.59 in theory)
//
// Each gate should be sized so:
//   C_in(stage_i) = C_in(stage_{i+1}) * g_i / f

// ================================================================
// (b) FO3 (Fan-Out of 3) -- definition and sizing
// ================================================================
//
// FO3 = Fan-Out of 3 (also sometimes called FO4 in some references)
// It represents a standard cell driving 3 identical copies of itself.
// FO3 delay is commonly used as a technology-independent timing unit.
//
//   FO3 configuration:
//     INV (size X) --> drives 3 identical INVs (each size X)
//     H = 3 * C_in / C_in = 3
//     G = 1 (inverter)
//     F = G * H = 1 * 3 = 3
//
// Sizing rule for FO3 chain (N stages):
//   Optimal size ratio between stages = F^(1/N) = 3^(1/N)
//   For a 2-stage FO3: each stage sized with effort = sqrt(3) ≈ 1.73
//   Input gate: size = 1
//   Output gate: size = 1 * H = 3
//
```

```
// Effect of making PMOS in the second stage LARGER:
//   - Drive strength of the second stage increases (PMOS pulls up harder)
//   - Rise time of the second stage OUTPUT decreases (faster rising edge)
//   - BUT: input capacitance of the second stage INCREASES
//     (larger PMOS gate = bigger gate cap)
//   - This increases the LOAD on the first stage -> first stage slows down
//   - Optimal point is when the stage effort is balanced:
//     making PMOS too large shifts delay from output rise to input-side load
//   - Also affects the switching threshold VM:
//     larger PMOS -> stronger pull-up -> VM shifts HIGHER (away from VDD/2)
//   Summary: larger PMOS in stage 2 trades output rise speed for
//   increased load on stage 1 and a higher VM. Must be co-optimized.
```

解释

Logical effort 的核心公式 F=GH 把延迟分解为门固有特性（G）和电路负载（H）两部分，

最优化路径只需让每级的"局部工作量"f=F^(1/N)相等。

FO3 的实际意义是：它是真实芯片时序分析中最常用的延迟单位，

比抽象的"门延迟"更接近真实工艺，工程师用 FO3 delay 来快速评估一个逻辑路径

能否在目标频率下工作。

第二级 PMOS 变大的权衡是经典的面试追问：

不能只看"输出变快"，还要看"输入负载增大导致上一级变慢"，

两者共同决定最优尺寸点，这正是 logical effort 方法的应用场景。

Q43. Analog/Mixed-Signal: Circuit Optimization Techniques

QUESTION

As a designer, list 5-6 techniques that can optimize circuit

performance (speed, power, or area).

ANSWER

```
// ================================================================
// 1. Reduce threshold voltage (Vth)
// ================================================================
//   Lower Vth -> transistor turns on with less gate overdrive
//   -> faster switching (higher Ion current)
//   Trade-off: higher leakage current (Ioff grows exponentially with lower Vth)
//   Technique: multi-Vth design -- use low-Vth on critical path,
//   high-Vth on non-critical paths to control leakage.

// ================================================================
// 2. Reduce supply voltage (VDD)
// ================================================================
//   Dynamic power  P = C * VDD^2 * f  --> quadratic reduction with VDD
//   Speed penalty: lower VDD reduces overdrive (Vgs-Vth), slowing transistors
//   Technique: DVFS (Dynamic Voltage and Frequency Scaling) -- lower VDD
//   when full performance is not needed, raise for peak demand.

// ================================================================
// 3. Shared diffusion (shared source/drain contacts)
// ================================================================
//   In standard cell layout, adjacent transistors share diffusion regions,
//   eliminating the isolation (dummy poly) between them.
//   This reduces: diffusion parasitic capacitance, cell width, and metal routing.
```

```
//   Critical for reducing parasitic drain caps on critical paths.

// ================================================================
// 4. Transistor sizing (upsizing/downsizing)
// ================================================================
//   Upsize transistors on the critical path: lower drive resistance,
//   faster transition. But increases gate capacitance for previous stage.
//   Optimal sizing uses logical effort to balance stage efforts.
//   Downsize non-critical cells to reduce power and area.

// ================================================================
// 5. Multi-finger transistor layout
// ================================================================
//   A wide transistor (e.g. W=8um) can be split into 4 fingers of W=2um each.
//   Benefits: reduces gate resistance (shorter poly per finger),
//   improves frequency response (lower Rg -> lower noise, faster RF performance),
//   more compact layout (better aspect ratio).

// ================================================================
// 6. Gate sizing for logical effort / topology change
// ================================================================
//   Replace complex gates with simpler decomposed forms on critical path:
//   e.g. replace slow NAND4 (g=6/3) with two NAND2s (g=4/3 each) in series
//   when path effort can absorb extra stage. AOI/OAI cells compound functions
//   efficiently for specific patterns. Also: restructure logic to reduce
//   the number of stacked transistors (reduces effective resistance).
```

解释

电路优化的六类技术覆盖了功耗（VDD降低、leakage控制）、

速度（Vth、sizing、shared diffusion）和面积（multi-finger、拓扑重组）三个维度。

面试中的关键回答策略：每个技术必须同时说出 benefit 和 trade-off，

只说"降低Vth变快"而不提"漏电增加"是不完整的。

Shared diffusion 是最容易被候选人遗漏的技术，但它对标准单元设计至关重要，

因为它直接影响寄生电容和布局面积，属于物理设计层面的优化。

Q44. Analog/Mixed-Signal: Two-Stage Inverter VDDh->VDDl vs VDDl->VDDh

QUESTION

A two-stage inverter chain crosses voltage domains:

Case 1: Stage 1 at VDDh, Stage 2 at VDDl (high to low)

Case 2: Stage 1 at VDDl, Stage 2 at VDDh (low to high)

Which case works correctly? What are the issues with Case 2?

Give 3-4 potential problems.

ANSWER

```
// ================================================================
// Case 1: VDDh -> VDDl (high voltage drives low voltage input)
// ================================================================
//
// Stage 1 output swings from 0 to VDDh.
// Stage 2 (at VDDl) receives this signal at its input.
//
```

```
// WORKS if VDDh > VDDl (typical case: 1.8V drives 1.0V domain).
// When Stage 1 output = VDDh:
//   - Input to Stage 2 is VDDh, which is ABOVE VDDl
//   - PMOS of Stage 2: Vgs = VDDh - VDDl (net positive for NMOS convention)
//     Wait: PMOS source=VDDl, gate=VDDh: Vgs = VDDh - VDDl > 0
//     For PMOS (enhancement): Vgs must be < -|Vtp| to be OFF
//     Vgs = +(VDDh-VDDl) > 0 -> PMOS is OFF (correct for logic 1 input)
//   - NMOS: Vgs = VDDh - 0 = VDDh > Vtn -> fully ON
//   -> Stage 2 output = 0 (correct inversion)
//
// When Stage 1 output = 0:
//   - PMOS of Stage 2: Vgs = 0 - VDDl = -VDDl -> ON (correct)
//   -> Stage 2 output = VDDl (correct)
//
// Result: Case 1 functions correctly. The overvoltage at the input
// (VDDh > VDDl) stresses the gate oxide slightly but is generally
// acceptable if VDDh is within the input voltage tolerance of Stage 2.

// ================================================================
// Case 2: VDDl -> VDDh (low voltage drives high voltage input)
// ================================================================
//
// Stage 1 output swings from 0 to VDDl (e.g. 1.0V).
// Stage 2 (at VDDh, e.g. 1.8V) receives this signal.
//
// Problem 1: INCOMPLETE LOGIC HIGH -- PMOS does not turn fully OFF
//   Stage 2 PMOS: source=VDDh=1.8V, gate=VDDl=1.0V
//   Vsg = VDDh - VDDl = 0.8V
//   If |Vtp| = 0.4V: Vsg = 0.8V > |Vtp| -> PMOS is still ON!
//   Both PMOS and NMOS conduct simultaneously -> short-circuit current,
//   undefined output level, excessive power dissipation (shoot-through).
//
// Problem 2: REDUCED NOISE MARGIN
//   Even if PMOS barely turns off (VDDl just above Vtp), the output
//   does not reach VDDh (full rail). Noise margin is degraded.
//   NM_H = VDDh - VOH is reduced because VOH < VDDh.
//
// Problem 3: SLOW TRANSITION / HIGH CROWBAR CURRENT
//   During switching, both PMOS and NMOS are simultaneously partially on.
//   This creates a direct path from VDDh to GND -> peak shoot-through
//   current. In a large driver this is a significant power/reliability concern.
//
// Problem 4: GATE OXIDE RELIABILITY
//   If a level shifter is NOT used, and Stage 2 has thin gate oxide
//   optimized for VDDl, receiving a signal that drives NMOS gate to VDDl
//   while the drain is at VDDh can create Vgd stress:
//   Vgd = VDDl - VDDh (magnitude = VDDh - VDDl)
//   This may exceed the reliable operating voltage for the thin oxide,
//   causing long-term reliability degradation (TDDB, hot carriers).
//
// FIX: Use a LEVEL SHIFTER between the two domains.
//   Level shifters are specifically designed to translate VDDl-swing
//   signals to full VDDh-swing signals safely, avoiding all four issues above.
```

解释

Case 1（高压驱动低压）通常可以工作，因为高电平（VDDh）送进低压域时，

NMOS 被强力打开（Vgs=VDDh > Vtn），PMOS 被强力关断（Vgs=VDDh-VDDl > 0），

逻辑功能正确，只需注意输入过压对栅氧的应力。

Case 2（低压驱动高压）的致命问题在于 Problem 1：

低压域的高电平（VDDl）不足以关断高压域的 PMOS（Vsg=VDDh-VDDl 仍然大于 |Vtp|），

导致两个管子同时导通，形成从电源到地的低阻通路（shoot-through），

既消耗大量静态功耗，又产生不确定的输出电平。

这是 SoC 多电压设计中最常见的设计错误类型，业界标准解决方案是插入专用的电平转换器。

Q45. Analog/Mixed-Signal: NAND2 Sizing from Corresponding Inverter

QUESTION

Given that nand2(2,2,2,2) corresponds to inverter(2,1),

what is the corresponding nand2 sizing for inverter(1,3)?

(Notation: nand2(Wp_A, Wn_A, Wp_B, Wn_B), inverter(Wp, Wn))

ANSWER

```
// ================================================================
// Derivation of the sizing rule
// ================================================================
//
// First, verify the given example to establish the rule:
//   nand2(2,2,2,2) <-> inverter(2,1)
//
//   NAND2 structure: two PMOS in parallel (A-path and B-path),
//                    two NMOS in SERIES (stacked).
//
//   For equivalent drive strength:
//   PMOS: In NAND2, each PMOS is a separate pull-up path (parallel).
//     Each PMOS sees the SAME Vgs as the inverter PMOS.
//     No stacking penalty for PMOS -> same W as inverter PMOS.
//     nand2 Wp = inverter Wp = 2  ✓
//
//   NMOS: In NAND2, the two NMOS are IN SERIES (stacked 2-deep).
//     Series resistance is doubled: 2 x (1/Wn) vs 1/Wn for inverter.
//     To match drive strength: double each NMOS width.
//     nand2 Wn = inverter Wn * 2 = 1 * 2 = 2  ✓
//     This confirms: nand2(2,2,2,2) <-> inverter(2,1)  ✓

// ================================================================
// Apply rule to inverter(1,3)
// ================================================================
//
//   inverter(Wp=1, Wn=3)
//
//   PMOS (no stack penalty):
//     nand2 Wp = inverter Wp = 1
//
//   NMOS (stacked 2-deep, double each):
//     nand2 Wn = inverter Wn * 2 = 3 * 2 = 6
//
//   Answer: nand2(1, 6, 1, 6)
//
// ================================================================
// Verification
// ================================================================
//
//   Drive strength check for NMOS (worst case, both inputs high):
//     Each NMOS: Wn=6, stacked 2 in series
//     Effective width = 6/2 = 3 = inverter Wn  ✓
//
//   Drive strength check for PMOS (one input low):
```

```
//     Each PMOS: Wp=1 (only ONE fires at a time in worst case)
//     = inverter Wp = 1  ✓
//
//   NAND2 sizing rule summary:
//   nand2(Wp, Wn) from inverter(Wp_inv, Wn_inv):
//     Wp = Wp_inv          (parallel PMOS, no stack factor)
//     Wn = Wn_inv * 2      (series NMOS, stack factor = 2)
//   For N-input NAND: Wn = Wn_inv * N
```

解释

NAND2 sizing 规则的物理来源：NAND2 的 PMOS 是并联的（两条独立拉高路径），

每个 PMOS 的驱动能力和等效 inverter 完全相同，所以宽度不变；

NMOS 是串联的（两个管子形成电阻串联），等效电阻翻倍，

要维持相同的驱动能力就必须把每个 NMOS 的宽度也翻倍。

这个规律延伸到 NAND3：NMOS 宽度乘以3；NAND4：乘以4。

NOR 的情况相反：NOR 的 PMOS 是串联的（要乘以2），NMOS 是并联的（不变），

这也是为什么 NOR 的面积/功耗比 NAND 大——PMOS 本身已经比 NMOS 宽1.5-2倍，

再乘以串联系数使 NOR 的 PMOS 非常大。

Q46. Analog/Mixed-Signal: Master-Slave DFF -- Component Purpose and Removal Analysis

QUESTION

Given the master-slave DFF circuit with components I1, I2, T1, T2

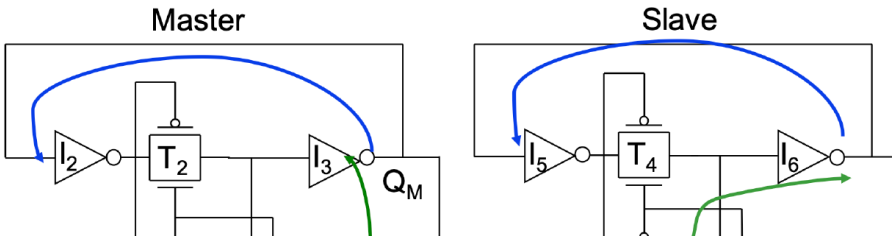
(master latch) and I4, I5, T3, T4, I6 (slave latch):

(a) What is the purpose of each component?

(b) What happens if we remove I1? What if we remove I6?

ANSWER

a. register circuit? what's the purpose of "each" component? ex. I1, I2, T1, T2, I6
what if we remove I1, I6 what's the outcome?(2~3 answers)




```
// ================================================================
// (a) Component roles
// ================================================================
//
// MASTER LATCH (left side, transparent when CLK=0):
//
//   I1  = Input inverter for D
//         Buffers/inverts the D input. Provides:
//         1. Signal restoration (drive strength for D)
//         2. Isolation between the D input and the T1 transmission gate
```

```
//         Drives the transmission gate T1 with a clean ~D signal.
//
//   T1  = Transmission gate, controlled by CLK/~CLK
//         PASSES ~D into the master latch loop when CLK=0 (transparent phase).
//         BLOCKS input when CLK=1 (hold phase).
//
//   I2  = Inverter within master latch feedback loop
//         Inverts the latched value to drive T2.
//         Part of the cross-coupled pair that HOLDS state.
//
//   T2  = Feedback transmission gate, controlled by ~CLK/CLK
//         When CLK=1 (hold phase): closes the feedback loop (T2 ON),
//         creating a static storage element (inverter loop).
//         When CLK=0 (transparent): T2 is OFF, loop is open, D can overwrite.
//
//   QM  = Master latch output (inverted value of D when transparent)
//
// SLAVE LATCH (right side, transparent when CLK=1):
//
//   I4  = Inverter, driven by QM
//         Buffers/inverts QM to drive T3. (Mirror of I1 in master)
//
//   T3  = Slave input transmission gate, controlled by ~CLK/CLK
//         PASSES data from master into slave when CLK=1.
//         BLOCKS when CLK=0.
//
//   T4  = Slave feedback transmission gate, controlled by CLK/~CLK
//         Closes slave feedback loop when CLK=0 (slave holds state).
//
//   I5  = Inverter within slave feedback loop (mirror of I2 in master)
//
//   I6  = Output buffer/inverter
//         1. Re-inverts so Q has same polarity as D.
//         2. Provides strong drive strength to external load.
//         3. Isolates the internal slave loop from external capacitive load.
//
// ================================================================
// (b) What happens when removing I1 or I6?
// ================================================================
//
// REMOVE I1 (input inverter before T1):
//   Effect 1: Drive strength -- D input must now directly drive T1
//   and the master latch feedback loop. If D is from a long wire or
//   weak driver, the latch may fail to capture the correct value
//   (setup time violation, data corruption).
//
//   Effect 2: Polarity -- I1 was inverting D before T1.
//   Without I1, the master latch now stores D directly (not ~D).
//   The final Q output polarity CHANGES (Q becomes inverted relative
//   to D) unless the circuit is redesigned to compensate.
//
//   Effect 3: Contention risk -- during the transparent phase (CLK=0),
//   D drives directly into the latch while the feedback (I2) is
//   partially on. Without I1 buffering, this creates a potential
//   write conflict (weak D fighting strong feedback).
//
// REMOVE I6 (output buffer):
//   Effect 1: No output drive strength -- the internal slave latch node
//   (weak) directly drives Q. External gates see only the weak
//   current from the transmission gate T3 path. Long wires or many
//   fanout loads will not be driven correctly.
//
//   Effect 2: Capacitive load pulls into the latch -- external load
```

```
//   capacitance now directly loads the slave latch storage node,
//   potentially corrupting the stored value or slowing tc2q significantly.
//
//   Effect 3: Polarity inversion -- I6 provides the final re-inversion
//   so that Q = D after two latch inversions. Without I6, Q = ~D
//   (inverted output). The flip-flop becomes an inverting flip-flop.
```

解释

Master-slave DFF 的工作原理是"两级锁存器，时钟互补"：

master 在 CLK=0 时透明（采样 D），slave 在 CLK=1 时透明（传播到 Q）。

传输门（T1/T2/T3/T4）是关键元件：CLK 和 ~CLK 同时控制 NMOS 和 PMOS，

使传输门在全 rail-to-rail 范围内导通（不像单 NMOS 传输门有 Vth 损失）。

去掉 I1 的核心影响是极性翻转+驱动力丢失；去掉 I6 的核心影响是驱动力丢失+极性翻转。

两者都会导致 Q = ~D（极性取反），这是面试中必须明确指出的关键点。

I1 和 I6 实际上在功能上是"一对"——它们合起来提供两次反相（净同相）和两次缓冲，

任意去掉一个都会导致净反相变成奇数次（Q = ~D）。

Q47. Analog/Mixed-Signal: tsu, tc2q Measurement in SPICE; tcq vs tdc Plot

QUESTION

(a) How do you measure setup time (tsu) and clock-to-Q delay (tc2q)

in SPICE?

(b) Describe the tcq vs. tdc (data-to-clock) plot.

ANSWER

```
// ================================================================
// (a) Measuring tsu and tc2q in SPICE
// ================================================================
//
// Setup time (tsu):
//   tsu is the minimum time D must be stable BEFORE the clock edge
//   for the flip-flop to reliably capture D.
//
//   SPICE measurement method:
//   1. Apply D transition, then CLK rising edge.
//   2. Parametrically sweep the time between D arrival and CLK edge
//      (delta_t = t_clk - t_D_stable).
//   3. Measure tc2q for each delta_t.
//   4. As delta_t decreases, tc2q starts to INCREASE sharply.
//      The point where tc2q starts rising above a reference
//      (e.g. nominal tc2q + 10%) defines the setup time constraint.
//   5. tsu = minimum delta_t at which tc2q is still "acceptable".
//
//   SPICE netlist approach:
//   .measure tran tD_stable when v(D)=VDD/2 rise=1
//   .measure tran tCLK       when v(CLK)=VDD/2 rise=1
//   .measure tran tc2q       when v(Q)=VDD/2   rise=1
//   Setup margin = tCLK - tD_stable
//   Sweep this margin from large (>0) to negative; record tc2q each time.
//
// Clock-to-Q delay (tc2q):
//   tc2q = time from CLK crossing 50% to Q crossing 50%.
```

```
//
//   SPICE measurement:
//   .measure tran tc2q TRIG v(CLK)=VDD/2 rise=1
//                      TARG v(Q)=VDD/2   rise=1
//   This gives one tc2q number for a given setup margin.

// ================================================================
// (b) tcq vs. tdc (data-to-clock) plot
// ================================================================
//
//   tdc = time from D transition to CLK edge
//         (positive tdc = D arrives BEFORE CLK, i.e. setup margin)
//         (negative tdc = D arrives AFTER CLK, i.e. setup violation)
//
//   tcq = clock-to-Q propagation delay (measured from CLK edge to Q)
//
//   Shape of tcq vs. tdc curve:
//
//   tcq
//    |
//    |  \           <- as tdc increases (more setup margin)
//    |   \          tcq initially decreases (FF captures cleanly, fast)
//    |    \_______  <- asymptote at "nominal" tc2q (large setup margin)
//    |
//    |              <- as tdc decreases (less setup margin):
//    |  ________    tcq increases (FF starts to metastabilize, slow)
//    | /
//    |/             <- tdc = -thold: FF captures wrong value (hold violation)
//     +-----------> tdc  (positive = D before CLK)
//
//   Key points on the curve:
//   1. tdc >> 0: tcq = nominal tc2q (constant, minimum delay)
//   2. tdc decreasing toward 0: tcq increases ("setup degradation region")
//   3. tdc = -tsu: tcq diverges to infinity (metastability boundary)
//   4. tdc < -tsu: FF may capture wrong data (undefined behavior)
//
//   The setup time (tsu) is defined at the point where tcq exceeds
//   a specified limit (e.g. nominal + 50ps, or simply "diverges").
```

解释

tsu 测量的关键是"扫描 D 到 CLK 的时间间隔，观察 tc2q 何时开始变慢"。

不能简单地定义为"tc2q 开始变化的点"，因为 tc2q 即使在充裕余量下也会随 D 时序略微变化；

业界标准是选一个 tc2q 的增量阈值（如增加 10%~50ps）作为 tsu 的定义点。

tcq vs tdc 曲线的形状体现了 FF 的"捕获特性"：

大余量时 tc2q 接近常数；余量减小时 tc2q 逐渐增大（FF 内部节点在亚稳态边缘工作更慢）；

在 tsu 点附近 tc2q 急剧上升趋向无穷（亚稳态：FF 无法决定输出0还是1）。

这条曲线是 STA 工具确定 FF 时序约束的基础，也是理解"为什么需要建立时间余量"的物理直觉。

Q48. Analog/Mixed-Signal: 6T SRAM Operation, SNM, and Sizing

QUESTION

(a) Explain 6T SRAM cell operation (read, write, hold).

(b) What is Static Noise Margin (SNM)?

(c) How do you size the cell transistors for read stability?

ANSWER

```
// ================================================================
// (a) 6T SRAM cell operation
// ================================================================
//
// Cell structure: 2 cross-coupled inverters (Q=M1/M2, QB=M3/M4) +
//                 2 access transistors (M5=BL side, M6=BLB side),
//                 both controlled by Word Line (WL).
//
//   HOLD (WL=0): access transistors OFF.
//     Cross-coupled inverters maintain state indefinitely (static power only).
//     Q=1, QB=0 (or Q=0, QB=1) -- stable state held by feedback.
//
//   READ (WL=1): access transistors ON, BL and BLB pre-charged to VDD.
//     Assume Q=1, QB=0.
//     M5 ON: BL side sees Q=1 -- M1 (PMOS) on, M2 (NMOS) off -> BL stays high.
//     Wait: M5 connects BL to storage node Q.
//     Since Q=1, BL (pre-charged to VDD) remains at VDD -- no discharge.
//     M6 ON: BLB side sees QB=0.
//     QB=0 -> M3 (PMOS) ON, M4 (NMOS) OFF --
//     Actually: M6 connects BLB to QB=0.
//     M4 (NMOS pull-down) in the QB inverter tries to discharge BLB through M6.
//     BLB starts to discharge toward 0 via M4-M6 stack.
//     RISK: as BLB drops, so does QB. If QB rises above Vtn, M3 can flip the cell!
//     Sense amplifier detects small differential: BL > BLB -> amplify to logic 1.
//
//   WRITE (WL=1, drive BL=0, BLB=1 to write Q=0):
//     BL driven to 0 by write driver -- overpowers the weak PMOS hold.
//     M5 ON: BL=0 pulls Q node toward 0, flipping the cell.
//     Once Q drops below Vtn of M3, QB flips high, state is written.
//     Write completes; WL goes back to 0, cell holds new state.

// ================================================================
// (b) Static Noise Margin (SNM)
// ================================================================
//
// SNM measures the ROBUSTNESS of the cell against noise during hold or read.
//
// Graphical definition:
//   Plot the VTC of both inverters in the cross-coupled pair on the same
//   graph (one forward, one mirrored/rotated).
//   The two curves form an "eye" or "butterfly" shape.
//   SNM = side length of the LARGEST SQUARE that fits inside one of the
//   two "eyes" of the butterfly curve.
//
//   Large SNM = robust cell (harder to flip with noise).
//   Small SNM = fragile cell (small noise can flip the state).
//
// SNM degrades during READ because:
//   The access transistor (M5/M6) creates a resistive divider with the
//   cell NMOS (M2/M4). This raises the "0" node voltage slightly
//   (read disturb), reducing the available noise margin.
//   Read SNM < Hold SNM.

// ================================================================
// (c) Sizing for read stability: Cell Ratio (CR)
// ================================================================
//
// Cell Ratio (CR) = W/L of pull-down NMOS (M2) / W/L of access NMOS (M5)
//
//   CR > 1 required for stable read.
//   Higher CR = stronger pull-down relative to access -> QB voltage stays low
//   during read -> larger read SNM.
```

```
//
//   Typical target: CR >= 2 for robust operation.
//
//   Rule:
//     M2 (cell NMOS, pull-down): W/L = 2 (or larger)
//     M5 (access NMOS):          W/L = 1
//     CR = 2 / 1 = 2
//
// Pull-up Ratio (PR) = W/L of access NMOS / W/L of load PMOS (M1)
//   PR < 1 for writeability (access transistor must overpower PMOS during write).
//   Typical: PR = 0.5 to 1.5.
//
//   Competing constraints:
//   - Read stability: wants large CR (strong M2) and small M5
//   - Write ability: wants M5 to be strong enough to flip the cell
//   - Area: wants all transistors as small as possible (dense array)
//   -> Standard 6T sizing: M2=2x, M4=2x, M5=M6=1x, M1=M3=1x
```

解释

6T SRAM 的读操作是最脆弱的：BLB 通过 M4-M6 串联放电，

但 QB 节点电压会随 BLB 的下降而升高，如果升高超过 M3 的 Vtp，

就会触发状态翻转（read disturb）——这正是需要 Cell Ratio > 1 的根本原因。

SNM 的"蝴蝶曲线"是 SRAM 设计文档里最常见的图形，

最大正方形的边长代表能容忍的最大静态噪声，是选择工艺角（SS/TT/FF）时的关键指标。

CR 和 PR 是 SRAM 设计的两个核心权衡：CR 保读稳定，PR 保写能力，

两者都要满足约束的同时尽量缩小单元面积——这是 SRAM 版图工程师的核心技能。

Q49. Analog/Mixed-Signal: ROM Types and Sense Amplifier; Timing Order

QUESTION

(a) Explain NAND-type and NOR-type ROM structure.

(b) What is a sense amplifier (SA)?

(c) What is the timing order of ROM operation:

decode_en, precharge, SA_EN?

ANSWER

```
// ================================================================
// (a) NAND-type vs NOR-type ROM
// ================================================================
//
// NOR-type ROM:
//   - Bit cells are in PARALLEL between BL and GND.
//   - WL controls one row of cells.
//   - Programmed cell: NMOS transistor connected (1 = transistor present)
//   - Read: if ANY cell in the selected column is ON -> BL discharges -> read 0
//   - Advantage: FAST read (any one transistor can pull down BL)
//   - Disadvantage: large cell area (each cell needs separate BL contact)
//   - Used for: fast-access lookup tables, microcode ROM
//
// NAND-type ROM:
//   - Bit cells are in SERIES (chain) between BL and GND.
//   - Multiple WLs control a chain of transistors.
```

```
//   - Programmed cell: WL always turns transistor ON (all in chain = series path)
//   - Erased cell: transistor removed (no path through that position)
//   - Read: all cells in the selected NAND string must be ON for BL to discharge
//   - Advantage: dense (cells can share source/drain contacts, very compact)
//   - Disadvantage: slower read (series resistance of the entire chain)
//   - Used for: high-density storage (Flash memory is NAND-based)

// ================================================================
// (b) Sense Amplifier (SA)
// ================================================================
//
// A sense amplifier is a circuit that AMPLIFIES the small differential
// voltage on the bit line pair (BL, BLB) into a full logic swing.
//
// Why needed: BL/BLB are large capacitance lines (many cells attached).
// Full discharge takes too long -> SA detects a SMALL differential
// (e.g. 50-100mV) and regeneratively amplifies it to VDD/0 in one step.
//
// Latch-type SA (most common):
//   When SA_EN=1: cross-coupled inverter pair (latch) is enabled.
//   The small differential on BL/BLB is amplified by positive feedback
//   until outputs reach full rail.
//   Speed: nanoseconds for amplification (vs. microseconds for full discharge).

// ================================================================
// (c) ROM operation timing order
// ================================================================
//
// Correct sequence:
//
//   Step 1: PRECHARGE (precharge=1)
//     BL and BLB are pre-charged to VDD (or VDD/2 for half-VDD sensing).
//     This ensures both bit lines start at known state before read.
//     Access transistors to SA are OFF during precharge.
//
//   Step 2: decode_en (address decode enable)
//     Address decoder selects the correct WL.
//     WL goes HIGH, activating the selected row of bit cells.
//     BL (or BLB) begins to discharge through the selected cell(s).
//     Must happen AFTER precharge so BL starts at VDD.
//
//   Step 3: SA_EN (sense amplifier enable)
//     After sufficient time for BL/BLB differential to develop
//     (typically 50-200mV), SA_EN fires the sense amplifier.
//     SA latches and amplifies the differential to full swing.
//     Must happen AFTER decode_en + cell discharge time.
//
//   Timeline:
//     |--precharge--|--decode_en--|--cell_discharge--|--SA_EN--|
//        (setup)      (WL high)     (BL develops)   (amplify)
//
//   If SA_EN fires too early: differential too small -> incorrect read.
//   If SA_EN fires too late: unnecessary latency wasted.
```

解释

NAND ROM 和 NOR ROM 的命名来自其位线结构：

NOR ROM 中每个存储单元和位线之间是"或"关系（任一单元导通就拉低 BL），

像一个大的 NOR 门；NAND ROM 中单元串联成链，像一个大的 NAND 门。

Sense amplifier 是存储器中最重要的外围电路之一：

它利用正反馈（交叉耦合反相器的闩锁特性）把几十毫伏的微小差分信号

放大到满摆幅，这个放大是在几个 ns 内完成的，而不是等 BL 慢慢放电到逻辑0。

ROM 操作时序顺序（precharge -> decode_en -> SA_EN）是不可逆的：

precharge 必须在 decode 之前（否则 BL 不在 VDD，结果不确定）；

SA_EN 必须在 decode 之后等待足够的放电时间（否则差分太小，SA 可能翻错方向）。

Q50. Analog/Mixed-Signal: Electromigration -- DC, AC, Iavg, Irms, Black's Equation

QUESTION

What is electromigration (EM)? Explain DC and AC EM, the significance of Iavg and Irms, and Black's equation.

ANSWER

```
// ================================================================
// What is electromigration?
// ================================================================
//
// Electromigration is the gradual displacement of metal atoms in a
// conductor caused by momentum transfer from conduction electrons
// ("electron wind").
//
// At high current density, electrons collide with metal ions and
// push them in the direction of electron flow (against conventional
// current direction). Over time this causes:
//   - VOIDS: metal depleted in some regions -> increased resistance
//            -> eventually OPEN CIRCUIT (wire breaks)
//   - HILLOCKS: metal piled up in other regions -> short circuit risk
//               with adjacent wires.
//
// EM is a reliability failure mechanism -- it degrades over years,
// not immediately. It is characterized statistically (MTTF = Mean Time
// to Failure).

// ================================================================
// DC vs. AC electromigration
// ================================================================
//
// DC EM (unidirectional current):
//   Electrons always flow in one direction. Metal atoms migrate
//   continuously in one direction. Damage accumulates monotonically.
//   Limit: Iavg (average current) -- "DC current rule"
//   EM limit for DC: J <= Jmax_DC (current density limit for DC)
//
// AC EM (bidirectional current, alternating):
//   Electrons alternately push atoms left and right.
//   Some self-healing occurs during the reverse phase.
//   Net damage is LESS than DC at the same peak current.
//   Limit: Irms (RMS current) -- captures heating effect
//   AND Iavg (average, for net drift direction)
//
//   Key insight: AC EM is less severe because reverse current partially
//   pushes atoms back (voiding partially heals during opposite phase).
//   However, joule heating (proportional to I^2*R = Irms^2 * R) still
//   accelerates EM even for zero-average AC signals.

// ================================================================
// Iavg and Irms significance
// ================================================================
```

```
//
//   Iavg (average current): determines the NET atomic drift direction.
//     - For DC: Iavg = Ipeak (worst case)
//     - For zero-mean AC: Iavg = 0 (no net drift, but heating remains)
//     - Used for: DC/unidirectional EM check
//
//   Irms (RMS current): determines JOULE HEATING (temperature rise).
//     - Temperature is the dominant acceleration factor for EM.
//     - Higher temperature -> exponentially faster EM (Arrhenius).
//     - Even zero-mean AC causes heating -> Irms check needed.
//     - For sinusoidal: Irms = Ipeak / sqrt(2)
//
//   EM sign-off rules (typically):
//     DC nets:  check Iavg <= limit
//     AC nets:  check Irms <= limit (and Iavg for any DC bias component)

// ================================================================
// Black's equation (MTTF model)
// ================================================================
//
//   MTTF = A * J^(-n) * exp(Ea / k * T)
//
//   Where:
//     A   = technology-dependent constant
//     J   = current density (A/m^2)
//     n   = current density exponent (typically n=1 for void-limited,
//           n=2 for nucleation-limited; commonly n=2 for interconnect)
//     Ea  = activation energy (eV): for aluminum ~0.6-0.7eV,
//           for copper ~0.8-0.9eV (Cu has better EM resistance)
//     k   = Boltzmann's constant (8.617e-5 eV/K)
//     T   = absolute temperature (Kelvin)
//
//   Key insights from Black's equation:
//   1. MTTF decreases EXPONENTIALLY with temperature -> thermal design critical
//   2. MTTF decreases as J^n -> doubling current density reduces MTTF by 4x (n=2)
//   3. Copper (higher Ea) has MUCH better EM lifetime than aluminum -> why
//      advanced nodes switched from Al to Cu for interconnects.
//   4. Every 10°C increase in temperature roughly HALVES the MTTF (Arrhenius).
```

解释

电迁移的物理本质是电子与金属离子的动量交换：电子流把金属原子推走，

在一侧形成空洞（开路），在另一侧堆积（潜在短路）。

DC 和 AC EM 的区别在于"净位移"：DC 是单向推移，损伤单调累积；

AC 的逆向电流提供了部分"自修复"，所以 AC EM 更宽松，

但焦耳加热（由 Irms 决定）仍然加速 EM，不能完全忽略。

Black 方程的工程含义：温度每升高 10°C MTTF 减半，说明散热设计对可靠性至关重要；

电流密度翻倍使 MTTF 缩短 4 倍（n=2），说明 EM 设计裕量必须包含 worst-case 电流估计。

铜互连替代铝的核心原因之一就是 EM 寿命（更高的 Ea），

这是先进工艺节点互连材料选择的关键技术依据。

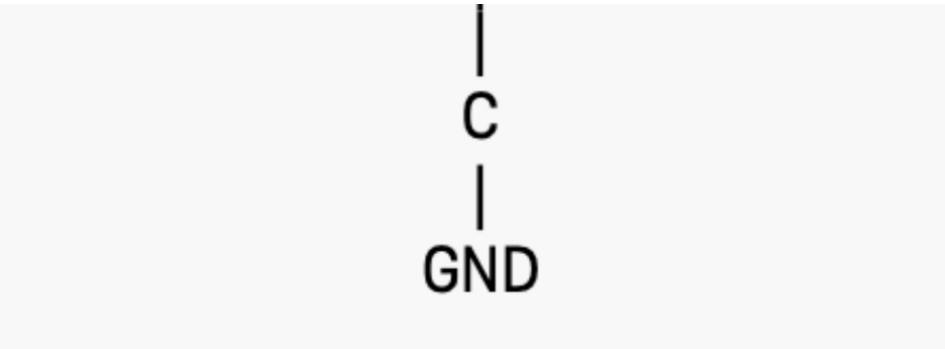
Q51. Analog/Mixed-Signal: RC Lowpass Filter -- Transfer Function and Cutoff Frequency

QUESTION

Analyze the RC circuit shown. What type of filter is it?

Derive the transfer function and find the cutoff frequency.

ANSWER




```
// Circuit: Vin -- R -- Vout
//                      |
//                      C
//                      |
//                     GND
//
// Vout is taken across C (capacitor to GND).
// This is a FIRST-ORDER RC LOWPASS FILTER.

// ================================================================
// Transfer function derivation
// ================================================================
//
// Using voltage divider in the frequency domain (s-domain / phasor):
//
//   Z_R = R              (resistor impedance)
//   Z_C = 1 / (sC)      (capacitor impedance in s-domain)
//
// Vout / Vin = Z_C / (Z_R + Z_C)          (voltage divider)
//           = (1/sC) / (R + 1/sC)
//           = 1 / (1 + sRC)
//
// Transfer function: H(s) = 1 / (1 + sRC)
//
// This is a standard 1st-order lowpass with one pole at s = -1/RC.

// ================================================================
// Frequency response (s = jw)
// ================================================================
//
// H(jw) = 1 / (1 + jwRC)
//
// Magnitude: |H(jw)| = 1 / sqrt(1 + (wRC)^2)
//
//   At w=0 (DC):    |H| = 1    (0 dB) -- DC passes through fully
//   At w -> inf:    |H| -> 0   (-inf dB) -- high freq blocked
//   At w = 1/RC:    |H| = 1/sqrt(2) = -3dB -- cutoff point

// ================================================================
// Cutoff frequency (-3dB frequency)
// ================================================================
//
// Definition: frequency where |H| = 1/sqrt(2) (power drops by half)
```

```
//
//   1 / sqrt(1 + (w_c * RC)^2) = 1/sqrt(2)
//   1 + (w_c * RC)^2 = 2
//   (w_c * RC)^2 = 1
//   w_c = 1/RC               (angular frequency, rad/s)
//
//   f_c = w_c / (2*pi)
//   f_c = 1 / (2 * pi * RC)  (Hz)
//
// IMPORTANT: The cutoff frequency is 1/(2*pi*RC), NOT 2*pi*RC.
//   The 2*pi is in the DENOMINATOR.
//   Common mistake: confusing w_c = 1/RC (angular) with f_c (Hz).
//   f_c = 1/(2*pi*RC) means: bigger R or C -> lower cutoff -> narrower passband.

// ================================================================
// Intuition: why it's a LOWPASS
// ================================================================
//
// At LOW frequency: capacitor impedance Z_C = 1/(jwC) is LARGE.
//   Most of Vin appears across the large Z_C -> Vout ≈ Vin. (passes)
//
// At HIGH frequency: Z_C becomes SMALL.
//   Most of Vin drops across R (now relatively large) -> Vout ≈ 0. (blocked)
//
// The "corner" between these two behaviors is at Z_R = Z_C:
//   R = 1/(w_c * C) -> w_c = 1/(RC)  -- same result as above.
```

解释

RC 低通滤波器的传递函数 H(s)=1/(1+sRC) 是模拟信号处理的基础，

一个极点位于 s=-1/RC（左半平面，稳定）。

截止频率 f_c = 1/(2πRC) 是面试中最常被问错的公式：

ω_c = 1/RC（弧度/秒），转换为赫兹还需要除以 2π，而不是乘以 2π。

记忆技巧：RC 越大 -> 截止频率越低（滤波器带宽越窄），

这符合物理直觉——大电容充放电慢，因此高频信号来不及被电容响应，被"拦住"了。

实际 DV 岗面试出现这道题的意图是测试候选人是否有基本的模拟/信号处理背景，

能回答出 H(s)=1/(1+sRC) 和 f_c=1/(2πRC) 的推导过程就足够了。

SystemVerilog -- Coding

Q52. SystemVerilog: 2-to-1 MUX and 4-to-1 MUX Implementation

QUESTION

(a) Implement a 2-to-1 multiplexer module.

(b) Follow-up: implement a 4-to-1 multiplexer. Show at least two

different implementation styles.

ANSWER

```
// (a) 2-to-1 MUX ------------------------------------------------------
```

```
module mux2_1 #(
    parameter int WIDTH = 8
)(
    input  logic             sel,
    input  logic [WIDTH-1:0] d0,
    input  logic [WIDTH-1:0] d1,
    output logic [WIDTH-1:0] y
);
    assign y = sel ? d1 : d0;
endmodule

// (b) 4-to-1 MUX -- Style 1: ternary chain ------------------------------

module mux4_1_ternary #(
    parameter int WIDTH = 8
)(
    input  logic [1:0]       sel,
    input  logic [WIDTH-1:0] d0, d1, d2, d3,
    output logic [WIDTH-1:0] y
);
    assign y = (sel == 2'b00) ? d0 :
               (sel == 2'b01) ? d1 :
               (sel == 2'b10) ? d2 : d3;
endmodule

// 4-to-1 MUX -- Style 2: case statement (always_comb) -------------------

module mux4_1_case #(
    parameter int WIDTH = 8
)(
    input  logic [1:0]       sel,
    input  logic [WIDTH-1:0] d0, d1, d2, d3,
    output logic [WIDTH-1:0] y
);
    always_comb begin
        case (sel)
            2'b00: y = d0;
            2'b01: y = d1;
            2'b10: y = d2;
            2'b11: y = d3;
            default: y = '0;   // avoid inferred latch
        endcase
    end
endmodule

// 4-to-1 MUX -- Style 3: built from two 2-to-1 MUXes (hierarchical) ------

module mux4_1_hierarchical #(
    parameter int WIDTH = 8
)(
    input  logic [1:0]       sel,
    input  logic [WIDTH-1:0] d0, d1, d2, d3,
    output logic [WIDTH-1:0] y
);
    logic [WIDTH-1:0] lo_mux, hi_mux;

    mux2_1 #(.WIDTH(WIDTH)) u_lo (.sel(sel[0]), .d0(d0), .d1(d1), .y(lo_mux));
    mux2_1 #(.WIDTH(WIDTH)) u_hi (.sel(sel[0]), .d0(d2), .d1(d3), .y(hi_mux));
    mux2_1 #(.WIDTH(WIDTH)) u_out(.sel(sel[1]), .d0(lo_mux), .d1(hi_mux), .y(y));
endmodule

// 4-to-1 MUX -- Style 4: array indexing (most scalable for N-to-1) -------
```

```
module muxN_1 #(
    parameter int WIDTH = 8,
    parameter int N     = 4
)(
    input  logic [$clog2(N)-1:0] sel,
    input  logic [WIDTH-1:0]     d [N],
    output logic [WIDTH-1:0]     y
);
    assign y = d[sel];
endmodule
```

解释

2-1 MUX 是最基础的组合逻辑单元，ternary 写法最简洁。

4-1 MUX 的四种实现风格各有侧重：

Style 1（三元链）和 Style 2（case）功能等价，case 配合 always_comb

更适合扩展到多路输入，default 分支避免综合出意外锁存器。

Style 3（层级化，用2个2-1 MUX拼出4-1）体现"分治"思想，

是面试官最常追问的延伸——N-1 MUX 可以用 log2(N) 级 2-1 MUX 树构建。

Style 4（数组索引）是最 scalable 的写法，change N 只需改参数，

本质上综合工具会把它展开成同样的 MUX 树结构。

Q53. SystemVerilog: Mixed Blocking/Non-Blocking Cycle-by-Cycle Value Analysis

QUESTION

Given the following code with mixed blocking and non-blocking

assignments inside a single always_ff block, trace the values of

A, B, and C for the first 4 clock cycles given input = 1, 2, 3, 4.

```
always_ff @(posedge clk) begin
    A = in;     // blocking
    B <= A;     // non-blocking
    C = B;      // blocking
end
```

ANSWER

```
// Key SystemVerilog scheduling rule:
//   Blocking (=) assignments execute IMMEDIATELY within the active region,
//   updating the variable right away -- subsequent statements in the
//   SAME block see the NEW value.
//
//   Non-blocking (<=) assignments: the RHS is evaluated IMMEDIATELY
//   (using whatever values are current at that point in execution),
//   but the LHS update is DEFERRED to the NBA (Non-blocking Assign)
//   region at the END of the time step.
//
// Trace through the always_ff block line by line:
//
//   Line 1: A = in;     -- BLOCKING. A is updated to 'in' IMMEDIATELY.
//   Line 2: B <= A;     -- NON-BLOCKING. RHS (=A) is read NOW (using
//                          the JUST-UPDATED A from line 1). The actual
```

```
//                          assignment to B is QUEUED for the NBA region
//                          -- B does NOT change yet at this point.
//   Line 3: C = B;      -- BLOCKING. B has NOT been updated yet
//                          (the NBA from line 2 hasn't applied --
//                          NBA updates happen AFTER all blocking
//                          statements in this time step complete).
//                          So C gets the OLD value of B (from the
//                          PREVIOUS clock cycle).
//
//   At the END of the time step: the queued NBA update applies:
//   B finally becomes the new value (which was = A from this cycle).

// Cycle-by-cycle trace (initial A=B=C=0):
//
//   Cyc | in | A  | B (NBA, deferred) | C (old B)
//   ----+----+----+------------------+-----------
//    0  |  1 |  1 | 1  (was 0)       |  0
//    1  |  2 |  2 | 2  (was 1)       |  1
//    2  |  3 |  3 | 3  (was 2)       |  2
//    3  |  4 |  4 | 4  (was 3)       |  3
//
// Final values after 4 cycles: A=4, B=4, C=3
//
// Key takeaway: C is effectively "B delayed by one MORE cycle" relative
// to what you might naively expect, because the blocking read of B in
// line 3 happens BEFORE the non-blocking write from line 2 takes effect.
// A and B end up SYNCHRONIZED to the same value at the end of each cycle
// (since B <= A reads the freshly-blocking-updated A), but C lags B
// by one additional cycle due to reading the PRE-update value of B.

// Why this is a DANGEROUS coding style (the real interview point):
//   Mixing blocking and non-blocking assignments to DIFFERENT variables
//   that DEPEND ON EACH OTHER within the same always_ff block creates
//   exactly this kind of confusing, order-dependent, easy-to-misread behavior.
//   It is NOT illegal SystemVerilog, but it is widely considered BAD PRACTICE.
//
//   Industry best practice: use NON-BLOCKING (<=) EXCLUSIVELY for all
//   sequential logic in always_ff blocks. This makes every register
//   update "see" the PREVIOUS cycle's values consistently, matching
//   real hardware behavior (all flops update simultaneously on the
//   clock edge) and eliminating order-dependent surprises like the
//   one in this example.
//
//   Corrected version (recommended):
//   always_ff @(posedge clk) begin
//       A <= in;
//       B <= A;     // uses A from BEFORE this cycle (correct pipeline)
//       C <= B;     // uses B from BEFORE this cycle (correct pipeline)
//   end
//   This produces a clean 3-stage shift register: A, B, C each delayed
//   by exactly one more cycle than the previous, with NO order-dependent
//   confusion.
```

解释

这道题的核心是逐行执行 SV 的赋值调度规则：blocking 立即生效，

non-blocking 的右值立即求值但左值更新推迟到本时间步结束（NBA区）。

陷阱在于 Line 3 (C=B) 读取的是"尚未被Line 2更新"的B（旧值），

因为 NBA 的实际写入要等到整个 always 块执行完才统一生效。

这导致 A 和 B 表现得"同步"（B<=A读取的是刚被blocking更新的新A），

但 C 却比 B "多延迟一拍"，形成一个不直观、容易出错的依赖链。

面试的真正考点不是"算出正确答案"，而是能否指出这种混用是危险的编码习惯：

正确的工业界做法是在 always_ff 里统一只用 non-blocking，

这样每个寄存器更新都基于"上一拍"的值，行为符合真实硬件的同步特性，

彻底消除这种顺序依赖的诡异行为。

Q54. SystemVerilog: Named fork-join Disable -- Selective Process Killing

QUESTION

(a) How do you disable a SPECIFIC named process within a fork-join

without killing other processes?

(b) Implement a watchdog timer using this technique.

(c) How do you use disable fork safely when OTHER processes should

continue running?

ANSWER

```
// ================================================================
// (a) Named fork-join: kill only a specific group
// ================================================================
//
// Problem with plain "disable fork": it kills ALL threads launched
// by the current process -- including unrelated monitors/drivers
// that should keep running.
//
// Solution: wrap the threads you WANT to be able to kill inside a
// NAMED fork-join block. "disable <name>" only kills threads
// within THAT named block, leaving everything else untouched.

// ================================================================
// (b) Watchdog timer using named fork-join
// ================================================================

task run_with_watchdog(int timeout_ns = 1000);
    fork : watchdog_group          // NAMED fork block
        // Thread 1: the actual work
        begin : work_thread
            do_main_task();
            `uvm_info("WDG", "Task completed normally", UVM_LOW)
        end : work_thread

        // Thread 2: the watchdog timer
        begin : timer_thread
            #(timeout_ns * 1ns);
            `uvm_error("WDG", $sformatf(
                "TIMEOUT: task did not complete in %0d ns", timeout_ns))
        end : timer_thread
    join_any  : watchdog_group     // unblock when EITHER thread finishes

    // Kill whichever thread is still running -- ONLY within this group
    disable watchdog_group;        // precise: only kills work_thread or timer_thread
endtask

// ================================================================
// (c) Safe disable fork when other processes must survive
```

```
// ================================================================
//
// WRONG approach (kills unrelated processes too):
task unsafe_version();
    fork
        begin : background_monitor
            forever @(posedge vif.clk) begin   // should NEVER be killed
                monitor_something();
            end
        end

        begin : timed_work
            do_some_work();
        end

        begin : timer
            #1us;
            `uvm_error("TMO", "timeout")
        end
    join_any

    disable fork;   // DANGER: also kills background_monitor!
endtask

// CORRECT approach: wrap only the killable threads in a named sub-fork
task safe_version();
    // Background monitor runs OUTSIDE the named fork -- immune to disable
    fork
        forever @(posedge vif.clk) monitor_something();  // never killed
    join_none

    // Only timed_work + timer are inside the named group
    fork : killable_group
        begin
            do_some_work();
        end

        begin
            #1us;
            `uvm_error("TMO", "timeout")
        end
    join_any : killable_group

    disable killable_group;   // only kills timed_work and timer,
                              // background monitor keeps running
endtask

// ================================================================
// Summary: three levels of "kill" precision
// ================================================================
//
//   disable fork;                  -- kills ALL threads in current process
//                                     (nuclear option, dangerous in UVM)
//
//   disable <named_fork_label>;    -- kills only threads in that named block
//                                     (surgical, recommended)
//
//   process handle + kill():       -- kills a specific process by handle
//   //   process p;
//   //   fork begin p = process::self(); ... end join_none
//   //   p.kill();                 (most surgical, most complex)
```

解释

"disable fork 的陷阱"是 SystemVerilog 并发编程里最常见的错误之一：

在 UVM 环境里，run_phase 内通常有多个后台线程（monitor、driver、coverage collector），

裸的 disable fork 会把它们全部杀死，导致测试提前终止或丢失覆盖数据。

解决方案的关键设计：把"想要可控杀死的线程"和"不想杀死的线程"分成不同的 fork 块，

用命名 fork 块（fork : label ... join_any : label + disable label）

精确控制哪些线程被 join_any 解锁、哪些被 disable 清理。

这正是题目里强调的"把想 disable 的 process 和 disable 句柄放在同一个 fork-join 里"——

这个 fork-join 是它们的容器，disable 时只清理容器里的内容。

process handle + process::self() + p.kill() 是更精细的第三种方法，

适合需要按条件选择性杀死某个特定线程的场景。

Q55. SystemVerilog: fork-join / join_any / join_none -- Shared Variable Trace

QUESTION

Given the code below, trace the value of variable 'a' at the end

for each of three variants: fork-join, fork-join_any, fork-join_none.

```
integer a;
initial begin
    a = 1;
    fork
        begin #1; a = a * 3; end   // Thread 1
        begin #2; a = a - 2; end   // Thread 2
    join           // change to join_any or join_none
    a = a * 4;
    #2; a = a + 8;
end
```

ANSWER

```
// Key rules:
// - Threads SHARE the variable 'a' -- each thread reads the CURRENT value
//   of 'a' at the moment it executes, not the value at fork time.
// - join:      parent BLOCKS until ALL threads finish.
// - join_any:  parent BLOCKS until ANY ONE thread finishes; others keep running.
// - join_none: parent continues IMMEDIATELY; all threads run in background.
// - #N after join exits: delay is relative to when parent resumes.

// ---- Variant 1: fork ... join (wait ALL) ---------------------------------
//
//   t=0:  a=1. fork launched, parent blocks.
//   t=1:  Thread1 wakes: a = a*3 = 1*3 = 3
//   t=2:  Thread2 wakes: a = a-2 = 3-2 = 1  [sees a=3 set by Thread1]
//   t=2:  join satisfied (both done), parent resumes.
//   t=2:  a = a*4 = 1*4 = 4
//   t=4:  #2 fires: a = a+8 = 4+8 = 12
//
//   Final a = 12

// ---- Variant 2: fork ... join_any (wait FIRST) ---------------------------
//
//   t=0:  a=1. fork launched, parent blocks.
//   t=1:  Thread1 wakes (FIRST): a = 1*3 = 3
```

```
//         join_any triggers: parent resumes at t=1. Thread2 is STILL ALIVE.
//   t=1:  a = a*4 = 3*4 = 12  (parent executes this immediately)
//   t=2:  Two events at t=2:
//         - Thread2 wakes: a = a-2 = 12-2 = 10  [sees a=12]
//         - Parent's #2 (from t=1) fires: a = a+8 -- BUT Thread2 already
//           ran at t=2 (simulator scheduling: Thread2 executes first since
//           it was launched before parent's #2 was scheduled*)
//         -> Thread2 at t=2: a = 12-2 = 10
//         -> Parent #2 at t=3 (from t=1, #2 means "wait 2 more time units"):
//   t=3:  a = 10+8 = 18
//
//   Final a = 18
//   (* Simulator may schedule these differently -- see note below)

// ---- Variant 3: fork ... join_none (don't wait) -------------------------
//
//   t=0:  a=1. fork launched. join_none exits IMMEDIATELY (no blocking).
//   t=0:  a = a*4 = 1*4 = 4  (parent continues right away)
//         #2 starts counting from t=0.
//   t=1:  Thread1 wakes: a = a*3 = 4*3 = 12  [sees a=4 set at t=0]
//   t=2:  RACE CONDITION: two events scheduled simultaneously:
//         - Thread2: a = a-2 = 12-2 = 10
//         - Parent's #2: a = a+8 = 12+8 = 20  (if it fires BEFORE Thread2)
//           or a = 10+8 = 18 (if Thread2 fires FIRST)
//         IEEE does NOT specify which of these executes first.
//   t=2:  Result is NON-DETERMINISTIC: a = 18 or a = 20

// ---- Summary table ------------------------------------------------------
//
//   Variant     | t=1   | t=2    | t=3/4  | Final a
//   ------------|-------|--------|--------|--------
//   join        | a=3   | a=1    | t=4:12 | 12
//   join_any    | a=12  | a=10   | t=3:18 | 18
//   join_none   | a=12  | RACE   |   --   | 18 or 20 (non-det.)

// ---- Why join_none is DANGEROUS for shared variables --------------------
//   join_none provides NO synchronization for 'a'.
//   Parent and background threads race to read/write the same variable.
//   Use join_none ONLY for fire-and-forget tasks where the background
//   threads do NOT share mutable state with the parent process.
//   If you need shared variables, use join or join_any with proper
//   disable/cleanup, or use a mailbox/semaphore for communication.
```

解释

这道题的核心是"线程共享变量：每个线程读到的是执行那一刻的当前值"，

而不是 fork 时的快照。这和其他语言的线程/闭包行为可能不同。

join_any 后父线程继续，但背景线程还在跑，两者共享 a 变量——

join_any 不是"停止其他线程"，只是"允许父线程继续"。

join_none 的竞争条件（t=2同时有两个写操作）体现了 SV 调度模型的非确定性：

两个不同进程在同一时间步写同一变量，顺序由仿真器决定，结果不可预测，

这正是为什么共享可变变量 + join_none 是危险的编程模式。

面试中能逐步画出时间轴并在 join_none 那格写上"race condition"是关键加分项。

Q56. SystemVerilog: Polymorphism, Typecasting, and Virtual Functions

QUESTION

Explain polymorphism, virtual functions, and typecasting in SystemVerilog.

How do they work together?

ANSWER

```
// ================================================================
// Polymorphism: one interface, multiple behaviors
// ================================================================
//
// A BASE CLASS handle can point to a DERIVED CLASS object.
// When a virtual method is called through the base handle,
// the DERIVED class's version executes (runtime dispatch).

class animal;
    virtual function string speak();
        return "...";   // base default
    endfunction
endclass

class dog extends animal;
    virtual function string speak();
        return "Woof";   // overrides base
    endfunction
endclass

class cat extends animal;
    virtual function string speak();
        return "Meow";
    endfunction
endclass

// Polymorphic usage:
animal zoo[3];
zoo[0] = new dog();   // base handle, derived object
zoo[1] = new cat();
zoo[2] = new animal();

foreach (zoo[i])
    $display(zoo[i].speak());  // Woof, Meow, ...
// The correct speak() is called based on the ACTUAL object type,
// not the handle type.

// ================================================================
// virtual keyword: required for polymorphism
// ================================================================
//
// WITHOUT virtual: static dispatch (base version always called)
class base_no_virt;
    function string name(); return "base"; endfunction
endclass
class derived_no_virt extends base_no_virt;
    function string name(); return "derived"; endfunction
endclass
// base_no_virt h = new derived_no_virt();
// h.name() -> "base"   (static dispatch: base version!)

// WITH virtual: dynamic dispatch (correct version called)
class base_virt;
    virtual function string name(); return "base"; endfunction
endclass
class derived_virt extends base_virt;
    virtual function string name(); return "derived"; endfunction
endclass
```

```
// base_virt h = new derived_virt();
// h.name() -> "derived"  (dynamic dispatch: derived version!)

// ============================================================
// Typecasting: $cast()
// ============================================================
//
// $cast() safely converts a base class handle to a derived class handle.
// It returns 1 (success) if the actual object IS of the derived type,
// returns 0 (failure) if not.

class my_txn extends uvm_sequence_item;
    rand int data;
endclass

class error_txn extends my_txn;
    rand bit inject_err;
endclass

// Correct usage:
my_txn    base_h;
error_txn derived_h;

base_h = new error_txn();   // base handle, derived object

// Try to access derived fields via base handle -> compile error
// base_h.inject_err = 1;   // ERROR: base_h doesn't have inject_err

// Use $cast to recover derived handle:
if ($cast(derived_h, base_h)) begin
    derived_h.inject_err = 1;   // now we can access derived fields
end else begin
    `uvm_error("CAST", "Object is not of type error_txn")
end

// In UVM this is common for clone():
// uvm_object obj = txn.clone();
// my_txn cloned_txn;
// if (!$cast(cloned_txn, obj))
//     `uvm_fatal("CAST", "Clone failed cast")
```

解释

三个概念的协作关系：

polymorphism（多态）是目标：让同一段代码对不同类型的对象表现出不同行为；

virtual function 是实现手段：在基类声明 virtual，派生类 override，调用时动态分发；

$cast 是工具：当基类句柄实际指向派生类对象，需要访问派生类特有字段时，安全地向下转型。

virtual 关键字不可省略：没有 virtual 的函数是静态分发，

即使基类句柄指向派生类对象，调用的永远是基类版本——这是最常见的 OOP 误解。

UVM 框架本身大量使用这三者：uvm_object 的 do_copy/do_compare/do_print 是 virtual 的，

每种 sequence_item 只需 override 这些方法，UVM 的 copy()/compare() 就能自动调用正确版本。

Q57. SystemVerilog: Generate a Delay Signal 0-1.25x One Clock Cycle

QUESTION

Write SV code to create a delayed version of a signal where the delay

is between 0 and 1.25x one clock cycle. The signal may change each

cycle (bit data, clk).

Reference code: always @(data) data_d <= #(calculated_delay) data;

ANSWER

```
// ============================================================
// Problem analysis
// ============================================================
//
// Requirement:
//   - data_d is a delayed copy of data
//   - Delay = 0 to 1.25 * T_clk (e.g. 0 to 12.5ns for a 10ns clock)
//   - data can change every cycle (not just on clock edges)
//   - Delay itself may vary per cycle (randomized)
//
// The reference pattern:
//   always @(data) data_d <= #(calculated_delay) data;
//
// This is a CONTINUOUS sensitivity on data (any edge, not just posedge clk).
// The non-blocking assignment with an intra-assignment delay (#N)
// schedules the update to data_d N time units in the future.

// ============================================================
// Implementation (testbench style, not synthesizable)
// ============================================================
module delay_gen #(
    parameter real T_CLK = 10.0    // clock period in ns
);
    logic data;
    logic data_d;
    logic clk;
    real  calculated_delay;

    // Generate randomized delay: 0 to 1.25 * T_CLK
    // Called each time data changes
    function real get_delay();
        // $urandom_range returns integer; scale to real
        int unsigned raw = $urandom_range(0, 125);   // 0 to 125 (units of 0.1ns)
        return real'(raw) * 0.1;    // 0.0 to 12.5ns (for 10ns clock)
    endfunction

    // Sensitivity on data: fires every time data changes (any transition)
    always @(data) begin
        calculated_delay = get_delay();
        data_d           <= #(calculated_delay) data;
    end

    // ============================================================
    // Key SV constructs used
    // ============================================================
    //
    // 1. always @(data): fires on ANY change of data (level-sensitive)
    //    NOT the same as always @(posedge data)!
    //
    // 2. Non-blocking with intra-assignment delay:
    //      data_d <= #(delay) data;
    //    RHS (data) is sampled NOW (at the current time).
    //    LHS (data_d) update is SCHEDULED for (current_time + delay).
    //    This is the standard way to model propagation delay in TB.
    //
    // 3. Real-valued delay:
```

```
    //    `timescale 1ns/1ps  (precision = 1ps = 0.001ns)
    //    #12.5 is legal (12500 ps = 12.5ns with this timescale).
    //
    // 4. $urandom_range(0, 125): generates uniform random integer [0,125]
    //    Scale to 0.0-12.5ns by multiplying by 0.1.

endmodule

// ================================================================
// If the delay must change every CLOCK CYCLE (not per-data-change):
// ================================================================
always @(posedge clk) begin
    // Pick new delay for this clock cycle
    calculated_delay = real'($urandom_range(0, 125)) * 0.1;
end

// Apply delay whenever data changes, using CURRENT calculated_delay
always @(data) begin
    data_d <= #(calculated_delay) data;
end

// ================================================================
// Alternative using $urandom for fractional delays
// ================================================================
// Constrained random delay in SV (class-based):
class delay_rand;
    rand real delay_ns;
    real      T_clk;
    constraint c_delay {
        delay_ns >= 0.0;
        delay_ns <= T_clk * 1.25;
    }
endclass
// Note: SV constraints on real variables have limited solver support.
// Use integer constraints (0..125 in 0.1ns steps) and cast to real.
```

解释

`always @(data) data_d <= #(N) data` 是 testbench 中模拟传播延迟的标准模式：

数据变化时立即采样 RHS（data 的当前值），然后把这个值在 N 纳秒后写入 LHS（data_d）。

关键的 `#(N)` 是"intra-assignment delay"（赋值内延迟），必须用 NBA（<=）赋值，

不能用 blocking（=）——因为 blocking 会阻塞整个 always 块直到延迟完成，

导致期间的其他信号变化被忽略。

0-1.25x 时钟周期的实现技巧：用整数随机（$urandom_range 0-125）再乘以 0.1 得到浮点纳秒，

这绕开了 SV 约束器对 real 类型支持有限的问题。

`timescale 1ns/1ps` 允许最小精度 1ps，因此 #12.5 是合法的（= 12500ps）。

Q58. SystemVerilog: Polymorphism and Virtual Functions -- Output Trace

QUESTION

Given the code below, what is the output?

Then modify write() in parent to be virtual and re-trace the output.

```
class parent;
    bit [7:0] addr;
```

```
    function new(); addr = 8'hFF; endfunction
    function void write();
        $display("Parent addr: %0h", addr);
    endfunction
endclass

class child extends parent;
    function new(); addr = 8'hAA; endfunction
    function void write();
        $display("Child addr: %0h", addr);
    endfunction
endclass

initial begin
    parent a;
    child  b;
    a = new();
    b = new();
    a.write();
    b.write();
    a = b;
    $display("a's address: %0h", a.addr);
    a.write();
end
```

ANSWER

```
// ================================================================
// Case 1: write() is NOT virtual (as written)
// ================================================================
//
//   a = new()    -> a is a parent object, addr=0xFF
//   b = new()    -> b is a child object,  addr=0xAA
//
//   a.write()    -> a declared as 'parent', write() not virtual
//                   STATIC DISPATCH: always calls parent::write()
//                   Output: "Parent addr: ff"
//
//   b.write()    -> b declared as 'child', calls child::write()
//                   Output: "Child addr: aa"
//
//   a = b        -> handle assignment: a now POINTS TO the child object
//                   (a is still TYPE parent, but the underlying object is child)
//
//   a.addr       -> accesses addr field through the parent handle
//                   The child object's addr = 0xAA (set in child::new())
//                   Output: "a's address: aa"
//
//   a.write()    -> a is declared as 'parent', write() NOT virtual
//                   STATIC DISPATCH: still calls parent::write()
//                   But a.addr = 0xAA (child's addr, since a points to child)
//                   Output: "Parent addr: aa"
//
// CASE 1 OUTPUT:
//   Parent addr: ff
//   Child addr: aa
//   a's address: aa
//   Parent addr: aa    <- static dispatch, but addr from child object

// ================================================================
// Case 2: write() IS virtual in parent
// ================================================================
```

```
//
//   Change: (virtual) function void write();  in parent class
//
//   a = new()    -> parent object, addr=0xFF
//   b = new()    -> child object,  addr=0xAA
//
//   a.write()    -> a points to parent object
//                   DYNAMIC DISPATCH: actual object is parent -> parent::write()
//                   Output: "Parent addr: ff"
//
//   b.write()    -> b points to child object
//                   DYNAMIC DISPATCH: actual object is child -> child::write()
//                   Output: "Child addr: aa"
//
//   a = b        -> a now points to the child object
//
//   a.addr       -> 0xAA (child object's addr)
//                   Output: "a's address: aa"
//
//   a.write()    -> a points to child object
//                   DYNAMIC DISPATCH: actual object is CHILD -> child::write()!
//                   Output: "Child addr: aa"  <-- KEY DIFFERENCE from Case 1
//
// CASE 2 OUTPUT:
//   Parent addr: ff
//   Child addr: aa
//   a's address: aa
//   Child addr: aa    <- dynamic dispatch: actual object is child

// ================================================================
// Summary: the ONE line that differs
// ================================================================
//
//   Non-virtual: a.write() after a=b -> "Parent addr: aa"
//     (static: declared type = parent, always calls parent version)
//
//   Virtual:     a.write() after a=b -> "Child addr: aa"
//     (dynamic: actual object = child, calls child version)
//
//   This is the essence of polymorphism:
//   "virtual" enables the METHOD to be determined by the OBJECT,
//   not the HANDLE type.
```

解释

这道题考察的核心：`a = b` 之后，a 的"声明类型"仍然是 parent，

但它指向的"实际对象"是 child。

非 virtual 情况：方法分发由**声明类型**决定（编译时绑定），

所以 a.write() 永远调用 parent::write()，即使 a 指向 child 对象。

virtual 情况：方法分发由**实际对象类型**决定（运行时绑定），

a 指向 child 对象，所以调用 child::write()。

`a.addr` 的值始终是 0xAA（无论 virtual 与否），因为 addr 是数据字段，

不是方法——字段访问取决于实际对象的内存布局，而不是声明类型。

这也解释了为什么 Case 1 最后打印 "Parent addr: aa" 而非 "Parent addr: ff"：

方法调用走 parent（静态），但读取的 addr 是 child 对象里的 0xAA。

Q59. SystemVerilog: OOP -- Shape / Circle / Rectangle with Virtual print_area()

QUESTION

Complete the following SV OOP skeleton. Define a base class Shape with a virtual print_area(). Derive Circle (radius=10) and Rectangle (l=2, w=4) that override print_area(). Create an array of Shape handles, populate with one Circle and one Rectangle, then loop and call print_area(). Write the expected output in comments.

```
class Shape;
    virtual function void print_area();
        $display("Shape: no area defined");
    endfunction
endclass

class Circle extends Shape;
    int radius = 10;
    virtual function void print_area();
        real area = 3.14159 * radius * radius;
        $display("Circle area: %.2f", area);
    endfunction
endclass

class Rectangle extends Shape;
    int l = 2;
    int w = 4;
    virtual function void print_area();
        $display("Rectangle area: %0d", l * w);
    endfunction
endclass

module top;
    Shape calculations[$];   // dynamic array of base-class handles

    initial begin
        Circle    c = new();
        Rectangle r = new();

        calculations.push_back(c);   // store Circle via Shape handle
        calculations.push_back(r);   // store Rectangle via Shape handle

        foreach (calculations[i])
            calculations[i].print_area();   // polymorphic dispatch
    end
endmodule
```

ANSWER

```
// ================================================================
// Expected output (VIRTUAL function -> dynamic dispatch):
// ================================================================
//
//   Circle area: 314.16
//   Rectangle area: 8
//
// Trace:
//   calculations[0] -> actual object is Circle
//     print_area() is virtual -> calls Circle::print_area()
//     area = 3.14159 * 10 * 10 = 314.159 -> formatted "%.2f" -> 314.16
```

```
//
//   calculations[1] -> actual object is Rectangle
//     print_area() is virtual -> calls Rectangle::print_area()
//     area = 2 * 4 = 8

// ================================================================
// Key OOP concepts demonstrated
// ================================================================
//
// 1. VIRTUAL FUNCTION: base class declares virtual, derived classes override.
//    Without virtual: all calls would print "Shape: no area defined"
//    (static dispatch on handle type = Shape).
//    With virtual: dispatch based on ACTUAL object type (dynamic).
//
// 2. BASE CLASS HANDLE ARRAY: Shape calculations[$]
//    Can hold any subclass object (Circle, Rectangle, or future subclasses).
//    This is the SV/OOP way to process heterogeneous collections uniformly.
//
// 3. push_back() into dynamic array:
//    Dynamic queue ($) grows automatically.
//    No need to pre-size; each push_back adds one element.
//
// 4. foreach loop:
//    foreach (calculations[i]) iterates i = 0, 1, ..., size-1.
//    Equivalent to: for (int i=0; i<calculations.size(); i++)
//
// ================================================================
// What if print_area() were NOT virtual?
// ================================================================
//
//   Output would be:
//     Shape: no area defined
//     Shape: no area defined
//
//   Both calls dispatch to Shape::print_area() (static, handle type = Shape).
//   The actual Circle/Rectangle implementations are never reached.
//   This is why virtual is essential for polymorphic behavior.
```

解释

这道题把 OOP 三要素都考察到了：继承（Circle/Rectangle extends Shape）、

封装（各自的 radius/l/w 字段）、多态（virtual print_area 动态分发）。

`Shape calculations[$]` 是 SV 中实现多态数组的标准写法：

基类句柄可以存储任何派生类对象，统一调用 virtual 方法时自动走正确的实现。

和 C++/Java 的区别：SV 中 `new()` 不带参数（构造函数可以是 function new()，

也可以利用成员变量的初始值）；动态队列用 `[$]` 声明，`push_back()` 添加元素。

面试常见追问：如果 Circle 没有 override print_area() 会怎样？

答案是调用 Shape 的基类版本（"no area defined"），因为派生类没有提供自己的实现，

virtual 方法沿着继承链向上查找第一个有实现的版本。

Q60. SystemVerilog: fork-join_none Tasks with #Delay -- Value Trace and Semaphore

QUESTION

(a) Two tasks each have a #delay and write to a shared variable.

What are the final values with join_none?

(b) If there are 10000 threads and you only want to wait for 3 to complete before unblocking, what mechanism do you use?

ANSWER

```
// ================================================================
// (a) fork-join_none with #delay tasks
// ================================================================
//
// task task1(); #1ns; val = 2; endtask
// task task2(); #2ns; val = 2; endtask
//
// initial begin
//   fork
//     task1();   // fires at t+1ns: val = 2
//     task2();   // fires at t+2ns: val = 2
//   join_none    // parent continues immediately
//   // parent runs here at t=0, tasks run in background
// end
//
// Result: val = 2 at t=1ns (task1), then val = 2 again at t=2ns (task2).
// Since both write the same value (2), no race condition visible here.
// Final val = 2 (deterministic because both tasks write 2).
//
// If they wrote different values (e.g. 2 and 3):
//   t=1ns: val = 2 (task1)
//   t=2ns: val = 3 (task2)  <- last writer wins
//   Final val = 3 (order is deterministic only because #1 < #2)
//   With join_none, no synchronization -> true race if delays were equal.

// ================================================================
// How to keep a SECOND fork-join_none from affecting others
// ================================================================
//
// Wrap the threads you want to kill in a named fork-join block.
// Other join_none threads outside that block are unaffected.
//
initial begin
    // Background thread that should NEVER be killed
    fork
        forever @(posedge clk) monitor_task();
    join_none

    // Killable group: wrap in named fork
    fork : killable
        task1();
        task2();
    join_none : killable

    // Later, kill only the killable group
    disable killable;   // kills task1/task2, monitor_task is SAFE
end

// ================================================================
// (b) 10000 threads, wait for only 3 to complete -> SEMAPHORE
// ================================================================
//
// Use a semaphore initialized to 0.
// Each thread does sem.put() when done.
// Main thread calls sem.get(3) to wait until 3 threads have finished.

semaphore sem = new(0);   // start empty
```

```
initial begin
    // Launch 10000 threads
    for (int i = 0; i < 10000; i++) begin
        fork
            automatic int tid = i;
            begin
                do_work(tid);
                sem.put(1);   // signal completion
            end
        join_none
    end

    // Block until exactly 3 threads have completed
    sem.get(3);    // waits until sem count reaches 3

    $display("3 threads completed -- proceeding");
    // Remaining 9997 threads continue in background
end
```

解释

join_none 中两个任务写同一个值（2）：最终结果确定为 2，没有竞争，

但如果两个任务写不同值，后完成的任务会覆盖先完成的（last-writer-wins）。

"防止 disable fork 杀死无关线程"的标准模式：把可杀死的线程包在命名 fork 块里，

监视线程在块外启动，`disable label` 只清理该命名块内的线程。

Semaphore 的 `get(N)` 会阻塞直到 semaphore 的内部计数器 >= N，

是"等待 M 个线程中的 N 个先完成"这类需求的标准解法。

Q61. SystemVerilog: Sliding Window -- Track Count of 1's (Parameterized)

QUESTION

Given a 4-bit data stream, a window of 10 values, track how many

of the last 10 values had exactly 3 ones in the nibble.

Build this as a SV class, then generalize to N ones, N of M-bit value,

and 32-bit data with parametrized counter.

ANSWER

```
// ============================================================
// Base: track count of 1's in 4-bit window of 10 values
// ============================================================
class ones_window_tracker;
    parameter int WINDOW   = 10;
    parameter int DATA_W   = 4;
    parameter int TARGET_N = 3;   // count values that have exactly 3 ones

    // Circular buffer of recent data values
    bit [DATA_W-1:0] buf [WINDOW];
    int unsigned     head = 0;    // next write position
    int unsigned     count = 0;   // how many values currently match target

    function new();
        foreach (buf[i]) buf[i] = '0;
    endfunction
```

```
    // Push a new value, evict the oldest, update match count
    function void push(bit [DATA_W-1:0] data);
        // Remove contribution of outgoing value
        if ($countones(buf[head]) == TARGET_N)
            count--;

        // Insert new value
        buf[head] = data;
        if ($countones(data) == TARGET_N)
            count++;

        head = (head + 1) % WINDOW;
    endfunction

    function int unsigned get_match_count();
        return count;
    endfunction

    function void print();
        $display("Window: matches with %0d ones = %0d / %0d",
                 TARGET_N, count, WINDOW);
    endfunction
endclass

// ============================================================
// Generalized: N ones in M-bit data, parametrized window
// ============================================================
class window_tracker #(
    parameter int WINDOW   = 10,
    parameter int DATA_W   = 4,
    parameter int TARGET_N = 3
);
    bit [DATA_W-1:0] buf  [WINDOW];
    int unsigned     head  = 0;
    int unsigned     count = 0;

    function new();
        foreach (buf[i]) buf[i] = '0;
    endfunction

    function void push(bit [DATA_W-1:0] data);
        if ($countones(buf[head]) == TARGET_N) count--;
        buf[head] = data;
        if ($countones(data) == TARGET_N)      count++;
        head = (head + 1) % WINDOW;
    endfunction

    function int unsigned get_count(); return count; endfunction
endclass

// Usage: track 32-bit values with exactly 8 ones, window of 20
// window_tracker #(.WINDOW(20), .DATA_W(32), .TARGET_N(8)) tracker32;

// ============================================================
// Generalized: track N occurrences of VALUE_M (not just ones count)
// ============================================================
class value_window_tracker #(
    parameter int            WINDOW  = 10,
    parameter int            DATA_W  = 4,
    parameter bit [DATA_W-1:0] TARGET_VAL = 4'b1010  // specific value to match
);
    bit [DATA_W-1:0] buf  [WINDOW];
    int unsigned     head  = 0;
```

```
    int unsigned     count = 0;

    function void push(bit [DATA_W-1:0] data);
        if (buf[head] == TARGET_VAL) count--;
        buf[head] = data;
        if (data == TARGET_VAL)      count++;
        head = (head + 1) % WINDOW;
    endfunction

    function int unsigned get_count(); return count; endfunction
endclass
```

解释

循环缓冲区（circular buffer）是 sliding window 的标准实现：

`head` 指向下一个写入位置，每次 push 先移除旧值（先判断旧值是否匹配，是则 count--），

再写入新值（判断新值是否匹配，是则 count++），最后 head = (head+1) % WINDOW。

这样 count 始终反映当前窗口内的匹配数，O(1) 时间更新，O(WINDOW) 空间。

参数化设计三步走：先固定具体参数（WINDOW=10, DATA_W=4, TARGET_N=3）验证逻辑，

再提取为 class parameter，最后把"比较规则"也参数化（从比较 ones count 到比较具体值）。

`$countones()` 是 SV 内置的 popcount 函数，可以用在 class function 中（不只限于 constraint/module）。

Q62. SystemVerilog: fork-join_none in for Loop -- Block Without Affecting Others

QUESTION

You have a for loop launching threads with fork-join_none.

How do you block (wait for all loop threads) without affecting

OTHER join_none threads running elsewhere?

ANSWER

```
// ================================================================
// Problem: for loop with join_none threads
// ================================================================
//
// Naive approach -- WRONG: disable fork kills ALL threads
initial begin
    // Background monitor (should never be killed)
    fork forever monitor(); join_none

    // For loop spawning N threads
    for (int i = 0; i < 10; i++) begin
        fork
            automatic int tid = i;
            thread_task(tid);
        join_none
    end
    disable fork;   // KILLS the monitor too! WRONG.
end

// ================================================================
// Correct: guard the for-loop threads in a named fork-join
// ================================================================
initial begin
    // Background monitor OUTSIDE the named fork -- never killed
```

```
    fork
        forever monitor();
    join_none

    // Wrap ALL for-loop threads in one named fork-join
    fork : loop_group          // named fork block
        for (int i = 0; i < 10; i++) begin
            fork
                automatic int tid = i;
                thread_task(tid);
            join_none
        end
    join : loop_group          // join waits for... nothing here directly

    // But we need to WAIT for the inner join_none threads to complete.
    // join_none inside join means the outer join finishes immediately.
    // Alternative: use join_any + disable pattern or semaphore:

    // --- Better: semaphore to wait for all N loop threads ---
    semaphore done = new(0);

    fork : loop_group2
        for (int i = 0; i < 10; i++) begin
            fork
                automatic int tid = i;
                begin
                    thread_task(tid);
                    done.put(1);   // signal completion
                end
            join_none
        end
    join_none : loop_group2

    // Wait for all 10 threads to complete
    done.get(10);     // blocks until 10 puts received
    $display("All 10 loop threads done, monitor still running");

    // If you want to kill the loop threads without waiting for all:
    // disable loop_group2;   -- kills only those threads, monitor is safe
end

// ================================================================
// Elegant pattern for N threads with guarded kill
// ================================================================
task run_N_threads_guarded(int N);
    semaphore sem = new(0);

    fork : thread_group        // named container
        for (int i = 0; i < N; i++) begin
            fork
                automatic int tid = i;
                begin
                    do_work(tid);
                    sem.put(1);
                end
            join_none
        end
    join_none : thread_group

    // Option A: wait for ALL
    sem.get(N);
    // Option B: wait for first M (e.g. 3)
    // sem.get(3); disable thread_group;
```

```
    // Option C: timeout kill
    // fork begin #100ns; disable thread_group; end join_none
endtask
```

解释

For 循环的 join_none 线程管理有三个层次：

等待全部完成用 semaphore.get(N)；等待部分完成用 semaphore.get(M)；

超时强制中止用 `#timeout; disable label`。

关键原则：`disable fork` 是核弹，`disable label` 是手术刀。

在任何有后台长生命周期线程的环境里（UVM monitor/driver），

都不应该裸用 `disable fork`，必须通过命名 fork 块限制 disable 的范围。

`automatic int tid = i` 在 fork-join_none 循环中是必须的——

没有 automatic，所有线程共享同一个 tid 变量，循环结束后所有线程看到的都是最后一个 i 的值。

Q63. SystemVerilog: |-> vs |=> -- Overlapping vs Non-Overlapping Implication

QUESTION

What is the difference between |-> and |=> in SVA?

Give examples showing when each is used.

ANSWER

```
// ================================================================
// |-> Overlapping implication
// ================================================================
//
// a |-> b
//
// Meaning: IF a is true at time T, THEN b must be true AT THE SAME TIME T.
// The antecedent (a) and consequent (b) are evaluated in the SAME clock cycle.
// "Overlapping" because they share the same time point.
//
// Example:
//   @(posedge clk) valid |-> !fifo_empty;
//   "If valid is asserted, then fifo_empty must be low IN THE SAME CYCLE."
//
// Timing diagram:
//   clk:   _|■|_|■|_|■|
//   valid:  _|■■■|_____
//   b:      _|■■■|_____  <- must be true in same cycle as valid
//             ^ same cycle

// ================================================================
// |=> Non-overlapping implication
// ================================================================
//
// a |=> b
//
// Meaning: IF a is true at time T, THEN b must be true at time T+1.
// The consequent is checked in the NEXT clock cycle.
// Equivalent to: a |-> ##1 b
//
// Example:
//   @(posedge clk) $rose(req) |=> gnt;
```

```
//   "If req rises at cycle T, then gnt must be high at cycle T+1."
//
// Timing diagram:
//   clk:   _|■|_|■|_|■|
//   req:   _|■■■■■■|___
//   gnt:   ___|■■■■■|__  <- checked in NEXT cycle after req rises
//              ^ T    ^ T+1

// ================================================================
// Common usage patterns
// ================================================================
//
//   Use |->  when consequent must hold NOW:
//     valid |-> data_stable    (data must be stable same cycle as valid)
//     ack   |-> !error_flag    (no error when ack is high)
//
//   Use |=>  when consequent is a REGISTERED/DELAYED response:
//     $rose(clk_en) |=> (q == d)    (FF output updates next cycle)
//     write_req |=> write_ack       (ack comes one cycle after request)
//     $rose(start) |=> busy         (busy registered, not combinational)
//
//   Key rule of thumb:
//     Combinational response? -> |->
//     Registered (clocked) response? -> |=>
```

解释

`|->` 和 `|=>` 的区别就是"同一拍"和"下一拍"，写法上 `|=>` 是 `|-> ##1` 的语法糖。

选择的判断规则：查信号是组合逻辑输出（当拍就能响应）还是寄存器输出（下一拍才稳定）。

最常见的错误：把时序逻辑的响应用 `|->` 检查，导致 assert 在数据刚刚被寄存的同一拍就检查，

此时 Q 还是旧值，必然 fail。改为 `|=>` 就能正确地在下一拍采样寄存器输出。

`##N` 扩展：如果响应需要 N 个周期（流水线），用 `|-> ##N result_valid`，

或者 `|-> ##[1:5]` 表示在 1~5 拍内某拍响应即可。

Q64. SystemVerilog: OOP -- Override vs Overload (C++ comparison)

QUESTION

What is the difference between method override and method overload?

How does SV handle each? Compare with C++.

ANSWER

```
// ================================================================
// Method OVERRIDE (SV and C++ both support)
// ================================================================
//
// Override: a DERIVED class provides a NEW IMPLEMENTATION of a method
//           that exists in the BASE class, with the SAME signature.
//
// In SV:
class Animal;
    virtual function string speak();
        return "...";
    endfunction
endclass
```

```
class Dog extends Animal;
    virtual function string speak();   // OVERRIDES Animal::speak
        return "Woof";
    endfunction
endclass
//
// Rules for override in SV:
//   - Must have the SAME function name, return type, and argument types.
//   - Base class function should be declared virtual (enables polymorphism).
//   - Derived class can optionally use 'virtual' keyword again.
//
// In C++: same concept, use 'override' keyword for safety:
//   string speak() override { return "Woof"; }

// ================================================================
// Method OVERLOAD (C++ supports; SV does NOT)
// ================================================================
//
// Overload: MULTIPLE methods with the SAME NAME but DIFFERENT signatures
//           (different parameter types or number of parameters).
//           Resolved at COMPILE TIME based on argument types.
//
// In C++ (supported):
//   void print(int x)    { cout << "int: "    << x; }
//   void print(double x) { cout << "double: " << x; }
//   void print(string s) { cout << "string: " << s; }
//   // All three exist simultaneously; compiler picks by argument type.
//
// In SystemVerilog (NOT supported):
//   SV does NOT support function overloading.
//   You cannot have two functions with the same name in the same scope,
//   even if their signatures differ.
//   Workaround: use different names, or use default argument values:
//
function void print_val(int x, string prefix = "");
    $display("%s%0d", prefix, x);
endfunction
// Call as: print_val(5) or print_val(5, "value: ")
// This approximates overloading with optional arguments.

// ================================================================
// Summary table
// ================================================================
//
//   Feature          | C++         | SystemVerilog
//   -----------------+-------------+------------------
//   Override         | Yes         | Yes (with virtual)
//   Overload         | Yes         | NO (not supported)
//   Polymorphism     | Yes         | Yes (virtual)
//   Default args     | Yes         | Yes (workaround for overload)
//   'override' kw    | Yes (C++11) | No keyword needed
```

解释

Override（覆写）是"同名同签名，派生类重新实现"，SV 完全支持，配合 virtual 实现运行时多态。

Overload（重载）是"同名不同签名，编译器根据参数类型选择"，C++ 支持，SV 不支持。

SV 的设计决策：不支持 overload 简化了语言规范和编译器实现，

但代价是 API 设计时必须用不同名字区分不同参数类型的版本，或用 default argument 模拟。

面试中能清晰说出"SV 不支持 overload"并给出 workaround 是关键加分点——

很多候选人只知道 override 而不清楚 overload 在 SV 中的限制。

Q65. SystemVerilog: Bit Field Extraction and Variable Bit Selection

QUESTION

Given a 32-bit value x, extract bits [15:8] and then access

the third bit of that slice (variable bit position).

How do you do this? Show the SV part-select syntax.

ANSWER

```
// ================================================================
// Problem setup
// ================================================================
//
// x is 32 bits.
// We want bits [15:8] -- the second byte.
// Then we want bit [2] of that slice (the third bit of the byte).
// Bit position can also be a variable (n).

logic [31:0] x = 32'hABCD_EF12;  // example value

// ================================================================
// Step 1: Extract fixed slice [15:8]
// ================================================================
logic [7:0] extract;
assign extract = x[15:8];   // direct part-select: bits 15 down to 8
// x = 0xABCD_EF12 -> x[15:8] = 0xEF = 8'b1110_1111

// ================================================================
// Step 2: Access a specific bit within the slice
// ================================================================
logic bit2;
assign bit2 = extract[2];         // bit[2] of extract = x[10]
// OR equivalently in one expression:
assign bit2_direct = x[8 + 2];   // same as x[10]
assign bit2_direct2 = x[10];     // explicit absolute bit index

// ================================================================
// Step 3: Variable bit select (bit position is a variable)
// ================================================================
logic [2:0] n;     // variable bit index (0-7)
logic bit_n;

// Fixed-width slice starting at a variable position:
// x[base +: width] = x[base + width - 1 : base]  (ascending)
// x[base -: width] = x[base : base - width + 1]  (descending)

// Get bit n of the [15:8] slice:
assign bit_n = x[8 + n];   // absolute index = base(8) + offset(n)

// Or using the extract:
assign bit_n_v2 = extract[n];   // same result

// ================================================================
// The +: operator (indexed part-select) -- key SV syntax
// ================================================================
//
// x[start +: width]  -- width bits starting at 'start' (ascending)
// x[start -: width]  -- width bits ending at 'start' (descending)
//
// Used when the STARTING BIT is a variable:
```

```
logic [7:0] byte_n;
int unsigned byte_idx;  // which byte to extract (0-3)
assign byte_n = x[byte_idx*8 +: 8];  // extracts 8 bits from byte_idx

// byte_idx=0: x[7:0]   (byte 0)
// byte_idx=1: x[15:8]  (byte 1)
// byte_idx=2: x[23:16] (byte 2)
// byte_idx=3: x[31:24] (byte 3)
```

解释

SV 的 part-select 有三种形式：

`x[15:8]`：固定范围，最常见；

`x[base +: width]`：从 base 开始向上取 width 位，适合 base 是变量的情况；

`x[base -: width]`：从 base 开始向下取 width 位（如 `x[15 -: 8]` == `x[15:8]`）。

关键规则：`+:` 和 `-:` 的 width 必须是常量，只有 base 可以是变量——

这是综合工具的限制，保证 mux 的选择宽度在编译时确定。

`x[8 + n]` vs `x[8 +: 1]`：取单个变量位时用 `x[8 + n]` 更简洁，

但如果要取多个位（如8位），必须用 `x[8*n +: 8]` 而不是
`x[(8*n+7):(8*n)]`（后者综合不支持变量边界）。

Q66. SystemVerilog: Virtual Class (Abstract Class) -- When to Use

QUESTION

What is a virtual class in SystemVerilog? When should you use one?

How is it different from a regular class with virtual functions?

ANSWER

```
// ============================================================
// Virtual class (abstract class)
// ============================================================
//
// A virtual class CANNOT be instantiated directly.
// It serves as a BASE CLASS template that derived classes MUST extend.
// It can contain:
//   - pure virtual functions (no implementation, MUST be overridden)
//   - regular virtual functions (have default implementation, CAN be overridden)
//   - regular methods and fields (concrete, inherited as-is)

virtual class BaseProtocol;
    // Pure virtual function: no body -- derived class MUST implement
    pure virtual function void drive(logic [31:0] data);
    pure virtual function logic [31:0] read();

    // Regular virtual function: has default, CAN be overridden
    virtual function void reset();
        $display("BaseProtocol: generic reset");
    endfunction

    // Concrete method: same for all protocols (not overridable)
    function void log(string msg);
        $display("[PROTO] %s", msg);
    endfunction
```

```
endclass

// Derived classes MUST implement the pure virtual functions
class APBDriver extends BaseProtocol;
    virtual function void drive(logic [31:0] data);
        $display("APB drive: data=%0h", data);
        // ... APB-specific driving logic
    endfunction

    virtual function logic [31:0] read();
        logic [31:0] val;
        // ... APB read logic
        return val;
    endfunction
endclass

class AXIDriver extends BaseProtocol;
    virtual function void drive(logic [31:0] data);
        $display("AXI drive: data=%0h", data);
    endfunction

    virtual function logic [31:0] read();
        return 32'h0;
    endfunction
endclass

// ================================================================
// What you CANNOT do with a virtual class
// ================================================================
//
// BaseProtocol proto = new();   // COMPILE ERROR: cannot instantiate virtual class
//
// You can only instantiate the concrete derived classes:
APBDriver apb = new();   // OK
AXIDriver axi = new();   // OK
BaseProtocol handle = apb;  // OK: base class HANDLE can point to derived object

// ================================================================
// When to use virtual class vs regular class with virtual functions
// ================================================================
//
//   Use VIRTUAL CLASS (abstract) when:
//   1. The base class has NO meaningful standalone implementation.
//      (A "shape" has no area by itself; only Circle/Rectangle have area)
//   2. You want to ENFORCE that all subclasses implement certain methods.
//      pure virtual = compiler error if subclass forgets to implement.
//   3. You want to prevent accidental instantiation of the base class.
//
//   Use REGULAR CLASS with virtual functions when:
//   1. The base class has a useful DEFAULT implementation.
//      (A default driver that subclasses may or may not override)
//   2. Instantiating the base class directly is sometimes valid.
//
//   In UVM:
//   uvm_object and uvm_component are effectively abstract base classes
//   (you always extend them, never instantiate directly).
//   uvm_sequence_item, uvm_sequence -- extend these with your own txn/seq.
```

解释

Virtual class（抽象类）和普通类加 virtual 方法的关键区别：

抽象类不能被实例化，且 `pure virtual` 方法强制子类必须实现——

如果子类忘了实现某个 pure virtual 方法，编译器直接报错，不会等到运行时才出问题。

选择时机：当"基类本身没有意义"时用 virtual class——

Shape 本身没有面积，BaseProtocol 本身没有驱动实现，

这种情况下允许 `new BaseProtocol()` 会是一个 bug（调用者会得到一个空壳对象）。

UVM 的实际应用：`uvm_sequence_item` 就像一个隐式的抽象基类——

你不会直接用 `uvm_sequence_item` 发送事务，总是继承它创建自己的 `my_txn`，

这就是 virtual class 设计思想的体现。

Q67. SystemVerilog: Blocking vs Non-Blocking Assignments -- Complete Guide

QUESTION

What is the difference between blocking (=) and non-blocking (<=)

assignments in Verilog/SystemVerilog? When should each be used?

ANSWER

```
// ================================================================
// Blocking assignment (=)
// ================================================================
//
// Executes IMMEDIATELY and SEQUENTIALLY within the always block.
// The next statement sees the UPDATED value right away.
// Analogous to a regular assignment in C/software.

always_comb begin   // correct use: combinational logic
    y = a & b;      // y immediately takes value of (a&b)
    z = y | c;      // z sees the NEW y (uses a&b, not old y)
end

// ================================================================
// Non-blocking assignment (<=)
// ================================================================
//
// Two-phase execution:
//   Phase 1 (Active region):   RHS is EVALUATED (reads current values)
//   Phase 2 (NBA region):      LHS is UPDATED (all simultaneously)
//
// Within one always block, ALL RHS are sampled BEFORE any LHS updates.
// This models flip-flop behavior: all inputs sampled at clock edge,
// all outputs updated simultaneously.

always_ff @(posedge clk) begin   // correct use: sequential logic
    a <= b;   // RHS: reads current b
    b <= a;   // RHS: reads current a (NOT the new a from line above!)
    // After NBA region: a gets old_b, b gets old_a -- perfect SWAP
end

// ================================================================
// The classic race condition example
// ================================================================
//
// WRONG (blocking in two concurrent always blocks):
always @(posedge clk) a = b;   // may run before or after block 2
always @(posedge clk) b = a;   // result DEPENDS on execution order
// -> Non-deterministic! Either a=b,b=b OR a=a,b=a depending on simulator
```

```
// CORRECT (non-blocking):
always_ff @(posedge clk) a <= b;   // both RHS sampled before any update
always_ff @(posedge clk) b <= a;   // -> always swaps correctly

// ================================================================
// The swap example (within one always block)
// ================================================================
//
// WRONG: blocking does NOT swap
always_ff @(posedge clk) begin
    a = b;   // a now = b (immediately)
    b = a;   // b = new_a = b (not the original a!)
    // result: a = b, b = b  -- NO SWAP
end

// CORRECT: non-blocking swaps correctly
always_ff @(posedge clk) begin
    a <= b;  // RHS = old b
    b <= a;  // RHS = old a
    // NBA update: a = old_b, b = old_a  -- CORRECT SWAP
end

// ================================================================
// Rules of thumb (golden rules)
// ================================================================
//
//   always_ff  (sequential): ALWAYS use <=
//   always_comb (combinational): ALWAYS use =
//   Never mix = and <= for the same variable
//   Never use = across concurrent always blocks for shared variables

// ================================================================
// SV Scheduling regions summary
// ================================================================
//
//   Time step:  Active -> NBA -> Observed -> Reactive
//   = fires in Active region (immediate)
//   <= RHS evaluated in Active; LHS update applied in NBA region
```

解释

非阻塞赋值的"两阶段"是理解 Verilog 调度模型的核心：

所有 always_ff 块在同一时刻（posedge clk）被触发，

它们的 RHS 在 Active 区全部求值（此时读的是旧值），

然后在 NBA 区统一更新 LHS——不管这些 always 块按什么顺序执行，结果都相同。

这完美模拟了真实触发器的行为：时钟沿到来时所有触发器同时采样，同时更新输出。

阻塞赋值的问题：在同一时刻触发的两个不同 always 块里，执行顺序由 simulator 决定，

IEEE 标准不规定这个顺序——这就是竞争条件（race condition）的根源。

Q68. SystemVerilog: Sequence Detector 10110 -- Moore / Mealy / Shift Register

QUESTION

Implement a sequence detector for the pattern "10110" using:

(a) Moore FSM

(b) Mealy FSM

(c) Shift register method

ANSWER

```
// ================================================================
// (a) Moore FSM: output depends only on current state
// ================================================================
//
// States track progress through "10110":
//   S0: initial / reset
//   S1: seen "1"
//   S2: seen "10"
//   S3: seen "101"
//   S4: seen "1011"
//   S5: seen "10110" -> output=1 (detected!)
//
// State transition table:
//   State | Input=0 | Input=1 | Output
//   ------+---------+---------+-------
//   S0    |  S0     |  S1     |  0
//   S1    |  S2     |  S1     |  0
//   S2    |  S0     |  S3     |  0
//   S3    |  S0     |  S4     |  0
//   S4    |  S5     |  S1     |  0
//   S5    |  S0     |  S1     |  1   <- detection

module seq_detect_moore (
    input  logic clk, rst_n, din,
    output logic detected
);
    typedef enum logic [2:0] {
        S0=0, S1=1, S2=2, S3=3, S4=4, S5=5
    } state_e;

    state_e state, next;

    // State register
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) state <= S0;
        else        state <= next;

    // Next-state logic
    always_comb begin
        unique case (state)
            S0: next = din ? S1 : S0;
            S1: next = din ? S1 : S2;
            S2: next = din ? S3 : S0;
            S3: next = din ? S4 : S0;
            S4: next = din ? S1 : S5;
            S5: next = din ? S1 : S0;
            default: next = S0;
        endcase
    end

    // Output (Moore: depends only on state)
    assign detected = (state == S5);

endmodule

// ================================================================
// (b) Mealy FSM: output depends on state AND input
// ================================================================
//
```

```
// Advantage: one fewer state, output asserted ONE CYCLE EARLIER than Moore.
// Output fires on the TRANSITION that completes the pattern, not after.
//
// States:
//   S0: initial
//   S1: seen "1"
//   S2: seen "10"
//   S3: seen "101"
//   S4: seen "1011"
//   (no S5: output fires on transition S4 -> S0 when din=0)
//
// State transition and output:
//   State | Input=0       | Input=1
//   ------+---------------+---------
//   S0    | S0 / out=0    | S1 / out=0
//   S1    | S2 / out=0    | S1 / out=0
//   S2    | S0 / out=0    | S3 / out=0
//   S3    | S0 / out=0    | S4 / out=0
//   S4    | S0 / out=1    | S1 / out=0  <- output on din=0 (completes "10110")

module seq_detect_mealy (
    input  logic clk, rst_n, din,
    output logic detected
);
    typedef enum logic [2:0] {S0,S1,S2,S3,S4} state_e;
    state_e state, next;

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) state <= S0;
        else        state <= next;

    // Combined next-state and output (Mealy)
    always_comb begin
        next     = S0;
        detected = 1'b0;
        unique case (state)
            S0: if (din) next = S1; else next = S0;
            S1: if (din) next = S1; else next = S2;
            S2: if (din) next = S3; else next = S0;
            S3: if (din) next = S4; else next = S0;
            S4: if (din) begin next = S1; end
                else     begin next = S0; detected = 1'b1; end
            default: next = S0;
        endcase
    end
endmodule

// ============================================================
// (c) Shift Register method: simplest implementation
// ============================================================
//
// Shift din into a 5-bit register every clock.
// Compare the register content against the target pattern.
// No state machine needed!

module seq_detect_shift (
    input  logic clk, rst_n, din,
    output logic detected
);
    logic [4:0] shift_reg;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
```

```
            shift_reg <= 5'b0;
        else
            shift_reg <= {shift_reg[3:0], din};  // shift left, insert din
    end

    // Detect when the last 5 bits match "10110"
    assign detected = (shift_reg == 5'b10110);

endmodule

// ================================================================
// Waveform trace for input "...10110..."
// ================================================================
//
//   Cycle | din | shift_reg | detected
//   ------+-----+-----------+---------
//     0   |  0  | 00000     |   0
//     1   |  1  | 00001     |   0
//     2   |  0  | 00010     |   0
//     3   |  1  | 00101     |   0
//     4   |  1  | 01011     |   0
//     5   |  0  | 10110     |   1  <- DETECTED (shift_reg == 5'b10110)
//     6   |  ?  | 1011?     |   0

// ================================================================
// Comparison: Moore vs Mealy vs Shift Reg
// ================================================================
//
//   Method      | States | Output latency | Overlap detection | Code complexity
//   ------------+--------+----------------+-------------------+----------------
//   Moore       |   6    | 1 cycle late   | Handled by FSM    | Medium
//   Mealy       |   5    | Same cycle     | Handled by FSM    | Medium
//   Shift Reg   |   N/A  | Same cycle     | Natural           | Simple
//
// Shift register is the simplest and handles overlapping sequences
// automatically (no special state needed for "101101" containing two
// overlapping occurrences of parts of "10110").
// FSM approaches are better when the pattern has complex overlap handling
// or when state meaning needs to be explicit for verification.
```

解释

三种方法的核心区别：

Moore FSM 的输出比 Mealy 晚一拍（输出在"检测到的下一个时钟"才出现），

因为输出只取决于状态，而状态在下一拍才更新。

Mealy FSM 减少了一个状态（检测状态 S5 被合并为 S4 的一个 transition output），

输出更及时，但组合路径更长（output = f(state, input)）。

移位寄存器是最简洁且最可靠的方法：不需要手工设计状态转移图，

天然支持重叠序列（overlapping pattern），且综合工具可以高效优化比较逻辑。

实际 DV 工作中，序列检测通常用 SVA `a ##1 b ##1 c ##1 d ##1 e` 来写，

比任何 FSM 都更简洁，面试时主动提出这个观点是加分项。

Q69. SystemVerilog: C++ Pointer Swap -- ref vs value, const pointer vs pointer to const

QUESTION

(a) Write a C++ function to swap two integer pointers.

Should the parameters be passed by reference or by value? Why?

(b) What is the difference between const int* val and int* const val?

ANSWER

```
// ================================================================
// (a) Swap integer pointers -- pass by value vs pass by reference
// ================================================================

// --- Version 1: pass pointers by VALUE (WRONG for swapping) ---
void swap_ptr_byval(int* a, int* b) {
    int* temp = a;
    a = b;
    b = temp;
    // This swaps LOCAL copies of a and b.
    // The original pointers in the CALLER are unchanged!
}

// Why it fails:
//   int x=1, y=2;
//   int* p = &x;   int* q = &y;
//   swap_ptr_byval(p, q);
//   // p still points to x, q still points to y -- NO SWAP!

// --- Version 2: pass pointers by REFERENCE (CORRECT) -----------
void swap_ptr_byref(int*& a, int*& b) {
    //             ^^ reference to pointer
    int* temp = a;
    a = b;
    b = temp;
    // This modifies the ACTUAL pointers in the caller.
}

// Correct usage:
//   int x=1, y=2;
//   int* p = &x;   int* q = &y;
//   swap_ptr_byref(p, q);
//   // Now p points to y, q points to x -- SWAPPED!

// --- Version 3: pass pointer-to-pointer (C style, no references) ---
void swap_ptr_pp(int** a, int** b) {
    int* temp = *a;
    *a = *b;
    *b = temp;
}
// Call: swap_ptr_pp(&p, &q);  -- must pass address of the pointer

// ================================================================
// Why reference is preferred in C++
// ================================================================
//
// By VALUE: a copy of the pointer is passed.
//   -> Changing a/b inside the function only changes the local copy.
//   -> Original pointers in caller are untouched. WRONG for swap.
//
// By REFERENCE (int*& a): the parameter IS the caller's pointer.
//   -> Changing a inside the function changes the caller's pointer.
//   -> The reference itself cannot be reseated (unlike a pointer).
//   -> Cleaner syntax than pointer-to-pointer (no & at call site).

// ================================================================
// (b) const int* val vs int* const val
// ================================================================
```

```
//
// Read declarations RIGHT TO LEFT ("clockwise/spiral rule"):
//
// const int* val    (or equivalently: int const* val)
//   = "val is a pointer to const int"
//   -> The VALUE pointed to is const (cannot change *val)
//   -> The POINTER itself is not const (can change val to point elsewhere)
//
//   const int x = 5, y = 10;
//   const int* val = &x;
//   *val = 20;      // ERROR: cannot modify the int through this pointer
//   val  = &y;      // OK:    val can point to a different const int

// int* const val
//   = "val is a const pointer to int"
//   -> The POINTER is const (cannot make val point elsewhere)
//   -> The VALUE pointed to is NOT const (can change *val)
//
//   int x = 5, y = 10;
//   int* const val = &x;
//   *val = 20;      // OK:    can modify the int (x becomes 20)
//   val  = &y;      // ERROR: cannot reseat the pointer

// const int* const val
//   = "val is a const pointer to const int"
//   -> BOTH the pointer and the value are const
//   *val = 20;      // ERROR
//   val  = &y;      // ERROR

// ================================================================
// Memory aid: read from RIGHT TO LEFT
// ================================================================
//
//   int* const val:    val is [const] [pointer to] [int]
//                      const pointer, non-const int
//
//   const int* val:    val is [pointer to] [const int]
//                      non-const pointer, const int
//
//   Mnemonic: "const on the LEFT of * = data is const"
//             "const on the RIGHT of * = pointer is const"
```

解释

指针 swap 的经典错误：传值只拷贝了指针本身的值（地址），

函数内改变这个副本对外部没有效果，就像在函数里 `x = 5` 不会改变外部的 `x` 一样。

`int*& a` 读法："a 是一个引用（&），它引用的东西是 int*（指向 int 的指针）"——

这是 C++ 最令人困惑的声明语法之一，读对了是面试亮点。

`const int*` vs `int* const` 的判断规则：看 const 在 `*` 的左边还是右边：

在左边（const 修饰 int）→ 指向的值不可变；在右边（const 修饰指针变量本身）→ 指针不可变。

`const int* const val` 是最严格的形式，两者都不可变，常用于函数参数中防止意外修改。

Verification -- Test Plan

Q70. Verification: FIFO Verification -- What to Verify, How, Corner Cases

QUESTION

How do you verify a FIFO? List what properties need to be verified, how to implement the verification environment, and what corner cases to cover (including interrupt-related behavior).

ANSWER

```
// ================================================================
// Properties to verify
// ================================================================
//
// Functional correctness:
//   F1. Data integrity: data read must equal data written (FIFO ordering)
//   F2. FIFO order (First-In-First-Out): read order matches write order
//   F3. Full flag: asserted exactly when all entries are occupied
//   F4. Empty flag: asserted exactly when no entries are present
//   F5. Count / occupancy: accurately reflects the number of valid entries
//   F6. Simultaneous push and pop: count is unchanged, data flows correctly
//
// Safety properties (bad things that must NEVER happen):
//   S1. No underflow: cannot pop from an empty FIFO
//   S2. No overflow: cannot push into a full FIFO (data must not be lost
//       or corrupt existing entries)
//   S3. No data corruption: entries already in the FIFO must not change
//       unless explicitly popped
//
// Reset behavior:
//   R1. After reset, FIFO is empty, full=0, empty=1, count=0
//   R2. Reset can occur at any time (mid-push, mid-pop, when full/empty)
//       and must produce a clean, consistent state

// ================================================================
// Verification environment (UVM scoreboard pattern)
// ================================================================
//
// class fifo_scoreboard extends uvm_scoreboard;
//     uvm_analysis_imp_push #(fifo_txn, fifo_scoreboard) imp_push;
//     uvm_analysis_imp_pop  #(fifo_txn, fifo_scoreboard) imp_pop;
//
//     fifo_txn expected_q[$];   // shadow queue mimicking DUT behavior
//
//     function void write_push(fifo_txn txn);
//         expected_q.push_back(txn.data);    // model the push
//     endfunction
//
//     function void write_pop(fifo_txn txn);
//         if (expected_q.size() == 0) begin
//             `uvm_error("SCB", "Pop on empty FIFO -- underflow!")
//             return;
//         end
//         fifo_txn exp = expected_q.pop_front();
//         if (txn.data !== exp.data)
//             `uvm_error("SCB", $sformatf("Data mismatch: got=%h exp=%h",
//                        txn.data, exp.data))
//     endfunction
//
//     function void check_phase(uvm_phase phase);
//         if (expected_q.size() > 0)
//             `uvm_error("SCB", $sformatf("%0d entries never read",
//                        expected_q.size()))
```

```
//     endfunction
// endclass

// ============================================================
// Corner cases to cover
// ============================================================
//
// Boundary / capacity:
//   C1. Fill to exactly full (depth entries written, depth+1 would overflow)
//   C2. Drain to exactly empty (all entries read, one more would underflow)
//   C3. Fill then drain completely (full fill-drain cycle)
//   C4. Single entry: push one, pop one (minimum occupancy)
//   C5. Back-to-back push+pop at depth-1 (1 slot before full): simultaneous
//
// Simultaneous push and pop:
//   C6. Push and pop on same cycle when FIFO has 1 entry (not full, not empty)
//   C7. Push and pop on same cycle when FIFO is FULL (pop makes room,
//       push should succeed; data should flow correctly)
//   C8. Push and pop on same cycle when FIFO is EMPTY (pop should be
//       ignored/flagged as error; push should succeed)
//
// Flag accuracy:
//   C9.  Transition full->not-full on exactly one pop (flag must deassert)
//   C10. Transition empty->not-empty on exactly one push (flag must deassert)
//   C11. Count accuracy across many rapid push/pop cycles
//
// Reset:
//   C12. Reset when FIFO is full (must clear all entries)
//   C13. Reset while simultaneously pushing (which wins?)
//   C14. Reset while simultaneously popping (which wins?)
//   C15. Repeated reset (no latch-up or corruption after multiple resets)
//
// Interrupt-related corner cases (for systems where FIFO feeds an interrupt):
//   C16. Interrupt fires when FIFO reaches a programmable threshold level
//        (e.g. "interrupt when count >= DEPTH/2") -- verify threshold crossing
//        triggers interrupt on the CORRECT cycle, not one cycle early/late
//   C17. Interrupt clears correctly when FIFO drains below threshold
//        (level-sensitive interrupt: must deassert when condition goes away)
//   C18. Interrupt PENDING during a reset: does reset clear the interrupt flag?
//   C19. Rapid push/pop causing the interrupt flag to toggle quickly --
//        verify no missed interrupts or spurious double-firings
//   C20. ISR (interrupt service routine) reads data from FIFO while new data
//        is still being pushed -- read-while-write collision coverage
```

解释

FIFO 验证的层次：功能正确性（数据完整性+顺序）-> 安全属性（不溢出/下溢）-> 复位行为。

Scoreboard 用 shadow queue（软件 FIFO 模型）追踪期望值：

push 时往 shadow queue 里放数据，pop 时从 shadow queue 取出预期值比对。

中断相关的 corner case 是这道题的扩展亮点：

FIFO 的阈值中断（half-full、almost-full 等）要求精确的时序——

中断必须在阈值被穿越的那一拍触发，而不是延迟一拍，

这种"edge vs level"语义的验证和普通的功能覆盖完全不同，

需要专门设计的定向测试来压制特定的阈值穿越场景，

并配合时序断言（SVA）验证中断触发的精确时钟周期。

Q71. Verification: Systolic Array UVM Testbench -- PE Level and Sub-System Level

QUESTION

Design a UVM testbench to verify a Systolic Array for matrix multiplication

C = A x B. The design has two levels:

Level 1: systolic_array module (NxN PE mesh, single sr_clk).

Level 2: sub_sys module (adds input/output FIFOs, dual-clock: sys_clk for

data I/O, sr_clk for computation, double-buffered weight storage).

Both levels must share UVM components as much as possible.

ANSWER

```
// ================================================================
// Background: Processing Element (PE)
// ================================================================
//
//   Each PE has:
//     Inputs:  a (activation, flows RIGHT to next PE in same row)
//              b (weight, preloaded, stays in PE)
//              cin (partial sum from PE ABOVE)
//     Outputs: a_out (forward a to the right)
//              cout = a * b + cin  (partial sum, passes DOWN to next row)
//
//   RTL sketch for a single PE:
//   module pe (input clk, rst_n, input [W-1:0] a, b, cin,
//              output reg [W-1:0] a_out, output reg [2W-1:0] cout);
//       always_ff @(posedge clk or negedge rst_n)
//           if (!rst_n) begin a_out<=0; cout<=0; end
//           else        begin a_out<=a; cout<= a*b + cin; end
//   endmodule

// ================================================================
// Timing: 2x2 systolic array compute cycles
// ================================================================
//
//   B is preloaded into each PE (b_ij in PE(i-1, j-1), 0-indexed).
//   A columns are fed in STAGGERED: row r starts at cycle r.
//   Partial sums flow DOWN; after 2N-1 cycles all C outputs appear.
//
//   Cycle 1: PE(0,0) computes a11*b11  (a11 enters row 0)
//   Cycle 2: PE(0,0) gets a21; PE(0,1) gets a11 (forwarded right)
//            PE(1,0) receives a11*b11 as cin, adds a12*b21 -> C11 out!
//   Cycle 3: PE(1,1) outputs C12, PE(1,0) outputs C21
//   Cycle 4: PE(1,1) outputs C22
//   Total output cycles: 2N-1 = 3 (for N=2)

// ================================================================
// Double buffer: preload next B while computing current C
// ================================================================
//
//   Problem: loading B takes N cycles -> systolic array sits idle.
//   Solution: each PE has TWO weight registers.
//     - Active register:  used for current computation
//     - Shadow register:  being loaded with next B matrix
//   After each C output, swap active/shadow -> zero idle time.

// ================================================================
// LEVEL 1: systolic_array module UVM testbench
// ================================================================
```

```
// Transaction class (shared between both levels)
class sa_txn extends uvm_sequence_item;
    `uvm_object_utils_begin(sa_txn)
        `uvm_field_int(N,       UVM_DEFAULT)
        `uvm_field_int(in_valid, UVM_DEFAULT)
        `uvm_field_sda(a_din,   UVM_DEFAULT)
        `uvm_field_sda(b_din,   UVM_DEFAULT)
        `uvm_field_int(out_valid, UVM_DEFAULT)
        `uvm_field_sda(c_dout,  UVM_DEFAULT)
    `uvm_object_utils_end

    parameter int DIN_W = 8;
    rand logic signed [DIN_W-1:0] a_din[];   // N elements of A column
    rand logic signed [DIN_W-1:0] b_din[];   // N elements of B row
    rand logic                    in_valid;
    logic [2*DIN_W-1:0]           c_dout[];  // output C column
    logic                         out_valid;
    int unsigned                  N;

    // Constraint: valid matrix elements, signed range
    constraint c_data_range {
        foreach (a_din[i]) a_din[i] inside {[-64:63]};
        foreach (b_din[i]) b_din[i] inside {[-64:63]};
    }
endclass

// Sequence: stream N columns of A and N rows of B, staggered
class sa_matmul_seq extends uvm_sequence #(sa_txn);
    `uvm_object_utils(sa_matmul_seq)

    parameter int N = 2;
    rand logic signed [7:0] A [N][N];   // full A matrix
    rand logic signed [7:0] B [N][N];   // full B matrix (preloaded to PEs)

    task body();
        sa_txn txn;
        // Feed A columns one per cycle; B is already preloaded
        // Row r of A is staggered by r cycles
        for (int cycle = 0; cycle < 2*N-1; cycle++) begin
            txn = sa_txn::type_id::create("txn");
            start_item(txn);
            assert(txn.randomize() with {
                // A input: column index = cycle (staggered)
                foreach (txn.a_din[r])
                    txn.a_din[r] = (cycle >= r && cycle-r < N) ?
                                   A[r][cycle-r] : 0;
                txn.in_valid = (cycle == N-1);   // valid on last A column
            });
            finish_item(txn);
        end
    endtask
endclass

// Driver: drives a_din, b_din (during preload), in_valid
class sa_driver extends uvm_driver #(sa_txn);
    `uvm_component_utils(sa_driver)
    virtual sa_if vif;

    task run_phase(uvm_phase phase);
        forever begin
            sa_txn txn;
            seq_item_port.get_next_item(txn);
```

```
            @(posedge vif.clk);
            vif.a_din    <= txn.a_din;
            vif.in_valid <= txn.in_valid;
            seq_item_port.item_done();
        end
    endtask
endclass

// Monitor: captures outputs when out_valid is high
class sa_monitor extends uvm_monitor;
    `uvm_component_utils(sa_monitor)
    virtual sa_if          vif;
    uvm_analysis_port #(sa_txn) ap;

    task run_phase(uvm_phase phase);
        forever begin
            @(posedge vif.clk);
            if (vif.out_valid) begin
                sa_txn txn = sa_txn::type_id::create("txn");
                txn.c_dout    = vif.c_dout;
                txn.out_valid = 1;
                ap.write(txn);
            end
        end
    endtask
endclass

// Scoreboard: golden model computes C = A*B, compares with DUT output
class sa_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(sa_scoreboard)
    uvm_analysis_imp #(sa_txn, sa_scoreboard) imp;

    // Golden model: compute C = A*B in software
    function automatic void compute_golden(
        input  logic signed [7:0]  A[][], B[][],
        output logic signed [15:0] C[][]
    );
        int N = A.size();
        foreach (C[i,j]) begin
            C[i][j] = 0;
            for (int k = 0; k < N; k++)
                C[i][j] += A[i][k] * B[k][j];
        end
    endfunction

    function void write(sa_txn txn);
        // Compare txn.c_dout against golden C column
        // (column index tracked by a counter)
        // If mismatch: `uvm_error
    endfunction
endclass

// ============================================================
// LEVEL 2: sub_sys -- dual-clock, FIFO-based wrapper
// ============================================================
//
// Additional challenges vs Level 1:
//   - TWO clocks: sys_clk (data I/O via FIFO) and sr_clk (compute)
//   - Input FIFO: write via wr_fifo on sys_clk; FIFO feeds systolic array on sr_clk
//   - Output FIFO: systolic array writes on sr_clk; read via rd_fifo on sys_clk
//   - in_fifo_full / out_fifo_empty flags
//   - M_minus_one[7:0] controls NxM matrix size
//   - Double-buffered weight storage: preload B while computing C
```

```
// Sub-system transaction (extends sa_txn, adds FIFO control)
class sub_sys_txn extends sa_txn;
    `uvm_object_utils(sub_sys_txn)

    rand logic [BUS_W-1:0] din;   // packed A column + B row on bus
    logic      wr_fifo;
    logic      rd_fifo;
    logic      in_fifo_full;
    logic      out_fifo_empty;
    logic [7:0] M_minus_one;
endclass

// Sub-system driver: write data to input FIFO on sys_clk,
// poll in_fifo_full before writing
class sub_sys_driver extends uvm_driver #(sub_sys_txn);
    `uvm_component_utils(sub_sys_driver)
    virtual sub_sys_if vif;

    task run_phase(uvm_phase phase);
        forever begin
            sub_sys_txn txn;
            seq_item_port.get_next_item(txn);
            // Wait for FIFO to not be full
            @(posedge vif.sys_clk);
            while (vif.in_fifo_full) @(posedge vif.sys_clk);
            vif.din     <= txn.din;
            vif.wr_fifo <= 1;
            @(posedge vif.sys_clk);
            vif.wr_fifo <= 0;
            seq_item_port.item_done();
        end
    endtask
endclass

// Sub-system monitor: read from output FIFO on sys_clk when not empty
class sub_sys_monitor extends uvm_monitor;
    `uvm_component_utils(sub_sys_monitor)
    virtual sub_sys_if vif;
    uvm_analysis_port #(sub_sys_txn) ap;

    task run_phase(uvm_phase phase);
        forever begin
            @(posedge vif.sys_clk);
            if (!vif.out_fifo_empty) begin
                vif.rd_fifo <= 1;
                @(posedge vif.sys_clk);
                sub_sys_txn txn = sub_sys_txn::type_id::create("txn");
                txn.din = vif.dout;
                ap.write(txn);
                vif.rd_fifo <= 0;
            end
        end
    endtask
endclass

// ================================================================
// Testbench reuse strategy (Level 1 -> Level 2)
// ================================================================
//
//   Reused without change:
//     sa_txn               -- core transaction fields (A, B, C data)
//     sa_scoreboard        -- golden model compute_golden() function
```

```
//                            (Level 2 scoreboard extends Level 1)
//     sa_matmul_seq logic  -- A staggering logic is identical
//
//   Extended for Level 2:
//     sub_sys_txn          extends sa_txn  (adds FIFO control fields)
//     sub_sys_driver       extends sa_driver (adds FIFO full polling)
//     sub_sys_monitor      new (reads on sys_clk, not sr_clk)
//     sub_sys_agent        wraps sub-system driver+monitor
//     sub_sys_scoreboard   extends sa_scoreboard (adds FIFO ordering)
//
//   Key difference: dual-clock domain crossing
//     sys_clk drives FIFO read/write
//     sr_clk  drives systolic array compute
//     Scoreboard must account for latency from FIFO -> compute -> FIFO:
//       total_latency = (N cycles to fill input FIFO)
//                     + (2N-1 cycles to compute on sr_clk)
//                     + (output FIFO propagation)
//     Use event-based ordering in scoreboard, not fixed-cycle comparison.

// ================================================================
// Corner cases for verification plan
// ================================================================
//
//   Module level (sa_matmul):
//     C1. 1x1 matrix (N=1): simplest case, cout = a*b
//     C2. All-zero A or B: result must be zero matrix
//     C3. Identity matrix: A*I = A, I*B = B
//     C4. Overflow: max signed values, verify 2*DIN_W output handles it
//     C5. Back-to-back matrix multiplications (double-buffer toggle)
//     C6. Reset mid-computation: output should be 0 after reset
//
//   Sub-system level (sub_sys):
//     C7.  Input FIFO full: backpressure handling, wr_fifo ignored
//     C8.  Output FIFO empty: rd_fifo when empty, dout should be 0
//     C9.  sys_clk >> sr_clk: FIFO fills fast, compute is bottleneck
//     C10. sys_clk << sr_clk: compute drains FIFO, possible starve
//     C11. M_minus_one=0 (M=1): NxN single column case
//     C12. M_minus_one=max (M=256): stress long-running computation
//     C13. Consecutive multiplications: verify double-buffer continuity
//     C14. Reset during FIFO fill: output FIFO must clear, counters reset
```

解释

这道题是完整的 UVM 环境设计题，考察三个层次的能力：

1. 理解硬件：PE 的 cout = a*b + cin，A 从左向右流动，B 预加载（或从上向下流动），

部分和从上往下累积——N=2 时需要 2N-1=3 个计算周期产生全部 C 输出。

2. UVM 架构：transaction、driver、monitor、scoreboard 各司其职；

golden model（software C = A*B）放在 scoreboard 里做自动比对。

3. 复用策略：Level 2（sub_sys）继承 Level 1 的 scoreboard 和 transaction，

只扩展 FIFO 相关的 driver/monitor，避免重复实现 golden model。

双时钟域（sys_clk vs sr_clk）是 Level 2 的核心挑战：

scoreboard 不能用固定延迟比较，必须基于 out_valid/out_fifo_empty 信号

做事件驱动的输出捕获，才能正确处理任意时钟比例下的输出时序。

Double buffer 验证的关键 corner case：

第一个 C 输出的最后一列（out_valid 最后一拍）和第二个 B 矩阵开始写入

在时间上应该是重叠的——验证这个重叠窗口是否导致数据污染是最重要的功能点。

Q72. Verification: Task vs Function in SystemVerilog Testbench

QUESTION

In a UVM testbench, when do you use a task vs a function?

What are the key differences?

ANSWER

```
// ================================================================
// Function
// ================================================================
//
// - Executes in ZERO simulation time (no time-consuming statements).
// - CANNOT contain: @(event), #delay, wait(), task calls.
// - CAN return a value (or void).
// - Called instantly -- no time passes in simulation.
//
// Use function for:
//   - Data computation / transformation (e.g. CRC calculation)
//   - Scoreboard comparison logic
//   - Constraint helpers / coverage utilities
//   - build_phase, connect_phase, end_of_elaboration_phase (must be functions)
//   - Callbacks that UVM calls as functions (write(), check(), report())
//
// Example:
function int compute_crc(logic [31:0] data);
    return data ^ 32'hDEADBEEF;   // simplified
endfunction

function void check_output(my_txn actual, my_txn expected);
    if (actual.data !== expected.data)
        `uvm_error("SCB", $sformatf("Mismatch: got %h exp %h",
                   actual.data, expected.data))
endfunction

// ================================================================
// Task
// ================================================================
//
// - CAN consume simulation time.
// - CAN contain: @(event), #delay, wait(), other task calls.
// - CANNOT return a value (use output/inout args instead).
//
// Use task for:
//   - Driving the DUT interface (wait for clocks, apply stimulus)
//   - Waiting for events or signals
//   - run_phase (must be a task -- it runs over simulation time)
//   - Sequences (body() is a task)
//   - Any operation that requires clock synchronization
//
// Example:
task drive_packet(my_txn txn);
    @(posedge vif.clk);           // wait for clock -- needs a task
    vif.valid <= 1;
    vif.data  <= txn.data;
    @(posedge vif.clk);
    vif.valid <= 0;
endtask
```

```
// ================================================================
// Quick decision rule
// ================================================================
//
//   Does it need to wait for time (clock, event, delay)?
//     YES -> TASK
//     NO  -> FUNCTION (and it CAN'T consume time even if you try)
//
//   Does it need to return a value?
//     YES -> FUNCTION (tasks use output ports instead)
//
//   UVM phase callbacks:
//     build_phase, connect_phase, etc.     -> FUNCTION (void)
//     run_phase, main_phase, etc.          -> TASK (time-consuming)
```

解释

最简单的判断：有没有"等待时间的操作"（@event、#delay、wait()）——

有就必须用 task，没有就用 function（且 function 中不能调用 task）。

UVM Phase 的规律：静态配置阶段（build、connect、configure）都是 function，

因为它们在仿真时间 0 执行；运行阶段（run、main、shutdown）都是 task，

因为它们需要在时钟中运行。

常见错误：在 function 里调用了一个包含 @posedge 的 task，编译器报错

"time-consuming statement not allowed in function"。

解决方法：把需要等待时钟的代码移到 task 里。

Q73. Verification: 512-bit Transaction Split-Reassemble Over 64-bit Link

QUESTION

A 512-bit transaction is wider than a 64-bit interconnect DUT link.

How do you split it into 64-bit chunks at the driver, and reassemble

at the output side in the monitor/scoreboard?

ANSWER

```
// ================================================================
// Transaction class: 512-bit payload
// ================================================================
class wide_txn extends uvm_sequence_item;
    `uvm_object_utils(wide_txn)
    rand logic [511:0] payload;
    rand logic [15:0]  txn_id;     // unique transaction identifier
    // ... other fields
endclass

// ================================================================
// Driver: split 512b into 8 x 64b flits and drive sequentially
// ================================================================
class wide_driver extends uvm_driver #(wide_txn);
    `uvm_component_utils(wide_driver)
    virtual link_if vif;

    task drive_item(wide_txn txn);
```

```
        // Split: 512 / 64 = 8 flits
        for (int i = 0; i < 8; i++) begin
            @(posedge vif.clk);
            vif.data    <= txn.payload[i*64 +: 64];  // 64-bit slice
            vif.flit_id <= i[2:0];
            vif.txn_id  <= txn.txn_id;
            vif.last    <= (i == 7);   // last flit marker
            vif.valid   <= 1;
        end
        @(posedge vif.clk);
        vif.valid <= 0;
    endtask

    task run_phase(uvm_phase phase);
        forever begin
            wide_txn txn;
            seq_item_port.get_next_item(txn);
            drive_item(txn);
            seq_item_port.item_done();
        end
    endtask
endclass

// ============================================================
// Monitor: reassemble 8 x 64b flits back into 512b transaction
// ============================================================
class wide_monitor extends uvm_monitor;
    `uvm_component_utils(wide_monitor)
    virtual link_if            vif;
    uvm_analysis_port #(wide_txn) ap;

    // Partial reassembly buffer, keyed by txn_id
    logic [511:0] reassembly_buf [logic [15:0]];
    int           flit_count     [logic [15:0]];

    task run_phase(uvm_phase phase);
        forever begin
            @(posedge vif.clk);
            if (vif.valid) begin
                int flit = vif.flit_id;
                logic [15:0] id = vif.txn_id;
                // Insert this flit into the correct 64-bit slot
                reassembly_buf[id][flit*64 +: 64] = vif.data;
                flit_count[id]++;

                if (vif.last) begin
                    // All 8 flits received -- emit complete transaction
                    wide_txn txn = wide_txn::type_id::create("txn");
                    txn.payload = reassembly_buf[id];
                    txn.txn_id  = id;
                    ap.write(txn);
                    // Clean up
                    reassembly_buf.delete(id);
                    flit_count.delete(id);
                end
            end
        end
    endtask
endclass

// ============================================================
// Scoreboard: match reassembled output against expected (by txn_id)
// ============================================================
```

```
class wide_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(wide_scoreboard)
    // Associative array: expected transactions keyed by txn_id
    wide_txn expected_aa [logic [15:0]];

    function void write_input(wide_txn txn);
        expected_aa[txn.txn_id] = txn;
    endfunction

    function void write_output(wide_txn actual);
        if (!expected_aa.exists(actual.txn_id)) begin
            `uvm_error("SCB", $sformatf("Unexpected txn_id=%0h", actual.txn_id))
            return;
        end
        wide_txn exp = expected_aa[actual.txn_id];
        if (actual.payload !== exp.payload)
            `uvm_error("SCB", $sformatf("Payload mismatch txn_id=%0h", actual.txn_id))
        expected_aa.delete(actual.txn_id);
    endfunction
endclass
```

解释

拆分逻辑的关键：`payload[i*64 +: 64]` 是 SV 的 part-select 写法，

从 bit i*64 开始取 64 位，比手写 `[511:448]` 等静态范围更简洁且不易出错。

拼装逻辑：monitor 维护一个以 txn_id 为 key 的 associative array 作为缓冲区，

每收到一个 flit 就填到对应位置，收到 last 标志时 emit 完整 transaction。

txn_id 是贯穿整个流程的"线索"：driver 附在每个 flit 上，

monitor 用它来归并乱序到来的 flit（互联 DUT 可能重排 flit 顺序），

scoreboard 用它做期望值匹配。

`last` 标志是协议设计的关键：如果没有这个标志，monitor 必须计数 8 个 flit，

不够灵活（如果 payload 大小可变就需要额外的 flit 计数字段）。

Q74. Verification: Out-of-Order Scoreboard -- Track and Match with Associative Array

QUESTION

An interconnect DUT can return responses out of order (different from the request order). How do you design the scoreboard to correctly track and match expected vs actual responses?

ANSWER

```
// ================================================================
// Core design: associative array keyed by transaction ID
// ================================================================
//
// Instead of a FIFO queue (which assumes in-order responses),
// use an associative array where the KEY is the transaction ID.
// When a response arrives, look up by ID -- order doesn't matter.

class ooo_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(ooo_scoreboard)
```

```
    // Expected responses, indexed by txn_id
    // Key: unique transaction ID assigned by the driver
    // Value: the expected response computed from the request
    my_response expected_aa [int unsigned];

    // Track all in-flight transaction IDs to detect missing responses
    int unsigned in_flight_ids[$];

    // Called when a REQUEST is sent (from input monitor)
    function void write_request(my_request req);
        my_response exp = compute_expected(req);   // golden model
        expected_aa[req.txn_id] = exp;
        in_flight_ids.push_back(req.txn_id);
        `uvm_info("SCB", $sformatf("Expecting response for txn_id=%0d", req.txn_id), UVM_HIGH)
    endfunction

    // Called when a RESPONSE arrives (from output monitor)
    function void write_response(my_response actual);
        // Step 1: Check if we were expecting this transaction ID
        if (!expected_aa.exists(actual.txn_id)) begin
            `uvm_error("SCB", $sformatf(
                "UNEXPECTED response: txn_id=%0d never sent or already matched",
                actual.txn_id))
            return;
        end

        // Step 2: Retrieve the expected response and compare
        my_response exp = expected_aa[actual.txn_id];
        if (actual.data  !== exp.data ||
            actual.status !== exp.status) begin
            `uvm_error("SCB", $sformatf(
                "MISMATCH txn_id=%0d: got data=%0h status=%0h, exp data=%0h status=%0h",
                actual.txn_id, actual.data, actual.status, exp.data, exp.status))
        end else begin
            `uvm_info("SCB", $sformatf("PASS txn_id=%0d", actual.txn_id), UVM_MEDIUM)
        end

        // Step 3: Remove from expected map (transaction complete)
        expected_aa.delete(actual.txn_id);
    endfunction

    // check_phase: detect any requests that never got a response
    function void check_phase(uvm_phase phase);
        super.check_phase(phase);
        if (expected_aa.num() > 0) begin
            `uvm_error("SCB", $sformatf(
                "%0d transactions never received a response!", expected_aa.num()))
            foreach (expected_aa[id])
                `uvm_info("SCB", $sformatf("  Missing response: txn_id=%0d", id), UVM_NONE)
        end
    endfunction

    // Golden model: compute expected response from request
    function automatic my_response compute_expected(my_request req);
        my_response exp = my_response::type_id::create("exp");
        // ... application-specific computation
        exp.data   = req.data ^ req.key;   // example
        exp.status = 2'b00;                // OKAY
        return exp;
    endfunction
endclass
```

解释

乱序 scoreboard 的设计核心：把 FIFO queue 换成以 txn_id 为 key 的 associative array。

三步匹配流程：request 到来时计算期望值存入 AA；response 到来时按 ID
查找期望值比对；比对完毕删除该 ID。

check_phase 的"未响应检测"是完整性的关键：

如果某个 request 的 response 永远没来（DUT 丢了响应），

在 check_phase 里 AA 里还有这个 ID，就会报错——这是常规 FIFO scoreboard 做不到的检查。

txn_id 的唯一性保证需要在 driver 层维护一个全局递增计数器（`static int unsigned next_id`），

并在每次 start_item 时赋给 txn.txn_id，确保 ID 不重复。

Q75. Verification: Debug Scoreboard Mismatch -- Step-by-Step Approach

QUESTION

A test fails and the scoreboard reports a data mismatch.

Walk through your debug process step by step.

ANSWER

```
// ================================================================
// Step 1: Understand WHAT mismatched
// ================================================================
//
//   Read the error message carefully:
//   - Which field mismatched? (data, status, address, ID?)
//   - What was the expected value vs actual value?
//   - At what simulation time did it occur?
//   - Which transaction ID / sequence number?
//
//   Example message:
//   [SCB] MISMATCH txn_id=42: got data=0xDEAD exp data=0xBEEF @ 1025ns

// ================================================================
// Step 2: Trace the ACTUAL value backward through the DUT
// ================================================================
//
//   Open the waveform (FSDB/VCD) and search for time 1025ns:
//   a. Look at the DUT output port at that time: does vif.data == 0xDEAD?
//   b. Trace 0xDEAD backward:  output register -> pipeline stage ->
//      computation unit -> input register -> ... until you find where
//      it diverges from what you expect.
//   c. Did the DUT compute the wrong value, or is the monitor reading
//      at the wrong time (e.g. sampling on the wrong clock edge)?

// ================================================================
// Step 3: Verify the EXPECTED value is correct
// ================================================================
//
//   Question your golden model before blaming the DUT:
//   a. Is the scoreboard's compute_expected() function correct?
//      - Check edge cases: signed vs unsigned, overflow, byte ordering.
//   b. Does the monitor capture the right signals at the right time?
//      - Check: is it sampling on posedge or negedge? correct port?
//   c. Is the expected value stale? (scoreboard stored the request value
//      but the DUT processes it N cycles later with a different operand)
//   d. Is the ordering assumption correct? (maybe txn_id=42's response
//      arrived before txn_id=41 and we're matching the wrong item)
```

```
// ================================================================
// Step 4: Narrow down to DUT logic vs testbench bug
// ================================================================
//
//   Directed test: write the simplest possible test that reproduces it.
//   - Use a fixed (non-random) transaction to eliminate randomness.
//   - Hand-calculate the expected output.
//   - Drive JUST this one transaction and check the raw DUT output.
//
//   If the directed test passes -> the issue is in constraint/randomization
//   (the random stimulus was creating an illegal/unexpected input).
//   If the directed test fails  -> the issue is in DUT RTL logic.

// ================================================================
// Step 5: Check for common testbench bugs
// ================================================================
//
//   - Off-by-one cycle: monitor samples one cycle too early/late.
//   - Byte endianness swap in the monitor unpacking code.
//   - Scoreboard used queue (assumes in-order) but DUT reorders responses.
//   - Golden model doesn't handle reset state correctly.
//   - Multiple drivers: did something else write the interface?
//   - X propagation: DUT output was X (not 0/1) and monitor read it as 0.

// ================================================================
// Step 6: Fix and regression-test
// ================================================================
//
//   Once root cause found:
//   - Fix the DUT RTL or the testbench golden model.
//   - Add a DIRECTED test that specifically covers this bug pattern.
//   - Re-run full regression to confirm no new failures.
//   - Document: what was the bug? what was the fix? how was it caught?
```

解释

Debug 流程的关键原则：先不要假设是 DUT 的问题，也不要假设是 testbench 的问题——

系统地排查两种可能性。

Step 3（质疑 golden model）是最容易被跳过但最重要的步骤：

scoreboard 的 compute_expected() 经常有符号/无符号处理错误、字节序错误，

这些"testbench bug"在压力测试下才暴露，和 DUT 没有关系。

Step 4（定向测试）是最高效的分离手段：

把随机种子换成固定输入，排除随机性，让问题可重复，再用波形确认 DUT 行为。

"先看波形，后看代码"是调试顺序的最佳实践——

直接看 RTL 代码往往会陷入细节，波形上的异常模式通常能更快定位问题层次。

Q76. Verification: Router UVM Testbench -- Complete Design

QUESTION

Write a complete UVM testbench for an N-port router DUT.

The router receives packets on N input ports and routes each packet

to one or more output ports based on the packet header (destination field).

Each port uses an AXI-Stream-like interface: valid/ready handshake,

with last signal to mark end of packet.

ANSWER

```
// ================================================================
// DUT Assumption: N-port AXI-Stream router
// Input  port i: in_tvalid[i], in_tready[i], in_tdata[i], in_tdest[i],
//                in_tid[i], in_tlast[i]
// Output port j: out_tvalid[j], out_tready[j], out_tdata[j],
//                out_tid[j], out_tlast[j]
// AXI-Stream rule: once tvalid asserted, must hold until tready=1.
// ================================================================

// ================================================================
// 1. Transaction (Sequence Item)
// ================================================================
class router_txn extends uvm_sequence_item;
    `uvm_object_utils_begin(router_txn)
        `uvm_field_int    (tid,      UVM_DEFAULT)
        `uvm_field_int    (src_port, UVM_DEFAULT)
        `uvm_field_int    (dst_port, UVM_DEFAULT)
        `uvm_field_da_int (payload,  UVM_DEFAULT)
    `uvm_object_utils_end

    parameter int N        = 4;
    parameter int DATA_W   = 32;
    parameter int MAX_BEAT = 16;

    static int unsigned next_tid = 0;   // global unique ID counter

    rand bit [$clog2(N)-1:0] src_port;
    rand bit [$clog2(N)-1:0] dst_port;
    rand bit [DATA_W-1:0]    payload [];
    bit [7:0]                 tid;       // assigned in pre_randomize

    constraint c_len     { payload.size() inside {[1:MAX_BEAT]}; }
    constraint c_dst     { dst_port < N; src_port < N; }
    constraint c_no_loop { dst_port != src_port; }

    function void pre_randomize();
        tid = next_tid++;   // unique before each randomize call
    endfunction

    function new(string name = "router_txn");
        super.new(name);
    endfunction
endclass

// ================================================================
// 2. Sequence (base + single-packet + stress)
// ================================================================

// Base sequence: sends one randomized packet on a given src_port
class router_single_seq extends uvm_sequence #(router_txn);
    `uvm_object_utils(router_single_seq)

    int unsigned port;     // which input port this seq runs on
    int unsigned dst = 0;  // override destination (0 = random)

    function new(string name = "router_single_seq");
        super.new(name);
    endfunction
```

```
    task body();
        router_txn txn = router_txn::type_id::create("txn");
        start_item(txn);
        if (dst > 0)
            assert(txn.randomize() with { txn.src_port == port;
                                          txn.dst_port == dst; })
            else `uvm_fatal("SEQ","Randomize failed")
        else
            assert(txn.randomize() with { txn.src_port == port; })
            else `uvm_fatal("SEQ","Randomize failed")
        finish_item(txn);
    endtask
endclass

// Virtual sequencer for multi-port coordination
class router_vseqr extends uvm_sequencer;
    `uvm_component_utils(router_vseqr)
    // Handles to each port's real sequencer (set in env.connect_phase)
    uvm_sequencer #(router_txn) port_seqr[4];
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
endclass

// Stress sequence: drives all ports concurrently
class router_stress_seq extends uvm_sequence;
    `uvm_object_utils(router_stress_seq)
    `uvm_declare_p_sequencer(router_vseqr)

    int num_pkts = 20;

    function new(string name = "router_stress_seq");
        super.new(name);
    endfunction

    task body();
        fork
            drive_port(0); drive_port(1);
            drive_port(2); drive_port(3);
        join
    endtask

    task drive_port(int port);
        router_txn txn;
        repeat (num_pkts) begin
            txn = router_txn::type_id::create("txn");
            txn.port = port;
            // Start sub-sequence on THIS port's sequencer
            router_single_seq seq = router_single_seq::type_id::create("seq");
            seq.port = port;
            seq.start(p_sequencer.port_seqr[port]);
        end
    endtask
endclass

// ================================================================
// 3. Driver (one per input port)
// AXI-Stream rule: assert tvalid, THEN wait for tready.
// Once tvalid=1, hold it until the handshake (valid & ready) completes.
// ================================================================
class router_driver extends uvm_driver #(router_txn);
    `uvm_component_utils(router_driver)
```

```
    virtual router_if vif;
    int unsigned      port_id;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db #(virtual router_if)::get(
                this, "", "router_vif", vif))
            `uvm_fatal("DRV", "Cannot get router_vif")
        if (!uvm_config_db #(int)::get(
                this, "", "port_id", port_id))
            `uvm_fatal("DRV", "Cannot get port_id")
    endfunction

    task run_phase(uvm_phase phase);
        drive_idle();
        // Wait for reset release
        @(negedge vif.rst_n);   // rst_n goes low
        @(posedge vif.rst_n);   // rst_n released (active-low)
        @(posedge vif.clk);

        forever begin
            router_txn txn;
            seq_item_port.get_next_item(txn);
            drive_packet(txn);
            seq_item_port.item_done();
        end
    endtask

    task drive_packet(router_txn txn);
        foreach (txn.payload[i]) begin
            // AXI-Stream: assert valid + data first
            @(posedge vif.clk);
            vif.in_tvalid[port_id] <= 1;
            vif.in_tdata [port_id] <= txn.payload[i];
            vif.in_tdest [port_id] <= txn.dst_port;
            vif.in_tid   [port_id] <= txn.tid;
            vif.in_tlast [port_id] <= (i == txn.payload.size() - 1);

            // THEN wait for ready -- hold valid until handshake completes
            @(posedge vif.clk);
            while (!vif.in_tready[port_id]) begin
                // Keep valid/data held (AXI-S requirement)
                @(posedge vif.clk);
            end
            // Handshake done: valid & ready both 1, beat transferred
        end
        // Deassert after last beat
        drive_idle();
    endtask

    task drive_idle();
        vif.in_tvalid[port_id] <= 0;
        vif.in_tdata [port_id] <= '0;
        vif.in_tlast [port_id] <= 0;
        vif.in_tdest [port_id] <= '0;
        vif.in_tid   [port_id] <= '0;
    endtask
endclass

// ============================================================
// 4. Monitor (parameterized: input side OR output side)
// ============================================================
// mode=INPUT:  monitors in_tvalid/ready/data/last (what DUT accepts)
```

```
// mode=OUTPUT: monitors out_tvalid/ready/data/last (what DUT emits)
typedef enum {MONITOR_INPUT, MONITOR_OUTPUT} mon_mode_e;

class router_monitor extends uvm_monitor;
    `uvm_component_utils(router_monitor)

    virtual router_if              vif;
    int unsigned                   port_id;
    mon_mode_e                     mode;
    uvm_analysis_port #(router_txn) ap;

    // Packet reassembly
    router_txn   cur_pkt;
    bit          assembling;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        ap = new("ap", this);   // <<< must instantiate ap here
        if (!uvm_config_db #(virtual router_if)::get(
                this, "", "router_vif", vif))
            `uvm_fatal("MON", "Cannot get router_vif")
    endfunction

    task run_phase(uvm_phase phase);
        assembling = 0;
        forever begin
            @(posedge vif.clk);
            capture_beat();
        end
    endtask

    task capture_beat();
        bit   valid, ready, last;
        bit [31:0] data;
        bit [7:0]  tid_val;
        bit [1:0]  dst_val;

        // Select signals based on mode
        if (mode == MONITOR_INPUT) begin
            valid   = vif.in_tvalid[port_id];
            ready   = vif.in_tready[port_id];
            data    = vif.in_tdata [port_id];
            last    = vif.in_tlast [port_id];
            tid_val = vif.in_tid   [port_id];
            dst_val = vif.in_tdest [port_id];
        end else begin
            valid   = vif.out_tvalid[port_id];
            ready   = vif.out_tready[port_id];
            data    = vif.out_tdata [port_id];
            last    = vif.out_tlast [port_id];
            tid_val = vif.out_tid   [port_id];
            dst_val = port_id;   // output port IS the destination
        end

        // Handshake: beat transferred only when both valid AND ready
        if (valid && ready) begin
            if (!assembling) begin
                cur_pkt          = router_txn::type_id::create("pkt");
                cur_pkt.tid      = tid_val;
                cur_pkt.dst_port = dst_val;
                cur_pkt.src_port = (mode == MONITOR_INPUT) ? port_id : '0;
                assembling       = 1;
            end
```

```
            // Append this beat
            cur_pkt.payload = new[cur_pkt.payload.size() + 1](cur_pkt.payload);
            cur_pkt.payload[$] = data;

            if (last) begin
                ap.write(cur_pkt);   // emit complete packet
                assembling = 0;
            end
        end
    endtask
endclass

// ============================================================
// 5. Scoreboard
// ============================================================
// Uses uvm_analysis_imp macros to have two write() methods:
//   write_input  <- from input monitors (feeds expected queue)
//   write_output <- from output monitors (does comparison)
//
`uvm_analysis_imp_decl(_input)
`uvm_analysis_imp_decl(_output)

class router_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(router_scoreboard)

    uvm_analysis_imp_input  #(router_txn, router_scoreboard) imp_in;
    uvm_analysis_imp_output #(router_txn, router_scoreboard) imp_out;

    // Key: {32'(dst_port), 32'(tid)} -> expected txn
    router_txn expected_aa [longint unsigned];

    int unsigned n_sent, n_received, n_errors;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        imp_in  = new("imp_in",  this);   // <<< must instantiate
        imp_out = new("imp_out", this);   // <<< must instantiate
    endfunction

    // Called by input monitor: store expected
    function void write_input(router_txn txn);
        longint unsigned key = {32'(txn.dst_port), 32'(txn.tid)};
        expected_aa[key] = txn;
        n_sent++;
    endfunction

    // Called by output monitor: compare against expected
    function void write_output(router_txn actual);
        longint unsigned key = {32'(actual.dst_port), 32'(actual.tid)};
        n_received++;

        if (!expected_aa.exists(key)) begin
            `uvm_error("SCB", $sformatf(
                "SPURIOUS pkt: tid=0x%0h at out_port=%0d -- not expected",
                actual.tid, actual.dst_port))
            n_errors++;
            return;
        end

        router_txn exp = expected_aa[key];

        // Payload length check
        if (actual.payload.size() != exp.payload.size()) begin
```

```
            `uvm_error("SCB", $sformatf(
                "LEN MISMATCH tid=0x%0h: got=%0d exp=%0d",
                actual.tid, actual.payload.size(), exp.payload.size()))
            n_errors++;
        end else begin
            // Payload content check (beat by beat)
            foreach (actual.payload[i])
                if (actual.payload[i] !== exp.payload[i]) begin
                    `uvm_error("SCB", $sformatf(
                        "DATA MISMATCH tid=0x%0h beat[%0d]: got=0x%0h exp=0x%0h",
                        actual.tid, i, actual.payload[i], exp.payload[i]))
                    n_errors++;
                end
        end

        expected_aa.delete(key);

        if (n_errors == 0)
            `uvm_info("SCB", $sformatf("PASS tid=0x%0h", actual.tid), UVM_HIGH)
    endfunction

    function void check_phase(uvm_phase phase);
        super.check_phase(phase);
        if (expected_aa.num() > 0) begin
            `uvm_error("SCB", $sformatf(
                "%0d packets were sent but never received!", expected_aa.num()))
            foreach (expected_aa[k])
                `uvm_info("SCB", $sformatf(
                    "  Lost: dst=%0d tid=0x%0h",
                    expected_aa[k].dst_port, expected_aa[k].tid), UVM_NONE)
        end
        `uvm_info("SCB", $sformatf(
            "RESULT: sent=%0d received=%0d errors=%0d",
            n_sent, n_received, n_errors), UVM_NONE)
    endfunction
endclass

// ============================================================
// 6. Agent (one per port)
// ============================================================
class router_agent extends uvm_agent;
    `uvm_component_utils(router_agent)

    router_driver              drv;
    router_monitor             mon_in;    // input side monitor
    router_monitor             mon_out;   // output side monitor
    uvm_sequencer #(router_txn) seqr;

    int unsigned port_id;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        seqr    = uvm_sequencer #(router_txn)::type_id::create("seqr",   this);
        drv     = router_driver::type_id::create("drv",     this);
        mon_in  = router_monitor::type_id::create("mon_in",  this);
        mon_out = router_monitor::type_id::create("mon_out", this);

        // Configure monitors with mode and port_id
        uvm_config_db #(int)::set(this, "drv",     "port_id", port_id);
        uvm_config_db #(int)::set(this, "mon_in",  "port_id", port_id);
        uvm_config_db #(int)::set(this, "mon_out", "port_id", port_id);
        mon_in.mode  = MONITOR_INPUT;
        mon_out.mode = MONITOR_OUTPUT;
```

```
    endfunction

    function void connect_phase(uvm_phase phase);
        drv.seq_item_port.connect(seqr.seq_item_export);
    endfunction
endclass

// ============================================================
// 7. Environment
// ============================================================
class router_env extends uvm_env;
    `uvm_component_utils(router_env)

    parameter int N = 4;
    router_agent      agents[N];
    router_scoreboard scb;
    router_vseqr      vseqr;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        for (int i = 0; i < N; i++) begin
            agents[i] = router_agent::type_id::create(
                $sformatf("agent%0d",i), this);
            agents[i].port_id = i;
        end
        scb   = router_scoreboard::type_id::create("scb",   this);
        vseqr = router_vseqr::type_id::create("vseqr", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        for (int i = 0; i < N; i++) begin
            // Input monitors  -> scoreboard expected queue
            agents[i].mon_in.ap.connect(scb.imp_in);
            // Output monitors -> scoreboard actual queue
            agents[i].mon_out.ap.connect(scb.imp_out);
            // Hook up virtual sequencer handles
            vseqr.port_seqr[i] = agents[i].seqr;
        end
    endfunction
endclass
```

解释

审查后修复的10处关键问题：

1. Driver AXI-Stream 协议：先断言 tvalid，再等 tready，握手完成后再移动到下一拍。

原版在检查 tready 之前才设 tvalid，可能导致 valid=0 时仍等待 ready，违反协议。

2. Monitor：增加了 build_phase 里的 ap = new() 实例化，以及通过 mode 参数

区分 INPUT/OUTPUT 侧，使用正确的信号集（in_t* vs out_t*）。

3. Scoreboard：增加了 build_phase 里 imp_in/imp_out 的 new() 实例化，

使用 `uvm_analysis_imp_decl 宏声明两个独立的 write 方法（write_input / write_output）。

4. Driver：增加了 build_phase 从 config_db 获取 vif 和 port_id。

5. Sequence：分离为三层——router_single_seq（单包）、

router_vseqr（虚拟 sequencer 持有各端口 seqr 句柄）、

router_stress_seq（用 `uvm_declare_p_sequencer 访问 p_sequencer.port_seqr[]）。

6. Agent：在 build_phase 里设置 monitor 的 mode 和 port_id，

确保 mon_in 监控输入侧信号，mon_out 监控输出侧信号。

7. tid 管理：改为 static 全局计数器 pre_randomize() 赋值，

保证每个 transaction 有唯一 ID，避免 scoreboard key 碰撞。

8. 复位等待：driver 改为等 rst_n 上升沿（active-low 复位释放后）再开始驱动。

Q77. Verification: req/ack Handshake Testbench with Latency Checking

QUESTION

Write a complete UVM testbench (driver, monitor, scoreboard) for:

- input: clk, req (req=1 for one cycle = one new request)

- output: ack (ack=1 for one cycle = one acknowledgement)

Rules:

1. Every req must receive exactly one ack within 1-5 cycles.

2. Back-to-back reqs are allowed (multiple in-flight simultaneously).

3. No spurious acks (ack without a pending req).

4. No missing acks (req that never gets acked).

ANSWER

```
// ================================================================
// Interface
// ================================================================
interface req_ack_if (input logic clk);
    logic req;
    logic ack;
    logic rst_n;

    clocking driver_cb @(posedge clk);
        default output #1;
        output req;
        input  ack;
    endclocking

    clocking monitor_cb @(posedge clk);
        input req, ack;
    endclocking
endinterface

// ================================================================
// Transaction
// ================================================================
class req_ack_txn extends uvm_sequence_item;
    `uvm_object_utils_begin(req_ack_txn)
        `uvm_field_int(req,   UVM_DEFAULT)
        `uvm_field_int(cycle, UVM_DEFAULT)
    `uvm_object_utils_end

    bit          req;     // 1 = request, 0 = idle
    int unsigned cycle;   // simulation cycle number (set by monitor)

    function new(string name = "req_ack_txn");
        super.new(name);
    endfunction
endclass
```

```
// ============================================================
// Sequence: drives randomized req patterns
// ============================================================
class req_seq extends uvm_sequence #(req_ack_txn);
    `uvm_object_utils(req_seq)

    int unsigned num_reqs   = 20;
    int unsigned max_gap    = 4;   // max idle cycles between reqs

    function new(string name = "req_seq");
        super.new(name);
    endfunction

    task body();
        req_ack_txn txn;
        int unsigned idle;

        repeat (num_reqs) begin
            // Random idle gap before each req (0 = back-to-back)
            idle = $urandom_range(0, max_gap);
            repeat (idle) begin
                txn = req_ack_txn::type_id::create("txn");
                start_item(txn);
                assert(txn.randomize() with { txn.req == 0; })
                    else `uvm_fatal("SEQ","Randomize failed")
                finish_item(txn);
            end

            // Drive one req
            txn = req_ack_txn::type_id::create("txn");
            start_item(txn);
            assert(txn.randomize() with { txn.req == 1; })
                else `uvm_fatal("SEQ","Randomize failed")
            finish_item(txn);
        end

        // Let DUT drain: wait 10 cycles after last req
        repeat (10) begin
            txn = req_ack_txn::type_id::create("txn");
            start_item(txn);
            assert(txn.randomize() with { txn.req == 0; })
                else `uvm_fatal("SEQ","Randomize failed")
            finish_item(txn);
        end
    endtask
endclass

// Back-to-back stress sequence (all reqs consecutive, no gaps)
class req_burst_seq extends req_seq;
    `uvm_object_utils(req_burst_seq)
    function new(string name = "req_burst_seq");
        super.new(name);
        max_gap  = 0;   // force back-to-back
        num_reqs = 10;
    endfunction
endclass

// ============================================================
// Driver: drives req onto the DUT input each cycle
// ============================================================
class req_ack_driver extends uvm_driver #(req_ack_txn);
    `uvm_component_utils(req_ack_driver)
```

```
    virtual req_ack_if vif;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db #(virtual req_ack_if)::get(
                this, "", "req_ack_vif", vif))
            `uvm_fatal("DRV","Cannot get req_ack_vif")
    endfunction

    task run_phase(uvm_phase phase);
        // Initialize
        vif.driver_cb.req <= 0;

        // Wait for reset release
        @(posedge vif.rst_n);
        @(posedge vif.clk);

        forever begin
            req_ack_txn txn;
            // get_next_item blocks until sequence provides a txn.
            // Each txn = exactly one clock cycle of driving.
            seq_item_port.get_next_item(txn);
            @(posedge vif.clk);
            vif.driver_cb.req <= txn.req;
            seq_item_port.item_done();
        end
    endtask
endclass

// ============================================================
// Monitor: observes BOTH req and ack, sends both to scoreboard
// ============================================================
// Uses two analysis ports:
//   ap_req -> scoreboard.imp_req  (when req seen)
//   ap_ack -> scoreboard.imp_ack  (when ack seen)

class req_ack_monitor extends uvm_monitor;
    `uvm_component_utils(req_ack_monitor)

    virtual req_ack_if vif;
    uvm_analysis_port #(req_ack_txn) ap_req;
    uvm_analysis_port #(req_ack_txn) ap_ack;

    int unsigned cycle_count;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        ap_req = new("ap_req", this);
        ap_ack = new("ap_ack", this);
        if (!uvm_config_db #(virtual req_ack_if)::get(
                this, "", "req_ack_vif", vif))
            `uvm_fatal("MON","Cannot get req_ack_vif")
    endfunction

    task run_phase(uvm_phase phase);
        cycle_count = 0;
        // Wait for reset
        @(posedge vif.rst_n);

        forever begin
            @(posedge vif.clk);
            #1;   // small delta after clock edge to sample stable values
```

```
            cycle_count++;

            if (vif.monitor_cb.req) begin
                req_ack_txn txn = req_ack_txn::type_id::create("req_txn");
                txn.req   = 1;
                txn.cycle = cycle_count;
                ap_req.write(txn);
                `uvm_info("MON", $sformatf("REQ seen at cycle %0d", cycle_count), UVM_HIGH)
            end

            if (vif.monitor_cb.ack) begin
                req_ack_txn txn = req_ack_txn::type_id::create("ack_txn");
                txn.req   = 0;   // this is an ack observation
                txn.cycle = cycle_count;
                ap_ack.write(txn);
                `uvm_info("MON", $sformatf("ACK seen at cycle %0d", cycle_count), UVM_HIGH)
            end
        end
    endtask
endclass

// ============================================================
// Scoreboard
// ============================================================
// Tracks pending reqs in a FIFO queue (oldest first).
// When ack arrives: pop oldest req, check latency in [1..5].
//
// Why a queue (not associative array)?
//   The spec doesn't give reqs unique IDs. We assume acks are issued
//   in FIFO order (oldest req acked first). If DUT is allowed to reorder,
//   we'd need a relaxed "any pending req within window" check instead.

`uvm_analysis_imp_decl(_req)
`uvm_analysis_imp_decl(_ack)

class req_ack_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(req_ack_scoreboard)

    uvm_analysis_imp_req #(req_ack_txn, req_ack_scoreboard) imp_req;
    uvm_analysis_imp_ack #(req_ack_txn, req_ack_scoreboard) imp_ack;

    // Queue of cycle-numbers of pending reqs (oldest first)
    int unsigned pending_reqs[$];

    // Latency bounds (from spec)
    localparam int MIN_LAT = 1;
    localparam int MAX_LAT = 5;

    // Counters
    int unsigned n_reqs, n_acks, n_errors;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        imp_req = new("imp_req", this);
        imp_ack = new("imp_ack", this);
    endfunction

    // Called by monitor when req is seen
    function void write_req(req_ack_txn txn);
        pending_reqs.push_back(txn.cycle);
        n_reqs++;
        `uvm_info("SCB", $sformatf(
            "REQ recorded @ cycle %0d  [%0d pending]",
```

```
            txn.cycle, pending_reqs.size()), UVM_MEDIUM)
    endfunction

    // Called by monitor when ack is seen
    function void write_ack(req_ack_txn txn);
        int unsigned ack_cycle = txn.cycle;
        n_acks++;

        // --- Check 1: No spurious ack ---
        if (pending_reqs.size() == 0) begin
            `uvm_error("SCB", $sformatf(
                "SPURIOUS ACK at cycle %0d -- no pending req!", ack_cycle))
            n_errors++;
            return;
        end

        // --- Check 2: Latency within [MIN_LAT, MAX_LAT] ---
        // Match against OLDEST pending req (FIFO assumption)
        int unsigned req_cycle = pending_reqs.pop_front();
        int unsigned latency   = ack_cycle - req_cycle;

        if (latency < MIN_LAT) begin
            `uvm_error("SCB", $sformatf(
                "ACK TOO EARLY: req@%0d ack@%0d latency=%0d < %0d",
                req_cycle, ack_cycle, latency, MIN_LAT))
            n_errors++;
        end else if (latency > MAX_LAT) begin
            `uvm_error("SCB", $sformatf(
                "ACK TOO LATE: req@%0d ack@%0d latency=%0d > %0d",
                req_cycle, ack_cycle, latency, MAX_LAT))
            n_errors++;
        end else begin
            `uvm_info("SCB", $sformatf(
                "PASS: req@%0d -> ack@%0d latency=%0d",
                req_cycle, ack_cycle, latency), UVM_MEDIUM)
        end
    endfunction

    // --- Check 3: No missing acks (all reqs eventually acked) ---
    function void check_phase(uvm_phase phase);
        super.check_phase(phase);

        if (pending_reqs.size() > 0) begin
            `uvm_error("SCB", $sformatf(
                "%0d req(s) were never acknowledged!", pending_reqs.size()))
            foreach (pending_reqs[i])
                `uvm_error("SCB", $sformatf(
                    "  Unacked req at cycle %0d", pending_reqs[i]))
            n_errors += pending_reqs.size();
        end

        `uvm_info("SCB", $sformatf(
            "SUMMARY: reqs=%0d acks=%0d errors=%0d",
            n_reqs, n_acks, n_errors), UVM_NONE)

        // Total acks must equal total reqs
        if (n_acks != n_reqs)
            `uvm_error("SCB", $sformatf(
                "COUNT MISMATCH: sent %0d reqs but got %0d acks",
                n_reqs, n_acks))
    endfunction
endclass
```

```
// ============================================================
// Environment
// ============================================================
class req_ack_env extends uvm_env;
    `uvm_component_utils(req_ack_env)

    req_ack_driver           drv;
    req_ack_monitor          mon;
    req_ack_scoreboard       scb;
    uvm_sequencer #(req_ack_txn) seqr;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        drv  = req_ack_driver::type_id::create("drv",  this);
        mon  = req_ack_monitor::type_id::create("mon", this);
        scb  = req_ack_scoreboard::type_id::create("scb", this);
        seqr = uvm_sequencer #(req_ack_txn)::type_id::create("seqr", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        drv.seq_item_port.connect(seqr.seq_item_export);
        mon.ap_req.connect(scb.imp_req);
        mon.ap_ack.connect(scb.imp_ack);
    endfunction
endclass

// ============================================================
// Test
// ============================================================
class req_ack_test extends uvm_test;
    `uvm_component_utils(req_ack_test)

    req_ack_env env;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = req_ack_env::type_id::create("env", this);
        uvm_config_db #(virtual req_ack_if)::set(
            this, "env.*", "req_ack_vif", /* vif from top */);
    endfunction

    task run_phase(uvm_phase phase);
        req_seq seq;
        phase.raise_objection(this);

        seq = req_seq::type_id::create("seq");
        seq.num_reqs = 20;
        seq.start(env.seqr);

        phase.drop_objection(this);
    endtask
endclass

// ============================================================
// What the scoreboard checks (summary)
// ============================================================
//
//   Check 1 (write_ack): SPURIOUS ACK
//     pending_reqs is empty when ack arrives -> ERROR
//
//   Check 2 (write_ack): LATENCY BOUNDS
//     ack_cycle - req_cycle < 1  -> ERROR: too early
//     ack_cycle - req_cycle > 5  -> ERROR: too late
```

```
//
//   Check 3 (check_phase): MISSING ACK
//     pending_reqs non-empty at end of test -> ERROR
//
//   Check 4 (check_phase): COUNT MATCH
//     n_acks != n_reqs -> ERROR
//
// ================================================================
// Corner cases covered by tests
// ================================================================
//
//   CC1. Back-to-back reqs (req_burst_seq): 10 consecutive high cycles
//        -> multiple reqs in-flight simultaneously, FIFO queue stress
//   CC2. Single req followed by long idle: simplest case
//   CC3. Req arrives when DUT is "busy" processing earlier req
//   CC4. Ack latency = 1 (minimum): ack on very next cycle
//   CC5. Ack latency = 5 (maximum): ack as late as allowed
//   CC6. Ack latency = 6 (> max): caught by scoreboard as LATE
//   CC7. Ack with no req pending: caught as SPURIOUS
//   CC8. Test ends with pending reqs: caught in check_phase
```

解释

这道题的核心挑战是"多个请求同时在飞"的追踪：

不能简单地"看到 req 就期待下一个 ack 是它的"，因为在第一个 ack 到来之前可能已经有多个 req。

Scoreboard 用 FIFO 队列（`pending_reqs[$]`）追踪所有未被 ack 的 req 及其到达时刻：

req 进队，ack 出队（弹出最老的 req），验证时间差在 [1,5] 范围内。

这里有一个关键假设：ack 按 FIFO 顺序发出（先来的 req 先被 ack）。

如果 DUT spec 允许乱序 ack，则需要换成"找队列中任意一个满足延迟约束的 req 来匹配"的更宽松逻辑。

Monitor 用两个独立的 analysis port（ap_req 和 ap_ack）分别上报请求和应答，

scoreboard 通过 `` `uvm_analysis_imp_decl `` 宏声明两个对应的 write 方法，

清晰分离"什么时候看到 req"和"什么时候看到 ack"的处理逻辑。

三条检查的覆盖完整性：spurious ack（假阳）、latency 越界（时序违规）、

missing ack（假阴）——三者结合才能完整覆盖 DUT 可能违反的所有 spec 条款。

Q78. Verification: Out-of-Order Packet Scoreboard with Reconstruction

QUESTION

A DUT receives packets, reorders them internally, and outputs them
in a different order. Each packet may also be split into fragments
at the input and must be reassembled at the output.
Design a scoreboard that handles both out-of-order delivery AND
packet reconstruction.

ANSWER

```
// ================================================================
// Problem: two layers of complexity
// ================================================================
//
// Layer 1 -- Reconstruction: packets are split into fragments
```

```
//   (e.g. input: frag[0], frag[1], frag[2] for packet_id=7)
//   Must accumulate fragments and reconstruct the full packet.
//
// Layer 2 -- Out-of-order: reconstructed packets may arrive
//   at the output in a different order than they were sent.
//   Must match each output packet to the correct expected packet
//   using a unique identifier, not arrival order.
//
// Solution: two-stage scoreboard
//   Stage A (Input side):  fragment accumulator -> build expected packets
//   Stage B (Output side): out-of-order matcher -> compare by ID

// ================================================================
// Transaction classes
// ================================================================
class frag_txn extends uvm_sequence_item;
    `uvm_object_utils(frag_txn)
    int unsigned pkt_id;    // which packet this fragment belongs to
    int unsigned frag_idx;  // fragment index (0, 1, 2 ...)
    int unsigned frag_total;// total fragments expected for this packet
    logic [31:0] data;      // this fragment's payload
endclass

class packet_txn extends uvm_sequence_item;
    `uvm_object_utils(packet_txn)
    int unsigned  pkt_id;
    logic [31:0]  payload[];  // all fragments concatenated
    function new(string name = "packet_txn");
        super.new(name);
    endfunction
endclass

// ================================================================
// Scoreboard: Stage A -- Fragment Reconstruction
// ================================================================
class recon_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(recon_scoreboard)

    // Analysis imports
    `uvm_analysis_imp_decl(_in_frag)   // input fragments
    `uvm_analysis_imp_decl(_out_pkt)   // output complete packets

    uvm_analysis_imp_in_frag #(frag_txn,    recon_scoreboard) imp_in;
    uvm_analysis_imp_out_pkt #(packet_txn,  recon_scoreboard) imp_out;

    // ---- Stage A: accumulate fragments per packet ID ----
    // Key = pkt_id, Value = array of received fragments
    frag_txn    in_frags [int unsigned][$];   // pkt_id -> fragment queue
    int         frag_count [int unsigned];    // pkt_id -> fragments received

    // ---- Stage B: expected fully-assembled packets ----
    // Key = pkt_id, Value = reconstructed expected packet
    packet_txn  expected_pkts [int unsigned];

    // ---- Statistics ----
    int unsigned n_pkts_built, n_pkts_matched, n_errors;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        imp_in  = new("imp_in",  this);
        imp_out = new("imp_out", this);
    endfunction
```

```
    // Called when an input FRAGMENT arrives
    function void write_in_frag(frag_txn frag);
        int id = frag.pkt_id;
        in_frags[id].push_back(frag);
        frag_count[id]++;

        // Check if all fragments for this packet have arrived
        if (frag_count[id] == frag.frag_total) begin
            // Reconstruct the expected packet (sort by frag_idx first)
            packet_txn exp = packet_txn::type_id::create("exp");
            exp.pkt_id  = id;
            exp.payload = new[frag.frag_total];

            // Sort fragments by index and assemble payload
            foreach (in_frags[id][i]) begin
                frag_txn f = in_frags[id][i];
                exp.payload[f.frag_idx] = f.data;
            end

            // Store in expected map for Stage B matching
            expected_pkts[id] = exp;
            n_pkts_built++;
            `uvm_info("SCB", $sformatf(
                "Reconstructed pkt_id=%0d (%0d frags)",
                id, frag.frag_total), UVM_MEDIUM)

            // Clean up fragment buffer
            in_frags.delete(id);
            frag_count.delete(id);
        end
    endfunction

    // Called when an OUTPUT packet arrives (may be out of order)
    function void write_out_pkt(packet_txn actual);
        int id = actual.pkt_id;
        n_pkts_matched++;

        // Wait: expected packet may not be built yet
        // (if input fragments are still arriving)
        // Use a timeout or a second pass in check_phase.
        if (!expected_pkts.exists(id)) begin
            `uvm_error("SCB", $sformatf(
                "OUTPUT pkt_id=%0d arrived but not yet reconstructed!", id))
            n_errors++;
            return;
        end

        packet_txn exp = expected_pkts[id];

        // Compare payload
        if (actual.payload.size() != exp.payload.size()) begin
            `uvm_error("SCB", $sformatf(
                "pkt_id=%0d: length mismatch got=%0d exp=%0d",
                id, actual.payload.size(), exp.payload.size()))
            n_errors++;
        end else begin
            foreach (actual.payload[i])
                if (actual.payload[i] !== exp.payload[i]) begin
                    `uvm_error("SCB", $sformatf(
                        "pkt_id=%0d word[%0d]: got=0x%h exp=0x%h",
                        id, i, actual.payload[i], exp.payload[i]))
                    n_errors++;
                end
```

```
        end

        if (n_errors == 0)
            `uvm_info("SCB", $sformatf("PASS pkt_id=%0d", id), UVM_MEDIUM)

        expected_pkts.delete(id);
    endfunction

    function void check_phase(uvm_phase phase);
        super.check_phase(phase);
        // Any leftover expected = output never arrived (missing packet)
        if (expected_pkts.num() > 0) begin
            `uvm_error("SCB", $sformatf(
                "%0d packet(s) built but never received at output",
                expected_pkts.num()))
            foreach (expected_pkts[id])
                `uvm_error("SCB", $sformatf("  Missing output: pkt_id=%0d", id))
        end
        // Any leftover fragments = packet never completed
        if (in_frags.num() > 0) begin
            `uvm_error("SCB", $sformatf(
                "%0d packet(s) have incomplete fragments", in_frags.num()))
        end
        `uvm_info("SCB", $sformatf(
            "RESULT: built=%0d matched=%0d errors=%0d",
            n_pkts_built, n_pkts_matched, n_errors), UVM_NONE)
    endfunction
endclass
```

解释

两层复杂度的处理策略：把问题分成两个独立的阶段，每个阶段只解决一个维度的问题。

Stage A（重组）：以 pkt_id 为 key 的 associative array of queues，

逐片段累积，当 frag_count == frag_total 时按 frag_idx 排序后拼接成完整包，

存入 expected_pkts 供 Stage B 使用。

Stage B（乱序匹配）：输出包到来时按 pkt_id 查 expected_pkts，O(1) 查找，

与到达顺序完全无关——这正是 associative array 相比 queue 的优势。

check_phase 双重检查：expected_pkts 有剩余说明输出丢包；

in_frags 有剩余说明某包的片段没有全部到达（输入侧丢片段）。

两者结合才能覆盖所有"数据不完整"的场景。

Q79. Verification: Credit Release Handshake Checker with SVA

QUESTION

Two IP blocks share a credit-based flow control protocol.

Write a complete SVA checker/model for IP1 that verifies:

reset/release handshake, no credit during reset, credit counter

underflow/overflow, and simultaneous credit handling.

See protocol description in comments.

ANSWER

```
module ip1_credit_checker #(
    parameter int unsigned NUM_CREDITS     = 16,
    parameter int unsigned INITIAL_CREDITS = 10
) (
    input logic clk,
    input logic ip1_rst_n,
    input logic ip2_rst_n,
    input logic ip1_credit_release,
    input logic ip1_credit_release_ack,
    input logic ip1_credit,
    input logic ip2_credit
);

    // ================================================================
    // State machine: tracks the handshake phase
    // ================================================================
    typedef enum logic [1:0] {
        IP1_RESET    = 2'd0,   // IP1 is in reset
        WAIT_ACK     = 2'd1,   // ip1_credit_release asserted, waiting for ack
        NORMAL_OP    = 2'd2    // handshake complete, credits flowing
    } state_e;

    state_e state, next;

    // ---- State register ------------------------------------------------
    always_ff @(posedge clk or negedge ip1_rst_n) begin
        if (!ip1_rst_n)
            state <= IP1_RESET;
        else
            state <= next;
    end

    // ---- Next-state logic ----------------------------------------------
    always_comb begin
        next = state;
        case (state)
            IP1_RESET: begin
                // Transition out once ip1_rst_n is high (already in FF)
                // We enter WAIT_ACK on the first cycle after reset release.
                // ip1_credit_release is driven by IP1; we track state here.
                if (ip1_credit_release)
                    next = WAIT_ACK;
            end

            WAIT_ACK: begin
                // Accept ack only when IP2 is also out of reset
                if (ip2_rst_n && ip1_credit_release_ack)
                    next = NORMAL_OP;
            end

            NORMAL_OP: begin
                // Reset re-asserted
                if (!ip1_rst_n)
                    next = IP1_RESET;
            end
        endcase
    end

    // Convenience aliases
    wire in_reset    = (state == IP1_RESET);
    wire in_handshake = (state == WAIT_ACK);
    wire in_normal   = (state == NORMAL_OP);
```

```
    // ================================================================
    // Credit counter model
    // ================================================================
    // Counts available credits IP1 holds.
    // Starts at INITIAL_CREDITS after handshake completes.
    // Send (ip1_credit=1): decrements.
    // Receive (ip2_credit=1): increments.
    // Simultaneous send+receive: net zero change.

    logic [$clog2(NUM_CREDITS+1)-1:0] avail_credits;

    always_ff @(posedge clk or negedge ip1_rst_n) begin
        if (!ip1_rst_n) begin
            avail_credits <= '0;
        end else begin
            case (state)
                // Load initial credits the cycle handshake completes
                WAIT_ACK: begin
                    if (ip2_rst_n && ip1_credit_release_ack)
                        avail_credits <= INITIAL_CREDITS[$clog2(NUM_CREDITS+1)-1:0];
                end

                NORMAL_OP: begin
                    // Simultaneous: net zero
                    if (ip1_credit && ip2_credit)
                        avail_credits <= avail_credits;
                    // Send only: decrement
                    else if (ip1_credit && !ip2_credit)
                        avail_credits <= avail_credits - 1'b1;
                    // Receive only: increment
                    else if (!ip1_credit && ip2_credit)
                        avail_credits <= avail_credits + 1'b1;
                end

                default: avail_credits <= avail_credits;
            endcase
        end
    end

    // ================================================================
    // SVA Properties
    // ================================================================

    // ---- P1: After IP1 reset deasserts, credit_release must eventually rise
    property p_release_after_reset;
        @(posedge clk)
        $rose(ip1_rst_n) |-> ##[1:$] ip1_credit_release;
    endproperty
    ap_release_after_reset: assert property (p_release_after_reset)
        else $error("P1 FAIL: ip1_credit_release never asserted after reset");

    // ---- P2: ip1_credit_release must stay high until ack is received
    property p_release_stays_high;
        @(posedge clk) disable iff (!ip1_rst_n)
        // Once release is asserted, it must not deassert
        // until a valid ack (ip2_rst_n=1 AND ack pulse) is seen
        (ip1_credit_release && !ip2_rst_n) |=>
            ip1_credit_release;   // keep holding while ip2 still in reset
    endproperty
    ap_release_stays_high: assert property (p_release_stays_high)
        else $error("P2 FAIL: ip1_credit_release dropped before valid ack");

    // Once release is asserted, must hold until ack from a live IP2
```

```
    property p_release_hold_until_ack;
        @(posedge clk) disable iff (!ip1_rst_n)
        ($rose(ip1_credit_release)) |->
            ip1_credit_release s_until (ip2_rst_n && ip1_credit_release_ack);
    endproperty
    ap_release_hold: assert property (p_release_hold_until_ack)
        else $error("P2b FAIL: credit_release dropped before receiving valid ack");

    // ---- P3: ack from IP2 is not valid while IP2 is in reset
    property p_no_ack_during_ip2_reset;
        @(posedge clk) disable iff (!ip1_rst_n)
        (!ip2_rst_n) |-> !ip1_credit_release_ack;
    endproperty
    ap_no_ack_during_ip2_reset: assert property (p_no_ack_during_ip2_reset)
        else $error("P3 FAIL: ack asserted while IP2 still in reset");

    // ---- P4: No credit traffic during IP1 reset or during handshake
    property p_no_ip1_credit_in_reset;
        @(posedge clk)
        (!ip1_rst_n || in_handshake) |-> !ip1_credit;
    endproperty
    ap_no_ip1_credit_in_reset: assert property (p_no_ip1_credit_in_reset)
        else $error("P4a FAIL: ip1_credit asserted during reset or handshake");

    property p_no_ip2_credit_in_reset;
        @(posedge clk)
        (!ip1_rst_n || in_handshake) |-> !ip2_credit;
    endproperty
    ap_no_ip2_credit_in_reset: assert property (p_no_ip2_credit_in_reset)
        else $error("P4b FAIL: ip2_credit asserted during reset or handshake");

    // ---- P5: Underflow -- IP1 must not send when credits == 0
    property p_no_underflow;
        @(posedge clk) disable iff (!ip1_rst_n || !in_normal)
        // If only sending (not receiving simultaneously), must have credits
        (ip1_credit && !ip2_credit) |-> (avail_credits > '0);
    endproperty
    ap_no_underflow: assert property (p_no_underflow)
        else $error("P5 FAIL: ip1_credit sent with avail_credits==0 (underflow)");

    // ---- P6: Overflow -- credit count must not exceed NUM_CREDITS
    // Check that the counter never exceeds NUM_CREDITS
    // (Checked as an immediate assertion in the counter update logic)
    always_ff @(posedge clk) begin
        if (ip1_rst_n && in_normal) begin
            ap_no_overflow: assert (avail_credits <= NUM_CREDITS[$clog2(NUM_CREDITS+1)-1:0])
                else $error("P6 FAIL: avail_credits=%0d exceeds NUM_CREDITS=%0d",
                            avail_credits, NUM_CREDITS);
        end
    end

    // ---- P7: Simultaneous credit -- no net change
    // Verified by the counter model itself (simultaneous: no update).
    // Add a cover to verify the scenario is exercised:
    cp_simultaneous_credit: cover property (
        @(posedge clk) disable iff (!ip1_rst_n)
        (in_normal && ip1_credit && ip2_credit)
    );

    // ---- P8: LIVENESS -- once release asserted and IP2 is out of reset,
    //          ack must eventually arrive
    property p_ack_liveness;
        @(posedge clk) disable iff (!ip1_rst_n)
```

```
        (ip1_credit_release && ip2_rst_n) |-> ##[1:$] ip1_credit_release_ack;
    endproperty
    ap_ack_liveness: assert property (p_ack_liveness)
        else $error("P8 FAIL: credit_release_ack never received after IP2 out of reset");

    // ============================================================
    // Cover properties (verify interesting scenarios are reached)
    // ============================================================
    cp_normal_op:      cover property (@(posedge clk) $rose(in_normal));
    cp_ip1_sends:      cover property (@(posedge clk) in_normal && ip1_credit && !ip2_credit);
    cp_ip2_sends:      cover property (@(posedge clk) in_normal && ip2_credit && !ip1_credit);
    cp_min_credits:    cover property (@(posedge clk) in_normal && avail_credits == 1);
    cp_max_credits:    cover property (@(posedge clk) in_normal && avail_credits == NUM_CREDITS[$clog2(NUM_C
    cp_ip2_rst_during: cover property (@(posedge clk) in_handshake && !ip2_rst_n);

endmodule
```

解释

这道题的核心是三阶段状态机（IP1_RESET -> WAIT_ACK -> NORMAL_OP）配合 credit 计数器模型：

状态机决定"当前处于哪个阶段"，每条 SVA 属性用 disable iff 或 |-> 精确限定其生效时机。

P2（ip1_credit_release 必须保持高直到有效 ack）有两层写法：

简单层用 |=> ip1_credit_release（IP2 仍在 reset 时必须继续持有），

更强的层用 s_until（强 until）——release 必须"一直高直到满足条件"，

这是形式验证中 liveness 和 safety 结合的经典模式。

P3（IP2 在 reset 中不应该发 ack）是防御性检查：

即使 IP2 RTL 不可能这样做，在验证中显式断言可以防止 testbench bug 或 X 传播导致的假阳性。

credit 计数器的同时发送/接收（simultaneous credit）处理：

净零变化在代码里最简单的实现是"两个条件都满足时不更新"，

避免先+1再-1带来的竞争（在同一 always_ff 里不会发生，但明确写出更易于审阅）。

Cover properties 不可忽略：它们验证"感兴趣的场景确实被仿真覆盖到"，

如果 cp_min_credits 从未被覆盖，说明测试没有把信用用完，

underflow 保护的 assert 就从未被真正考验过——这是验证完整性的关键检查。

Q80. Verification: Lane Redundancy Remapping -- SV Constraint Generator

QUESTION

A link has 10 lanes total: 8 regular lanes (0-7) and 2 redundant lanes (8-9).

The DUT accepts 8 active lanes and remaps any failed regular lanes

to the redundant lanes for yield improvement.

Write a SystemVerilog generator that exhaustively tests all valid

combinations: 0, 1, or 2 failed regular lanes.

ANSWER

```
// ============================================================
// Problem analysis
// ============================================================
```

```
//
// Regular lanes:    8 (indices 0-7)
// Redundant lanes:  2 (indices 8-9)
// Allowed failures: 0, 1, or 2 regular lanes
//
// Combinations:
//   0 failures: C(8,0) =  1 (no remapping needed)
//   1 failure:  C(8,1) =  8 (failed lane -> redundant[0])
//   2 failures: C(8,2) = 28 (2 failed lanes -> redundant[0], redundant[1])
//   Total:             = 37 unique lane configurations

// ============================================================
// Transaction: captures one lane configuration
// ============================================================
class lane_config_txn extends uvm_sequence_item;
    `uvm_object_utils_begin(lane_config_txn)
        `uvm_field_int(failed_lanes_mask, UVM_DEFAULT)
        `uvm_field_int(num_failures,      UVM_DEFAULT)
        `uvm_field_int(active_map,        UVM_DEFAULT)
    `uvm_object_utils_end

    parameter int TOTAL_LANES     = 10;
    parameter int REGULAR_LANES   = 8;
    parameter int REDUNDANT_LANES = 2;

    // Which regular lanes are failed (bit mask, bits 0-7)
    rand bit [7:0] failed_lanes_mask;

    // How many lanes failed (derived)
    int unsigned num_failures;

    // Final active lane map: which physical lane drives which logical channel
    // active_map[logical] = physical lane index (0-9)
    int unsigned active_map [REGULAR_LANES];

    // Constraint: at most 2 lanes can fail (we have only 2 redundant)
    constraint c_max_failures {
        $countones(failed_lanes_mask) <= REDUNDANT_LANES;
    }

    function void post_randomize();
        num_failures = $countones(failed_lanes_mask);
        build_active_map();
    endfunction

    // Build the active lane map: failed regular lanes -> redundant lanes
    function void build_active_map();
        int unsigned redundant_idx = REGULAR_LANES;  // first redundant = lane 8
        for (int i = 0; i < REGULAR_LANES; i++) begin
            if (failed_lanes_mask[i]) begin
                // Remap failed lane i to next available redundant
                active_map[i] = redundant_idx++;
            end else begin
                active_map[i] = i;    // healthy: use regular lane directly
            end
        end
    endfunction

    function string sprint_map();
        string s = $sformatf("failed=%08b [", failed_lanes_mask);
        for (int i = 0; i < REGULAR_LANES; i++)
            s = {s, $sformatf("L%0d->P%0d ", i, active_map[i])};
        return {s, "]"};
```

```
    endfunction
endclass

// ============================================================
// Sequence: exhaustive combinatorial sweep
// (covers all 37 failure combinations systematically)
// ============================================================
class lane_remap_exhaustive_seq extends uvm_sequence #(lane_config_txn);
    `uvm_object_utils(lane_remap_exhaustive_seq)

    parameter int REGULAR_LANES   = 8;
    parameter int REDUNDANT_LANES = 2;

    function new(string name = "lane_remap_exhaustive_seq");
        super.new(name);
    endfunction

    task body();
        lane_config_txn txn;

        // ---- 0 failures: no remapping --------------------------------
        txn = lane_config_txn::type_id::create("txn");
        start_item(txn);
        assert(txn.randomize() with { txn.failed_lanes_mask == 8'h00; })
            else `uvm_fatal("SEQ","Randomize failed")
        finish_item(txn);
        `uvm_info("SEQ", $sformatf("0 failures: %s", txn.sprint_map()), UVM_LOW)

        // ---- 1 failure: C(8,1) = 8 combinations ----------------------
        for (int i = 0; i < REGULAR_LANES; i++) begin
            txn = lane_config_txn::type_id::create("txn");
            start_item(txn);
            assert(txn.randomize() with {
                txn.failed_lanes_mask == (8'h1 << i);
            }) else `uvm_fatal("SEQ","Randomize failed")
            finish_item(txn);
            `uvm_info("SEQ", $sformatf("1 fail (lane%0d): %s", i, txn.sprint_map()), UVM_LOW)
        end

        // ---- 2 failures: C(8,2) = 28 combinations ------------------
        for (int i = 0; i < REGULAR_LANES; i++) begin
            for (int j = i+1; j < REGULAR_LANES; j++) begin
                txn = lane_config_txn::type_id::create("txn");
                start_item(txn);
                assert(txn.randomize() with {
                    txn.failed_lanes_mask == ((8'h1 << i) | (8'h1 << j));
                }) else `uvm_fatal("SEQ","Randomize failed")
                finish_item(txn);
                `uvm_info("SEQ", $sformatf("2 fail (lanes%0d,%0d): %s",
                    i, j, txn.sprint_map()), UVM_LOW)
            end
        end
    endtask
endclass

// ============================================================
// Randomized sequence: weighted toward failure scenarios
// (for regression / coverage closure after exhaustive)
// ============================================================
class lane_remap_random_seq extends uvm_sequence #(lane_config_txn);
    `uvm_object_utils(lane_remap_random_seq)

    int unsigned num_transactions = 100;
```

```
    task body();
        lane_config_txn txn;
        repeat (num_transactions) begin
            txn = lane_config_txn::type_id::create("txn");
            start_item(txn);
            assert(txn.randomize() with {
                // Bias: 50% no failure, 30% one failure, 20% two failures
                $countones(txn.failed_lanes_mask) dist {
                    0 := 50,
                    1 := 30,
                    2 := 20
                };
            }) else `uvm_fatal("SEQ","Randomize failed")
            finish_item(txn);
        end
    endtask
endclass

// ============================================================
// Scoreboard: verify remap is correct
// ============================================================
class lane_remap_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(lane_remap_scoreboard)

    uvm_analysis_imp #(lane_config_txn, lane_remap_scoreboard) imp;

    parameter int REGULAR_LANES   = 8;
    parameter int REDUNDANT_LANES = 2;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        imp = new("imp", this);
    endfunction

    function void write(lane_config_txn txn);
        int unsigned redundant_used = 0;

        // Rule 1: number of failures <= REDUNDANT_LANES
        assert (txn.num_failures <= REDUNDANT_LANES)
            else `uvm_error("SCB", $sformatf(
                "Too many failures: %0d > %0d", txn.num_failures, REDUNDANT_LANES))

        // Rule 2: each failed regular lane is mapped to a unique redundant
        for (int i = 0; i < REGULAR_LANES; i++) begin
            if (txn.failed_lanes_mask[i]) begin
                int phys = txn.active_map[i];
                // Must map to redundant lane range (8 or 9)
                assert (phys >= REGULAR_LANES && phys < REGULAR_LANES + REDUNDANT_LANES)
                    else `uvm_error("SCB", $sformatf(
                        "Failed lane %0d mapped to non-redundant physical %0d", i, phys))
                redundant_used++;
            end else begin
                // Healthy lane must map to itself
                assert (txn.active_map[i] == i)
                    else `uvm_error("SCB", $sformatf(
                        "Healthy lane %0d wrongly mapped to %0d", i, txn.active_map[i]))
            end
        end

        // Rule 3: no two lanes map to the same physical lane (no collision)
        for (int i = 0; i < REGULAR_LANES; i++)
            for (int j = i+1; j < REGULAR_LANES; j++)
```

```
                assert (txn.active_map[i] != txn.active_map[j])
                    else `uvm_error("SCB", $sformatf(
                        "Collision: logical %0d and %0d both -> physical %0d",
                        i, j, txn.active_map[i]))

        `uvm_info("SCB", $sformatf("PASS: %s", txn.sprint_map()), UVM_HIGH)
    endfunction
endclass
```

解释

Lane remapping 的核心是排列组合：最多2个冗余lane，

所以只需要测试 C(8,0)+C(8,1)+C(8,2)=37 种失败组合，数量有限可以穷举。

穷举序列（exhaustive_seq）用双层 for 循环显式生成所有 C(8,2) 组合，

比随机化更可靠——保证不遗漏任何角落 case，这是硬件接口验证的常见策略。

约束设计：`failed_lanes_mask` 用 bit mask 表示，$countones 约束最大失败数，

post_randomize 里 build_active_map 从左到右遍历，依次把失败 lane 映射到下一个冗余 lane。

Scoreboard 的三条规则覆盖了 remap 逻辑的完整性：

规则1 防止"失败太多但冗余不够"的越界；

规则2 验证每个失败 lane 确实被映射到冗余范围（8或9）；

规则3 防止两个 logical lane 映射到同一个 physical lane（冲突）。

Q81. Verification: PHY Link Training -- Delay Code Sweep with PRBS Error Injection

QUESTION

A PHY link init training sweeps delay codes 0-192.

Each code has a 20-cycle sampling window where 32-bit PRBS data

is received and compared with a locally generated PRBS.

Inject errors such that codes 0-63 exceed the error threshold

and codes 64-192 do not. The DUT should identify code 64 as the

training result. Write the verification sequence and checker.

ANSWER

```
// ================================================================
// Protocol overview
// ================================================================
//
// Delay code sweep: code 0 -> 1 -> ... -> 192
// For each code:
//   - DUT applies the delay setting to the PHY Rx path
//   - 20 sampling cycles occur
//   - Each cycle: receive 32-bit PRBS data, XOR with local PRBS
//     -> any bit difference = bit error
//   - Count total bit errors across 20 cycles (max = 20*32 = 640 bits)
//   - If error_count > threshold: code is "bad" (too much noise)
//   - If error_count <= threshold: code is "good" (sampling eye open)
//
// Expected result: DUT should select code 64 (first good code)
//   or the center of the good window [64,192] for best margin.
```

```
// ================================================================
// PRBS generator model (PRBS-31)
// ================================================================
class prbs_gen;
    bit [30:0] state;

    function new(bit [30:0] seed = 31'h1);
        state = seed;
    endfunction

    function bit [31:0] next32();
        bit [31:0] out;
        for (int i = 0; i < 32; i++) begin
            bit new_bit = state[30] ^ state[27];  // PRBS-31 taps: x^31+x^28+1
            out[i] = new_bit;
            state   = {state[29:0], new_bit};
        end
        return out;
    endfunction
endclass

// ================================================================
// Transaction: one full delay code sweep result
// ================================================================
class training_txn extends uvm_sequence_item;
    `uvm_object_utils(training_txn)

    parameter int MAX_CODE      = 192;
    parameter int WINDOW_CYCLES = 20;
    parameter int DATA_BITS     = 32;
    parameter int ERROR_THRESHOLD = 10;  // <= 10 errors = good code

    // Observed error count per delay code
    int unsigned error_counts [MAX_CODE+1];

    // DUT's chosen best delay code
    int unsigned chosen_code;

    function new(string name = "training_txn");
        super.new(name);
    endfunction
endclass

// ================================================================
// Sequence: drives the delay sweep with controlled error injection
// ================================================================
class training_sweep_seq extends uvm_sequence #(training_txn);
    `uvm_object_utils(training_sweep_seq)

    parameter int MAX_CODE         = 192;
    parameter int WINDOW_CYCLES    = 20;
    parameter int DATA_BITS        = 32;
    parameter int ERROR_THRESHOLD  = 10;
    parameter int BAD_CODE_LIMIT   = 63;   // codes 0..63 = bad zone

    // Error injection rate for bad zone (inject enough to exceed threshold)
    // e.g. inject ~5 bit errors per cycle = 100 errors per window >> threshold
    parameter int INJECT_ERRORS_PER_CYCLE = 5;

    function new(string name = "training_sweep_seq");
        super.new(name);
    endfunction
```

```
    task body();
        training_txn txn;
        prbs_gen local_prbs  = new(31'hABCD_EF01);
        prbs_gen remote_prbs = new(31'hABCD_EF01);  // same seed = aligned
        bit [31:0] local_word, remote_word, corrupted_word;
        int unsigned error_count;

        txn = training_txn::type_id::create("txn");
        start_item(txn);
        assert(txn.randomize()) else `uvm_fatal("SEQ","Randomize failed")

        // Sweep all 193 delay codes
        for (int code = 0; code <= MAX_CODE; code++) begin
            error_count = 0;

            // Apply delay code to DUT (via interface write)
            // vif.delay_code <= code;
            // @(posedge vif.clk);

            // 20-cycle sampling window
            for (int cyc = 0; cyc < WINDOW_CYCLES; cyc++) begin
                local_word  = local_prbs.next32();
                remote_word = remote_prbs.next32();

                // ---- Error injection ----------------------------------
                if (code <= BAD_CODE_LIMIT) begin
                    // Bad zone: corrupt remote data to inject bit errors
                    // Flip INJECT_ERRORS_PER_CYCLE random bits
                    corrupted_word = remote_word;
                    for (int b = 0; b < INJECT_ERRORS_PER_CYCLE; b++) begin
                        int bit_pos = $urandom_range(0, DATA_BITS-1);
                        corrupted_word[bit_pos] = ~corrupted_word[bit_pos];
                    end
                    // Drive corrupted data onto interface
                    // vif.rx_data <= corrupted_word;
                end else begin
                    // Good zone: clean data (no injection)
                    // vif.rx_data <= remote_word;
                    corrupted_word = remote_word;
                end

                // Count bit errors: XOR with local, count ones
                error_count += $countones(corrupted_word ^ local_word);
            end

            txn.error_counts[code] = error_count;

            `uvm_info("SEQ", $sformatf(
                "Code=%0d errors=%0d (%s)",
                code, error_count,
                (error_count <= ERROR_THRESHOLD) ? "GOOD" : "BAD"), UVM_HIGH)
        end

        // Model expects DUT to choose first good code (64)
        // or center of good window
        finish_item(txn);
    endtask
endclass

// ============================================================
// Scoreboard: verify training result
// ============================================================
class training_scoreboard extends uvm_scoreboard;
```

```
    `uvm_component_utils(training_scoreboard)

    parameter int MAX_CODE        = 192;
    parameter int ERROR_THRESHOLD = 10;
    parameter int EXPECTED_FIRST_GOOD = 64;   // first code in good zone
    parameter int EXPECTED_LAST_GOOD  = 192;  // last code in good zone

    uvm_analysis_imp #(training_txn, training_scoreboard) imp;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        imp = new("imp", this);
    endfunction

    function void write(training_txn txn);
        int unsigned first_good = -1;
        int unsigned last_good  = -1;
        int unsigned n_errors   = 0;

        // Check 1: bad zone codes 0-63 must ALL exceed threshold
        for (int code = 0; code < EXPECTED_FIRST_GOOD; code++) begin
            if (txn.error_counts[code] <= ERROR_THRESHOLD) begin
                `uvm_error("SCB", $sformatf(
                    "Code %0d in BAD zone has errors=%0d <= threshold=%0d",
                    code, txn.error_counts[code], ERROR_THRESHOLD))
                n_errors++;
            end
        end

        // Check 2: good zone codes 64-192 must ALL be below threshold
        for (int code = EXPECTED_FIRST_GOOD; code <= MAX_CODE; code++) begin
            if (txn.error_counts[code] > ERROR_THRESHOLD) begin
                `uvm_error("SCB", $sformatf(
                    "Code %0d in GOOD zone has errors=%0d > threshold=%0d",
                    code, txn.error_counts[code], ERROR_THRESHOLD))
                n_errors++;
            end
        end

        // Find actual good window boundaries
        foreach (txn.error_counts[code]) begin
            if (txn.error_counts[code] <= ERROR_THRESHOLD) begin
                if (first_good == -1) first_good = code;
                last_good = code;
            end
        end

        // Check 3: DUT selected code is within the good window
        if (txn.chosen_code < first_good || txn.chosen_code > last_good) begin
            `uvm_error("SCB", $sformatf(
                "DUT chose code=%0d outside good window [%0d,%0d]",
                txn.chosen_code, first_good, last_good))
            n_errors++;
        end

        // Report good window statistics
        `uvm_info("SCB", $sformatf(
            "Good window: [%0d,%0d] width=%0d, DUT chose %0d, errors=%0d",
            first_good, last_good, last_good-first_good+1,
            txn.chosen_code, n_errors), UVM_LOW)
    endfunction
endclass
```

解释

Link training 验证的核心是"受控错误注入 + 边界验证"：

bad zone (0-63) 每周期注入5个比特错误，20个窗口累计 ≥100 个错误，远超阈值（10），

确保 DUT 正确把这些 code 标记为"不良"；

good zone (64-192) 注入零错误，DUT 应该看到接近零的错误计数。

PRBS 对齐是关键假设：local 和 remote 使用相同的种子，所以在没有错误注入时

XOR 结果为全零，任何1都代表真实的比特错误，这正是 PRBS 训练的物理原理。

Scoreboard 的三层检查：

Check 1 验证 bad zone 全部超阈（错误注入有效）；

Check 2 验证 good zone 全部不超阈（clean data path）；

Check 3 验证 DUT 选出的最终 code 在有效窗口内

（DUT 可能选第一个好 code 64，或窗口中心 128，都是合法的）。

$countones(corrupted ^ local) 是计算汉明距离的标准 SV 写法，

每个不同的比特位置都会让 XOR 对应位为1，$countones 直接统计差异比特数。

Q82. Verification: 12-Channel 32-bit Coverage Strategy

QUESTION

A DUT has 12 channels, each carrying 32-bit data.

How do you design the functional coverage for this DUT?

The full value space (12 x 2^32) is too large to exhaust.

ANSWER

```
// ================================================================
// Problem: state space too large to enumerate
// ================================================================
//
// 12 channels x 2^32 values = ~5.15 x 10^10 combinations.
// Exhaustive coverage is IMPOSSIBLE. Strategy: partition into
// MEANINGFUL BINS that cover interesting behaviors, not all values.

// ================================================================
// Coverage group design
// ================================================================
covergroup channel_cg @(posedge clk);

    // --- 1. Per-channel value partitioning (key ranges, not all values) ---
    cp_channel_id: coverpoint ch_id {
        bins ch[] = {[0:11]};    // all 12 channels exercised
    }

    cp_data_range: coverpoint data {
        bins zero          = {32'h0000_0000};
        bins all_ones      = {32'hFFFF_FFFF};
        bins max_pos       = {32'h7FFF_FFFF};   // max positive signed
        bins min_neg       = {32'h8000_0000};   // min negative signed (2's comp)
        bins low_range     = {[32'h0000_0001 : 32'h0000_00FF]};
        bins mid_range     = {[32'h0000_0100 : 32'hFFFE_FFFF]};
        bins high_range    = {[32'hFFFF_0000 : 32'hFFFF_FFFE]};
```

```
        bins walking_ones  = {32'h0000_0001, 32'h0000_0002, 32'h0000_0004,
                              32'h0000_0008, 32'h0000_0010, 32'h0000_0020,
                              32'h0000_0040, 32'h0000_0080};  // bit walk
    }

    // --- 2. Cross coverage: every channel must exercise every data range ---
    cx_ch_data: cross cp_channel_id, cp_data_range;
    // This creates 12 x 8 = 96 cross bins -- achievable!

    // --- 3. Consecutive channel activity ---
    cp_same_ch_consecutive: coverpoint ch_id {
        bins ch_repeat[] = ([0:11] => [0:11]);  // back-to-back same channel
    }

    // --- 4. Multi-channel simultaneous activity ---
    cp_active_channels: coverpoint active_mask {
        // active_mask is a 12-bit mask: which channels are valid this cycle
        bins single_ch  = {12'h001, 12'h002, 12'h004, 12'h008,
                           12'h010, 12'h020, 12'h040, 12'h080,
                           12'h100, 12'h200, 12'h400, 12'h800};
        bins two_ch     = {[12'h003 : 12'hFFE]};  // at least 2 active
        bins all_ch     = {12'hFFF};              // all 12 simultaneously
    }

    // --- 5. Data transition coverage (delta between consecutive transactions) ---
    cp_data_transition: coverpoint data {
        bins zero_to_nonzero = (32'h0 => [32'h1:32'hFFFF_FFFF]);
        bins nonzero_to_zero = ([32'h1:32'hFFFF_FFFF] => 32'h0);
        bins pos_to_neg      = ([32'h0:32'h7FFF_FFFF] => [32'h8000_0000:32'hFFFF_FFFF]);
        bins neg_to_pos      = ([32'h8000_0000:32'hFFFF_FFFF] => [32'h0:32'h7FFF_FFFF]);
    }

endgroup

// ============================================================
// Additional channel-pair stress (optional advanced coverage)
// ============================================================
// For protocols where channel ordering matters:
covergroup ch_pair_cg @(posedge clk);
    // Previous channel vs current channel
    cp_prev: coverpoint prev_ch_id { bins ch[] = {[0:11]}; }
    cp_curr: coverpoint ch_id      { bins ch[] = {[0:11]}; }
    cx_ch_sequence: cross cp_prev, cp_curr;
    // 12 x 12 = 144 bins: covers all ch->ch transitions
endgroup

// ============================================================
// Coverage philosophy summary
// ============================================================
//
// Instead of: enumerate all 2^32 x 12 values (impossible)
// Do this:
//
//   1. Partition 32-bit space into MEANINGFUL BINS (boundary values,
//      zero, all-ones, sign boundary, bit walk patterns)
//      -> ~8-10 bins instead of 4 billion values
//
//   2. CROSS every data bin with every channel ID
//      -> 12 ch x 8 data bins = 96 cross bins (achievable in hours)
//
//   3. Cover ACTIVITY PATTERNS (single channel, multi-channel, all-channel)
//      -> tests arbitration, ordering, backpressure behavior
//
```

```
//   4. Cover DATA TRANSITIONS (0->non0, neg->pos, etc.)
//      -> tests signed arithmetic edge cases, overflow detection
//
//   5. RANDOM + directed: use constrained random to hit all coverage bins,
//      then add directed tests for specific corner cases
```

解释

12 channel x 2^32 的核心问题是"如何把无限大的验证空间变成有限且有意义的覆盖目标"。

答案不是"缩小随机范围让它自然命中"，而是"主动设计 meaningful bins"：

边界值（0、全1、最大正数、最小负数）、走位模式（walking ones）、

以及特定的符号边界（32'h7FFF_FFFF / 32'h8000_0000）。

cross coverage 的关键价值在这道题里尤其明显：

单独覆盖每个 channel 和单独覆盖每个 data range 是不够的——

必须确保每个 channel 都覆盖到了所有 data range（包括边界值），

这是避免"channel X 从未传输过 0"这类遗漏的唯一方式。

96 个 cross bin（12×8）是可以在几小时压力测试里关闭的目标，

而不是 5×10^10 个穷举组合——这种"化不可能为可行"的分析能力本身就是答案的加分项。

Q83. Verification: UVM Backdoor Access -- HDL Path and Usage

QUESTION

What is UVM backdoor access? How do you use HDL paths for backdoor reads and writes? When should you use backdoor vs frontdoor?

ANSWER

```
// ================================================================
// Frontdoor vs Backdoor
// ================================================================
//
// Frontdoor access: uses the actual bus protocol (APB/AXI/etc.)
//   - Goes through the driver -> bus -> DUT interface
//   - Takes real simulation time (clock cycles)
//   - Tests the bus interface itself
//   - Used for: normal functional stimulus, protocol testing
//
// Backdoor access: directly reads/writes DUT internal signals
//   - Bypasses the bus, uses the simulator's HDL path mechanism
//   - Takes ZERO simulation time (instant)
//   - Does NOT test the bus interface
//   - Used for: fast initialization, state injection, debug

// ================================================================
// HDL path for backdoor in UVM RAL
// ================================================================
//
// Step 1: Set the HDL path in the register model
class my_ctrl_reg extends uvm_reg;
    `uvm_object_utils(my_ctrl_reg)
    uvm_reg_field EN;
    uvm_reg_field MODE;
```

```
    function void build();
        EN   = uvm_reg_field::type_id::create("EN");
        MODE = uvm_reg_field::type_id::create("MODE");
        EN.configure  (this, 1, 0, "RW", 0, 0, 1, 1, 0);
        MODE.configure(this, 2, 1, "RW", 0, 0, 1, 1, 0);

        // Add HDL path for backdoor access
        // Argument: HDL path string pointing to the register in the DUT
        add_hdl_path_slice("tb.dut.u_ctrl.ctrl_reg", 0, 32);
        //                  ^-- simulator path      ^lsb ^size
    endfunction
endclass

// Step 2: Use backdoor in a sequence or test
class fast_init_seq extends uvm_sequence;
    `uvm_object_utils(fast_init_seq)

    my_reg_block regmodel;

    task body();
        uvm_status_e status;
        uvm_reg_data_t val;

        // --- Backdoor WRITE: set register state instantly ---
        // No bus transaction generated, zero simulation time
        regmodel.CTRL.write(
            .status(status),
            .value(32'h0000_0003),
            .path(UVM_BACKDOOR)     // <-- key parameter
        );

        // --- Backdoor READ: read DUT internal state instantly ---
        regmodel.CTRL.read(
            .status(status),
            .value(val),
            .path(UVM_BACKDOOR)
        );
        `uvm_info("SEQ", $sformatf("CTRL = 0x%0h", val), UVM_LOW)

        // --- Field-level backdoor access ---
        regmodel.CTRL.EN.write(
            .status(status),
            .value(1'b1),
            .path(UVM_BACKDOOR)
        );

        // --- Mirror check via backdoor ---
        // Read hardware value and compare to mirror (expected value)
        regmodel.CTRL.mirror(
            .status(status),
            .check(UVM_CHECK),
            .path(UVM_BACKDOOR)    // fast check without bus overhead
        );
    endtask
endclass

// ================================================================
// Direct HDL path access (without RAL)
// ================================================================
//
// For non-register signals, use $hdl_path() or force/release:
//
// Read:
```

```
//   bit [7:0] dut_val;
//   uvm_hdl_read("tb.dut.u_fifo.count", dut_val);
//
// Write:
//   uvm_hdl_deposit("tb.dut.u_ctrl.state", 2'b01);  // deposit (like force)
//   uvm_hdl_force  ("tb.dut.u_clk_gate.enable", 1); // force (override)
//   uvm_hdl_release("tb.dut.u_clk_gate.enable");    // release
//
// Note: uvm_hdl_* functions are simulation-specific and may
// not work in all simulators. Path format varies (VCS uses "top.x.y",
// Questa may differ).

// ============================================================
// When to use backdoor
// ============================================================
//
//   Backdoor IS appropriate for:
//   1. FAST RESET/INIT: set DUT to a known state without wasting
//      thousands of clock cycles driving bus transactions.
//   2. STATE INJECTION: force the DUT into a corner case state
//      that would take too long to reach via normal stimulus.
//   3. GOLDEN REFERENCE CHECK: read internal state to verify DUT
//      computed the correct intermediate value (debug).
//   4. COVERAGE CLOSURE: force rare states that constrained-random
//      can't reach efficiently.
//
//   Backdoor is NOT appropriate for:
//   1. Verifying the BUS INTERFACE ITSELF (bypass = no test).
//   2. SIGN-OFF testing (must demonstrate end-to-end protocol).
//   3. When access ordering or side-effects of the bus write matter.
```

解释

Backdoor 的核心价值是"跳过协议层直接操作 DUT 内部状态"：

使用 HDL 路径字符串（如 "tb.dut.u_ctrl.ctrl_reg"）让仿真器直接定位到网表信号，

read 和 write 在时间 t=0 完成（zero simulation time）。

add_hdl_path_slice() 是连接 RAL register model 和 DUT 实际信号的关键调用：

参数是 HDL 路径字符串、起始 bit 位置、宽度，

允许一个寄存器字段对应多个 HDL 切片（用于跨多个寄存器的宽字段）。

uvm_hdl_read/deposit/force/release 是不依赖 RAL 的底层 backdoor API，

适合访问任意 DUT 内部信号（不只是寄存器）。

工程实践中的典型用法：test 的 run_phase 开始时用 backdoor 快速初始化所有寄存器

到测试所需的起始状态，节省几百个时钟周期的 frontdoor 初始化时间，

然后切换到 frontdoor 进行真正的功能验证。

Q84. Verification: Firmware Writes 1 to enable_reg but Device Does Not Start -- Debug

QUESTION

Firmware writes a 1 to enable_reg to start a device, but the device

does not activate. How do you systematically debug this?

ANSWER

```
// ================================================================
// Problem: firmware write 1 -> enable_reg, device stays inactive
// ================================================================
//
// Possible root causes (systematic elimination):
//
//   Layer 0: Did the WRITE actually happen?
//   Layer 1: Did the write REACH the register?
//   Layer 2: Did the register CAPTURE the value?
//   Layer 3: Did the register OUTPUT propagate to the device?
//   Layer 4: Is the device responding correctly to enable?

// ================================================================
// Step 1: Verify the firmware write was issued correctly
// ================================================================
//
//   a. Add $display or UVM_INFO in the driver/sequence at the
//      exact point of the write:
//      `uvm_info("FW", $sformatf("Writing 1 to enable_reg @ addr=0x%0h", ADDR), UVM_LOW)
//
//   b. Check: is the address correct?
//      Common bug: off-by-one in address map, or wrong byte offset.
//      Verify: addr of enable_reg in RTL register map vs what firmware uses.
//
//   c. Check: is the data correct?
//      Is bit[0] being set, or is firmware writing to the wrong bit field?
//      e.g. writing 32'h0000_0001 vs 32'h0000_0002 (wrong bit).

// ================================================================
// Step 2: Verify the write reached the register -- waveform check
// ================================================================
//
//   Open the simulator waveform (FSDB/VCD) and check:
//
//   a. APB/AXI bus signals at the time of the write:
//      - PSEL=1, PENABLE=1, PWRITE=1, PADDR=correct, PWDATA=1 ?
//      - PREADY=1 (slave accepted)? If PREADY stays 0: slave is stalling.
//
//   b. Check for BUS ERRORS:
//      - PSLVERR=1? Slave returned an error response.
//      - Bus arbitration: was the bus granted to firmware's master?
//
//   c. Check timing:
//      - Was the write in RESET state? If rst_n=0 during write,
//        the register ignores the write and resets to 0.

// ================================================================
// Step 3: Check the register itself (backdoor read)
// ================================================================
//
//   Use UVM RAL backdoor read to check what the register actually holds:
//
//   uvm_status_e status;
//   uvm_reg_data_t val;
//   regmodel.ENABLE_REG.read(.status(status), .value(val), .path(UVM_BACKDOOR));
//   `uvm_info("DBG", $sformatf("enable_reg actual value = 0x%0h", val), UVM_NONE)
//
//   If val == 0 after the write:
//     -> Write never reached the register, OR register was reset after the write.
//   If val == 1 after the write:
//     -> Register is correct; problem is DOWNSTREAM (step 4).
//
//   Also check the RAL mirror vs hardware:
```

```
//   regmodel.ENABLE_REG.mirror(.status(status), .check(UVM_CHECK));
//   -> If mismatch: hardware state != predicted state. Hardware bug.

// ================================================================
// Step 4: Check if enable propagates to the device logic
// ================================================================
//
//   In the waveform, trace the enable signal from register output
//   to the device:
//
//   a. Register output:
//      Check: dut.u_reg_block.enable_reg_q == 1 ?
//      (If 0: register not storing / wrong flop being read)
//
//   b. Clock gating:
//      Is there a clock gate between enable_reg and the device?
//      enable_reg might control a clock enable -- check if the
//      device's clock is actually switching after enable=1.
//
//   c. Additional enable conditions:
//      Many devices require MULTIPLE conditions: enable AND !reset AND mode==RUN.
//      Check all conditions are satisfied simultaneously.
//      e.g.: device_start = enable_reg & ~soft_reset & (mode == 2'b01)
//
//   d. Timing:
//      Is there a pipeline delay between writing enable=1 and the device
//      seeing it? Some registers take effect after N cycles (next frame,
//      next packet, after a specific trigger).

// ================================================================
// Step 5: Write an SVA checker to catch this automatically
// ================================================================
//
//   property p_enable_starts_device;
//       @(posedge clk) disable iff (!rst_n)
//       $rose(enable_reg) |-> ##[1:5] device_active;
//   endproperty
//   assert property (p_enable_starts_device)
//       else $error("enable_reg went high but device never activated within 5 cycles");
//
//   This assertion would have CAUGHT the bug automatically in simulation,
//   without requiring manual waveform inspection.

// ================================================================
// Summary: root cause categories and corresponding checks
// ================================================================
//
//   Root Cause               | Check
//   -------------------------+---------------------------------
//   Wrong address            | Compare addr in waveform vs map
//   Wrong bit field          | Check PWDATA bit pattern
//   Bus error / no grant     | PSLVERR, arbitration signals
//   Write during reset       | rst_n state at write time
//   Register not storing     | Backdoor read after write
//   Enable not propagating   | Trace signal path in waveform
//   Additional enable needed | Check multi-condition AND logic
//   Pipeline delay           | Check N cycles after write
```

解释

这道题考察的是系统化调试思路，不是一个具体的 bug 答案，

正确的回答策略是"从最外层到最内层逐层排查"——这和 debug 的通用方法论一致：

先确认"输入发出了"，再确认"输入被接收了"，再确认"被正确处理了"，最后确认"输出正确"。

Backdoor read 在这里是关键工具：它把"register 有没有存到值"和"register 下游有没有响应"

完全分开，一次 backdoor read 可以立刻判断问题在 register 层还是在 register 以后的逻辑层。

多条件 AND 逻辑是最常见的真实 bug 类型：

工程师写 enable 检查时经常只验证 enable_reg=1，

但 RTL 里 device_start = enable & ~soft_reset & (mode==RUN)，

如果 soft_reset 是 1 或 mode 不对，enable=1 根本不够。

最后的 SVA 属性是这道题最重要的 takeaway：

如果从一开始就写了 `$rose(enable_reg) |-> ##[1:5] device_active`，

这个 bug 在第一次仿真就会被自动捕获，根本不需要手动查波形。

Q85. Verification: DV Environment for Software-Programmed Threshold Transaction Limiter

QUESTION

Design a DV environment for a DUT that is a "threshold transaction limiter":

Software programs a threshold register (e.g. max_count=5).

The DUT counts incoming transactions and blocks/drops new ones

once the count reaches the threshold.

How do you structure the testbench and what do you verify?

ANSWER

```
// ================================================================
// DUT behavior
// ================================================================
//
// 1. Software writes threshold via register interface (APB/AXI).
// 2. Transactions arrive at input port (valid/ready handshake).
// 3. DUT counts accepted transactions.
// 4. When count >= threshold: new transactions are BLOCKED (ready=0)
//    or DROPPED (ready=1 but data discarded, with drop_flag output).
// 5. Software can reset the counter (write 0 to count_reg).

// ================================================================
// UVM Environment structure
// ================================================================
//
// env
//   ■■■ reg_agent         (APB/AXI: programs threshold, reads count)
//   ■     ■■■ reg_driver
//   ■     ■■■ reg_monitor
//   ■■■ txn_agent         (drives input transactions)
//   ■     ■■■ txn_driver
//   ■     ■■■ txn_monitor (observes which txns are accepted/dropped)
//   ■■■ scoreboard        (tracks count, predicts when blocking starts)

// ================================================================
// Key scoreboard model
// ================================================================
class threshold_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(threshold_scoreboard)
```

```
    int unsigned threshold;     // programmed value
    int unsigned txn_count = 0; // accepted transaction count

    // Called when threshold register is written
    function void set_threshold(int unsigned val);
        threshold = val;
        `uvm_info("SCB", $sformatf("Threshold set to %0d", threshold), UVM_LOW)
    endfunction

    // Called when a transaction is observed at the input
    function void observe_txn(bit accepted);
        if (txn_count < threshold) begin
            // Should be accepted
            if (!accepted)
                `uvm_error("SCB", $sformatf(
                    "Txn WRONGLY DROPPED: count=%0d threshold=%0d",
                    txn_count, threshold))
            else
                txn_count++;
        end else begin
            // At or above threshold: should be blocked/dropped
            if (accepted)
                `uvm_error("SCB", $sformatf(
                    "Txn WRONGLY ACCEPTED: count=%0d >= threshold=%0d",
                    txn_count, threshold))
        end
    endfunction

    // Called when counter reset
    function void reset_counter();
        txn_count = 0;
    endfunction
endclass

// ============================================================
// Test scenarios
// ============================================================
//
// TC1: Basic threshold: program threshold=5, send 7 transactions.
//      Verify first 5 accepted, last 2 dropped.
//
// TC2: Threshold=0: all transactions should be immediately blocked.
//
// TC3: Threshold=MAX_INT: no blocking ever occurs.
//
// TC4: Counter reset mid-stream:
//      Send 5 txns (hits threshold), reset counter, send 5 more.
//      Verify all 10 accepted (reset re-opens the gate).
//
// TC5: Change threshold WHILE transactions are in-flight:
//      Send 3 txns, lower threshold to 2 mid-stream.
//      Verify DUT immediately starts blocking.
//
// TC6: Burst of transactions exactly AT threshold boundary:
//      Simultaneous valid pulses when count = threshold-1.
//      Only one should get through (race condition check).
//
// TC7: Back-to-back threshold reprogramming:
//      Rapidly alternate threshold between 1 and 10.
//      Verify counter and blocking behavior stays consistent.

// ============================================================
// SVA properties
```

```
// ================================================================
//
// Once threshold is hit, no more acceptances until reset:
//   property p_no_accept_above_threshold;
//       @(posedge clk) disable iff (!rst_n)
//       (txn_count >= threshold) |-> !txn_accepted;
//   endproperty
//
// Accepted transaction increments count:
//   property p_count_increments;
//       @(posedge clk)
//       $rose(txn_accepted) |=> (txn_count == $past(txn_count) + 1);
//   endproperty
```

解释

Threshold transaction limiter 验证的关键是"边界测试"：

恰好 threshold-1 个（全部通过）、恰好 threshold 个（刚好满）、threshold+1 个（必须有一个被拦）。

Scoreboard 维护软件侧的 expected count，和 DUT 的实际接受/拒绝行为比对。

TC6（临界点并发事务）是最难覆盖的场景：当 count=threshold-1 时，

如果两个事务在同一时钟周期同时到达，DUT 必须只让一个通过——

这是一个经典的"临界区竞争"，需要专门的定向测试来验证 DUT 的仲裁逻辑。

TC5（运行中修改 threshold）测试的是配置更新的原子性：

新 threshold 生效的时机是否正确（当前周期还是下个周期？），这需要时序分析。

Q86. Verification: 128-bit Parallel Bus -- Stuck-at Fault and Short Wire Test Patterns

QUESTION

A DUT has a 128-bit parallel input bus. Design efficient test patterns

to detect stuck-at-0, stuck-at-1 faults, and short circuits between wires.

ANSWER

```
// ================================================================
// Fault models
// ================================================================
//
// Stuck-at-0 (SA0): bit always reads as 0 regardless of input.
//   To detect: apply 1 to that bit, observe if output is still 0.
//
// Stuck-at-1 (SA1): bit always reads as 1 regardless of input.
//   To detect: apply 0 to that bit, observe if output is still 1.
//
// Short circuit (bridging fault): two wires are shorted together.
//   When one wire drives 0 and the other 1, both read the same value.
//   To detect: apply patterns where adjacent wires have different values.

// ================================================================
// Pattern 1: Walking 1's (detects SA0 on each bit)
// ================================================================
//
// Apply 128 patterns, each with exactly ONE bit set:
//   0x00000000...00000001  (bit 0 = 1)
//   0x00000000...00000002  (bit 1 = 1)
//   0x00000000...00000004  (bit 2 = 1)
```

```
//   ...
//   0x80000000...00000000  (bit 127 = 1)
//
// If bit[k] is stuck-at-0: when the pattern has bit[k]=1,
// the output reads 0 instead of 1 -> SA0 detected on bit k.

task walking_ones(ref logic [127:0] dut_in, ref logic [127:0] dut_out);
    for (int k = 0; k < 128; k++) begin
        dut_in = (128'h1 << k);   // set only bit k
        @(posedge clk);
        #1;
        if (dut_out[k] !== 1'b1)
            $error("SA0 detected on bit %0d: expected 1 got %0b", k, dut_out[k]);
    end
endtask

// ================================================================
// Pattern 2: Walking 0's (detects SA1 on each bit)
// ================================================================
//
// Apply 128 patterns, each with exactly ONE bit cleared:
//   0xFFFFF...FFFFFFFE  (bit 0 = 0)
//   0xFFFFF...FFFFFFFD  (bit 1 = 0)
//   ...

task walking_zeros(ref logic [127:0] dut_in, ref logic [127:0] dut_out);
    for (int k = 0; k < 128; k++) begin
        dut_in = ~(128'h1 << k);   // all 1s except bit k
        @(posedge clk);
        #1;
        if (dut_out[k] !== 1'b0)
            $error("SA1 detected on bit %0d: expected 0 got %0b", k, dut_out[k]);
    end
endtask

// ================================================================
// Pattern 3: Checkerboard (detects adjacent shorts)
// ================================================================
//
// Alternating 0/1 pattern ensures adjacent bits always differ.
// If two adjacent bits are shorted, one will pull the other to its value.
// Two complementary patterns cover both polarities:

task checkerboard(ref logic [127:0] dut_in, ref logic [127:0] dut_out);
    // Pattern A: 01010101...
    dut_in = 128'h5555_5555_5555_5555_5555_5555_5555_5555;
    @(posedge clk); #1;
    if (dut_out !== dut_in)
        $error("Checkerboard A mismatch: expected %0h got %0h", dut_in, dut_out);

    // Pattern B: 10101010... (complement)
    dut_in = 128'hAAAA_AAAA_AAAA_AAAA_AAAA_AAAA_AAAA_AAAA;
    @(posedge clk); #1;
    if (dut_out !== dut_in)
        $error("Checkerboard B mismatch: expected %0h got %0h", dut_in, dut_out);
endtask

// ================================================================
// Pattern 4: Binary tree / LFSR pseudo-random (wide coverage)
// ================================================================
//
// For detecting complex coupling faults between non-adjacent bits,
// use pseudo-random patterns (PRBS/LFSR):
```

```
//   - N random patterns cover 1-(1-p)^N of all possible fault conditions
//   - For 128-bit bus: 256 random patterns give very high coverage
//   - Much faster than exhaustive 2^128 enumeration

// ================================================================
// Summary: minimum test set for complete stuck-at coverage
// ================================================================
//
//   Pattern            | Patterns | Detects
//   -------------------+----------+--------------------------
//   Walking 1's        |    128   | All SA0 faults
//   Walking 0's        |    128   | All SA1 faults
//   Checkerboard A+B   |      2   | Adjacent short circuits
//   All-0 / All-1      |      2   | Global SA0/SA1
//   ------------------------------------
//   Total minimum:           260 patterns for full coverage
```

解释

Walking 1's 是测试 SA0 最高效的模式：每次只有一个 bit 为1，

如果那个 bit 卡死在0，一个测试向量就能立刻发现。128位总线只需128个测试向量，

而不是穷举 2^128 个组合。

Checkerboard 模式是检测相邻导线短路的关键：

只有当相邻 bit 的值不同时，短路才会导致可观测的差异（一个拉向另一个）；

如果两个相邻 bit 都是1或都是0，即使短路也看不出来。

因此需要两个互补的 checkerboard 模式，确保每对相邻 bit 都经历了"一个0一个1"的组合。

260 个模式 vs 2^128 穷举：这道题考察的是"如何用最少的测试向量达到最高的故障覆盖率"，

这正是 ATPG（Automatic Test Pattern Generation）的核心问题，

面试中能说出 walking 1's/0's + checkerboard 的组合及其覆盖的故障类型是满分回答。

Q87. Verification: Using AI-Generated Code in Verification -- Risks and Workflow

QUESTION

How do you use AI-generated code in a DV flow?

What bugs or issues have you encountered?

ANSWER

```
// ================================================================
// Where AI-generated code is useful in DV
// ================================================================
//
// 1. Boilerplate UVM component skeletons (driver/monitor/scoreboard)
//    AI can produce correct class structure, port declarations,
//    build_phase/connect_phase scaffolding fast.
//
// 2. Sequence/stimulus generation: simple directed test sequences,
//    constraint templates, common protocol transaction patterns.
//
// 3. Golden model / reference model: algorithmic functions
//    (CRC, checksum, encoding) where the math is well-defined.
//
// 4. Scripting: log parsing, coverage analysis scripts (Python).
```

```
// ================================================================
// Common bugs and issues in AI-generated DV code
// ================================================================
//
// Bug class 1: WRONG PROTOCOL TIMING
//   AI often gets 1-cycle off in handshake protocols.
//   Example: driver samples data on posedge but the spec says
//   data must be stable before posedge (setup time assumed).
//   Fix: always verify wave diagram against the spec manually.
//
// Bug class 2: MISSING RESET HANDLING
//   AI-generated drivers frequently skip the reset phase:
//   they start driving immediately without waiting for rst_n to deassert.
//   Result: X propagation from DUT, false assertions fire.
//   Fix: always add @(posedge rst_n) before driving.
//
// Bug class 3: SHALLOW COVERAGE GROUPS
//   AI generates cp_data and cp_addr separately but misses
//   cross coverage between them (the most important coverage).
//   Result: coverage closure looks high but key corner cases missed.
//   Fix: review cover groups for missing cross bins.
//
// Bug class 4: INCORRECT BLOCKING/NON-BLOCKING MIX
//   AI sometimes uses = inside always_ff or mixes = and <= on
//   the same variable -- the classic race condition bug.
//   Fix: always audit always_ff blocks for = vs <=.
//
// Bug class 5: MISSING item_done() IN DRIVER
//   AI-generated UVM drivers sometimes forget seq_item_port.item_done()
//   after driving, causing the sequence to hang (deadlock).
//   Fix: add a checklist: get_next_item -> drive -> item_done.
//
// ================================================================
// Recommended workflow for AI-generated DV code
// ================================================================
//
// Step 1: Generate skeleton -> review structure (naming, hierarchy)
// Step 2: Simulate with simplest directed test, check waveform
// Step 3: Add reset sequence, verify X-free startup
// Step 4: Review coverage groups for cross coverage
// Step 5: Run lint / compile checks (blocking = in always_ff)
// Step 6: Do NOT let AI write the scoreboard golden model
//         without independent verification of the reference math.
```

解释

AI 生成 DV 代码最大的价值在于"消除重复劳动"（boilerplate），

但最大的风险在于"看起来对但时序细节错误"——AI 理解协议的深度有限，

容易在 cycle 边界、握手时序、reset 处理上犯错。

最常被忽视的问题是 coverage group 太浅：

AI 很容易生成独立的 coverpoint 但不会主动识别"哪些 cross 是业务关键的"，

这需要人工根据 spec 和验证计划来设计。

实际工程使用建议：把 AI 当成"快速产生第一稿"的工具，

但每个输出都必须经过人工的协议时序审查 + 波形验证，

不能直接信任 AI 输出的正确性。

Q88. Verification: Memory Debug -- Frontdoor Write + Backdoor Peek to Find Swap Bug

QUESTION

A memory decoder has a swap bug: two addresses are swapped.

Describe how to find it using frontdoor write + backdoor peek.

ANSWER

```
// ================================================================
// The bug: address decoder swaps two locations
// ================================================================
//
// Intended: write to addr A stores data at memory[A]
// Bug: write to addr A stores data at memory[B] (addresses A and B swapped)
//
// Example:
//   Intended: mem[0x10] = X, mem[0x20] = Y
//   With bug:  mem[0x10] = Y, mem[0x20] = X  (swapped!)

// ================================================================
// Debug strategy: frontdoor write + backdoor peek
// ================================================================
//
// Step 1: Write DISTINCT sentinel values via frontdoor (bus protocol)
//         Use values that are obviously different and don't repeat.

task check_decoder_swap(int addr_a, int addr_b);
    logic [31:0] sentinel_a = 32'hAAAA_0000 | addr_a;  // encodes addr in value
    logic [31:0] sentinel_b = 32'hBBBB_0000 | addr_b;
    logic [31:0] peek_val;

    // Frontdoor writes: go through the full bus -> decoder -> memory path
    reg_model.MEM[addr_a].write(.status(status), .value(sentinel_a), .path(UVM_FRONTDOOR));
    reg_model.MEM[addr_b].write(.status(status), .value(sentinel_b), .path(UVM_FRONTDOOR));

    // Step 2: Backdoor peek: BYPASS the decoder, read raw memory cells
    //         directly via HDL path (zero simulation time)
    uvm_hdl_read($sformatf("tb.dut.u_mem.mem[%0d]", addr_a), peek_val);

    // Step 3: Compare -- if swap bug exists:
    //   peek at raw mem[addr_a] should have sentinel_a
    //   If it has sentinel_b -> addr_a and addr_b are SWAPPED in decoder
    if (peek_val == sentinel_b) begin
        `uvm_error("MEM_DBG", $sformatf(
            "SWAP BUG: frontdoor write to addr 0x%0h stored at mem[0x%0h]",
            addr_a, addr_b))
    end else if (peek_val == sentinel_a) begin
        `uvm_info("MEM_DBG", $sformatf(
            "OK: addr 0x%0h -> mem[0x%0h] is correct", addr_a, addr_a), UVM_LOW)
    end else begin
        `uvm_error("MEM_DBG", $sformatf(
            "UNEXPECTED VALUE at mem[0x%0h]: got 0x%0h", addr_a, peek_val))
    end

    // Also peek at addr_b for complete check
    uvm_hdl_read($sformatf("tb.dut.u_mem.mem[%0d]", addr_b), peek_val);
    if (peek_val == sentinel_a)
        `uvm_error("MEM_DBG", $sformatf("CONFIRMED SWAP: sentinel_a found at mem[0x%0h]", addr_b))
endtask

// ================================================================
```

```
// Why frontdoor write + backdoor peek (not all frontdoor)?
// ================================================================
//
// All-frontdoor (write then read):
//   - Write addr A -> read addr A.
//   - If decoder is CONSISTENTLY swapped (A->B always), the read also
//     goes through the same swapped decoder and returns the same wrong value.
//   - You would read sentinel_a (but from the wrong location) -- MISS THE BUG!
//
// Frontdoor write + BACKDOOR peek:
//   - Write goes through the buggy decoder -> lands at mem[B] instead of mem[A].
//   - Backdoor peek reads mem[A] DIRECTLY -- bypasses the decoder.
//   - mem[A] is still empty (or has old data) -> you see the MISMATCH.
//   - The inconsistency between "where you wrote" and "what's in that location"
//     reveals the decoder swap.
//
// This is the classic use case for backdoor access:
// "Verify that the PHYSICAL STORAGE matches the LOGICAL ADDRESS"
// -- something only possible when you can read both layers independently.
```

解释

这道题的核心洞察：如果 decoder swap 是对称的（A总是去B，B总是去A），

那么纯 frontdoor write + frontdoor read 会因为两次都经过同样的 bug decoder 而互相抵消，

你永远看不到错误（写A其实存到B，读A其实也读B，结果匹配但地址全错了）。

Backdoor peek 打破这个对称性：它绕过 decoder 直接读物理地址，

写操作的 intended 地址和物理存储的 actual 地址之间的差异就暴露了。

Sentinel 值的设计：把地址信息编码进数据值（`0xAAAA_0000 | addr`），

这样即使不知道哪两个地址被 swap，看到一个位置里的数据包含了另一个地址的编码就能立刻识别。

Q89. Verification: 1-Input 8-Output Router -- sequence_item, Hash Function, Scoreboard

QUESTION

A router has 1 input port and 8 output ports.

Each packet contains:

- destination_id [31:0]

- protocol [7:0]

- data [511:0]

A hash function maps (destination_id, protocol) to one of 8 output ports.

Write: (a) the sequence_item, (b) the hash function, (c) the scoreboard.

Then describe the testbench connection.

ANSWER

```
// ================================================================
// (a) Sequence Item
// ================================================================
class router_pkt extends uvm_sequence_item;
    `uvm_object_utils_begin(router_pkt)
        `uvm_field_int(destination_id, UVM_DEFAULT)
        `uvm_field_int(protocol,       UVM_DEFAULT)
        `uvm_field_int(data,           UVM_DEFAULT)
```

```
    `uvm_field_int(pkt_id,         UVM_DEFAULT)
    `uvm_field_int(exp_port,       UVM_DEFAULT)
`uvm_object_utils_end

// Packet fields matching DUT interface
rand logic [31:0]  destination_id;
rand logic [7:0]   protocol;
rand logic [511:0] data;

// Metadata (not driven to DUT, used by TB)
static int unsigned next_id = 0;
int unsigned        pkt_id;    // unique ID for scoreboard matching
int unsigned        exp_port;  // expected output port (set in post_randomize)

// Constraints
constraint c_proto_valid {
    // Only test known protocols: TCP=6, UDP=17, ICMP=1, HTTP raw=0x50
    protocol inside {8'h01, 8'h06, 8'h11, 8'h50};
}
constraint c_dest_range {
    // Cover both low and high destination IDs
    destination_id dist {
        [32'h0000_0000 : 32'h0000_00FF] := 30,   // small IDs
        [32'h0000_0100 : 32'hFFFE_FFFF] := 50,   // mid range
        [32'hFFFF_0000 : 32'hFFFF_FFFF] := 20    // large IDs
    };
}

function new(string name = "router_pkt");
    super.new(name);
endfunction

function void post_randomize();
    pkt_id   = next_id++;
    exp_port = hash(destination_id, protocol);  // predict expected port
endfunction

// ================================================================
// (b) Hash function -- maps (dest_id, protocol) -> port [0:7]
// ================================================================
//
// Design rationale:
//   - XOR-based: fast, balanced, and easy to verify in RTL
//   - Multiplying protocol by a prime (0x9E...) spreads bit patterns
//     to avoid clustering when protocol values are small (1,6,17)
//   - Final AND 3'h7 gives port in [0,7]
//
static function int unsigned hash(
    logic [31:0] dest_id,
    logic [7:0]  proto
);
    logic [31:0] mixed;
    // Mix dest_id and protocol with a multiplicative hash constant
    // Fibonacci hashing constant: 0x9E3779B9
    mixed = dest_id ^ (32'(proto) * 32'h9E37_79B9);
    // Fold upper and lower halves for better mixing
    mixed = mixed ^ (mixed >> 16);
    // Take lower 3 bits -> port in [0,7]
    return int'(mixed[2:0]);
endfunction

// Simpler alternative hash (easier to debug):
// return int'(dest_id[2:0] ^ proto[2:0]);
```

```
    function string convert2string();
        return $sformatf(
            "pkt_id=%0d dest=0x%08h proto=0x%02h data[31:0]=0x%08h exp_port=%0d",
            pkt_id, destination_id, protocol, data[31:0], exp_port);
    endfunction
endclass

// ================================================================
// (c) Scoreboard
// ================================================================
//
// Design:
//   - Input monitor feeds expected pkts into scoreboard
//   - 8 output monitors each feed actual received pkts
//   - Scoreboard: key = pkt_id, value = expected pkt
//   - On each output port arrival: look up pkt_id, verify port & data
//
`uvm_analysis_imp_decl(_input)
`uvm_analysis_imp_decl(_output)

class router_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(router_scoreboard)

    // Analysis ports
    uvm_analysis_imp_input  #(router_pkt, router_scoreboard) imp_in;
    uvm_analysis_imp_output #(router_pkt, router_scoreboard) imp_out;

    // Expected packet table: pkt_id -> expected pkt
    router_pkt expected_aa [int unsigned];

    // Statistics
    int unsigned n_sent, n_received, n_errors;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        imp_in  = new("imp_in",  this);
        imp_out = new("imp_out", this);
    endfunction

    // Called by INPUT monitor when packet enters the router
    function void write_input(router_pkt pkt);
        expected_aa[pkt.pkt_id] = pkt;
        n_sent++;
        `uvm_info("SCB", $sformatf("IN: %s", pkt.convert2string()), UVM_HIGH)
    endfunction

    // Called by OUTPUT monitors when packet exits any of the 8 ports
    function void write_output(router_pkt actual);
        int unsigned id = actual.pkt_id;
        n_received++;

        // Check 1: was this packet expected?
        if (!expected_aa.exists(id)) begin
            `uvm_error("SCB", $sformatf(
                "SPURIOUS PKT: pkt_id=%0d at port %0d was never sent",
                id, actual.exp_port))
            n_errors++;
            return;
        end

        router_pkt exp = expected_aa[id];
```

```
        // Check 2: correct output port?
        if (actual.exp_port !== exp.exp_port) begin
            `uvm_error("SCB", $sformatf(
                "WRONG PORT: pkt_id=%0d dest=0x%08h proto=0x%02h: "
                "got port=%0d exp port=%0d",
                id, exp.destination_id, exp.protocol,
                actual.exp_port, exp.exp_port))
            n_errors++;
        end

        // Check 3: data integrity (payload not corrupted)
        if (actual.data !== exp.data) begin
            `uvm_error("SCB", $sformatf(
                "DATA CORRUPTION: pkt_id=%0d: "
                "got data[31:0]=0x%08h exp data[31:0]=0x%08h",
                id, actual.data[31:0], exp.data[31:0]))
            n_errors++;
        end

        // Check 4: destination_id and protocol preserved
        if (actual.destination_id !== exp.destination_id ||
            actual.protocol       !== exp.protocol) begin
            `uvm_error("SCB", $sformatf(
                "HEADER CORRUPTION: pkt_id=%0d", id))
            n_errors++;
        end

        if (n_errors == 0)
            `uvm_info("SCB", $sformatf(
                "PASS pkt_id=%0d -> port %0d", id, actual.exp_port), UVM_MEDIUM)

        expected_aa.delete(id);
    endfunction

    function void check_phase(uvm_phase phase);
        super.check_phase(phase);
        // Packets sent but never received (dropped or lost)
        if (expected_aa.num() > 0) begin
            `uvm_error("SCB", $sformatf(
                "%0d packets never arrived at any output port!",
                expected_aa.num()))
            foreach (expected_aa[id])
                `uvm_error("SCB", $sformatf(
                    "  Lost: pkt_id=%0d dest=0x%08h proto=0x%02h exp_port=%0d",
                    id, expected_aa[id].destination_id,
                    expected_aa[id].protocol, expected_aa[id].exp_port))
        end
        `uvm_info("SCB", $sformatf(
            "RESULT: sent=%0d received=%0d errors=%0d",
            n_sent, n_received, n_errors), UVM_NONE)
    endfunction
endclass

// ============================================================
// Testbench connection (env.connect_phase)
// ============================================================
//
// env
//   ■■■ input_agent            (1 agent for the single input port)
//   ■     ■■■ input_driver     drives dest_id, protocol, data, valid
//   ■     ■■■ input_monitor    observes input packets, feeds SCB
//   ■■■ output_agent[8]        (1 passive agent per output port)
//   ■     ■■■ output_monitor   observes output data, feeds SCB
```

```
//   ■■■ scoreboard
//   ■■■ virtual_sequencer      (coordinates input traffic sequences)
//
// connect_phase:
//   input_agent.monitor.ap.connect(scoreboard.imp_in);
//   for (int p = 0; p < 8; p++)
//       output_agents[p].monitor.ap.connect(scoreboard.imp_out);
//
// Output monitor must observe WHICH PORT a packet exited on.
// Two approaches:
//   A. Each port has its own monitor instance, which sets actual.exp_port = port_id.
//   B. Single monitor with a port_id parameter set at build time.

// ============================================================
// Coverage: ensure all 8 ports and all protocols are exercised
// ============================================================
covergroup router_cg @(posedge clk);
    cp_port: coverpoint actual_port {
        bins port[] = {[0:7]};    // all 8 output ports
    }
    cp_proto: coverpoint protocol {
        bins tcp  = {8'h06};
        bins udp  = {8'h11};
        bins icmp = {8'h01};
        bins other = default;
    }
    // Cross: every protocol routed to every port
    cx_port_proto: cross cp_port, cp_proto;
    // Consecutive packets to SAME port (stress same-port queue)
    cp_port_repeat: coverpoint actual_port {
        bins same_port_twice = (0 => 0), (1 => 1), (2 => 2),
                               (3 => 3), (4 => 4), (5 => 5),
                               (6 => 6), (7 => 7);
    }
endgroup
```

解释

Sequence item 的设计要点：packet 结构和 DUT 接口一一对应（dest_id/protocol/data），

`pkt_id` 是 TB 内部的唯一标识符（不来自 DUT），用于 scoreboard 的 associative array 查找；

`exp_port` 在 post_randomize 里通过同一个 hash 函数预先计算，

这样 scoreboard 可以直接比对"DUT 实际输出的 port"和"TB 预测的 port"是否一致。

Hash 函数设计：使用 Fibonacci hashing 常数 `0x9E3779B9`（2^32/phi，phi 为黄金比例），

与 protocol 相乘可以把小整数值（1/6/17/80）散布到整个 32-bit 空间，

避免多个 protocol 哈希到相同端口的聚集现象。

Scoreboard 的 4 条检查体现了完整的路由验证：

端口正确性（最核心）、数据完整性（payload 未被损坏）、头部完整性（dest_id/protocol 未篡改）、

以及 `check_phase` 里的"丢包检测"（sent 但 never received）。

Coverage 的 cross bin `cp_port x cp_proto` 确保每种协议都经过了每个端口，

`cp_port_repeat` 验证了同一端口背靠背接收两个包（队列/缓冲压力测试）。

Q90. Verification: Memory Verification -- Complete Strategy and UVM Environment

QUESTION

How do you verify a memory DUT? Cover: direct testing, random access, address aliasing, performance, X-propagation, power, and GLS. Also describe the UVM environment setup.

ANSWER

```
// ================================================================
// UVM Environment structure for memory verification
// ================================================================
//
// env
//   ■■■ mem_agent (AXI/APB master)
//   ■     ■■■ mem_driver    -- drives read/write transactions
//   ■     ■■■ mem_monitor   -- observes bus, sends to scoreboard
//   ■■■ mem_scoreboard      -- golden model: shadow memory array
//   ■■■ mem_coverage        -- address/data/pattern coverage
//   ■■■ mem_reg_model       -- RAL model for register-mapped memories
//
// Scoreboard golden model:
//   logic [DATA_W-1:0] shadow_mem [ADDR_W]; // mirrors DUT memory
//   On write: shadow_mem[addr] = data;
//   On read:  assert(dut_data == shadow_mem[addr]);

// ================================================================
// 1. Direct Testing (directed tests)
// ================================================================
//
// Write then Read-Back (W-R-B):
//   Write a known pattern to address A, read back, compare.
//   Purpose: basic functionality, data integrity.
//
// Key patterns:
//   All-zeros:       0x00000000  (stuck-at-1 detection)
//   All-ones:        0xFFFFFFFF  (stuck-at-0 detection)
//   Checkerboard:    0x55555555 / 0xAAAAAAAA (adjacent bit coupling)
//   Walking-ones:    0x00000001, 0x00000002, 0x00000004...
//   Address-as-data: write addr=0x100 with data=0x100 (easy to spot swap)
//
// March test (industry standard for SRAM):
//   M0: (w0)              -- write 0 to all addresses ascending
//   M1: (r0, w1)          -- read 0 (verify), write 1, ascending
//   M2: (r1, w0)          -- read 1 (verify), write 0, descending
//   M3: (r0)              -- read 0 (verify) ascending
//   Detects: SAF, TF (Transition Fault), CF (Coupling Fault)

// ================================================================
// 2. Random Access Testing
// ================================================================
//
// Constrained-random stimulus:
//   - Random addresses (full range, boundary, near 0 and max)
//   - Random data values
//   - Random read/write mix ratios (pure write, pure read, 50/50)
//   - Back-to-back: write immediately followed by read to same addr
//   - Out-of-order: write addr A, write addr B, read addr A
//
// Coverage targets:
//   cp_addr: coverpoint addr {
//       bins low  = {[0 : DEPTH/4]};
//       bins mid  = {[DEPTH/4 : 3*DEPTH/4]};
//       bins high = {[3*DEPTH/4 : DEPTH-1]};
//       bins exact_zero = {0};
```

```
//       bins exact_max  = {DEPTH-1};
//   }
//   cp_rw: coverpoint is_write { bins wr={1}; bins rd={0}; }
//   cx_addr_rw: cross cp_addr, cp_rw;

// ================================================================
// 3. Address Aliasing
// ================================================================
//
// Aliasing bug: two different addresses map to the same physical cell.
// Write unique sentinel to addr A, read addr B -> if data matches, aliased.
//
// Detection method (frontdoor + backdoor):
//   for each pair (addr_a, addr_b):
//     write sentinel_a to addr_a (via bus)
//     write sentinel_b to addr_b (via bus)
//     backdoor peek addr_a -> should be sentinel_a, not sentinel_b
//     backdoor peek addr_b -> should be sentinel_b, not sentinel_a
//
// Sentinel encoding: data = 32'hA000_0000 | addr  (address embedded in data)
//   -> Makes it obvious which address was "supposed to" land where.
//
// Coverage: address alias check covers all address pairs in suspicious ranges
//   (typically boundaries between SRAM tiles, decoder boundaries)

// ================================================================
// 4. Performance Verification
// ================================================================
//
// Measure latency and throughput:
//   Read latency:     cycles from valid read request to valid data
//   Write latency:    cycles from valid write to acknowledge
//   Throughput:       transactions per clock (burst efficiency)
//
// Test scenarios:
//   - Back-to-back reads: request N reads consecutively, measure total time
//   - Read-after-write: write then immediately read (checks forwarding)
//   - Simultaneous read/write (if dual-port)
//   - ECC overhead: measure extra latency when ECC is enabled
//
// SVA performance check:
//   property p_read_latency;
//       @(posedge clk)
//       $rose(read_req) |-> ##[1:MAX_LAT] read_valid;
//   endproperty

// ================================================================
// 5. X-Propagation (Xprop)
// ================================================================
//
// Memory outputs should never be X after reset or during normal operation.
// Common X sources:
//   - Reading uninitialized cells (before any write)
//   - Enable/select signal is X (due to missing reset or CDC issue)
//   - ECC decoder receiving X on data bus
//
// Testing approach:
//   a. Simulation with Xprop mode (VCS: -xprop=tmerge or flowthrough)
//      Simulator propagates X more aggressively, exposing hidden issues.
//   b. Directed test: read before write to uninitialized location.
//      DUT should return 0 (if spec says initialized to 0) or X (if undefined).
//      Verify scoreboard doesn't compare against stale 0 data.
//   c. SVA: assert (!$isunknown(data_out)) in normal operation mode.
```

```
// ================================================================
// 6. Power Verification
// ================================================================
//
// Memory power modes: active, retention, power-down, deep sleep.
//
// Test scenarios:
//   a. Entry/exit sequence: verify control signals follow spec order.
//      (e.g. must gate clock before asserting shutdown, not simultaneously)
//   b. Retention test: enter retention, wait N cycles, exit.
//      Data written before retention must match data read after exit.
//   c. Power-domain isolation: verify memory outputs are held at known
//      value (not X) when the domain is off (isolation cells active).
//   d. Wakeup latency: cycles from power-up request to memory ready.

// ================================================================
// 7. Gate-Level Simulation (GLS)
// ================================================================
//
// RTL-to-GLS equivalence:
//   Run the same test suite on the gate-level netlist (post-synthesis/PnR).
//   Compare output waveforms against RTL golden reference.
//
// GLS-specific checks:
//   a. Timing: run with annotated SDF delays.
//      Checks that critical paths (address decode, sense amp) meet timing.
//   b. Functional: verify gate-level netlist produces same results as RTL.
//   c. X-initialization: in GLS, flip-flops start at X (not 0).
//      Test must verify reset sequence brings all outputs to known state.
//   d. Back-annotation: use actual extracted RC parasitics from PnR.
//      Catches timing issues invisible in RTL simulation.
```

解释

Memory 验证的层次从"基本功能"到"可靠性"再到"实现正确性"递进：

Direct testing（March test）覆盖基本故障模型（SAF/TF/CF）；

Random 测试覆盖统计意义上的完整性；

Aliasing 测试专门针对地址解码的结构性 bug；

Performance/Xprop/Power 是超越功能正确性的高级验证维度。

GLS 是整个验证流程的终点：RTL simulation 验证功能逻辑，

GLS 验证"综合+布局布线后的实现"是否还保持相同的功能和时序——

很多 timing bug（比如 hold 违例导致的 race condition）只在 GLS 阶段才能被发现。

UVM

Q91. UVM: Highest Priority Completed Sequence

QUESTION

In UVM, when multiple sequences are running on the same sequencer,

how does the sequencer decide which sequence item to send next?

What does "highest priority completed sequence" mean?

ANSWER

```
// UVM Sequencer Arbitration
//
// When multiple sequences compete for the SAME sequencer/driver,
// the sequencer uses an ARBITRATION algorithm to pick which sequence's
// next item to execute.
//
// How sequences request access:
//   start_item(req) -> sequence sends a GRANT REQUEST to the sequencer
//   The sequencer arbitrates among all pending requests and grants
//   access to one sequence at a time.
//   After finish_item(req), the sequence releases the grant.
//
// Default arbitration mode: SEQ_ARB_FIFO (first-come, first-served)
//   Sequences that called start_item first get granted first.
//   Simple, fair, no priority concept.
//
// Priority-based arbitration: SEQ_ARB_WEIGHTED or SEQ_ARB_STRICT_FIFO
//   Each sequence can be assigned a PRIORITY (integer, higher = more urgent)
//   sequencer.set_arbitration(UVM_SEQ_ARB_WEIGHTED);
//   seq1.set_priority(200);   // higher priority
//   seq2.set_priority(100);   // lower priority
//
// "Highest priority completed sequence":
//   In UVM_SEQ_ARB_STRICT_FIFO or similar modes, the sequencer picks
//   the sequence that:
//   1. Has the HIGHEST PRIORITY (among all sequences with pending requests), AND
//   2. Has COMPLETED a full start_item / finish_item handshake cycle
//      (i.e., the sequence item is fully prepared and ready to be driven)
//   The "completed" qualifier means the sequence has finished randomizing
//   and packaging its item -- the sequencer will NOT grant to a sequence
//   that requested access but hasn't finished building its item yet.

// ---- Arbitration modes summary -----------------------------------------
//
//   UVM_SEQ_ARB_FIFO          : first-come-first-served (default)
//   UVM_SEQ_ARB_WEIGHTED      : grant proportional to priority weight
//                               (higher priority = more grants over time)
//   UVM_SEQ_ARB_STRICT_FIFO   : strict priority; within same priority: FIFO
//   UVM_SEQ_ARB_STRICT_RANDOM : strict priority; within same priority: random
//   UVM_SEQ_ARB_RANDOM        : random selection ignoring priority
//   UVM_SEQ_ARB_USER          : user-defined arbitration function

// ---- Example: two sequences competing ----------------------------------
//
// class high_prio_seq extends uvm_sequence;
//     task body();
//         my_txn txn = my_txn::type_id::create("txn");
//         start_item(txn);          // request access from sequencer
//         assert(txn.randomize());  // prepare item (this is the "complete" step)
//         finish_item(txn);         // grant released; item sent to driver
//     endtask
// endclass
//
// In the test:
//   high_prio_seq seq_h = high_prio_seq::type_id::create("seq_h");
//   low_prio_seq  seq_l = low_prio_seq::type_id::create("seq_l");
//   sequencer.set_arbitration(UVM_SEQ_ARB_STRICT_FIFO);
//   seq_h.set_priority(200);
//   seq_l.set_priority(100);
```

```
//   fork
//       seq_h.start(sequencer);
//       seq_l.start(sequencer);
//   join
//   // seq_h items will be driven first (higher priority)
//   // seq_l gets turns only when seq_h has no pending request
```

解释

UVM sequencer 的仲裁机制是多 sequence 场景下的核心知识点。

"highest priority completed sequence" 的含义是双重条件：

不仅要优先级最高，还要是"已经 finish randomize 准备好了的"——

sequencer 不会等一个慢的高优先级 sequence 完成 randomize，

而是选"已完成准备的所有 sequence 中优先级最高的那个"。

这是面试中容易混淆的点：priority 只在多个 sequence "同时"准备好且竞争同一 grant 时才起作用，

如果高优先级 sequence 还没 start_item，低优先级已完成 start_item 的会先得到 grant。

实际工程中最常用的是默认的 FIFO 模式，只有在需要精细控制（如压力测试 + 错误注入并发）时才切换到 WEIGHTED。

Q92. UVM: Virtual Sequencer -- Two Usage Patterns

QUESTION

What is a virtual sequencer in UVM? Describe the TWO main ways to use it.

ANSWER

```
// ================================================================
// What is a virtual sequencer?
// ================================================================
//
// A virtual sequencer is a uvm_sequencer that does NOT drive a
// specific interface directly. Instead, it HOLDS HANDLES to multiple
// real sequencers and coordinates sequences across them.
//
// It acts as an ORCHESTRATOR: one virtual sequence can simultaneously
// control an AXI agent, an I2C agent, and a GPIO agent, keeping them
// in sync without each needing to know about the others.

// ================================================================
// Usage Pattern 1: Virtual Sequence + Virtual Sequencer (classic)
// ================================================================
//
// Step 1: Define the virtual sequencer holding handles to real seqrs.
class my_virtual_sequencer extends uvm_sequencer;
    `uvm_component_utils(my_virtual_sequencer)

    // Handles to the real sequencers (set in env.connect_phase)
    axi_sequencer  axi_seqr;
    i2c_sequencer  i2c_seqr;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
endclass
```

```
// Step 2: Define a virtual sequence that uses those handles.
class my_virtual_sequence extends uvm_sequence;
    `uvm_object_utils(my_virtual_sequence)

    function new(string name = "my_virtual_sequence");
        super.new(name);
    endfunction

    task body();
        axi_write_seq  axi_seq = axi_write_seq::type_id::create("axi_seq");
        i2c_read_seq   i2c_seq = i2c_read_seq::type_id::create("i2c_seq");

        // Access real sequencers through the virtual sequencer handle
        // p_sequencer is automatically cast to my_virtual_sequencer
        fork
            axi_seq.start(p_sequencer.axi_seqr);
            i2c_seq.start(p_sequencer.i2c_seqr);
        join
    endtask
endclass
// NOTE: virtual sequence must declare `uvm_declare_p_sequencer(my_virtual_sequencer)
// to get a typed p_sequencer handle:
// class my_virtual_sequence extends uvm_sequence;
//     `uvm_declare_p_sequencer(my_virtual_sequencer)

// Step 3: In the test, start the virtual sequence on the virtual sequencer.
// my_virt_seq.start(env.virt_seqr);

// ================================================================
// Usage Pattern 2: Virtual Sequence WITHOUT a virtual sequencer
//   (start on null, pass sequencer handles via config_db)
// ================================================================
//
// Some teams skip creating a virtual sequencer entirely.
// Instead, the virtual sequence gets real sequencer handles
// from uvm_config_db and starts sub-sequences directly.
class my_virtual_seq_no_vseqr extends uvm_sequence;
    `uvm_object_utils(my_virtual_seq_no_vseqr)

    axi_sequencer  axi_seqr;
    i2c_sequencer  i2c_seqr;

    function new(string name = "my_virtual_seq_no_vseqr");
        super.new(name);
    endfunction

    task body();
        // Get real sequencer handles from config_db
        if (!uvm_config_db #(axi_sequencer)::get(null, "*", "axi_seqr", axi_seqr))
            `uvm_fatal("VSEQ", "Cannot get axi_seqr from config_db")
        if (!uvm_config_db #(i2c_sequencer)::get(null, "*", "i2c_seqr", i2c_seqr))
            `uvm_fatal("VSEQ", "Cannot get i2c_seqr from config_db")

        axi_write_seq axi_seq = axi_write_seq::type_id::create("axi_seq");
        i2c_read_seq  i2c_seq = i2c_read_seq::type_id::create("i2c_seq");

        fork
            axi_seq.start(axi_seqr);   // start directly on real sequencer
            i2c_seq.start(i2c_seqr);
        join
    endtask
endclass
```

```
// This virtual sequence is started with null sequencer:
// my_virt_seq.start(null);   // see next question for what null means

// ================================================================
// Comparison of the two patterns
// ================================================================
//
//   Pattern                 | Virtual Seqr + Virtual Seq | No Virtual Seqr (null)
//   ------------------------+----------------------------+-----------------------
//   Infrastructure          | Extra vseqr component      | Leaner, fewer components
//   Sequencer access        | Via p_sequencer handles    | Via config_db or field
//   Arbitration support     | Yes (can add arbitration)  | No arbitration layer
//   Reusability             | High (portable seqr setup) | Lower (depends on cfg)
//   Common in industry      | Traditional/large projects | Lean/modern projects
```

解释

Virtual sequencer 的核心价值是"跨 agent 协调"：

真实 sequencer 只管自己的协议（AXI、I2C、SPI），

virtual sequencer 提供一个"指挥层"，让一个 virtual sequence 能同时操控多个 agent。

两种模式的选择依据：

Pattern 1（有 virtual sequencer）：适合大型环境，需要仲裁（arbitration）、

或者多个 virtual sequence 共享同一套 sequencer 句柄时。

Pattern 2（null 启动）：适合轻量环境，virtual sequence 自己通过 config_db 获取所需句柄，

不需要额外的基础设施，代码更精简，很多现代 UVM 项目偏好这种方式。

`uvm_declare_p_sequencer(T)` 宏是 Pattern 1 的关键：

它给 virtual sequence 提供一个类型化的 p_sequencer 句柄，

避免手动 $cast，并在 sequence 和 sequencer 类型不匹配时给出清晰的错误信息。

Q93. UVM: seq.start(null) -- What Happens When Sequencer is null?

QUESTION

In UVM, what happens when you call seq.start(null)?

When is this valid and when does it cause problems?

ANSWER

```
// seq.start(sequencer) API:
//   The sequencer argument tells the sequence WHERE to send its items.
//   start(null) means "run this sequence without binding it to any sequencer".

// ================================================================
// What seq.start(null) DOES:
// ================================================================
//
// When you call seq.start(null):
//   1. The sequence's body() task is called normally.
//   2. m_sequencer (the sequence's internal sequencer pointer) is set to null.
//   3. p_sequencer (if declared via uvm_declare_p_sequencer) is also null.
//
// Any call to start_item() / finish_item() INSIDE the sequence will FAIL
// at runtime because start_item() needs a valid sequencer to send items to.
```

```
//
// THEREFORE: start(null) is ONLY valid for sequences that do NOT call
// start_item() / finish_item() themselves -- i.e., VIRTUAL SEQUENCES
// that only launch sub-sequences via sub_seq.start(some_real_sequencer).

// ================================================================
// Valid use case: virtual sequence (no direct item generation)
// ================================================================
class my_virtual_seq extends uvm_sequence;
    `uvm_object_utils(my_virtual_seq)

    task body();
        axi_write_seq axi_seq = axi_write_seq::type_id::create("axi_seq");
        i2c_read_seq  i2c_seq = i2c_read_seq::type_id::create("i2c_seq");

        // Sub-sequences are started on REAL sequencers -- start(null) is fine here
        // because THIS sequence never calls start_item() itself
        fork
            axi_seq.start(axi_seqr);   // axi_seqr obtained from config_db or field
            i2c_seq.start(i2c_seqr);
        join
    endtask
endclass

// In the test:
// my_virtual_seq vseq = my_virtual_seq::type_id::create("vseq");
// vseq.start(null);   // VALID: vseq never calls start_item()

// ================================================================
// INVALID use case: regular sequence calling start(null)
// ================================================================
class my_regular_seq extends uvm_sequence #(my_txn);
    task body();
        my_txn txn = my_txn::type_id::create("txn");
        start_item(txn);          // <<< CRASH if started with null!
        assert(txn.randomize());
        finish_item(txn);         // <<< CRASH if started with null!
    endtask
endclass
//
// my_regular_seq.start(null) -> UVM_FATAL at start_item():
// "Error: 'start_item' called on sequence with null sequencer"
// This is a common bug when a regular sequence is accidentally
// started on null instead of a valid sequencer.

// ================================================================
// How null interacts with m_sequencer and parent_sequence
// ================================================================
//
// m_sequencer hierarchy propagation:
//   When seq.start(seqr) is called, the sequencer is stored in
//   m_sequencer. Sub-sequences inherit this if not explicitly overridden:
//     sub_seq.start(null, this)  // inherits parent's m_sequencer
//
//   BUT in the virtual sequence case, m_sequencer IS null, so
//   sub-sequences MUST be explicitly passed a real sequencer handle:
//     axi_seq.start(axi_seqr)    // explicit real sequencer

// ================================================================
// Quick rule summary
// ================================================================
//
//   Sequence type        | start(null) | start(real_seqr)
```

```
//   ---------------------+-------------|------------------
//   Virtual sequence      | VALID       | Usually wrong
//   (no start_item calls) |             | (would bind to wrong seqr)
//   Regular sequence      | CRASH at    | CORRECT
//   (calls start_item)    | start_item  |
```

解释

start(null) 的本质是"允许 sequence 的 body() 运行，但不绑定任何 sequencer"。

只要 body() 里没有 start_item()/finish_item()，这就完全合法——

virtual sequence 恰好满足这个条件（它只启动子 sequence，不自己产生 item）。

最容易犯的错误：把一个普通 sequence 误传 null，在 start_item() 处运行时崩溃。

这个错误在大型项目里很常见，因为 sequence 层级深，顶层 virtual sequence 用 null 启动，

某个中间层无意间直接 start_item 而没有 sub-seq 化，就会出现这个 fatal error。

另一个容易混淆的点：sub_seq.start(null, this) 中的 this 是 parent_sequence，

这会让 sub_seq 继承 parent 的 m_sequencer（包括 null），而不是自动获得真实 sequencer。

所以在 virtual sequence 里必须显式传入真实的 sequencer 句柄给每个 sub-sequence。

Q94. UVM: Factory Override -- Instance Override and Type Override

QUESTION

Explain the UVM factory mechanism. How do you write an instance override

vs a type override? Where should overrides be registered?

ANSWER

```
// ================================================================
// UVM Factory mechanism
// ================================================================
//
// The UVM factory is a REGISTRY that maps class names (strings) to
// class types, enabling RUNTIME TYPE SUBSTITUTION without changing
// the environment code. When any component/object calls
// T::type_id::create("name", parent), the factory intercepts this
// and can redirect it to return a DERIVED class instead.
//
// This implements the Open-Closed Principle:
//   - Open for extension (add new derived classes)
//   - Closed for modification (no changes to env/agent/driver code)
//
// Registration (required for factory to know the class):
class my_driver extends uvm_driver #(my_txn);
    `uvm_component_utils(my_driver)   // registers my_driver with factory
    // ...
endclass
//
class error_driver extends my_driver;
    `uvm_component_utils(error_driver) // registers error_driver with factory
    // override run_phase to inject errors ...
endclass

// ================================================================
// Type Override: replace ALL instances of base with derived
// ================================================================
```

```
//
// Syntax:
//   <base_type>::type_id::set_type_override(<derived_type>::get_type());
//
// Effect: every call to my_driver::type_id::create() ANYWHERE in the
//   testbench will return an error_driver object instead.
//
class error_injection_test extends base_test;
    function void build_phase(uvm_phase phase);
        // Type override: replace ALL my_driver with error_driver globally
        my_driver::type_id::set_type_override(error_driver::get_type());
        super.build_phase(phase);   // env builds AFTER override is set
    endfunction
endclass

// ================================================================
// Instance Override: replace ONE specific instance by hierarchical path
// ================================================================
//
// Syntax:
//   <base_type>::type_id::set_inst_override(
//       <derived_type>::get_type(), "<full_instance_path>");
//
// Effect: only the driver at the specified hierarchical path is overridden;
//   other instances of my_driver are unaffected.
//
class selective_error_test extends base_test;
    function void build_phase(uvm_phase phase);
        // Instance override: only env.agent0.driver gets error_driver
        // env.agent1.driver remains my_driver
        my_driver::type_id::set_inst_override(
            error_driver::get_type(),
            "uvm_test_top.env.agent0.driver"  // exact hierarchical path
        );
        super.build_phase(phase);
    endfunction
endclass

// ================================================================
// WHERE to write overrides -- critical timing rule
// ================================================================
//
// Overrides MUST be set BEFORE the object/component is created.
// The typical place is build_phase() of the TEST, BEFORE calling
// super.build_phase() (which triggers env -> agent -> driver build).
//
// Correct order:
//   1. Test::build_phase() starts
//   2. Set override (factory registers the substitution)  <-- HERE
//   3. super.build_phase() -> env.build_phase() -> agent.build_phase()
//      -> driver = my_driver::type_id::create() -> factory returns error_driver
//
// WRONG order (override too late):
//   1. super.build_phase() called first -> driver already created as my_driver
//   2. Override set after -> has NO effect on already-created components
//
// Type override:    set in test's build_phase BEFORE super
// Instance override: set in test's build_phase BEFORE super
//
// You can also use the +uvm_set_type_override plusarg at runtime:
//   +uvm_set_type_override=my_driver,error_driver
// (UVM framework parses this and calls set_type_override automatically)
```

解释

Factory 的核心价值：测试代码只写一次，通过 override 就能切换 DUT 激励行为，
无需修改 env/agent 代码——符合"用例驱动测试"的 DV 设计原则。
Type override vs Instance override 的选择：
Type override 是全局的，适合"这个 test 里所有 driver 都换"的场景（如全局错误注入）；
Instance override 是精准的，适合"只有某一个 agent 的 driver 换"的场景
（如多个 agent 中只有一个需要特殊行为）。
最关键的陷阱：override 必须在 create 之前调用。
在 test::build_phase 里，如果先调用 super.build_phase()，
env 就已经开始 build 并调用 create()，此时再设 override 已经来不及。
标准写法是：先设 override，再调 super.build_phase()，让 create 走到工厂时
已经有了正确的替换规则。

Q95. UVM: Mailbox vs Queue vs Associative Array -- When to Use Which

QUESTION

In UVM/SystemVerilog verification, when do you use a mailbox, a queue,
and an associative array? Compare their use cases.

ANSWER

```
// ================================================================
// Mailbox (uvm_tlm_analysis_fifo / built-in mailbox)
// ================================================================
//
// A mailbox is a THREAD-SAFE FIFO for INTER-PROCESS COMMUNICATION.
//
// Key properties:
//   - Blocking: get() blocks the calling process until data is available.
//   - put() can also block if the mailbox has a bounded capacity.
//   - Thread-safe: safe for concurrent put/get from different processes.
//   - Ordering: FIFO (first in, first out).
//
// Use mailbox when:
//   1. Two concurrent processes (e.g. monitor and scoreboard) need to
//      exchange data safely across a time boundary.
//   2. You want BLOCKING behavior: consumer waits until producer has data.
//   3. Multiple producers write to one consumer (fan-in communication).
//
// Example: monitor captures transactions, scoreboard consumes them.
//   mailbox #(my_txn) mbx = new();
//   // Monitor (producer):  mbx.put(txn);
//   // Scoreboard (consumer): mbx.get(txn); // blocks until txn available

// ================================================================
// Queue (dynamic array with push/pop)
// ================================================================
//
// A queue is a SINGLE-THREADED DATA STRUCTURE (not thread-safe).
//
// Key properties:
//   - Non-blocking: peek/pop do not block if empty (must check size first).
```

```
//   - Can push_front / push_back / pop_front / pop_back.
//   - Random access by index: q[i].
//   - Ordering: maintained as inserted.
//
// Use queue when:
//   1. Within a SINGLE process (no concurrency concerns).
//   2. You need ORDERED storage where items arrive in sequence.
//   3. Scoreboard expected-value tracking in a simple in-order protocol.
//   4. You need index access (mailbox does not support indexing).
//
// Example: scoreboard for in-order DUT.
//   my_txn expected_q[$];
//   // On input:   expected_q.push_back(expected_txn);
//   // On output:  actual = dut_q.pop_front();
//   //             compare(actual, expected_q.pop_front());

// ================================================================
// Associative Array
// ================================================================
//
// An associative array is a KEY-VALUE MAP (like a dictionary/hash map).
//
// Key properties:
//   - O(1) average access by key (any type: int, string, struct).
//   - No implicit ordering (insertion order not preserved).
//   - exists(key) checks presence; delete(key) removes entry.
//
// Use associative array when:
//   1. Responses can arrive OUT-OF-ORDER: use transaction ID as key.
//   2. You need to LOOK UP a specific item without knowing its position.
//   3. Sparse storage: only store items for keys that actually exist.
//   4. Per-channel or per-ID tracking in multi-stream environments.
//
// Example: out-of-order scoreboard.
//   my_txn expected_aa [int];   // key = transaction ID
//   // On input:   expected_aa[txn.id] = expected_txn;
//   // On output:  if (expected_aa.exists(actual.id)) begin
//   //               compare(actual, expected_aa[actual.id]);
//   //               expected_aa.delete(actual.id);
//   //             end

// ================================================================
// Decision table
// ================================================================
//
//   Need                           | Use
//   -------------------------------+--------------------
//   Inter-process (concurrent) comm | mailbox
//   Blocking consumer waits for data| mailbox
//   In-order, single-process FIFO   | queue
//   Index access within single proc | queue
//   Out-of-order by ID lookup       | associative array
//   Sparse / conditional key lookup | associative array
//   Multi-stream per-channel track  | associative array
```

解释

三种数据结构覆盖了 UVM 验证中的三种典型场景：

Mailbox 解决"跨进程通信"问题，阻塞语义让消费者不需要轮询，是 monitor→scoreboard 的标准通道。

Queue 解决"单进程有序缓冲"问题，适合简单的in-order协议验证，支持下标访问。

Associative array 解决"乱序匹配"问题，用 transaction ID 做 key 可以在 O(1) 时间内找到对应的期望值，

是验证 AXI、PCIe 等支持乱序响应协议的必备数据结构。

混合使用也很常见：mailbox 负责跨进程传递数据，

associative array 在 scoreboard 内部做乱序匹配存储。

Q96. UVM: Component Hierarchy and Driver-Sequencer Handshake

QUESTION

Walk through the UVM component hierarchy. Then explain the

driver-sequencer handshake mechanism step by step.

ANSWER

```
// ================================================================
// UVM Component Hierarchy
// ================================================================
//
// uvm_test
//   ■■■ uvm_env
//         ■■■ uvm_agent (active)
//         ■     ■■■ uvm_sequencer   -- arbitrates sequences, sends items to driver
//         ■     ■■■ uvm_driver      -- drives items onto the DUT interface
//         ■     ■■■ uvm_monitor     -- passively observes DUT interface
//         ■■■ uvm_agent (passive)   -- monitor only (no driver/sequencer)
//         ■     ■■■ uvm_monitor
//         ■■■ uvm_scoreboard        -- compares expected vs actual
//         ■■■ uvm_coverage_collector-- functional coverage sampling
//
// Connections (made in connect_phase):
//   - Driver's  seq_item_port  <--> Sequencer's seq_item_export
//   - Monitor's analysis_port  -->  Scoreboard's analysis_imp
//   - Monitor's analysis_port  -->  Coverage's  analysis_imp

// ================================================================
// Driver-Sequencer Handshake (TLM-based pull model)
// ================================================================
//
// The sequencer-driver interface uses a PULL model: the driver
// requests items from the sequencer when it is ready.
//
// Step-by-step flow:
//
// SEQUENCE SIDE (body() task):
//   1. Sequence calls: start_item(req)
//      -> Request sent to sequencer: "I have an item ready to send"
//      -> Sequence BLOCKS here until sequencer grants access
//         (arbitration: sequencer picks among competing sequences)
//
//   2. Sequencer grants access -> sequence unblocks.
//
//   3. Sequence calls: assert(req.randomize()) -- randomize the item
//
//   4. Sequence calls: finish_item(req)
//      -> Item handed to sequencer's export
//      -> Sequence BLOCKS again until driver calls item_done()
//
// DRIVER SIDE (run_phase task):
//   5. Driver calls: seq_item_port.get_next_item(req)
//      -> Driver BLOCKS until sequencer has an item ready
//      -> When item available, req is populated
```

```
//
//   6. Driver drives req onto the DUT interface (via virtual interface):
//      vif.addr  <= req.addr;
//      vif.data  <= req.data;
//      @(posedge vif.clk);   -- wait for clock cycles as needed
//
//   7. Driver calls: seq_item_port.item_done()
//      -> Signals sequencer: "item has been driven, I'm ready for next"
//      -> Sequence's finish_item() unblocks
//      -> Sequence can now prepare and randomize the next item
//
// Repeat from step 1 for each transaction.
//
// Key methods summary:
//   Sequence:  start_item(req) -> randomize -> finish_item(req)
//   Driver:    get_next_item(req) -> drive -> item_done()
//
// get_next_item vs try_next_item:
//   get_next_item: BLOCKING (driver stalls until item available)
//   try_next_item: NON-BLOCKING (returns null if nothing pending)
//                  Use when driver needs to generate idle cycles.
```

解释

UVM 的 pull model 设计理念：driver 主动拉取（get_next_item），

而不是 sequencer 主动推送，这让 driver 完全控制驱动节奏，

不需要关心 sequence 什么时候产生数据。

start_item/finish_item 的两次阻塞各有意义：

start_item 等待仲裁（当多个 sequence 竞争同一 sequencer 时决定谁先）；

finish_item 等待 driver 确认驱动完成，确保 item 被完整消化后 sequence 才继续产生下一个。

item_done() 是"握手完成"的信号，缺少它会导致 driver 和 sequence 死锁，

这是 UVM driver 实现中最常见的 bug 之一。

Q97. UVM: Modify Driver for One Test Without Touching Other Tests

QUESTION

You want one specific test to use a modified driver behavior without

changing the base driver class or affecting other tests. How?

ANSWER

```
// Answer: Use the UVM factory INSTANCE OVERRIDE in that test's build_phase.
//
// Step 1: Create a derived driver with the modified behavior.
class error_injection_driver extends my_driver;
    `uvm_component_utils(error_injection_driver)

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    // Override only the specific task that needs modification
    task drive_item(my_txn txn);
        // Inject errors: flip a random bit 10% of the time
        if ($urandom_range(0,9) == 0)
```

```
            txn.data ^= (1 << $urandom_range(0, 31));
        super.drive_item(txn);   // then call original drive logic
    endtask
endclass

// Step 2: In ONLY this test's build_phase, override the instance.
class error_injection_test extends base_test;
    `uvm_component_utils(error_injection_test)

    function void build_phase(uvm_phase phase);
        // Instance override: ONLY env.agent.driver gets the error version.
        // All other tests are unaffected (no global change).
        my_driver::type_id::set_inst_override(
            error_injection_driver::get_type(),
            "uvm_test_top.env.agent.driver"
        );
        // Call super AFTER override -- so driver is created with new type.
        super.build_phase(phase);
    endfunction
endclass

// Other tests remain completely unaffected:
class normal_test extends base_test;
    // No override -> driver remains my_driver (original behavior)
endclass

// ================================================================
// Alternative: config_db flag to modify behavior
// ================================================================
// If driver already has a "error_mode" field:
class configurable_driver extends uvm_driver #(my_txn);
    bit error_mode = 0;   // default off
    // In drive_item: if (error_mode) inject error;
endclass

// In the test's build_phase:
// uvm_config_db #(bit)::set(this, "env.agent.driver", "error_mode", 1);
// (config_db approach -- no subclassing needed, simpler for flag-based changes)
```

解释

这道题考察对 UVM 可扩展性设计的理解，有两种主流解法：

Factory instance override（推荐用于行为差异大的场景）：

创建派生类 override 特定方法，在目标 test 里精确替换那一个实例，

其他 test 完全不知道这个替换的存在。

config_db flag（推荐用于简单的开关型差异）：

driver 里预埋一个 `error_mode` 控制字段，

test 通过 config_db 设置这个 flag，driver 根据 flag 改变行为。

两者的选择依据：行为差异大（需要重写整段逻辑）→ factory override；

行为差异小（只是开关某个特性）→ config_db flag，更简洁，不需要额外类。

两种方法都遵循"不修改已有代码"的开闭原则，是 UVM 设计的核心思想。

Q98. UVM: APB Driver -- Full Implementation

QUESTION

Write a complete UVM APB driver that implements:

- APB write transaction
- APB read transaction
- Proper 2-phase (SETUP + ACCESS) protocol timing
- try_next_item / drive_idle for bus idle cycles

ANSWER

```
// ============================================================
// APB Transaction (sequence item)
// ============================================================
class apb_txn extends uvm_sequence_item;
    `uvm_object_utils_begin(apb_txn)
        `uvm_field_int(addr,  UVM_DEFAULT)
        `uvm_field_int(data,  UVM_DEFAULT)
        `uvm_field_int(write, UVM_DEFAULT)
        `uvm_field_int(strb,  UVM_DEFAULT)
    `uvm_object_utils_end

    rand logic [31:0] addr;
    rand logic [31:0] data;
    rand logic        write;   // 1=write, 0=read
    rand logic [3:0]  strb;    // byte strobes (PSTRB)

    // Response fields (filled by driver on read)
    logic [31:0] rdata;
    logic        slverr;

    constraint c_word_align { addr[1:0] == 2'b00; }
    constraint c_strb       { write -> strb inside {4'hF, 4'h3, 4'hC, 4'h1, 4'h2, 4'h4, 4'h8}; }
endclass

// ============================================================
// APB virtual interface
// ============================================================
interface apb_if (input logic PCLK, PRESETn);
    logic        PSEL;
    logic        PENABLE;
    logic        PWRITE;
    logic [31:0] PADDR;
    logic [31:0] PWDATA;
    logic [3:0]  PSTRB;
    logic [31:0] PRDATA;
    logic        PREADY;
    logic        PSLVERR;

    // Clocking block for driver (output from master's perspective)
    clocking driver_cb @(posedge PCLK);
        default output #1;
        output PSEL, PENABLE, PWRITE, PADDR, PWDATA, PSTRB;
        input  PRDATA, PREADY, PSLVERR;
    endclocking

    // Clocking block for monitor (input only)
    clocking monitor_cb @(posedge PCLK);
        input PSEL, PENABLE, PWRITE, PADDR, PWDATA, PSTRB;
        input PRDATA, PREADY, PSLVERR;
    endclocking
endinterface

// ============================================================
```

```
// APB Driver
// ================================================================
class apb_driver extends uvm_driver #(apb_txn);
    `uvm_component_utils(apb_driver)

    virtual apb_if vif;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db #(virtual apb_if)::get(
                this, "", "apb_vif", vif))
            `uvm_fatal("DRV", "Cannot get APB virtual interface")
    endfunction

    task run_phase(uvm_phase phase);
        // Initialize bus to idle state
        drive_idle();
        // Wait for reset deassertion
        @(posedge vif.PRESETn);
        @(posedge vif.PCLK);

        forever begin
            apb_txn txn;

            // Non-blocking: maintain idle if no transaction pending
            seq_item_port.try_next_item(txn);

            if (txn != null) begin
                if (txn.write)
                    apb_write(txn);
                else
                    apb_read(txn);
                seq_item_port.item_done();
            end else begin
                drive_idle();
                @(posedge vif.PCLK);
            end
        end
    endtask

    // ---- APB Write (SETUP -> ACCESS -> IDLE) ----------------------
    task apb_write(apb_txn txn);
        // --- SETUP phase (1 cycle) ---
        @(posedge vif.PCLK);
        vif.driver_cb.PSEL    <= 1;
        vif.driver_cb.PENABLE <= 0;
        vif.driver_cb.PWRITE  <= 1;
        vif.driver_cb.PADDR   <= txn.addr;
        vif.driver_cb.PWDATA  <= txn.data;
        vif.driver_cb.PSTRB   <= txn.strb;

        // --- ACCESS phase: wait for PREADY ---
        @(posedge vif.PCLK);
        vif.driver_cb.PENABLE <= 1;

        // Poll PREADY: slave may insert wait states
        while (!vif.driver_cb.PREADY)
            @(posedge vif.PCLK);
```

```
        // Capture error response
        txn.slverr = vif.driver_cb.PSLVERR;

        // --- End of transaction: IDLE ---
        @(posedge vif.PCLK);
        drive_idle();

        `uvm_info("DRV", $sformatf(
            "APB WRITE addr=0x%08h data=0x%08h strb=%04b slverr=%0b",
            txn.addr, txn.data, txn.strb, txn.slverr), UVM_MEDIUM)
    endtask

    // ---- APB Read (SETUP -> ACCESS -> IDLE) -----------------------
    task apb_read(apb_txn txn);
        // --- SETUP phase (1 cycle) ---
        @(posedge vif.PCLK);
        vif.driver_cb.PSEL    <= 1;
        vif.driver_cb.PENABLE <= 0;
        vif.driver_cb.PWRITE  <= 0;
        vif.driver_cb.PADDR   <= txn.addr;
        vif.driver_cb.PWDATA  <= '0;
        vif.driver_cb.PSTRB   <= '0;

        // --- ACCESS phase: wait for PREADY ---
        @(posedge vif.PCLK);
        vif.driver_cb.PENABLE <= 1;

        while (!vif.driver_cb.PREADY)
            @(posedge vif.PCLK);

        // Capture read data and error
        txn.rdata  = vif.driver_cb.PRDATA;
        txn.slverr = vif.driver_cb.PSLVERR;

        // --- End of transaction: IDLE ---
        @(posedge vif.PCLK);
        drive_idle();

        `uvm_info("DRV", $sformatf(
            "APB READ  addr=0x%08h rdata=0x%08h slverr=%0b",
            txn.addr, txn.rdata, txn.slverr), UVM_MEDIUM)
    endtask

    // ---- Drive bus to idle (no transaction) -----------------------
    task drive_idle();
        vif.driver_cb.PSEL    <= 0;
        vif.driver_cb.PENABLE <= 0;
        vif.driver_cb.PWRITE  <= 0;
        vif.driver_cb.PADDR   <= '0;
        vif.driver_cb.PWDATA  <= '0;
        vif.driver_cb.PSTRB   <= '0;
    endtask

endclass

// ============================================================
// Example sequence: write then read-back to verify APB FIFO
// ============================================================
class apb_fifo_test_seq extends uvm_sequence #(apb_txn);
    `uvm_object_utils(apb_fifo_test_seq)

    task body();
        apb_txn txn;
```

```
        // Poll STATUS until not full, then write to DATA_REG (0x00)
        repeat (4) begin
            txn = apb_txn::type_id::create("txn");
            start_item(txn);
            assert(txn.randomize() with {
                txn.addr  == 32'h0000_0000;  // DATA_REG
                txn.write == 1;
                txn.strb  == 4'hF;
            });
            finish_item(txn);
        end

        // Read back from DATA_REG (0x00)
        repeat (4) begin
            txn = apb_txn::type_id::create("txn");
            start_item(txn);
            assert(txn.randomize() with {
                txn.addr  == 32'h0000_0000;
                txn.write == 0;
            });
            finish_item(txn);
            `uvm_info("SEQ", $sformatf("Read: 0x%08h", txn.rdata), UVM_LOW)
        end

        // Check STATUS_REG (0x04) - should show empty after 4 reads
        txn = apb_txn::type_id::create("txn");
        start_item(txn);
        assert(txn.randomize() with {
            txn.addr  == 32'h0000_0004;  // STATUS_REG
            txn.write == 0;
        });
        finish_item(txn);
        `uvm_info("SEQ", $sformatf("Status: full=%0b empty=%0b count=%0d",
            txn.rdata[0], txn.rdata[1], txn.rdata[9:2]), UVM_LOW)
    endtask
endclass
```

解释

APB driver 的核心是正确实现两段式握手：

SETUP 阶段（PSEL=1, PENABLE=0）建立地址和控制信号；

ACCESS 阶段（PSEL=1, PENABLE=1）等待 PREADY——这是等待从设备就绪的关键，

如果从设备需要多周期处理（如访问慢速存储），它会拉低 PREADY 插入等待周期。

`try_next_item` 而非 `get_next_item` 的选择：APB 总线空闲时必须驱动 PSEL=0，

非阻塞方式让 driver 在没有事务时能主动维持 idle 状态，

如果用 `get_next_item` 阻塞，总线信号会悬空（X 态传播）。

Clocking block 的使用：`driver_cb` 中的 `output #1` 指定输出在时钟后1个时间单位发出，

避免 setup/hold 违例，这是 interface-based testbench 的标准实践。

读事务的 rdata 回写：`txn.rdata = vif.driver_cb.PRDATA` 在 PREADY=1 时捕获，

并通过 item_done() 返回给 sequence——这是 APB register 验证的基础。

Q99. UVM: Register Abstraction Layer (RAL) -- Working Principle and Usage

QUESTION

Explain the UVM Register Abstraction Layer (RAL / uvm_reg):

(a) What problem does it solve and how does it work?

(b) How do you define a register model?

(c) How do you use it in a sequence (frontdoor vs backdoor access)?

(d) How do you connect it to the DUT and verify registers?

ANSWER

```
// ================================================================
// (a) What problem RAL solves
// ================================================================
//
// Without RAL: every sequence that touches a register must manually
// construct a bus transaction (APB/AXI write to a specific address).
//   - The address is hardcoded everywhere.
//   - Field extraction (bit slicing) is repeated and error-prone.
//   - No automatic read-back verification.
//   - Changing the register map breaks every sequence that uses it.
//
// With RAL: registers are modeled as objects with named fields.
//   - Sequences write by NAME ("reg.CTRL.EN = 1"), not by address.
//   - RAL automatically constructs the correct bus transaction.
//   - RAL maintains a MIRROR (predicted value) of every register.
//   - Built-in register tests (hw_reset, bit_bash, aliasing) come free.
//   - Changing the address map only requires updating the RAL model.

// ================================================================
// (b) Defining a register model
// ================================================================

// Step 1: Define each register with its fields
class ctrl_reg extends uvm_reg;
    `uvm_object_utils(ctrl_reg)

    // Field declarations (name, access type, volatile, reset value)
    uvm_reg_field EN;      // bit[0]: enable
    uvm_reg_field MODE;    // bits[2:1]: operating mode
    uvm_reg_field RSVD;    // bits[31:3]: reserved

    function new(string name = "ctrl_reg");
        // Width = 32 bits, no coverage by default
        super.new(name, 32, UVM_NO_COVERAGE);
    endfunction

    function void build();
        EN   = uvm_reg_field::type_id::create("EN");
        MODE = uvm_reg_field::type_id::create("MODE");
        RSVD = uvm_reg_field::type_id::create("RSVD");

        // configure(parent, size, lsb_pos, access, volatile, reset, has_reset,
        //           is_rand, individually_accessible)
        EN.configure  (this, 1, 0, "RW", 0, 1'b0, 1, 1, 0);
        MODE.configure(this, 2, 1, "RW", 0, 2'b00, 1, 1, 0);
        RSVD.configure(this, 29, 3, "RO", 0, 29'b0, 1, 0, 0);
    endfunction
endclass

class status_reg extends uvm_reg;
    `uvm_object_utils(status_reg)
    uvm_reg_field BUSY;    // bit[0]: read-only status
    uvm_reg_field ERR_CNT; // bits[7:1]: error counter
```

```
    function new(string name = "status_reg");
        super.new(name, 32, UVM_NO_COVERAGE);
    endfunction

    function void build();
        BUSY    = uvm_reg_field::type_id::create("BUSY");
        ERR_CNT = uvm_reg_field::type_id::create("ERR_CNT");
        BUSY.configure   (this, 1, 0, "RO", 1, 1'b0, 1, 0, 0);
        ERR_CNT.configure(this, 7, 1, "RO", 1, 7'b0, 1, 0, 0);
    endfunction
endclass

// Step 2: Define the register block (address map)
class my_reg_block extends uvm_reg_block;
    `uvm_object_utils(my_reg_block)

    ctrl_reg   CTRL;     // offset 0x00
    status_reg STATUS;   // offset 0x04

    uvm_reg_map default_map;

    function new(string name = "my_reg_block");
        super.new(name, UVM_NO_COVERAGE);
    endfunction

    function void build();
        // Instantiate registers
        CTRL   = ctrl_reg::type_id::create("CTRL");
        STATUS = status_reg::type_id::create("STATUS");
        CTRL.build();
        STATUS.build();
        CTRL.configure  (this, null, "");
        STATUS.configure(this, null, "");

        // Create the address map
        // create_map(name, base_addr, n_bytes, endianness)
        default_map = create_map("default_map", 0, 4, UVM_LITTLE_ENDIAN);

        // Add registers to the map with their byte offsets
        default_map.add_reg(CTRL,   'h00, "RW");
        default_map.add_reg(STATUS, 'h04, "RO");

        // Lock the model (finalizes configuration)
        lock_model();
    endfunction
endclass

// ======================================================================
// (c) Using RAL in a sequence
// ======================================================================

class reg_config_seq extends uvm_sequence;
    `uvm_object_utils(reg_config_seq)

    my_reg_block regmodel;   // handle set by test before starting

    task body();
        uvm_status_e status;
        uvm_reg_data_t rdata;

        // --- Frontdoor write (generates a real bus transaction) ----------
        // Writes via the bus adapter -> APB/AXI write to CTRL address
```

```
        regmodel.CTRL.write(
            .status(status),
            .value(32'h0000_0003),   // EN=1, MODE=1
            .path(UVM_FRONTDOOR)
        );
        if (status != UVM_IS_OK)
            `uvm_error("SEQ", "CTRL write failed")

        // Field-level write (more readable, writes the whole register)
        regmodel.CTRL.EN.set(1'b1);      // stage the value
        regmodel.CTRL.MODE.set(2'b10);   // stage the value
        regmodel.CTRL.update(status);    // write only if mirror != desired

        // --- Frontdoor read (generates a real bus read transaction) -----
        regmodel.STATUS.read(
            .status(status),
            .value(rdata),
            .path(UVM_FRONTDOOR)
        );
        `uvm_info("SEQ", $sformatf("STATUS = 0x%0h, BUSY=%0b ERR_CNT=%0d",
            rdata,
            regmodel.STATUS.BUSY.get(),
            regmodel.STATUS.ERR_CNT.get()), UVM_LOW)

        // --- Backdoor write (HDL path, no bus, zero simulation time) ----
        // Useful for quickly initializing state without bus overhead
        regmodel.CTRL.write(
            .status(status),
            .value(32'h0000_0001),
            .path(UVM_BACKDOOR)
        );

        // --- Mirror check (read back and compare to predicted value) -----
        regmodel.CTRL.mirror(
            .status(status),
            .check(UVM_CHECK)   // compare hardware value to mirror
        );
    endtask
endclass

// ================================================================
// (d) Connecting RAL to the DUT: adapter + predictor
// ================================================================

// Step 1: Write a register adapter for your bus protocol (e.g. APB)
class apb_reg_adapter extends uvm_reg_adapter;
    `uvm_object_utils(apb_reg_adapter)

    function new(string name = "apb_reg_adapter");
        super.new(name);
        supports_byte_enable = 0;
        provides_responses   = 1;
    endfunction

    // Convert RAL bus_op -> APB transaction
    function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw);
        apb_txn txn = apb_txn::type_id::create("txn");
        txn.addr  = rw.addr;
        txn.write = (rw.kind == UVM_WRITE);
        txn.data  = rw.data;
        return txn;
    endfunction
```

```
    // Convert APB response -> RAL bus_op status
    function void bus2reg(uvm_sequence_item bus_item,
                          ref uvm_reg_bus_op rw);
        apb_txn txn;
        $cast(txn, bus_item);
        rw.kind   = txn.write ? UVM_WRITE : UVM_READ;
        rw.addr   = txn.addr;
        rw.data   = txn.rdata;
        rw.status = UVM_IS_OK;
    endfunction
endclass

// Step 2: Connect in env.connect_phase
// regmodel.default_map.set_sequencer(apb_agent.seqr, adapter);
// regmodel.default_map.set_auto_predict(1); // auto-update mirror on access

// Step 3: Built-in register tests (no extra code needed!)
// uvm_reg_hw_reset_seq  -- verifies reset values of all registers
// uvm_reg_bit_bash_seq  -- walks bits 0/1 through each RW field
// uvm_reg_access_seq    -- write/read-back every register
// uvm_mem_walk_seq      -- walks through memory locations

// Run built-in test:
// uvm_reg_hw_reset_seq reset_seq = new();
// reset_seq.model = regmodel;
// reset_seq.start(null);
```

解释

RAL 的核心价值是把"寄存器是什么"（字段、地址、访问属性）和"如何访问它"（APB/AXI/AHB）分离：

register model 只描述寄存器结构，adapter 负责协议转换，两者解耦。

Mirror（镜像）机制是 RAL 的关键：每次 write 后 RAL 自动更新 mirror 预测值，

mirror() 调用会读回硬件实际值并与 mirror 比对——这是寄存器验证的自动化核心。

Frontdoor vs Backdoor：

Frontdoor 通过真实总线事务访问，测试总线接口本身；

Backdoor 通过 HDL 路径（`vif.hdl_path`）直接强制写值，0 时间，用于快速初始化或调试。

built-in 测试序列（uvm_reg_hw_reset_seq、uvm_reg_bit_bash_seq）是 RAL 最大的生产力优势：

一旦 register model 建好，这些通用测试"免费"覆盖所有寄存器的复位值和读写完整性，

不需要为每个寄存器单独写测试用例。

adapter 的 reg2bus/bus2reg 是连接 RAL 和实际总线的桥梁，

每换一种总线协议只需替换 adapter，register model 和 sequence 代码完全不变。

Q100. UVM: uvm_object vs uvm_component -- Differences

QUESTION

What is the difference between uvm_object and uvm_component in UVM?

ANSWER

```
// ============================================================
// uvm_component
// ============================================================
```

```
//
// - Has a PARENT and a fixed position in the UVM component HIERARCHY.
// - Created with: T::type_id::create("name", parent)
//   The two-argument create() registers it in the hierarchy tree.
// - Participates in UVM PHASES (build, connect, run, check, report...).
// - Lives for the ENTIRE simulation duration (not garbage collected).
// - Has a full hierarchical path name:
//   e.g. "uvm_test_top.env.agent.driver"
// - Registered with: `uvm_component_utils(T)
//
// Examples: uvm_driver, uvm_monitor, uvm_scoreboard,
//           uvm_agent, uvm_env, uvm_test, uvm_sequencer

// ============================================================
// uvm_object
// ============================================================
//
// - Has NO parent, NO phase participation, NO hierarchical position.
// - Created with: T::type_id::create("name")   (one argument only)
// - Lives dynamically: can be created, used, and garbage collected.
// - Used to represent DATA: transactions, configurations, sequences.
// - Registered with: `uvm_object_utils(T)
//
// Examples: uvm_sequence_item, uvm_sequence, uvm_reg,
//           uvm_reg_block (register models are objects, not components)

// ============================================================
// Side-by-side comparison
// ============================================================
//
//   Property             | uvm_component          | uvm_object
//   ---------------------+------------------------+-------------------
//   Hierarchy            | Yes (parent/child)     | No
//   Phase participation  | Yes (build, run, etc.) | No
//   Lifetime             | Entire simulation      | Dynamic (GC eligible)
//   create() signature   | ("name", parent)       | ("name")
//   Registration macro   | `uvm_component_utils   | `uvm_object_utils
//   Typical use          | Env structure          | Data / transactions
//   get_full_name()      | Returns hierarchy path | Returns just name
//
// Key rule: if it DOES something over time -> component.
//           if it IS data being passed around -> object.
```

解释

Component vs Object 的本质区别是"结构 vs 数据"：

component 是测试环境的固定基础设施，在仿真开始前 build，贯穿整个仿真存在；

object 是流动的数据包，由 sequence 产生，经 driver 驱动，被 monitor 捕获，由 scoreboard 消耗。

创建方式是最直观的区分：两个参数（name+parent）用 component，一个参数（name alone）用 object。

错用 `uvm_object_utils` 注册一个 component 会导致工厂找不到它的 phase 回调，

最常见的症状是 build_phase 和 connect_phase 不被调用，组件"静默失效"。

Q101. UVM: Test Entry Point -- run_test() to Phase Execution

QUESTION

Walk through what happens from run_test("my_test") in the top-level

testbench all the way through UVM phase execution.

ANSWER

```
// ================================================================
// Step 1: run_test() call in top module
// ================================================================
//
// module top;
//     initial begin
//         uvm_config_db #(virtual dut_if)::set(null, "uvm_test_top.*",
//                                              "dut_vif", dut_vif);
//         run_test("my_test");   // <-- entry point
//     end
// endmodule
//
// run_test() does three things:
//   a. Looks up the test class name in the UVM factory registry.
//      If "+UVM_TESTNAME=other_test" plusarg is set, it overrides
//      the argument to run_test() -- plusarg wins.
//   b. Creates an instance of the test:
//      uvm_test_top = my_test::type_id::create("uvm_test_top", null)
//      "uvm_test_top" is ALWAYS the name of the root test instance.
//   c. Starts the phase scheduler.

// ================================================================
// Step 2: Build phase -- top-down recursive construction
// ================================================================
//
// UVM calls build_phase() top-down (parent before child):
//   my_test::build_phase()
//     -> creates my_env::build_phase()
//        -> creates my_agent::build_phase()
//           -> creates driver, monitor, sequencer
//
// Key point: super.build_phase(phase) must be called, typically LAST,
// so that apply_config_settings() processes uvm_config_db entries
// before child components are created.

// ================================================================
// Step 3: Connect phase -- bottom-up
// ================================================================
//
// connect_phase() runs BOTTOM-UP (children before parents):
//   agent::connect_phase: drv.seq_item_port.connect(seqr.seq_item_export)
//   env::connect_phase:   mon.ap.connect(scb.analysis_export)
//
// Bottom-up ensures that by the time parent connects, all children's
// ports already exist and are ready to be connected.

// ================================================================
// Step 4: Start of simulation phases
// ================================================================
//
//   start_of_simulation_phase() -- called after elaboration, before run.
//   Use for: printing topology, checking connections.
//
//   run_phase() -- THE main simulation phase.
//     - All components' run_phase() tasks run CONCURRENTLY (fork-join).
//     - Simulation time advances here.
//     - Ends when ALL objections are dropped.
//     - Objection protocol: test raises at start, drops at end of stimulus.

// ================================================================
// Step 5: Drain / cleanup phases (after run_phase)
// ================================================================
```

```
//
//   extract_phase()   -> collect data from DUT (backdoor reads)
//   check_phase()     -> compare expected vs actual (scoreboard)
//   report_phase()    -> print pass/fail summary
//   final_phase()     -> last-chance cleanup
//
//   These are FUNCTIONS (no time), run top-down.

// ================================================================
// Summary: full phase order
// ================================================================
//
//   build_phase         (top-down,  function)
//   connect_phase       (bottom-up, function)
//   end_of_elaboration  (bottom-up, function)
//   start_of_simulation (bottom-up, function)
//   run_phase           (parallel,  task -- time advances here)
//     pre_reset / reset / post_reset
//     pre_configure / configure / post_configure
//     pre_main / main / post_main
//     pre_shutdown / shutdown / post_shutdown
//   extract_phase       (bottom-up, function)
//   check_phase         (bottom-up, function)
//   report_phase        (bottom-up, function)
//   final_phase         (bottom-up, function)
//
// The sub-phases of run_phase (pre_reset .. shutdown) allow
// fine-grained coordination between test layers without explicit
// event synchronization.
```

解释

run_test() 的核心动作是三步：找到测试类（工厂查找）、创建根实例（固定名 uvm_test_top）、
启动 phase 调度器——phase 调度器是整个 UVM 执行引擎的入口。
build 和 connect 的方向不同有其逻辑：build 必须自顶向下（父先建子，
否则子组件不存在）；connect 必须自底向上（子先连好自己的端口，父才能把它们串起来）。
run_phase 的并发模型：所有组件的 run_phase 是 fork-join 并发运行，
不是串行调用。因此 driver、monitor、scoreboard 同时在跑，
仿真时间在这里推进，直到没有组件持有 objection。
"objection 协议"是 UVM 控制仿真结束的机制：
任何人 raise_objection 就让仿真继续，全部 drop_objection 后 phase 才结束。

Q102. UVM: Factory Override -- Why super.build_phase() Must Come AFTER the Override

QUESTION

You have two classes: clk_driver and jitter_clk_driver (extends clk_driver).
Write the registration, build_phase, and factory override correctly.
Explain: why must super.build_phase(phase) be called AFTER setting
the factory override, not before?

ANSWER

```
// ================================================================
```

```
// Step 1: Register both classes with the factory
// ================================================================
class clk_driver extends uvm_driver #(clk_txn);
    `uvm_component_utils(clk_driver)   // registers with factory

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);      // must call (applies config_db settings)
    endfunction

    task run_phase(uvm_phase phase);
        // normal clock driving ...
    endtask
endclass

class jitter_clk_driver extends clk_driver;
    `uvm_component_utils(jitter_clk_driver)  // registers with factory

    real jitter_ps = 50.0;   // additional field

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);   // call parent's build_phase too
    endfunction

    task run_phase(uvm_phase phase);
        // clock driving with jitter injection ...
    endtask
endclass

// ================================================================
// Step 2: In the test, set the override BEFORE super.build_phase()
// ================================================================
class jitter_test extends base_test;
    `uvm_component_utils(jitter_test)

    function void build_phase(uvm_phase phase);
        // --- CORRECT ORDER ---

        // 1. Set factory override FIRST
        clk_driver::type_id::set_type_override(
            jitter_clk_driver::get_type()
        );

        // 2. Call super.build_phase() AFTER the override
        super.build_phase(phase);   // this triggers env -> agent -> driver creation
    endfunction
endclass

// ================================================================
// WHY must override come before super.build_phase()?
// ================================================================
//
// super.build_phase(phase) triggers the ENTIRE build chain:
//
//   jitter_test::build_phase()
//     super.build_phase()          <- this starts the chain
```

```
//       base_test::build_phase()
//         env = my_env::create()
//           env::build_phase()
//             agent = my_agent::create()
//               agent::build_phase()
//                 drv = clk_driver::type_id::create("drv", this)
//                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//                 THIS is where the factory looks up clk_driver
//                 and returns jitter_clk_driver (if override is set)
//
// If you call super.build_phase() FIRST:
//   The factory creates clk_driver BEFORE you had a chance to set the override.
//   The override is registered too late -- clk_driver already exists.
//   jitter_clk_driver is NEVER instantiated.
//
// If you set the override FIRST, then call super.build_phase():
//   The factory registry already has the substitution rule.
//   When agent tries to create clk_driver, factory returns jitter_clk_driver.
//   Override works correctly.
//
// One-line summary:
//   "Set the rule before the factory uses it."
//   super.build_phase() is the trigger that causes factory::create() to fire.
//   Any override must be registered before that trigger.

// ================================================================
// WRONG (override too late -- has no effect):
// ================================================================
//   function void build_phase(uvm_phase phase);
//       super.build_phase(phase);   // factory creates clk_driver HERE
//       // Override set AFTER creation -- too late!
//       clk_driver::type_id::set_type_override(jitter_clk_driver::get_type());
//   endfunction
```

解释

这道题的核心是理解 UVM build 阶段的执行顺序：

`super.build_phase(phase)` 不只是一个简单的父类调用，

它触发了整个组件树的向下建造过程——每个 env 的 build_phase 创建 agent，

每个 agent 的 build_phase 创建 driver，每次 create() 都要查询工厂。

Factory override 是"在工厂里注册替换规则"，create() 是"工厂根据规则生产产品"。

必须先注册规则，再调用生产流程，否则工厂按默认规则（旧类）生产，替换规则永远不生效。

`super.build_phase(phase)` 在派生 driver 里仍然必须调用，

因为 uvm_component 的 build_phase 包含 `apply_config_settings()`，

这个方法处理所有 `uvm_config_db::get()` 的自动映射——

跳过它会导致配置数据库的设置全部失效（vif 获取失败、参数不传递等）。

Q103. UVM: Register Aliasing Detection Using Frontdoor Only

QUESTION

Two registers at different addresses are aliased (they share the same

physical storage). How do you detect register aliasing using ONLY

frontdoor read/write operations (no backdoor, no HDL path access)?

ANSWER

```
// ============================================================
// What is register aliasing?
// ============================================================
//
// Register aliasing: writing to address A also changes the value
// at address B (they share the same physical flip-flops).
// This can be an INTENTIONAL design feature (mirrored registers)
// or a BUG (address decoder overlap).

// ============================================================
// Detection strategy: frontdoor only
// ============================================================
//
// Algorithm:
//   For each register pair (REG_A, REG_B) that might be aliased:
//   1. Write UNIQUE_VAL_1 to REG_A.
//   2. Read REG_B -> if it changed to UNIQUE_VAL_1, aliasing detected!
//   3. Write UNIQUE_VAL_2 to REG_B.
//   4. Read REG_A -> if it changed to UNIQUE_VAL_2, aliasing confirmed!
//
// Key: use UNIQUE, DISTINCTIVE values so you can't confuse
//      "REG_B happened to have this value" with "aliasing".

class reg_alias_check_seq extends uvm_sequence;
    `uvm_object_utils(reg_alias_check_seq)

    my_reg_block regmodel;

    // Sentinel values for aliasing detection
    localparam logic [31:0] SENTINEL_A = 32'hA5A5_A5A5;
    localparam logic [31:0] SENTINEL_B = 32'h5A5A_5A5A;

    task check_alias_pair(uvm_reg reg_a, uvm_reg reg_b);
        uvm_status_e  status;
        uvm_reg_data_t val;

        // Step 1: Save original values
        uvm_reg_data_t orig_a, orig_b;
        reg_a.read(.status(status), .value(orig_a), .path(UVM_FRONTDOOR));
        reg_b.read(.status(status), .value(orig_b), .path(UVM_FRONTDOOR));

        // Step 2: Write sentinel to REG_A, check if REG_B changes
        reg_a.write(.status(status), .value(SENTINEL_A), .path(UVM_FRONTDOOR));
        reg_b.read(.status(status),  .value(val),        .path(UVM_FRONTDOOR));

        if (val == SENTINEL_A)
            `uvm_error("ALIAS", $sformatf(
                "ALIASING: write to %s changed %s to 0x%0h",
                reg_a.get_name(), reg_b.get_name(), val))

        // Step 3: Write sentinel to REG_B, check if REG_A changes
        reg_b.write(.status(status), .value(SENTINEL_B), .path(UVM_FRONTDOOR));
        reg_a.read(.status(status),  .value(val),        .path(UVM_FRONTDOOR));

        if (val == SENTINEL_B)
            `uvm_error("ALIAS", $sformatf(
                "ALIASING: write to %s changed %s to 0x%0h",
                reg_b.get_name(), reg_a.get_name(), val))

        // Step 4: Restore original values
        reg_a.write(.status(status), .value(orig_a), .path(UVM_FRONTDOOR));
        reg_b.write(.status(status), .value(orig_b), .path(UVM_FRONTDOOR));
```

```
    endtask

    task body();
        // Check all suspicious pairs
        check_alias_pair(regmodel.CTRL_REG,   regmodel.STATUS_REG);
        check_alias_pair(regmodel.CONFIG_REG,  regmodel.ENABLE_REG);

        // Or: exhaustive sweep of all register pairs
        uvm_reg all_regs[$];
        regmodel.get_registers(all_regs);

        for (int i = 0; i < all_regs.size(); i++) begin
            for (int j = i+1; j < all_regs.size(); j++) begin
                // Only check writable register pairs
                if (all_regs[i].get_rights() != "RO" &&
                    all_regs[j].get_rights() != "RO")
                    check_alias_pair(all_regs[i], all_regs[j]);
            end
        end
    endtask
endclass

// UVM RAL built-in: uvm_reg_access_seq also catches some aliasing
// by doing write-then-read on every register and checking mirrors.
// But explicit pair testing is more targeted for known suspicious pairs.
```

解释

纯 frontdoor 检测别名的核心思路是"写 A 看 B 有没有变"——

如果写 A 导致 B 改变，说明 A 和 B 共享存储（别名）。

Sentinel 值选取要有辨识度：`0xA5A5A5A5` 和 `0x5A5A5A5A` 是经典选择，

因为它们是互补的交替模式，同时出现在寄存器里的概率极低，不会和正常值混淆。

Exhaustive sweep（`get_registers()` 遍历所有对）是全覆盖方案，

但 $O(n^2)$ 对数量在大寄存器文件（几百个寄存器）时可能很慢。

工程实践中先针对"地址解码逻辑边界处"的可疑对进行检查，

再做全量扫描——这和 UVM 内置的 `uvm_reg_access_seq` 互补，后者不做跨寄存器的别名检查。

Q104. UVM: 5 Sequences on 1 Sequencer -- Track Which Transaction Came from Which Sequence

QUESTION

5 sequences run simultaneously on 1 sequencer. How do you make the

entire transaction flow "opaque" (trackable), so you always know which

transaction came from which sequence?

ANSWER

```
// ================================================================
// Problem: 5 sequences compete for 1 sequencer
// ================================================================
//
// When multiple sequences run on the same sequencer, the sequencer
// arbitrates between them (round-robin by default, or by priority).
// The driver and monitor see only individual transactions --
// they lose the sequence context. We need to restore this context.
```

```
// ================================================================
// Solution 1: Tag each transaction with a sequence_id (best practice)
// ================================================================
class tagged_txn extends uvm_sequence_item;
    `uvm_object_utils(tagged_txn)

    // UVM built-in: get_sequence_id() returns the ID of the parent sequence
    // Also add human-readable name for debug:
    string seq_name;     // e.g. "traffic_seq_0", "error_seq"
    int    seq_id;       // unique ID per sequence instance

    rand logic [31:0] addr;
    rand logic [31:0] data;
    rand logic        is_write;

    function string convert2string();
        return $sformatf("[%s|id=%0d] %s addr=0x%0h data=0x%0h",
                seq_name, seq_id, is_write ? "WR" : "RD", addr, data);
    endfunction
endclass

// ================================================================
// In each sequence: stamp every transaction before sending
// ================================================================
class my_seq extends uvm_sequence #(tagged_txn);
    `uvm_object_utils(my_seq)

    string my_name;   // set by test before start()
    int    my_id;

    task body();
        tagged_txn txn;
        repeat (10) begin
            txn = tagged_txn::type_id::create("txn");
            start_item(txn);
            assert(txn.randomize());
            // Stamp the transaction with THIS sequence's identity
            txn.seq_name = my_name;
            txn.seq_id   = my_id;
            // UVM built-in: txn.get_sequence_id() also works if preferred
            finish_item(txn);
        end
    endtask
endclass

// ================================================================
// Solution 2: UVM built-in get_sequence_id() + get_parent_sequence()
// ================================================================
//
// Every uvm_sequence_item has:
//   txn.get_sequence_id()    -> int: the sequence's arbitration ID
//   txn.get_parent_sequence()-> uvm_sequence_base: handle to parent seq
//
// In the driver or monitor, recover sequence context:
//
class tagged_driver extends uvm_driver #(tagged_txn);
    task run_phase(uvm_phase phase);
        forever begin
            tagged_txn txn;
            seq_item_port.get_next_item(txn);

            // Option A: use custom stamp
```

```
            `uvm_info("DRV", $sformatf("Driving txn from seq '%s'",
                txn.seq_name), UVM_MEDIUM)

            // Option B: use UVM built-in
            `uvm_info("DRV", $sformatf("Driving txn seq_id=%0d",
                txn.get_sequence_id()), UVM_MEDIUM)

            drive(txn);
            seq_item_port.item_done();
        end
    endtask
endclass

// ============================================================
// Solution 3: Separate sequencer per sequence (cleanest isolation)
// ============================================================
//
// If transactions from different sequences must NEVER interleave,
// assign each sequence its own sequencer+driver pair:
//
// env:
//   agent[0] -> sequencer[0] -> driver[0] (traffic_seq_0 only)
//   agent[1] -> sequencer[1] -> driver[1] (traffic_seq_1 only)
//   ...
//   agent[4] -> sequencer[4] -> driver[4] (error_seq only)
//
// Use a virtual sequencer to coordinate all 5 agents.
// This guarantees zero interleaving and perfect traceability.
// Cost: 5x hardware agents (5 drivers, 5 physical interfaces).

// ============================================================
// Solution 4: Sequence priority + unique transaction IDs
// ============================================================
//
// Assign unique priorities to each sequence:
//   seq0.start(seqr, null, 100);  // highest priority
//   seq1.start(seqr, null, 90);
//   ...
//   seq4.start(seqr, null, 60);
//
// Transactions from higher-priority sequences go first.
// With known priority ordering, scoreboard can infer sequence origin
// from transaction ordering patterns.
//
// Combine with custom tid field:
//   seq_id[2:0] embedded in txn.addr or txn.data[31:29]
//   Recoverable by scoreboard/monitor without extra protocol fields.

// ============================================================
// In the scoreboard: track per-sequence statistics
// ============================================================
class mem_scoreboard extends uvm_scoreboard;
    int unsigned per_seq_count [string];   // key = seq_name

    function void write(tagged_txn txn);
        per_seq_count[txn.seq_name]++;
        // Can now report which sequence produced which transactions
    endfunction

    function void report_phase(uvm_phase phase);
        foreach (per_seq_count[name])
            `uvm_info("SCB", $sformatf("Seq '%s': %0d txns",
                name, per_seq_count[name]), UVM_NONE)
```

```
    endfunction
endclass
```

解释

多 sequence 单 sequencer 的追踪难点在于：sequencer 仲裁后，

driver 和 monitor 只看到一个"混合"的事务流，失去了"这个事务来自哪个 sequence"的上下文。

最可靠的解法是 Solution 1（手动 stamp）：在每个 sequence 的 body() 里，

在 `start_item` 和 `finish_item` 之间把 seq_name 和 seq_id 写入 txn 对象，

这样 txn 在整个生命周期（driver → scoreboard → coverage）里都携带来源信息。

`get_sequence_id()` 是 UVM 内置的仲裁 ID，可以直接使用，

但它是数字 ID 而非描述性名称，debug 时不如自定义 seq_name 直观。

Solution 3（独立 agent）是最干净的隔离方案，适合"5条流必须完全独立不交叉"的场景，

代价是硬件接口数量增加——需要 DUT 支持多个物理端口。

Algorithms & Scripting

Q105. Algorithms: Fibonacci -- Recursive vs Iterative, Time Complexity

QUESTION

Implement Fibonacci using (a) recursion and (b) iteration.

What is the time complexity of each?

ANSWER

```
# (a) Recursive implementation
def fib_recursive(n: int) -> int:
    """
    Naive recursive Fibonacci.

    Time:  O(2^n) -- exponential! Each call spawns 2 more calls,
           forming a binary tree of depth n with ~2^n total calls.
           (More precisely O(phi^n) where phi=1.618, golden ratio,
           but commonly approximated/quoted as O(2^n).)
    Space: O(n) -- recursion call stack depth
    """
    if n <= 1:
        return n
    return fib_recursive(n - 1) + fib_recursive(n - 2)

# (b) Iterative implementation
def fib_iterative(n: int) -> int:
    """
    Bottom-up iterative Fibonacci using two running variables.

    Time:  O(n) -- single loop, n-1 iterations
    Space: O(1) -- only two variables tracked (no array, no recursion stack)
    """
    if n <= 1:
        return n
```

```
    a, b = 0, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b

# -- Examples --------------------------------------------------------------
# fib_recursive(10) -> 55     (both give the same result)
# fib_iterative(10) -> 55
#
# fib_iterative(0) -> 0
# fib_iterative(1) -> 1
# fib_iterative(9) -> 34

# (c) Bonus: memoized recursion -- best of both worlds
def fib_memo(n: int, cache: dict = None) -> int:
    """
    Top-down recursion with memoization -- avoids recomputation.
    Time:  O(n)  -- each subproblem computed once
    Space: O(n)  -- cache dict + recursion stack
    """
    if cache is None:
        cache = {}
    if n <= 1:
        return n
    if n in cache:
        return cache[n]
    cache[n] = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
    return cache[n]
```

解释

朴素递归的 O(2^n) 来自重复计算：fib(n)调用fib(n-1)和fib(n-2)，

但fib(n-1)内部又会重新计算fib(n-2)，大量子问题被反复求解。

迭代版本用两个变量滚动更新，彻底消除重复计算，O(n)时间O(1)空间，

是面试中期望的最优解。

Memoization（记忆化递归）是介于两者之间的方案：

保留递归的直观写法，用 cache 避免重复计算，同样达到 O(n)，

但需要额外 O(n) 空间存缓存，外加递归栈开销。

面试追问：能否把迭代版本的 O(1) 空间也用在 memo 版本？

答案是不能直接做到，因为递归本身需要调用栈，这是两种范式的本质区别。

Q106. Algorithms: Reverse a Linked List

QUESTION

Reverse a singly linked list. Implement both iterative and recursive solutions.

ANSWER

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# ---- Iterative solution -------------------------------------------------
```

```
def reverse_list_iterative(head: ListNode) -> ListNode:
    """
    Reverse a singly linked list using three pointers.

    Time:  O(n) -- single pass through the list
    Space: O(1) -- only a few pointer variables, no extra data structure
    """
    prev: ListNode = None
    curr: ListNode = head

    while curr:
        next_node = curr.next   # save next before overwriting curr.next
        curr.next = prev        # reverse the link
        prev = curr             # advance prev
        curr = next_node        # advance curr

    return prev   # prev is now the new head

# ---- Recursive solution ------------------------------------------------
def reverse_list_recursive(head: ListNode) -> ListNode:
    """
    Reverse a singly linked list using recursion.

    Time:  O(n) -- each node visited once
    Space: O(n) -- recursion call stack depth
    """
    if head is None or head.next is None:
        return head   # base case: empty list or single node

    new_head = reverse_list_recursive(head.next)  # reverse the rest first
    head.next.next = head    # make the next node point back to current
    head.next = None         # break the original forward link

    return new_head

# -- Example ------------------------------------------------------------
# Input:  1 -> 2 -> 3 -> 4 -> 5 -> None
# Output: 5 -> 4 -> 3 -> 2 -> 1 -> None
#
# Both solutions produce the same result; iterative is generally preferred
# in interviews for its O(1) space complexity.
```

解释

迭代法用三个指针（prev, curr, next）逐个翻转链接方向，

关键是必须先保存 next 再修改 curr.next，否则会丢失后续节点的引用。

递归法的思路是"先递归翻转后面的部分，再处理当前节点"：

reverse(head.next) 返回新的头节点，然后让 head.next.next = head

把当前节点接到已翻转链表的末尾，最后断开 head.next 避免成环。

空间复杂度的权衡是面试常问点：迭代 O(1) vs 递归 O(n)（调用栈），

生产代码通常选迭代版本以避免长链表导致的栈溢出风险。

Q107. Algorithms: Sliding Window -- Find Starting Index of Maximum Sum

QUESTION

Given an integer array and a window size k, return the STARTING INDEX

of the contiguous subarray of length k with the maximum sum.

Discuss corner cases.

ANSWER

```
def max_sum_start_index(arr: list[int], k: int) -> int:
    """
    Find the starting index of the length-k contiguous subarray
    with the maximum sum.

    Args:
        arr: input integer array
        k:   window length

    Returns:
        starting index of the best window, or -1 if invalid input

    Time:  O(n) -- single pass sliding window
    Space: O(1)
    """
    n = len(arr)

    # ---- Corner cases ------------------------------------------------
    if n == 0 or k <= 0 or k > n:
        return -1   # invalid: empty array, non-positive k, or k too large

    # Compute sum of the first window
    window_sum: int = sum(arr[:k])
    best_sum:   int = window_sum
    best_idx:   int = 0

    # Slide the window one element at a time
    for i in range(k, n):
        window_sum += arr[i] - arr[i - k]   # add new, remove oldest
        if window_sum > best_sum:
            best_sum = window_sum
            best_idx = i - k + 1

    return best_idx

# -- Examples ------------------------------------------------------------
# max_sum_start_index([1,3,-1,-3,5,3,6,7], 3) -> 5  (window [5,6... wait)
# Let's verify: windows of size 3:
#   [1,3,-1]=3 [3,-1,-3]=-1 [-1,-3,5]=1 [-3,5,3]=5 [5,3,6]=14 [3,6,7]=16
#   max sum=16 at start_idx=5 (window [3,6,7])
#
# max_sum_start_index([2,1,5,1,3,2], 3) -> 2  (window [5,1,3]=9)
#
# -- Corner cases to discuss in interview ----------------------------------
#
# 1. k == len(arr): only ONE possible window -- return index 0 immediately
#    (loop body never executes, which is correct -- best_idx stays 0)
#
# 2. k > len(arr): invalid -- no window of that size exists, return -1
#    (or could raise an exception depending on spec)
#
# 3. k <= 0: invalid window size, return -1
#
# 4. Empty array: no data, return -1
#
# 5. All negative numbers: e.g. [-5,-1,-3,-2], k=2
#    Algorithm still works correctly -- "maximum" sum can be negative,
#    finds the LEAST negative window, e.g. [-1,-3] sum=-4 is the best
```

```
#    among all size-2 windows. Must NOT special-case "ignore negatives".
#
# 6. Multiple windows tie for maximum sum: e.g. [1,2,1,2], k=2
#    [1,2]=3 and [1,2]=3 (positions 0 and 2) tie.
#    Current implementation returns the FIRST occurrence (uses strict >,
#    not >=, so later ties don't overwrite best_idx).
#    Clarify with interviewer: "first" vs "last" vs "any" valid max --
#    this is a common ambiguity to raise proactively.
#
# 7. k == 1: degenerates to "find index of maximum single element"
#    Algorithm still works (window of size 1, no sliding needed beyond
#    standard max-finding logic).
```

解释

滑动窗口的标准模板：先求首窗口和，之后每步"减去最左，加上最新"。

这道题相比基础版多了一个"返回起始索引"的要求，需要额外维护 best_idx，

在更新 best_sum 时同步记录当前窗口的起始位置 i-k+1。

Corner case 的系统性讨论体现工程严谨性：

k==n（只有一个窗口）、k>n（无效）、k<=0（无效）、空数组（无效）这四个是输入合法性检查；

全负数数组（验证算法不会"误判"应该跳过负数）和并列最大值（明确返回第一个还是任意一个）

这两个是算法行为的边界条件，最容易被面试官追问但容易被候选人忽略。

主动提出"并列时返回哪个"的问题本身就是加分项，体现需求澄清的意识。

Q108. Algorithms: Parse Mixed UVM/Custom Simulation Log File

QUESTION

You have a simulation log file containing a MIX of standard UVM_INFO output AND custom testbench output. Write a Python script to extract specific module information -- e.g. APB bus transactions and I2C transactions -- from the log. UVM lines already contain hierarchical instance names; use this structure to your advantage.

ANSWER

```
# Sample log format:
#   UVM_INFO @ 105: uvm_test_top.env.apb_agent.monitor [APB_MON] APB Write addr=0x1000 data=0xDEAD
#   UVM_INFO @ 110: uvm_test_top.env.i2c_agent.monitor [I2C_MON] I2C Start condition detected
#   [CUSTOM_LOG] 115ns: Custom debug message from testbench
#   UVM_INFO @ 120: uvm_test_top.env.apb_agent.driver [APB_DRV] APB Read addr=0x2000

import re
from collections import defaultdict

def parse_log(filepath: str) -> dict[str, list[str]]:
    """
    Parse a mixed UVM/custom simulation log, grouping lines by module
    type inferred from the UVM instance hierarchy path.

    Args:
        filepath: path to the log file

    Returns:
```

```
        dict mapping module category (e.g. 'apb', 'i2c') to list of
        matching log lines

    Time:  O(n) -- single pass over n lines
    Space: O(n) -- stores matched lines
    """
    results: dict[str, list[str]] = defaultdict(list)

    with open(filepath) as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

            # Only process UVM_INFO lines (skip custom/non-UVM lines)
            if not line.startswith('UVM_INFO'):
                continue

            # UVM lines have format:
            # UVM_INFO @ <time>: <instance_path> [<TAG>] <message>
            # Split on ':' to separate metadata from instance path
            parts = line.split(':', 2)   # split into at most 3 parts
            if len(parts) < 3:
                continue

            # parts[1] = " uvm_test_top.env.apb_agent.monitor [APB_MON] message..."
            # Extract instance path: everything before the first space-bracket
            instance_and_rest = parts[1].strip()
            # Instance path is the first whitespace-delimited token
            instance_path = instance_and_rest.split()[0]

            # Use the instance hierarchy to classify the line
            # Lowercase comparison for robustness against naming variations
            path_lower = instance_path.lower()

            if 'apb' in path_lower:
                results['apb'].append(line)
            elif 'i2c' in path_lower:
                results['i2c'].append(line)
            # Add more protocols as needed:
            elif 'axi' in path_lower:
                results['axi'].append(line)
            elif 'spi' in path_lower:
                results['spi'].append(line)

    return dict(results)

# -- Usage ---------------------------------------------------------------
# parsed = parse_log('sim.log')
# for category, lines in parsed.items():
#     print(f"=== {category.upper()} ({len(lines)} lines) ===")
#     for line in lines[:5]:   # show first 5
#         print(f"  {line}")

# -- Alternative: using regex for more robust instance extraction -----------
def parse_log_regex(filepath: str) -> dict[str, list[str]]:
    """Regex-based variant -- more robust to format variations."""
    results: dict[str, list[str]] = defaultdict(list)
    instance_pattern = re.compile(r'UVM_INFO @ \d+: (\S+)')

    with open(filepath) as f:
        for line in f:
            m = instance_pattern.match(line)
```

```
            if not m:
                continue
            instance_path = m.group(1).lower()
            for category in ('apb', 'i2c', 'axi', 'spi'):
                if category in instance_path:
                    results[category].append(line.strip())
                    break   # avoid double-classification

    return dict(results)

# -- Extracting specific data fields (e.g. addr/data from APB lines) --------
def extract_apb_fields(line: str) -> dict[str, str]:
    """Extract addr and data hex values from an APB log line."""
    fields = {}
    addr_m = re.search(r'addr=(0x[0-9A-Fa-f]+)', line)
    data_m = re.search(r'data=(0x[0-9A-Fa-f]+)', line)
    if addr_m: fields['addr'] = addr_m.group(1)
    if data_m: fields['data'] = data_m.group(1)
    return fields

# Example: extract_apb_fields("... APB Write addr=0x1000 data=0xDEAD")
#          -> {'addr': '0x1000', 'data': '0xDEAD'}
```

解释

UVM 日志格式的关键特征是固定结构：UVM_INFO @ time: instance_path [TAG] message。

利用 split(':', 2) 限制分割次数（maxsplit=2），避免消息内容里如果出现冒号

（比如时间戳 "10:30:00"）导致过度分割。

分类逻辑核心：instance_path 里天然包含模块名（apb_agent, i2c_agent），

直接用字符串包含判断（'apb' in path）就能定位，

不需要正则的复杂模式匹配，这也是题目强调"UVM已有instance name"的用意——

不用额外解析消息文本，直接利用现有层级路径分类。

Regex 版本提供了更严格的匹配（确保只匹配真正的 UVM_INFO 行首），

生产代码中两种方式都可用，regex 在日志格式有细微变化时更稳健。

Q109. Algorithms: Check if a Number is a Power of 2 (Bit Manipulation)

QUESTION

Write a function to check if a given integer is a power of 2.

Use bit manipulation (no loops, no log functions).

ANSWER

```
def is_power_of_2(n: int) -> bool:
    """
    Check if n is a power of 2 using the classic bit trick.

    Key insight: a power of 2 in binary has EXACTLY ONE bit set
    (e.g. 8 = 0b1000, 16 = 0b10000).
    n & (n-1) clears the LOWEST set bit of n.
    If n is a power of 2, clearing its only set bit gives 0.
    If n is NOT a power of 2, n has multiple set bits, so
    n & (n-1) leaves at least one bit still set (non-zero).
```

```
    Args:
        n: integer to check

    Returns:
        True if n is a positive power of 2, False otherwise

    Time:  O(1) -- single bitwise AND operation
    Space: O(1)
    """
    return n > 0 and (n & (n - 1)) == 0

# -- Why n > 0 is required ------------------------------------------------
# n=0:  0 & (-1) = 0 in two's complement -- would incorrectly return True
#       without the n>0 check! 0 is NOT a power of 2.
# n<0:  negative numbers are never powers of 2 by definition.

# -- Step-by-step trace ---------------------------------------------------
# n=12 (0b1100): n-1=11 (0b1011)
#   0b1100 & 0b1011 = 0b1000 (=8, non-zero) -> NOT power of 2  [correct: 12 isn't]
#
# n=8  (0b1000): n-1=7  (0b0111)
#   0b1000 & 0b0111 = 0b0000 (=0)           -> IS power of 2   [correct: 8=2^3]

# -- Examples -------------------------------------------------------------
# is_power_of_2(1)    -> True   (2^0)
# is_power_of_2(2)    -> True   (2^1)
# is_power_of_2(3)    -> False
# is_power_of_2(4)    -> True   (2^2)
# is_power_of_2(1024) -> True   (2^10)
# is_power_of_2(0)    -> False  (not a power of 2 by definition)
# is_power_of_2(-4)   -> False  (negative, excluded by n>0 check)

# -- Alternative bit-trick: n & (-n) isolates the lowest set bit ------------
def is_power_of_2_alt(n: int) -> bool:
    """
    Alternative: isolate the lowest set bit using n & (-n),
    then check if it equals n itself (meaning ONLY that bit is set).
    """
    return n > 0 and (n & (-n)) == n

# -- Hardware equivalent (RTL bit trick) ----------------------------------
# function automatic bit is_pow2(logic [31:0] n);
#     return (n != 0) && ((n & (n - 1)) == 0);
# endfunction
```

解释

核心技巧 n & (n-1)：减1操作会把最低位的1变成0，并把它右边所有的0变成1。

对于只有1个bit为1的数（即2的幂），这个操作会把那唯一的1清零，结果为0。

对于有多个bit为1的数，清除最低位的1后，更高位的1仍然存在，结果非0。

n>0 检查是必须的陷阱点：0 在补码表示下 0&(0-1) 在某些语言/位宽下意外等于0，

会被误判为"是2的幂"，必须显式排除。

这是面试中最经典的位操作技巧之一，同样的 n&(n-1) trick 还能用于

"统计一个数中1的个数"（每次清除一个1，循环次数=1的个数）。

Q110. Algorithms: Maximum Subarray Sum with Subset Tracking (Kadane's)

QUESTION

Given an integer array, find the contiguous subarray with the maximum sum. Return both the maximum sum AND the actual subarray elements.

What is the time complexity?

ANSWER

```
def max_subarray(nums: list[int]) -> tuple[int, list[int], int, int]:
    """
    Kadane's algorithm: find contiguous subarray with maximum sum.
    Also tracks start/end indices to recover the actual subarray.

    Time:  O(n)  -- single pass
    Space: O(1)  -- only a few scalar variables (plus O(k) for output)

    Key idea: at each position i, the best subarray ENDING at i is either:
      (a) nums[i] alone (start fresh), or
      (b) extend the best subarray ending at i-1
      Whichever is larger is curr_sum.
      Track the global max across all positions.
    """
    if not nums:
        return 0, [], -1, -1

    max_sum   = curr_sum  = nums[0]
    start     = end       = 0
    temp_start            = 0   # tracks where the current window started

    for i in range(1, len(nums)):
        # Should we start fresh at nums[i], or extend the running window?
        if curr_sum + nums[i] < nums[i]:
            curr_sum   = nums[i]
            temp_start = i       # reset window start
        else:
            curr_sum  += nums[i]

        # Update global best if this window is better
        if curr_sum > max_sum:
            max_sum = curr_sum
            start   = temp_start
            end     = i

    return max_sum, nums[start : end + 1], start, end

# -- Examples ------------------------------------------------------------
# max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4])
#   -> sum=6, subarray=[4,-1,2,1], idx=[3:6]
#
# max_subarray([5, 4, -1, 7, 8])
#   -> sum=23, subarray=[5,4,-1,7,8], idx=[0:4]  (entire array)
#
# max_subarray([-1, -2, -3])
#   -> sum=-1, subarray=[-1], idx=[0:0]  (least negative = best we can do)

# -- Corner cases --------------------------------------------------------
# All negative:   return the single largest (least negative) element
# Single element: trivially that element is the answer
# All positive:   entire array is the optimal subarray
```

解释

Kadane 算法的核心判断："把当前元素加进现有窗口" vs "从当前元素重新开始"。

当 curr_sum + nums[i] < nums[i] 时，说明前面积累的和是负的，

拖累大于帮助，不如从 nums[i] 重新开始——这是 O(n) 解法的精髓。

追踪子数组本身（不只是最大值）需要额外维护 temp_start：

每次决定重新开始时更新 temp_start，每次全局最大值被刷新时将 start=temp_start、end=i。

全负数数组是常见的 corner case：算法仍然正确，

因为重新开始的条件保证了每个负数都被单独考察过，最终返回最大的那个（最小绝对值的负数）。

Q111. Algorithms: Count Number of 1 Bits in Binary (Popcount)

QUESTION

Given an integer n, return the number of 1 bits in its binary

representation (popcount / Hamming weight).

Implement using the n & (n-1) bit trick. What is the time complexity?

ANSWER

```
def count_ones(n: int) -> int:
    """
    Count the number of 1 bits in n using the bit-clearing trick.

    Key insight: n & (n-1) CLEARS the lowest set bit of n.
    (n-1) flips all bits from the lowest 1 downward:
      e.g. n=0b10110100: n-1=0b10110011
           n & (n-1)  = 0b10110000  (lowest 1 bit cleared)
    Repeat until n becomes 0; count = number of iterations = number of 1s.

    Time:  O(k)  where k = number of 1 bits  (NOT O(log n) bits!)
           For sparse integers (few 1s) this is much faster than
           shifting through all 32/64 bits one by one.
    Space: O(1)
    """
    count = 0
    while n:
        n &= (n - 1)   # clear lowest set bit
        count += 1
    return count

# -- Examples ------------------------------------------------------------
# count_ones(0)          -> 0    (0b00000000)
# count_ones(1)          -> 1    (0b00000001)
# count_ones(7)          -> 3    (0b00000111)
# count_ones(8)          -> 1    (0b00001000)
# count_ones(255)        -> 8    (0b11111111)
# count_ones(0b10110101) -> 5

# -- Alternative 1: shift-and-count (always O(log n) iterations) ----------
def count_ones_shift(n: int) -> int:
    count = 0
    while n:
        count += n & 1   # check lowest bit
        n >>= 1          # shift right
    return count
# Simpler to understand, but processes ALL bits including leading zeros.

# -- Alternative 2: Python built-in (for reference) ----------------------
# bin(n).count('1')   or   int.bit_count(n)  [Python 3.10+]

# -- Hardware RTL equivalent ---------------------------------------------
```

```
# In RTL, popcount is often implemented as a REDUCTION TREE of full adders.
# SystemVerilog built-in: $countones(vec) counts 1s in a vector.
# Example: $countones(8'b10110101) = 5
```

解释

n & (n-1) 技巧的时间复杂度是 O(k) 而非 O(log n)，

这里 k 是1的个数（汉明重量）——对于稀疏整数（大量0，少量1）

比逐位移位快得多，这是面试中区分"知道这个技巧"和"真正理解它"的关键。

这道题和上面"判断2的幂"（is_power_of_2）使用的是同一个底层 bit-trick：

n&(n-1) 清除最低位的1。is_power_of_2 只需检查这个操作后是否得到0（只有一个1）；

popcount 需要统计这个操作能执行多少次（循环计次）。

RTL 中对应的是 $countones() 系统函数，

在 UVM 约束里也常用到：`foreach (arr[i]) unique_check.sum() with (int'(item==arr[i])) == 1`

等写法本质上都在做某种形式的 popcount。

Q112. Algorithms: Longest Consecutive Sequence (LC 128)

QUESTION

Given an unsorted array of integers, return the length of the longest

consecutive sequence. Must run in O(n) time.

(LeetCode 128)

ANSWER

```
def longest_consecutive(nums: list[int]) -> int:
    """
    Find the longest consecutive sequence in an unsorted array.

    Args:
        nums: unsorted list of integers (may contain duplicates)

    Returns:
        length of the longest consecutive sequence

    Time:  O(n) -- each element visited at most twice
    Space: O(n) -- hash set for O(1) lookups

    Key insight: only START counting from a number n where (n-1) is
    NOT in the set. This ensures each sequence is counted exactly once
    from its true starting point. Without this check, we'd recount
    overlapping sequences and degrade to O(n^2).
    """
    num_set: set[int] = set(nums)   # O(n) build, O(1) lookup
    best: int = 0

    for n in num_set:
        # Only start a sequence from its true beginning
        if n - 1 not in num_set:
            length = 1
            while n + length in num_set:
                length += 1
            best = max(best, length)
```

```
    return best

# -- Why O(n) despite the nested while loop? ------------------------------
# The while loop only runs for numbers that are SEQUENCE STARTS (n-1 not in set).
# Each number n is visited AT MOST TWICE across the entire algorithm:
#   - Once as a potential start check (the if condition)
#   - Once as an extension of some sequence (the while loop)
# Total inner loop iterations across ALL outer iterations <= n.
# Therefore total work = O(n), NOT O(n^2).

# -- Examples -------------------------------------------------------------
# longest_consecutive([100, 4, 200, 1, 3, 2])  -> 4  (sequence: 1,2,3,4)
# longest_consecutive([0,3,7,2,5,8,4,6,0,1])   -> 9  (sequence: 0..8)
# longest_consecutive([])                       -> 0
# longest_consecutive([1])                      -> 1

# -- Corner cases to discuss ----------------------------------------------
# Duplicates:  set() deduplication handles this automatically.
#              [1,2,2,3] has sequence 1,2,3 of length 3 (not 4).
# All same:    [5,5,5,5] -> sequence length 1.
# Already sorted: algorithm still O(n), no optimization needed.
# Negative numbers: works correctly (n-1, n+1 arithmetic handles negatives).
#   [-3,-2,-1,0,1] -> sequence of length 5.
```

解释

O(n) 解法的关键是"只从序列的真正起点开始计数"：

用 n-1 not in set 来判断 n 是否是某个连续序列的起点，

这保证了每个序列只被完整遍历一次，避免了从序列中间重复计数。

嵌套 while 看起来是 O(n^2) 但实际是 O(n) 的原因：

每个数字在整个算法里最多被"碰到"两次（一次外层 for 循环，一次内层 while 扩展），

摊销（amortized）后总操作数是 O(n)——这是面试中必须能清楚解释的复杂度分析。

这道题的 O(n^2) 朴素解法是：对每个数字 n 往上遍历找连续数，

但没有 n-1 not in set 的过滤，同一序列会被从每个元素重新开始统计。

能主动说出"这个 while 看起来嵌套但实际上 amortized O(1) per element"是关键加分项。

Q113. Algorithms: Longest Substring Without Duplicates (C++)

QUESTION

Given a string, find the length of the longest substring without

repeating characters. Implement in C++ using sliding window.

(LeetCode 3)

ANSWER

```
#include <unordered_map>
#include <string>
#include <algorithm>
using namespace std;

int lengthOfLongestSubstring(string s) {
    /*
```

```
     * Sliding window with hash map tracking the LAST SEEN index of each char.
     *
     * left:  left boundary of the current valid window (no duplicates)
     * right: current character being considered
     *
     * When s[right] was seen before AND its last position >= left
     * (meaning it's inside the current window), we must shrink the
     * window by moving left to last_pos[s[right]] + 1.
     *
     * Time:  O(n)  -- each character visited at most twice
     * Space: O(min(m, n))  where m = charset size (e.g. 128 for ASCII)
     */
    unordered_map<char, int> last_pos;  // char -> last seen index
    int left = 0, best = 0;

    for (int right = 0; right < (int)s.size(); ++right) {
        char c = s[right];

        // If c is in the current window [left, right-1], move left past it
        if (last_pos.count(c) && last_pos[c] >= left) {
            left = last_pos[c] + 1;
        }

        last_pos[c] = right;                        // update last seen
        best = max(best, right - left + 1);          // window size
    }
    return best;
}

/*
 * Trace for "abcabcbb":
 *   r=0 'a': window=[0,0]="a",      best=1
 *   r=1 'b': window=[0,1]="ab",     best=2
 *   r=2 'c': window=[0,2]="abc",    best=3
 *   r=3 'a': 'a' last at 0>=left(0) -> left=1, window=[1,3]="bca", best=3
 *   r=4 'b': 'b' last at 1>=left(1) -> left=2, window=[2,4]="cab", best=3
 *   r=5 'c': 'c' last at 2>=left(2) -> left=3, window=[3,5]="abc", best=3
 *   r=6 'b': 'b' last at 4>=left(3) -> left=5, window=[5,6]="cb",  best=3
 *   r=7 'b': 'b' last at 6>=left(5) -> left=7, window=[7,7]="b",   best=3
 *   Result: 3  (substring "abc")
 *
 * Corner cases:
 *   ""         -> 0  (empty string: loop never executes)
 *   "bbbbb"    -> 1  (all same: window always shrinks to 1)
 *   "abcdef"   -> 6  (no duplicates: window grows to full string)
 *   " "        -> 1  (space is a valid character)
 */
```

解释

滑动窗口的关键判断：`last_pos[c] >= left` 不能省略。

如果字符 c 曾经出现过但在当前窗口左边界之外（已经被排除了），

就不需要移动 left——这避免了 left 向左倒退（left 只能单调向右）。

C++ 实现用 `unordered_map` 做 O(1) 平均查找，对比 Python 的 dict 原理相同。

另一种写法用长度 128 的 `int last[128] = {-1}` 替代 hash map（ASCII 字符集），

省去 hash 开销，适合面试手写时更简洁：

`if (last[c] >= left) left = last[c] + 1;`

Q114. Algorithms: Elevator Cargo -- Minimum Trips with Weight Limit

QUESTION

An elevator carries cargo from floor 1 to the top floor.

It can carry at most 2 items per trip, and the combined weight must

not exceed max_weight. Find the minimum number of trips needed.

ANSWER

```
def min_trips(weights: list[int], max_weight: int) -> int:
    """
    Find the minimum number of elevator trips to move all cargo.
    Each trip carries 1 or 2 items; combined weight <= max_weight.

    Strategy: sort + two pointers (greedy).
      - Sort weights ascending.
      - Try to pair the LIGHTEST remaining item with the HEAVIEST.
        If they fit together, take both (left++ and right--).
        If they don't fit, the heaviest must go alone (right-- only).
      - Each iteration = one trip.

    Time:  O(n log n)  -- dominated by sort
    Space: O(1)        -- sort in-place, two pointer variables
    """
    weights.sort()
    left, right = 0, len(weights) - 1
    trips = 0

    while left <= right:
        if weights[left] + weights[right] <= max_weight:
            left += 1    # lightest can ride with heaviest
        right -= 1       # heaviest always goes this trip (alone or paired)
        trips += 1

    return trips

# -- Examples ------------------------------------------------------------
# min_trips([3, 2, 2, 1], max_weight=3)
#   sorted: [1, 2, 2, 3]
#   Trip 1: 1+3=4 > 3 -> 3 goes alone.          right=2, trips=1
#   Trip 2: 1+2=3 <= 3 -> pair (1,2).            left=1, right=1, trips=2
#   Trip 3: left==right -> 2 goes alone.          trips=3
#   Result: 3 trips
#
# min_trips([1, 2, 3, 4], max_weight=5)
#   sorted: [1, 2, 3, 4]
#   Trip 1: 1+4=5 <= 5 -> pair (1,4).            left=1, right=2, trips=1
#   Trip 2: 2+3=5 <= 5 -> pair (2,3).            left=2, right=1 -> done, trips=2
#   Result: 2 trips
#
# min_trips([3, 3, 3], max_weight=6)
#   sorted: [3, 3, 3]
#   Trip 1: 3+3=6 <= 6 -> pair (3,3).            left=1, right=1, trips=1
#   Trip 2: left==right -> 3 goes alone.          trips=2
#   Result: 2 trips

# -- Corner cases --------------------------------------------------------
# Single item:      min_trips([5], 10) -> 1 trip (only one item)
# All heavy (none can pair): min_trips([6,7,8], 9) -> 3 trips (all solo)
# All light (all pair):      min_trips([1,1,1,1], 5) -> 2 trips
# max_weight == heaviest:    any item heavier than max_weight -> INFEASIBLE
```

```
#                           (add a validation check in production code)
```

解释

贪心策略的正确性证明：最重的货物必须占用一个电梯槽位，

唯一能优化的是它是否能和另一件货物配对——能配对的最优选择是最轻的那件。

因为最轻的货物和任何人配对都是最省槽的：和最重的配不进就和任何人都配不了，

而和最重的配进了就省了一次单独运最轻货物的机会。

双指针：left 找最轻，right 找最重。

如果能配对：left++ right-- trips++；

如果不能配对：最重的单独走 right-- trips++。

每次循环 right 一定减少，所以最多 n 次循环，O(n) 扫描 + O(n log n) 排序。

面试中主动说出"这是经典的贪心+双指针，类似于接雨水/救生艇问题"是加分项。

Q115. Algorithms: Python -- Remove Vowels / Remove Consonants from String

QUESTION

Write Python functions to:

(a) Remove all consonants from a string.

(b) Remove all vowels from a string.

ANSWER

```
import re

# ================================================================
# (a) Remove consonants -- keep vowels, digits, spaces, punctuation
# ================================================================
def remove_consonants(s: str) -> str:
    """
    Remove all consonant letters from the string.
    Keep: vowels (a,e,i,o,u -- upper and lower), digits, spaces, punctuation.
    Remove: any letter that is NOT a vowel.
    """
    # Match any letter ([a-zA-Z]) that is NOT a vowel -> remove it
    return re.sub(r'[^aeiouAEIOU\s\d\W]', '', s)

# Simpler alternative using character class complement:
def remove_consonants_v2(s: str) -> str:
    # Keep only vowels + non-letter characters
    return re.sub(r'[b-df-hj-np-tv-zB-DF-HJ-NP-TV-Z]', '', s)

# -- Examples --
# remove_consonants("Hello World") -> "eo o"
# remove_consonants("Python")      -> "o"
# remove_consonants("AEIOUaeiou")  -> "AEIOUaeiou"  (all vowels, nothing removed)

# ================================================================
# (b) Remove vowels -- keep consonants, digits, spaces, punctuation
# ================================================================
def remove_vowels(s: str) -> str:
    """
    Remove all vowel letters from the string.
```

```
    Keep: consonants, digits, spaces, punctuation.
    """
    return re.sub(r'[aeiouAEIOU]', '', s)

# -- Examples --
# remove_vowels("Hello World") -> "Hll Wrld"
# remove_vowels("Python")      -> "Pythn"
# remove_vowels("AEIOUaeiou")  -> ""  (all vowels removed)

# ================================================================
# Non-regex alternative (for reference)
# ================================================================
def remove_vowels_no_regex(s: str) -> str:
    vowels = set('aeiouAEIOU')
    return ''.join(c for c in s if c not in vowels)

def remove_consonants_no_regex(s: str) -> str:
    vowels = set('aeiouAEIOU')
    # Keep: vowels OR non-letter characters
    return ''.join(c for c in s if c in vowels or not c.isalpha())
```

解释

Regex 版本的关键是字符类的取反：

`[^aeiouAEIOU\s\d\W]` 匹配"不是元音、不是空白、不是数字、不是非单词字符的字符"

——也就是辅音字母（consonant），然后用空字符串替换之。

`\W` 是"非单词字符"（标点、空格等），配合 `^`（取反）确保只删字母里的辅音，

不误删空格、数字和标点。

非 regex 版本逻辑更直观：维护一个 vowels 集合，用 set.in 判断，

适合面试时快速手写，不依赖记忆 regex 语法。

注意 remove_consonants 的补集逻辑：保留 `c in vowels or not c.isalpha()`，

也就是"是元音"或"根本不是字母"的字符都保留，其余（辅音）删去。

Q116. Algorithms: Sort Python Dictionary by Value; Regex Match Two Strings

QUESTION

(a) Sort a Python dictionary by its values (ascending).

(b) Write a regular expression that matches both:

"SCKLNQTD2T67" and "CKHLQD6T87"

ANSWER

```
# ================================================================
# (a) Sort dictionary by value
# ================================================================
d = {'banana': 3, 'apple': 1, 'cherry': 2}

# Method 1: sorted() with lambda (most common in interviews)
sorted_items = sorted(d.items(), key=lambda x: x[1])
sorted_dict  = dict(sorted_items)
print(sorted_dict)   # {'apple': 1, 'cherry': 2, 'banana': 3}

# Method 2: operator.itemgetter (faster for large dicts)
```

```
import operator
sorted_dict2 = dict(sorted(d.items(), key=operator.itemgetter(1)))

# Descending order:
sorted_desc = dict(sorted(d.items(), key=lambda x: x[1], reverse=True))

# ============================================================
# (b) Regex to match SCKLNQTD2T67 and CKHLQD6T87
# ============================================================
#
# Analyze the two strings:
#   SCKLNQTD2T67
#   CKHLQD6T87
#
# Pattern breakdown:
#   - Starts with one or more UPPERCASE letters
#   - Followed by one or more DIGITS
#   - Then the letter T
#   - Followed by one or more DIGITS
#   - End of string
#
import re

pattern = r'^[A-Z]+\d+T\d+$'

test_strings = ['SCKLNQTD2T67', 'CKHLQD6T87',
                'abc123T45',    # lowercase -- should NOT match
                'ABCT12',       # missing second digit group -- no match
                'AB12T']        # missing trailing digits -- no match

for s in test_strings:
    match = bool(re.match(pattern, s))
    print(f"  '{s}': {match}")
# SCKLNQTD2T67: True
# CKHLQD6T87:   True
# others:       False

# Pattern explanation:
#   ^       start of string
#   [A-Z]+  one or more uppercase letters
#   \d+     one or more digits
#   T       literal 'T'
#   \d+     one or more digits
#   $       end of string
```

解释

字典排序最简洁的写法是 `sorted(d.items(), key=lambda x: x[1])`，

返回的是 sorted list of tuples，再用 dict() 包装回字典。

注意 Python 3.7+ 保证 dict 的插入顺序，所以这个转换是有意义的。

正则表达式分析的关键是先"分解"两个字符串的公共结构：

SCKLNQTD2T67 = [字母部分SCKLNQTD] + [数字2] + [T] + [数字67]

CKHLQD6T87 = [字母部分CKHLQD] + [数字6] + [T] + [数字87]

共同模式：`^[A-Z]+\d+T\d+$`

面试技巧：先列出两个字符串的结构，找共同规律，再写 pattern，

比直接猜 pattern 更系统，也更容易和面试官解释。

Q117. Algorithms: Sort Log File by Age and Write to Output

QUESTION

Given a log file where each line has format "Name Age",

sort the entries by age (ascending) and write to a sorted output file.

ANSWER

```
# Input log file (log.txt):
#   Alice 30
#   Bob 25
#   Carol 35
#   Dave 28

def sort_log_by_age(input_file: str, output_file: str) -> None:
    """
    Read a log file of "Name Age" entries, sort by age, write sorted output.

    Time:  O(n log n) -- sort
    Space: O(n)       -- store all lines in memory
    """
    with open(input_file, 'r') as f:
        lines = [line.strip() for line in f if line.strip()]

    # Sort by the second field (age), converted to int for numeric sort
    sorted_lines = sorted(lines, key=lambda x: int(x.split()[1]))

    with open(output_file, 'w') as f:
        f.write('\n'.join(sorted_lines) + '\n')

# -- Output (sorted_log.txt) ----------------------------------------
# Bob 25
# Dave 28
# Alice 30
# Carol 35

# -- Robust version: handle extra whitespace and missing fields ----------
def sort_log_robust(input_file: str, output_file: str) -> None:
    entries = []
    with open(input_file, 'r') as f:
        for lineno, line in enumerate(f, 1):
            parts = line.strip().split()
            if len(parts) < 2:
                print(f"Warning: line {lineno} malformed: {line.strip()}")
                continue
            try:
                name, age = parts[0], int(parts[1])
                entries.append((name, age))
            except ValueError:
                print(f"Warning: age not integer on line {lineno}")

    entries.sort(key=lambda x: x[1])   # sort by age

    with open(output_file, 'w') as f:
        for name, age in entries:
            f.write(f"{name} {age}\n")
```

解释

核心是 `sorted(lines, key=lambda x: int(x.split()[1]))` :

`x.split()[1]` 取每行第二个字段（年龄字符串），`int()` 转换为整数比较（避免字符串排序把 "9" 排在 "25" 后面）。

Robust 版本的关键改进：捕获格式错误（字段缺失、年龄非整数），

这在实际 log 解析任务中必不可少，面试中主动提出鲁棒性是加分项。

Q118. Algorithms: Python -- Find Unpaired Numbers in a List

QUESTION

Given a list of numbers where every number appears twice except for

some unpaired ones, return all numbers that appear an odd number of times.

ANSWER

```
from collections import Counter

def find_unpaired(nums: list[int]) -> list[int]:
    """
    Find all numbers that appear an odd number of times (unpaired).

    Time:  O(n)
    Space: O(n)
    """
    counts = Counter(nums)
    return [num for num, cnt in counts.items() if cnt % 2 == 1]

# -- Examples ------------------------------------------------------------
# find_unpaired([1, 2, 3, 2, 3])    -> [1]
# find_unpaired([4, 1, 2, 1, 2])    -> [4]
# find_unpaired([1, 1, 2, 2, 3])    -> [3]
# find_unpaired([1, 2, 3])          -> [1, 2, 3]  (all unpaired)
# find_unpaired([1, 1])             -> []          (all paired)

# -- O(1) space version (XOR trick -- only works when exactly ONE unpaired) --
def find_single_unpaired(nums: list[int]) -> int:
    """
    If exactly ONE number is unpaired, XOR of all elements gives that number.
    Paired numbers XOR to 0 (x ^ x = 0); unpaired survives.

    Time:  O(n)
    Space: O(1)
    """
    result = 0
    for n in nums:
        result ^= n
    return result

# find_single_unpaired([1, 2, 3, 2, 3]) -> 1
# find_single_unpaired([4, 1, 2, 1, 2]) -> 4

# -- Corner cases --------------------------------------------------------
# Empty list:       find_unpaired([])    -> []
# All paired:       find_unpaired([1,1]) -> []
# Multiple unpaired: find_unpaired([1,2,3]) -> [1,2,3]
```

解释

Counter 版本最通用：统计每个数字出现次数，筛出奇数次的——

能处理"多个数字都是奇数次"的情况（如 [1,2,3] 全都出现1次）。

XOR 版本是经典 bit trick，空间最优（O(1)），但只适用于"恰好1个数字是奇数次"的特殊情况：

`x ^ x = 0`（同一个数 XOR 自身等于0），`0 ^ x = x`（任何数 XOR 0 等于自身），

所以所有成对的数消消乐，最后只剩下那一个孤单的数。

面试最佳策略：先说通用版（Counter），再主动提出 XOR trick 并说明适用条件，

展示知道不同方案的权衡而不是死记硬背一个答案。

Q119. Algorithms: Find the 3rd Smallest Value in an Integer Stream

QUESTION

Given a continuous integer stream, output the 3rd smallest value

seen so far at each step. The stream can be arbitrarily long.

Generalize to the K-th smallest.

ANSWER

```
# ================================================================
# Algorithm: Max-Heap of size K
# ================================================================
#
# Maintain a max-heap of the K smallest values seen so far.
# The root of the max-heap is always the K-th smallest element.
#
# For each new element x:
#   - If heap size < K: add x (we don't have K values yet)
#   - Else if x < heap.top (x is smaller than current Kth):
#       pop heap.top, push x  (replace the largest of our K smallest)
#   - Else: ignore x (x >= Kth, doesn't change the answer)
#
# Time:  O(n log K) -- log K per element for heap operations
# Space: O(K)       -- only K elements in the heap

import heapq   # Python: min-heap by default; negate for max-heap

def kth_smallest_stream(stream: list[int], k: int = 3):
    """
    Yield (input_value, kth_smallest_so_far) for each element.
    kth_smallest is None until at least k elements have been seen.
    """
    heap = []   # max-heap stored as negated values

    for x in stream:
        if len(heap) < k:
            heapq.heappush(heap, -x)          # add until we have k values
        elif x < -heap[0]:                     # x smaller than current kth
            heapq.heapreplace(heap, -x)        # O(log k): pop max, push x

        kth = -heap[0] if len(heap) == k else None
        yield x, kth

# -- Trace: stream = [5, 3, 8, 1, 2, 9, 4], k=3 ----------------------
# Input | Heap (max of 3 smallest) | 3rd smallest
#   5   | [-5]                     | None  (only 1 seen)
#   3   | [-5, -3]                 | None  (only 2 seen)
#   8   | [-8, -3, -5]             | 8     (3rd smallest of {5,3,8} = 8)
#   1   | [-5, -3, -1]  1 < 8 -> replace 8 with 1  -> 3rd = 5
#   2   | [-5, -3, -2]  2 < 5 -> replace 5 with 2  -> 3rd = 3  (wrong order above, heap rebalances)
#   9   | [-5, -3, -2]  9 >= 5 -> ignore            -> 3rd = 3  wait: heap has {1,2,3}? Let's retrace
```

```
#
# Correct trace with {5,3,8,1,2}:
#   After 5,3,8: heap = {5,3,8}, max=8, 3rd=8
#   See 1: 1<8 -> replace 8 with 1: heap={5,3,1}, max=5, 3rd=5
#   See 2: 2<5 -> replace 5 with 2: heap={3,2,1}, max=3, 3rd=3
#   See 9: 9>3 -> ignore:           heap={3,2,1}, max=3, 3rd=3
#   See 4: 4>3 -> ignore:           heap={3,2,1}, max=3, 3rd=3
#   Final answer: 3rd smallest in [5,3,8,1,2,9,4] = 3 ✓  (sorted: 1,2,3,4,5,8,9)

# -- Corner cases ------------------------------------------------------
# Stream has < k elements:  return None until k elements seen
# All same values:          kth = that value
# Sorted ascending stream:  heap fills at position k, no replacements after
# Sorted descending stream:  every element replaces the current kth (worst case for replacements
```

解释

用大小为 K 的最大堆（max-heap）维护"目前见过的最小 K 个数"是解决流式第 K 小问题的标准方法。

堆顶始终是这 K 个数里最大的那个——即当前的第 K 小值。

当新元素比堆顶小时，它应该进入"最小 K 个"的集合，同时把原来的第 K 小（堆顶）踢出去。

当新元素 >= 堆顶时，它不可能是前 K 小，直接忽略。

`heapreplace` 是 Python 里弹出并推入的原子操作，比先 pop 再 push 快（只需一次堆化）。

Python 的 `heapq` 是 min-heap，用负值实现 max-heap 是标准 trick。

Q120. Algorithms: Fibonacci -- Recursive, Iterative, Return Entire List

QUESTION

Write a Python Fibonacci function that returns the ENTIRE list

up to the nth element. Show recursive, iterative, and generator versions.

ANSWER

```
# ================================================================
# Version 1: Iterative (RECOMMENDED -- O(n) time, O(n) space)
# ================================================================
def fib_list(n: int) -> list[int]:
    """
    Return Fibonacci sequence as a list: [F(0), F(1), ..., F(n-1)]
    F(0)=0, F(1)=1, F(2)=1, F(3)=2, F(4)=3, ...
    """
    if n <= 0: return []
    if n == 1: return [0]
    result = [0, 1]
    for _ in range(2, n):
        result.append(result[-1] + result[-2])
    return result

# fib_list(0) -> []
# fib_list(1) -> [0]
# fib_list(2) -> [0, 1]
# fib_list(5) -> [0, 1, 1, 2, 3]
# fib_list(8) -> [0, 1, 1, 2, 3, 5, 8, 13]

# ================================================================
# Version 2: Recursive (elegant but slow -- O(2^n), avoid for large n)
# ================================================================
```

```python
def fib_recursive(n: int) -> int:
    """Returns the nth Fibonacci number (0-indexed)."""
    if n <= 1: return n
    return fib_recursive(n-1) + fib_recursive(n-2)

def fib_list_recursive(n: int) -> list[int]:
    return [fib_recursive(i) for i in range(n)]
# O(n * 2^n) -- very slow due to repeated subproblem computation

# ============================================================
# Version 3: Recursive with memoization (O(n) time, O(n) space)
# ============================================================
from functools import lru_cache

@lru_cache(maxsize=None)
def fib_memo(n: int) -> int:
    if n <= 1: return n
    return fib_memo(n-1) + fib_memo(n-2)

def fib_list_memo(n: int) -> list[int]:
    return [fib_memo(i) for i in range(n)]
# Now O(n) time -- each subproblem computed once and cached

# ============================================================
# Version 4: Generator (memory efficient for very large n)
# ============================================================
def fib_generator():
    """Yields Fibonacci numbers indefinitely: 0, 1, 1, 2, 3, 5, ..."""
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b  # simultaneous update (no temp variable needed)

def fib_list_gen(n: int) -> list[int]:
    """Use generator to get first n Fibonacci numbers."""
    gen = fib_generator()
    return [next(gen) for _ in range(n)]

# ============================================================
# Comparison
# ============================================================
#
#   Version     | Time     | Space | Notes
#   ------------+----------+-------+------------------------
#   Iterative   | O(n)     | O(n)  | Recommended for interviews
#   Recursive   | O(2^n)   | O(n)  | Simple but slow
#   Memoized    | O(n)     | O(n)  | Best recursive approach
#   Generator   | O(n)     | O(1)  | Best for streaming/large n
```

解释

迭代版本是面试首选：简洁、O(n) 时间、无递归栈溢出风险。

`result[-1] + result[-2]` 是 Python 负索引的优雅应用，始终引用最后两个元素。

Generator 版本的亮点是 `a, b = b, a + b`——Python 的元组赋值是原子的，

右边同时求值，左边同时赋值，无需中间变量，完美表达了斐波那契的递推关系。

面试常见追问：递归为什么慢？因为 `fib(5)` 调用 `fib(4)` 和 `fib(3)`，

而 `fib(4)` 也调用 `fib(3)`——同一个子问题被反复计算，

总调用次数是 O(2^n)，用 `lru_cache` 记忆化后降为 O(n)。

Brain Teasers

Q121. Brain Teaser: Find the Lighter Stone Among 9 (Balance Scale)

QUESTION

You have 9 identical-looking stones. One is lighter than the others.

Using a balance scale (no weights), what is the minimum number of

weighings needed to guarantee finding the lighter stone?

ANSWER

```
// Answer: 2 weighings.
//
// Strategy: divide into groups of 3.
//
// Weighing 1: Put 3 stones on each side, leave 3 aside.
//   Case A: Left side goes UP  -> lighter stone is in the LEFT group (3 stones)
//   Case B: Right side goes UP -> lighter stone is in the RIGHT group (3 stones)
//   Case C: Balance is EQUAL   -> lighter stone is in the SET-ASIDE group (3 stones)
//   After weighing 1: we have narrowed down to exactly 3 suspect stones.
//
// Weighing 2: Take the 3 suspect stones.
//             Put 1 on each side, leave 1 aside.
//   Case A: Left goes UP   -> LEFT stone is lighter. FOUND.
//   Case B: Right goes UP  -> RIGHT stone is lighter. FOUND.
//   Case C: Balance EQUAL  -> SET-ASIDE stone is lighter. FOUND.
//
// Total: 2 weighings to guarantee finding the lighter stone.
//
// Why 2 is optimal (cannot do in 1):
//   With 1 weighing you get 3 possible outcomes (left up, right up, equal).
//   Each outcome can at most eliminate 1 candidate per outcome, so
//   1 weighing can distinguish at most 3 stones.
//   With 9 stones, 1 weighing is insufficient (9 > 3).
//
// Generalization: with k weighings, you can find the lighter among 3^k stones.
//   k=1: up to 3 stones
//   k=2: up to 9 stones
//   k=3: up to 27 stones
//   Formula: ceil(log3(N)) weighings for N stones.
```

解释

这道题的核心是信息论视角：一次天平称量有3种可能结果（左重/右重/平衡），

等价于获得 log2(3) ≈ 1.58 比特信息。

N 个石头中找一个，需要的信息量是 log2(N) 比特。

9个石头 = log2(9) ≈ 3.17 比特，2次称量提供 2 × 1.58 = 3.16 比特，刚好够。

实际上用三分法（每次把候选集三等分）恰好让每次称量的信息量最大化，

这是"最优策略"的直觉来源。

面试中能说出"每次称量结果有3种，所以以3为底数计算"就是满分回答。

Q122. Brain Teaser: Blue and Red Balls -- What Color Is the Last Ball?

QUESTION

You have a bag with N blue balls and M red balls.

Each turn, randomly pick 2 balls and apply these rules:

- 2 blue -> put back 1 blue (remove 1 blue)

- 2 red -> put back 1 blue (remove both red, add 1 blue)

- 1 blue + 1 red -> put back 1 red (remove 1 blue)

Repeat until 1 ball remains.

What color is the last ball? Does the answer depend on N?

ANSWER

```
// ================================================================
// Answer (deterministic -- does NOT depend on N, only on M % 2)
// ================================================================
//
//   If M (red count) is ODD  -> last ball is always RED
//   If M (red count) is EVEN -> last ball is always BLUE
//
//   The number of blue balls N has NO effect on the final color.

// ================================================================
// Proof: Red count parity is an invariant
// ================================================================
//
// Examine how red count changes in each case:
//
//   Case 1: Pick 2 blue (B,B) -> put back 1 blue
//     Red count change: 0  (no red involved)
//     Parity change: none
//
//   Case 2: Pick 2 red (R,R) -> put back 1 blue
//     Red count change: -2  (remove 2 red, add 0 red)
//     Parity change: -2 -> SAME parity (even - even = even, odd - even = odd)
//
//   Case 3: Pick 1 blue + 1 red (B,R) -> put back 1 red
//     Red count change: 0  (remove 1 red, add 1 red back)
//     Parity change: none
//
// In ALL three cases, red count changes by 0 or -2.
// Subtracting an even number never changes parity.
// Therefore: parity of red count is INVARIANT throughout the process.
//
// When only 1 ball remains:
//   The ball is either RED (red count = 1, odd)
//            or BLUE (red count = 0, even)
//
// Since parity is preserved:
//   Initial M odd  -> final red count is odd  -> last ball is RED
//   Initial M even -> final red count is even -> last ball is BLUE (0 red)

// ================================================================
// Worked examples
// ================================================================
//
// Example 1: 5 blue, 3 red (M=3, odd -> RED)
//   Step: pick R,R -> put B -> (6 blue, 1 red)
```

```
//   Step: pick B,R -> put R -> (5 blue, 1 red)
//   Step: pick B,B -> put B -> (4 blue, 1 red)
//   ... eventually: (1 blue, 1 red)
//   Step: pick B,R -> put R -> (0 blue, 1 red)
//   Last ball: RED ✓
//
// Example 2: 4 blue, 2 red (M=2, even -> BLUE)
//   The process always ends with 1 BLUE ball regardless of order.
//   Last ball: BLUE ✓
//
// Example 3: N blue, 0 red (M=0, even -> BLUE)
//   No red balls at all. Only case (B,B)->B applies.
//   Blue count decreases by 1 each step. Always ends with 1 BLUE.
//   Last ball: BLUE ✓

// ============================================================
// Key insight for the interview
// ============================================================
//
// The trick is identifying the INVARIANT:
//   Not the total count (changes every step),
//   not the blue count (changes freely),
//   but the PARITY of the red count.
//
// This is a classic invariant-based puzzle.
// The answer is completely DETERMINISTIC:
//   No matter what order balls are picked, the final color is fixed
//   by M mod 2 alone. N (blue count) is irrelevant.
```

解释

这道题的关键是找到"不变量"（invariant）：红球数量的奇偶性在整个过程中保持不变。

三种操作对红球数量的影响：B+B→(-0)，R+R→(-2)，B+R→(-1+1=0)。

变化量只有 0 和 -2，都是偶数，所以奇偶性永远不变。

最终只剩1个球时：红球数是1（奇数）就是红球，是0（偶数）就是蓝球。

初始奇偶性决定终态——这就是答案。

蓝球数量完全不影响结果：因为蓝球可以在三种操作中自由增减（+1, -1, -1），

它不携带任何"守恒信息"，只有红球数的奇偶性是守恒量。

面试中答出"找不变量"的思路比直接猜答案更重要：

不变量分析是计算机科学和数学中处理离散过程的核心方法，

能主动说出"我要找一个在每步操作中保持不变的量"是思维方式的体现。